ORBIT · PYTHON

Python 十套卷

全部 10 套 · 599 题 · 完整题目与答案

本文件包含十套卷的全部题目、参考答案与解析。题目按套和题型排列，答案紧随题目。

第 1 套

60 题

一、单项选择题

01 • 单项选择

在Python中，以下哪个方法用于创建类的实例？

A. __new__()
B. __init__()
C. __create__()
D. __instance__()

答案 B. __init__()

解析 Python中创建实例涉及两个方法：
• __new__() ：真正的构造方法，负责创建对象实例，返回实例对象
• __init__() ：初始化方法，在对象创建后调用，用于初始化对象属性
执行顺序： __new__() → __init__()
示例：

```
class MyClass:
    def __new__(cls):
        print("调用__new__，创建实例")
        return super().__new__(cls)
    def __init__(self):
        print("调用__init__，初始化实例")
```

知识拓展：通常我们说的"构造方法"指 __init__() ，因为 __new__() 很少需要重写。 __new__() 的主要应用场景是实现单例模式或继承不可变类型（如int、str）。

02 • 单项选择

关于Python的 self 参数，以下说法正确的是：

A. self 是Python的关键字
B. self 必须作为实例方法的第一个参数
C. self 可以用任何名称替代
D. B和C都正确

答案 D. B和C都正确

解析 • self 不是Python关键字，只是一个约定俗成的命名
• 可以用其他名称（如 this 、 me ）替代，但不推荐
• 实例方法必须将实例本身作为第一个参数
示例：

```
class Test:
    def method1(self):
        # 标准用法
        print(self)
    def method2(this):
        # 可以，但不推荐
        print(this)
```

知识拓展：类方法使用 cls 作为第一个参数，静态方法不需要特殊的第一个参数。

03 • 单项选择

以下哪个装饰器用于定义静态方法？

A. @staticmethod

B. @classmethod

C. @property

D. @static

答案 A. @staticmethod

解析 Python提供三种方法装饰器：

- @staticmethod ：静态方法，不需要访问类或实例
- @classmethod ：类方法，第一个参数是类本身（cls）
- @property ：属性装饰器，将方法转为属性访问

对比示例：

```
class MyClass:
    @staticmethod
    def static_method():
        return "静态方法"
    @classmethod
    def class_method(cls):
        return f"类方法，类名：{cls.__name__}"
    @property
    def my_property(self):
        return "属性"
```

04 • 单项选择

Python中， __str__() 和 __repr__() 的主要区别是：

A. 完全相同

B. __str__() 面向用户， __repr__() 面向开发者

C. __str__() 面向开发者， __repr__() 面向用户

D. 两者都不能被重写

答案 B. __str__() 面向用户， __repr__() 面向开发者

解析 • __str__() ：返回用户友好的字符串，用于 print() 和 str()

- __repr__() ：返回开发者友好的字符串，用于调试和 repr()
- 理想情况下， eval(repr(obj)) == obj

示例：

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __str__(self):
        return f"点({self.x}, {self.y})"
    # 用户友好
    def __repr__(self):
        return f"Point({self.x}, {self.y})"
# 开发者友好
p = Point(3, 4)
print(str(p))
```

```
# 点(3, 4)
print(repr(p))
# Point(3, 4)
```

05 • 单项选择

以下代码的输出是什么？

```
class Counter:
    count = 0
    def __init__(self):
        Counter.count += 1
c1 = Counter()
c2 = Counter()
print(Counter.count)
```

A. 0
B. 1
C. 2
D. 报错

答案 C. 2

解析 这是类变量（类属性）的使用示例：
- count 是类变量，所有实例共享
- 每次创建实例时， __init__() 中 Counter.count += 1 执行一次
- 创建c1时：count = 0 + 1 = 1
- 创建c2时：count = 1 + 1 = 2

类变量vs实例变量：

```
class Demo:
    class_var = 0
    # 类变量，所有实例共享
    def __init__(self):
        self.instance_var = 0
        # 实例变量，每个实例独有
        Demo.class_var += 1
```

06 • 单项选择

关于Python的继承，以下说法错误的是：

A. Python支持多重继承
B. 子类会继承父类的所有属性和方法
C. 子类必须重写父类的所有方法
D. 使用 super() 可以调用父类方法

答案 C. 子类必须重写父类的所有方法

解析 Python继承的特点：
- 支持单继承和多重继承
- 子类自动继承父类的所有属性和方法
- 子类可以选择性
  可以选择性重写父类方法（方法覆盖）
- 使用 super() 调用父类方法

示例：

```
class Parent:
```

```
    def method1(self):
        print("父类方法1")
    def method2(self):
        print("父类方法2")
class Child(Parent):
    def method1(self):
        # 重写method1
        print("子类方法1")
# method2不重写，直接继承
```

07 • 单项选择

列表推导式 `[i**2 for i in range(6) if i % 2 == 1]` 的结果是：

A. `[1, 9, 25]`
B. `[0, 4, 16]`
C. `[1, 3, 5]`
D. `[0, 1, 4, 9, 16, 25]`

答案 A. `[1, 9, 25]`

解析 列表推导式的执行流程：

`[i**2 for i in range(6) if i % 2 == 1]`

逐步分析：

- `range(6)` 生成：0, 1, 2, 3, 4, 5
- `if i % 2 == 1` 过滤奇数：1, 3, 5
- `i**2` 计算平方：$1^2=1$, $3^2=9$, $5^2=25$
- 结果：`[1, 9, 25]`

08 • 单项选择

以下哪个不是可迭代对象？

A. 列表
B. 字符串
C. 整数
D. 字典

答案 C. 整数

解析 可迭代对象必须实现 `__iter__()` 方法或 `__getitem__()` 方法：

- 列表、元组、字符串、字典、集合：都是可迭代对象
- 整数、浮点数：不是可迭代对象

验证方法：

```
from collections.abc import Iterable
print(isinstance([1, 2], Iterable))
# True
print(isinstance("abc", Iterable))
# True
print(isinstance(42, Iterable))
# False
```

09 • 单项选择

关于生成器，以下说法正确的是：

A. 生成器会一次性生成所有值

B. 生成器使用 yield 关键字

C. 生成器比列表占用更多内存

D. 生成器不能用于循环

答案 B. 生成器使用 yield 关键字

解析 生成器的特点：

- 使用 yield 关键字定义
- 惰性求值

  惰性求值：按需生成值，不会一次性生成所有值
- 内存友好

  内存友好：只保存当前状态，不存储所有值
- 只能迭代一次

生成器vs列表：

```
        列表：
# 列表：立即生成所有值
list_comp = [x**2 for x in range(10000)]
# 占用大量内存
# 生成器：按需生成
gen_exp = (x**2 for x in range(10000))
# 内存占用小
```

10 • 单项选择

以下代码的输出是什么？

```
def gen():
    for i in range(3):
        yield i
g = gen()
print(next(g))
```

A. 0 1

B. 1 2

C. 0 0

D. 报错

答案 A. 0 1

解析 生成器的执行流程：

```
def gen():
    for i in range(3):
        # i = 0, 1, 2
        yield i
```

执行过程：

1. g = gen() ：创建生成器对象，不执行函数体
2. next(g) ：执行到第一个 yield ，返回0
3. next(g) ：继续执行到下一个 yield ，返回1

11 • 单项选择

关于迭代器协议，以下说法正确的是：

A. 只需实现 __iter__() 方法

B. 只需实现 __next__() 方法

C. 需要同时实现 __iter__() 和 __next__() 方法

D. 不需要实现任何特殊方法

答案 C. 需要同时实现 `__iter__()` 和 `__next__()` 方法

解析 迭代器协议要求实现两个方法：

- `__iter__()`：返回迭代器对象本身
- `__next__()`：返回下一个值，无值时抛出 StopIteration

自定义迭代器：

```
class Counter:
    def __init__(self, max_num):
        self.max_num = max_num
        self.num = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.num < self.max_num:
            self.num += 1
            return self.num
        raise StopIteration
```

12 • 单项选择

装饰器本质上是：

A. 一个类

B. 一个函数

C. 一个模块

D. 一个关键字

答案 B. 一个函数

解析 装饰器是一个高阶函数：

- 接收一个函数作为参数
- 返回一个新函数
- 本质是函数的包装器

基本结构：

```
def decorator(func):
    def wrapper(*args, **kwargs):
        # 前置处理
        result = func(*args, **kwargs)
        # 后置处理
        return result
    return wrapper
@decorator
def my_function():
    pass
# 等价于：my_function = decorator(my_function)
```

13 • 单项选择

以下代码的输出是什么？

```
def decorator(func):
    def wrapper():
        return func() * 2
    return wrapper
```

```
@decorator
def get_num():
    return 5
print(get_num())
```

A. 5
B. 10
C. 25
D. 报错

答案 B. 10

解析 装饰器的执行流程：

```
def double(func):
    def wrapper():
        return func() * 2
    return wrapper
@double
def get_num():
    return 5
```

执行过程：

1. get_num() 被 double 装饰
2. 调用 get_num() 实际调用 wrapper()
3. wrapper() 执行： func() * 2 = 5 * 2 = 10

14 • 单项选择

关于 *args 和 **kwargs ，以下说法正确的是：

A. *args 用于接收位置参数， **kwargs 用于接收关键字参数
B. *args 用于接收关键字参数， **kwargs 用于接收位置参数
C. 两者完全相同
D. 只能使用这两个名称

答案 A. *args 用于接收位置参数， **kwargs 用于接收关键字参数

解析 Python的可变参数：

- *args ：接收任意数量的位置参数，存为元组
- **kwargs ：接收任意数量的关键字参数，存为字典

示例：

```
def func(*args, **kwargs):
    print(f"args: {args}, type: {type(args)}")
    print(f"kwargs: {kwargs}, type: {type(kwargs)}")
func(1, 2, 3, a=4, b=5)
# args: (1, 2, 3), type: <class 'tuple'>
# kwargs: {'a': 4, 'b': 5}, type: <class 'dict'>
```

15 • 单项选择

以下代码的输出是什么？

```
a, b, *c = [1, 2, 3, 4, 5]
print(c)
```

A. [3, 4, 5]
B. [1, 2]
C. 3

D. [1, 2, 3, 4, 5]

答案 A. [3, 4, 5]

解析 解包赋值（序列解包）：

a, b, *c = [1, 2, 3, 4, 5]

执行过程：

- a = 1 （第一个元素）
- b = 2 （第二个元素）
- *c 接收剩余元素： c = [3, 4, 5]

其他用法：

```
a, *b, c = [1, 2, 3, 4, 5]
# a=1, b=[2,3,4], c=5
*a, b, c = [1, 2, 3, 4, 5]
# a=[1,2,3], b=4, c=5
```

16 • 单项选择

在Pandas中，DataFrame是：

A. 一维数据结构

B. 二维数据结构

C. 三维数据结构

D. 零维数据结构

答案 B. 二维数据结构

解析 Pandas核心数据结构：

- Series：一维标签数组，类似带索引的列表
- DataFrame：二维表格数据，类似Excel表格
- Panel：三维数据（已弃用）

DataFrame特点：

特点：

- 行索引（index）
- 列索引（columns）
- 可包含不同数据类型的列

17 • 单项选择

Pandas中，以下哪个方法用于读取CSV文件？

A. pd.read_csv()

B. pd.load_csv()

C. pd.import_csv()

D. pd.open_csv()

答案 A. pd.read_csv()

解析 Pandas常用读取函数：

- pd.read_csv() ：读取CSV文件
- pd.read_excel() ：读取Excel文件
- pd.read_json() ：读取JSON文件
- pd.read_sql() ：从数据库读取

18 • 单项选择

关于Pandas的 loc 和 iloc ，以下说法正确的是：

A. loc 使用整数位置索引

B. iloc 使用标签索引

C. loc 使用标签索引， iloc 使用整数位置索引

D. 两者完全相同

答案 C. loc 使用标签索引， iloc 使用整数位置索引

解析 Pandas索引方法对比：

- loc ：基于标签
  标签的索引，使用行列名称
- iloc ：基于整数位置
  整数位置的索引，使用数字下标

示例：

```
df = pd.DataFrame({'A': [1,2,3]}, index=['x','y','z'])
df.loc['x', 'A']
# 使用标签
df.iloc[0, 0]
# 使用位置
```

19 • 单项选择

以下哪个Pandas方法用于删除缺失值？

A. dropna()

B. fillna()

C. removena()

D. deletena()

答案 A. dropna()

解析 Pandas处理缺失值的方法：

- dropna() ：删除包含缺失值的行/列
- fillna() ：填充缺失值
- isna() / isnull() ：检测缺失值

20 • 单项选择

Pandas中， groupby() 操作返回的是：

A. DataFrame

B. Series

C. GroupBy对象

D. 列表

答案 C. GroupBy对象

解析 groupby() 返回GroupBy对象，需要进一步操作：

```
df.groupby('column').mean()
# 计算均值
df.groupby('column').sum()
# 计算总和
df.groupby('column').agg({})
# 聚合操作
```

21-25. Path对象属性：

21 • 单项选择

使用 pathlib.Path ，以下哪个属性返回文件名（不含路径）？

A. path.filename
B. path.name
C. path.basename
D. path.file

答案 B. path.name

解析

22 • 单项选择

Path 对象的 suffix 属性返回的是：

A. 文件路径
B. 文件扩展名
C. 文件名
D. 父目录

答案 B. 文件扩展名

解析

23 • 单项选择

以下哪个方法用于检查路径是否存在？

A. path.exists()
B. path.check()
C. path.is_exist()
D. path.valid()

答案 A. path.exists()

解析

24 • 单项选择

关于 Path 对象的 / 运算符，以下说法正确的是：

A. 用于除法运算
B. 用于路径拼接
C. 用于路径分割
D. 不能使用

答案 B. 用于路径拼接

解析

25 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('/home/user/file.txt')
print(p.stem)
```

A. file.txt
B. file
C. .txt
D. /home/user

答案 B. file

解析 Path对象示例：

```
from pathlib import Path
p = Path('/home/user/document.txt')
print(p.name)
# 'document.txt'
print(p.suffix)
# '.txt'
print(p.stem)
# 'document'
print(p.parent)
# '/home/user'
```

26 • 单项选择

关于Python的 lambda 函数，以下说法正确的是：

A. 可以包含多条语句
B. 只能包含一个表达式
C. 必须有函数名
D. 不能作为参数传递

答案 B. 只能包含一个表达式

解析 Lambda函数的限制：
• 只能包含一个表达式
• 不能包含语句（如赋值、循环）
• 自动返回表达式的结果
对比：

```
# lambda：单一表达式
lambda x: x * 2
# 普通函数：可包含多条语句
def func(x):
    result = x * 2
    return result
```

27 • 单项选择

以下代码的输出是什么？

```
nums = [1, 2, 3, 4]
result = list(map(lambda x: x * 2, nums))
print(result)
```

A. [1, 2, 3, 4]
B. [2, 4, 6, 8]
C. [1, 4, 9, 16]
D. 报错

答案 B. [2, 4, 6, 8]

解析 map() 函数将函数应用于序列的每个元素：

```
list(map(lambda x: x * 2, [1, 2, 3]))
# 结果：[2, 4, 6]
```

28 • 单项选择

字典推导式 {k: v**2 for k, v in {'a': 1, 'b': 2}.items()} 的结果是：

A. {'a': 1, 'b': 2}
B. {'a': 1, 'b': 4}
C. {'a': 2, 'b': 4}
D. 报错

答案 B. {'a': 1, 'b': 4}

解析 字典推导式：

```
{k: v**2 for k, v in {'a': 2, 'b': 3}.items()}
# 结果：{'a': 4, 'b': 9}
```

29 • 单项选择

关于 zip() 函数，以下说法错误的是：

A. 返回一个迭代器
B. 可以同时迭代多个序列
C. 以最长的序列为准
D. 返回元组的迭代器

答案 C. 以最长的序列为准

解析 zip() 以最短序列
最短序列为准：

```
list(zip([1,2,3], ['a','b']))
                # [(1,'a'), (2,'b')]
# 第三个元素3被丢弃
```

30 • 单项选择

以下代码的输出是什么？

```
list1 = [1, 2, 3]
list2 = ['a', 'b', 'c']
result = list(zip(list1, list2))
print(len(result))
```

A. 2
B. 3
C. 6
D. 报错

答案 B. 3

解析 两个长度为3的列表zip后，结果长度为3：

```
list(zip([1,2,3], [4,5,6]))
# [(1,4), (2,5), (3,6)]
```

二、不定项选择题

31 • 不定项选择

关于Python类的特殊方法，以下说法正确的有：

A. __init__() 是构造方法
B. __del__() 是析构方法

C. __len__() 用于返回对象长度

D. __call__() 使对象可以像函数一样被调用

答案 A. __init__() 是构造方法；B. __del__() 是析构方法；C. __len__() 用于返回对象长度；D. __call__() 使对象可以像函数一样被调用

解析 @property 装饰器的作用：

- A. 将方法转换为属性

将方法转换为属性 正确，让方法像属性一样访问

- B. 可以实现

可以实现getter和setter 正确，配合 @property.setter 使用

- C. 提供只读属性

提供只读属性 正确，不定义setter时即为只读

- D. 可以添加数据验证逻辑

可以添加数据验证逻辑 正确，在setter中可以验证

完整示例：

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius
    @property
    def celsius(self):
        """Getter: 获取温度"""
        return self._celsius
    @celsius.setter
    def celsius(self, value):
        """Setter: 设置温度并验证"""
        if value < -273.15:
            raise ValueError("温度不能低于绝对零度")
        self._celsius = value
    @property
    def fahrenheit(self):
        """只读属性：摄氏转华氏"""
        return self._celsius * 9/5 + 32
```

32 • 不定项选择

以下哪些是Python的访问控制约定：

A. 单下划线开头表示受保护的成员

B. 双下划线开头表示私有成员

C. 双下划线开头和结尾表示特殊方法

D. Python有严格的访问控制机制

答案 A. 单下划线开头表示受保护的成员；B. 双下划线开头表示私有成员；C. 双下划线开头和结尾表示特殊方法

解析 Python访问控制约定：

- A. 单下划线

单下划线 _ 表示保护属性

表示保护属性 约定，不强制

- B. 双下划线

双下划线 __ 触发名称改变

触发名称改变 正确，但不是真正的私有

- C. 没有严格的访问控制

没有严格的访问控制 Python信任程序员

- D. 可以实现真正的私有属性

可以实现真正的私有属性　错误，Python无法完全隐藏
名称改变机制：

```
class MyClass:
    def __init__(self):
        self._protected = 1
        # 保护属性（约定）
        self.__private = 2
# 私有属性（名称改变）
obj = MyClass()
print(obj._protected)
# 1 - 可以访问
print(obj._MyClass__private)
# 2 - 仍然可以访问！
```

33 • 不定项选择

关于Python的多重继承，以下说法正确的有：

A. Python支持多重继承
B. 使用MRO（方法解析顺序）确定方法调用顺序
C. 可以使用 __mro__ 属性查看继承顺序
D. 多重继承可能导致菱形继承问题

答案 A. Python支持多重继承；B. 使用MRO（方法解析顺序）确定方法调用顺序；C. 可以使用 __mro__ 属性查看继承顺序；D. 多重继承可能导致菱形继承问题

解析 生成器的优点：

- A. 节省内存
  节省内存　不一次性生成所有值
- B. 惰性求值
  惰性求值　按需计算
- C. 适合处理大数据集
  适合处理大数据集　内存占用小
- D. 可以表示无限序列
  可以表示无限序列　不需要预先生成所有值

对比示例：

```
import sys
# 列表：立即生成所有值
list_data = [x**2 for x in range(1000000)]
print(f"列表内存：{sys.getsizeof(list_data)} 字节")
# 生成器：按需生成
gen_data = (x**2 for x in range(1000000))
print(f"生成器内存：{sys.getsizeof(gen_data)} 字节")
# 无限序列示例
def infinite_sequence():
    num = 0
    while True:
        yield num
        num += 1
```

34 • 不定项选择

以下哪些是生成器的创建方式：

A. 使用 yield 关键字的函数

B. 使用生成器表达式 ()

C. 使用列表推导式 []

D. 使用 iter() 函数

答案 A. 使用 yield 关键字的函数；B. 使用生成器表达式 ()

解析 生成器的创建方式：

- A. 使用
  使用 yield 的函数
  的函数 生成器函数
- B. 生成器表达式
  生成器表达式 (x for x in ...)
- C. 列表推导式
  列表推导式 生成列表，不是生成器
- D. 实现
  实现 __iter__() 和 __next__() 这是迭代器，不是生成器

对比：

```
# A. 生成器函数
def gen_func():
    yield 1
    yield 2
# B. 生成器表达式
gen_exp = (x for x in range(3))
# C. 列表推导式（不是生成器）
list_comp = [x for x in range(3)]
```

35 • 不定项选择

关于迭代器和可迭代对象，以下说法正确的有：

A. 所有迭代器都是可迭代对象

B. 所有可迭代对象都是迭代器

C. 迭代器实现了 __iter__() 和 __next__() 方法

D. 可迭代对象只需实现 __iter__() 方法

答案 A. 所有迭代器都是可迭代对象；C. 迭代器实现了 __iter__() 和 __next__() 方法；D. 可迭代对象只需实现 __iter__() 方法

解析 • A. 可迭代对象可以用于
可迭代对象可以用于for循环
循环 正确

- B. 所有可迭代对象都是迭代器
  所有可迭代对象都是迭代器 错误，列表是可迭代但不是迭代器
- C. 迭代器必然是可迭代对象
  迭代器必然是可迭代对象 正确
- D. 可迭代对象实现
  可迭代对象实现 __iter__() 正确

区别：

```
# 可迭代对象：实现__iter__()
my_list = [1, 2, 3]
# 可迭代，但不是迭代器
# 迭代器：实现__iter__()和__next__()
my_iter = iter(my_list)
# 迭代器
```

36 • 不定项选择

以下哪些是装饰器的应用场景：

A. 日志记录
B. 性能测试
C. 权限验证
D. 缓存

答案 A. 日志记录；B. 性能测试；C. 权限验证；D. 缓存

解析 装饰器的常见应用：

• A. 计时统计

计时统计　测量函数执行时间

• B. 权限检查

权限检查　验证用户权限

• C. 缓存结果

缓存结果　避免重复计算

• D. 日志记录

日志记录　记录函数调用

应用示例：

```
import time
from functools import wraps
# 计时装饰器
def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} 耗时: {time.time()-start:.4f}秒")
        return result
    return wrapper
# 缓存装饰器
def cache(func):
    cached_results = {}
    @wraps(func)
    def wrapper(*args):
        if args not in cached_results:
            cached_results[args] = func(*args)
        return cached_results[args]
    return wrapper
```

7-15. 答案：

答案：ABCD

这些题目所有选项都正确，分别涉及Python高级特性、Pandas操作、pathlib模块和函数式编程的各个方面。

14. 特别说明：

特别说明：ABC

• D. 列表推导式总是比

列表推导式总是比for循环快

循环快　不一定，复杂逻辑时可能更慢

37 • 不定项选择

关于函数参数，以下说法正确的有：

A. 位置参数必须在关键字参数之前
B. *args 必须在 **kwargs 之前
C. 默认参数必须在位置参数之后
D. 可以使用 * 强制后面的参数必须使用关键字传递

答案 A. 位置参数必须在关键字参数之前；B. *args 必须在 **kwargs 之前；C. 默认参数必须在位置参数之后；D. 可以使用 * 强制后面的参数必须使用关键字传递

解析 （原卷合并作答）

38 • 不定项选择

关于解包操作，以下说法正确的有：

A. * 用于解包序列
B. ** 用于解包字典
C. 可以在函数调用时使用解包
D. 可以在赋值时使用解包

答案 A. * 用于解包序列；B. ** 用于解包字典；C. 可以在函数调用时使用解包；D. 可以在赋值时使用解包

解析 （原卷合并作答）

39 • 不定项选择

Pandas中，以下哪些方法可以用于数据选择：

A. loc[]
B. iloc[]
C. at[]
D. iat[]

答案 A. loc[]；B. iloc[]；C. at[]；D. iat[]

解析 （原卷合并作答）

40 • 不定项选择

关于Pandas的缺失值处理，以下说法正确的有：

A. dropna() 删除缺失值
B. fillna() 填充缺失值
C. isna() 检测缺失值
D. notna() 检测非缺失值

答案 A. dropna() 删除缺失值；B. fillna() 填充缺失值；C. isna() 检测缺失值；D. notna() 检测非缺失值

解析 （原卷合并作答）

41 • 不定项选择

Pandas中，以下哪些方法用于数据合并：

A. merge()
B. concat()
C. join()
D. append()

答案 A. merge()；B. concat()；C. join()；D. append()

解析 （原卷合并作答）

42 • 不定项选择

关于 pathlib.Path ，以下说法正确的有：

A. 提供面向对象的路径操作
B. 跨平台兼容
C. 可以使用 / 运算符拼接路径
D. 比 os.path 更现代

答案 A. 提供面向对象的路径操作；B. 跨平台兼容；C. 可以使用 / 运算符拼接路径；D. 比 os.path 更现代

解析 （原卷合并作答）

43 • 不定项选择

Path 对象的以下哪些方法用于路径判断：

A. is_file()
B. is_dir()
C. exists()
D. is_absolute()

答案 A. is_file()；B. is_dir()；C. exists()；D. is_absolute()

解析 （原卷合并作答）

44 • 不定项选择

关于列表推导式，以下说法正确的有：

A. 可以包含条件判断
B. 可以嵌套使用
C. 比for循环更简洁
D. 总是比for循环快

答案 A. 可以包含条件判断；B. 可以嵌套使用；C. 比for循环更简洁；D. 总是比for循环快

解析 （原卷合并作答）

45 • 不定项选择

以下哪些是Python的高阶函数：

A. map()
B. filter()
C. reduce()
D. sorted()

答案 A. map()；B. filter()；C. reduce()；D. sorted()

解析 （原卷合并作答）

三、填空题

46 • 填空 / 简答

在Python类中，使用 _____ 装饰器可以定义类方法，使用 _____ 装饰器可以定义静态方法。

答案 @classmethod ， @staticmethod

解析 Python中定义类方法和静态方法的装饰器：
类方法 @classmethod ：
- 第一个参数是类本身（约定为 cls ）
- 可以访问类属性和类方法
- 常用于工厂方法、替代构造函数

静态方法 @staticmethod ：
- 不需要 self 或 cls 参数
- 无法访问实例或类属性
- 适合工具函数，逻辑上属于类

对比示例：

```
class Date:
    def __init__(self, year, month, day):
        self.year = year
        self.month = month
        self.day = day
    @classmethod
    def from_string(cls, date_str):
        """类方法：从字符串创建对象"""
        year, month, day = map(int, date_str.split('-'))
        return cls(year, month, day)
    @staticmethod
    def is_valid_date(year, month, day):
        """静态方法：验证日期"""
        return 1 <= month <= 12 and 1 <= day <= 31
# 使用
d1 = Date.from_string('2024-01-15')
# 类方法
valid = Date.is_valid_date(2024, 1, 15)
# 静态方法
```

47 • 填空 / 简答

生成器函数使用 _____ 关键字返回值，每次调用 _____ 函数可以获取生成器的下一个值。

答案 yield ， next()

解析 生成器的核心关键字和函数：
yield 关键字：
关键字：
- 用于定义生成器函数
- 暂停函数执行并返回值
- 保存函数状态，下次调用从暂停处继续

next() 函数：
函数：
- 获取生成器的下一个值
- 无值时抛出 StopIteration 异常

完整示例：

```
def fibonacci(n):
    """生成前n个斐波那契数"""
    a, b = 0, 1
    count = 0
    while count < n:
```

```
        yield a
        # 返回当前值并暂停
        a, b = b, a + b
        count += 1
# 使用next()
fib = fibonacci(5)
print(next(fib))
# 0
print(next(fib))
# 1
print(next(fib))
# 1
# 使用for循环
for num in fibonacci(5):
    print(num, end=' ')
# 0 1 1 2 3
```

48 • 填空 / 简答

在Pandas中，_____ 用于基于标签的索引，_____ 用于基于整数位置的索引。

答案 loc ， iloc

解析 Pandas两种主要的索引方式：

loc - 基于标签的索引：

- 使用行名和列名
- 包含结束位置（闭区间）
- 支持布尔索引

iloc - 基于整数位置的索引：

- 使用整数位置（0开始）
- 不包含结束位置（左闭右开）
- 类似Python列表切片

对比示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
}, index=['x', 'y', 'z'])
# loc: 使用标签
df.loc['x', 'A']
# 1
df.loc['x':'y', 'A']
# 包含'y'
# iloc: 使用位置
df.iloc[0, 0]
# 1
df.iloc[0:2, 0]
# 不包含位置2
# 布尔索引（只能loc）
df.loc[df['A'] > 1, 'B']
# 条件筛选
```

49 • 填空 / 简答

使用 _____ 符号可以解包序列，使用 _____ 符号可以解包字典。

答案 * ， **

解析 Python参数解包和打包符号：

* - 位置参数：

- 函数定义中： *args 打包为元组
- 函数调用中：解包序列为位置参数
- 赋值中：收集剩余元素

** - 关键字参数：

- 函数定义中： **kwargs 打包为字典
- 函数调用中：解包字典为关键字参数

应用场景：

```
# 1. 函数定义：接收任意参数
def func(*args, **kwargs):
    print(f"args: {args}")
    # 元组
    print(f"kwargs: {kwargs}")
# 字典
func(1, 2, 3, a=4, b=5)
# args: (1, 2, 3)
# kwargs: {'a': 4, 'b': 5}
# 2. 函数调用：解包参数
values = [1, 2, 3]
params = {'x': 10, 'y': 20}
def add(a, b, c, x=0, y=0):
    return a + b + c + x + y
result = add(*values, **params)
# 解包
# 3. 赋值中：收集元素
a, *b, c = [1, 2, 3, 4, 5]
# a=1, b=[2,3,4], c=5
```

50 • 填空 / 简答

pathlib.Path 对象的 _____ 属性返回文件扩展名，_____ 属性返回不含扩展名的文件名。

答案 suffix ， stem

解析 肴析： pathlib.Path 对象的文件名相关属性：

suffix - 文件扩展名：

- 返回包括点号的扩展名（如 .txt ）
- 空字符串表示无扩展名
- suffixes 属性返回所有扩展名列表

stem - 不含扩展名的文件名：

- 只返回文件主名部分
- 不包含路径和扩展名

完整对比：

```
from pathlib import Path
# 单个扩展名
p1 = Path('/home/user/document.txt')
print(p1.name)
# 'document.txt' - 完整文件名
```

```
print(p1.suffix)
# '.txt' - 扩展名
print(p1.stem)
# 'document' - 不含扩展名
# 多个扩展名
p2 = Path('/home/user/archive.tar.gz')
print(p2.suffix)
# '.gz' - 最后一个扩展名
print(p2.suffixes)
# ['.tar', '.gz'] - 所有扩展名
print(p2.stem)
# 'archive.tar' - 不含最后扩展名
# 无扩展名
p3 = Path('/home/user/README')
print(p3.suffix)
# '' - 空字符串
print(p3.stem)
# 'README'
# 应用：修改扩展名
old_path = Path('document.txt')
new_path = old_path.with_suffix('.md')
print(new_path)
                # 'document.md'
```

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class BankAccount:
    def __init__(self, balance=0):
        self._balance = balance
    @property
    def balance(self):
        return self._balance
    @balance.setter
    def balance(self, value):
        if value < 0:
            raise ValueError("余额不能为负数")
        self._balance = value
    def deposit(self, amount):
        self.balance += amount
account = BankAccount(100)
account.deposit(50)
print(account.balance)
```

问题：

- (1) 输出结果是什么？
- (2) @property 装饰器的作用是什么？
- (3) 如果执行 account.balance = -10 会发生什么？

答案 • (1) 输出：150
• (2) @property 将方法转换为属性，可以像访问属性一样调用方法，同时可以添加getter和setter进行控制
• (3) 会抛出 ValueError 异常，提示"余额不能为负数"

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
def fibonacci_gen(n):
    a, b = 0, 1
    count = 0
    while count < n:
        yield a
        a, b = b, a + b
        count += 1
fib = fibonacci_gen(5)
print(list(fib))
```

问题：
• (1) 第一个 print 输出什么？
• (2) 第二个 print 输出什么？为什么？
• (3) 生成器相比列表的优势是什么？

答案 • (1) 输出： [0, 1, 1, 2, 3]
• (2) 输出： [] 。因为生成器只能迭代一次，第一次迭代后已经耗尽
• (3) 生成器按需生成值，不会一次性占用大量内存，适合处理大数据集

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie', 'David'],
    'age': [25, 30, 35, 40],
    'salary': [5000, 6000, 7000, 8000]
})
result = df[df['age'] > 30]['salary'].mean()
print(result)
```

问题：
• (1) 输出结果是什么？
• (2) df['age'] > 30 返回的是什么类型？
• (3) 如何使用 loc 改写这个查询？

答案 数据筛选
• (1) 输出：7500.0（35岁Charlie的7000和40岁David的8000的平均值）
• (2) 返回布尔型Series
• (3) df.loc[df['age'] > 30, 'salary'].mean()

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
from pathlib import Path
p = Path('/home/user/documents/report.pdf')
print(p.parent)
print(p.name)
print(p.suffix)
print(p.stem)
```

问题：

- (1) 四个 print 语句分别输出什么？
- (2) 如何获取该文件的绝对路径？
- (3) 如何检查该文件是否存在？

答案 对象操作

- (1)
  ```
  /home/user/documents
  report.pdf
  .pdf
  report
  ```
- (2) 使用 p.absolute() 或 p.resolve()
- (3) 使用 p.exists()

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
def apply_operations(*funcs):
    def wrapper(x):
        result = x
        for func in funcs:
            result = func(result)
        return result
    return wrapper
double = lambda x: x * 2
add_ten = lambda x: x + 10
operation = apply_operations(double, add_ten)
print(operation(5))
```

问题：

- (1) 输出结果是什么？
- (2) 这段代码体现了什么设计模式？
- (3) *funcs 的作用是什么？

答案 • (1) 输出：20（5*2=10，10+10=20）

- (2) 装饰器模式或函数组合模式
- (3) *funcs 接收任意数量的函数作为参数

解析

五、程序设计题

56 • 填空 / 简答

设计一个 Student 类，要求：

- 属性：姓名、学号、成绩字典（科目：分数）
- 方法：
  add_score(subject, score) ：添加成绩
  get_average() ：计算平均分
  __str__() ：返回学生信息
  使用 @property 实现 total_score 属性
- 创建3个学生实例，使用列表推导式找出平均分大于80的学生

答案

```
class Student:
    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id
        self.scores = {}
    def add_score(self, subject, score):
        self.scores[subject] = score
    def get_average(self):
        if not self.scores:
            return 0
        return sum(self.scores.values()) / len(self.scores)
    @property
    def total_score(self):
        return sum(self.scores.values())
    def __str__(self):
        return f"学生：{self.name}，学号：{self.student_id}，平均分：{self.get_average():.2f}"
s1 = Student("张三", "001")
s1.add_score("语文", 85)
s1.add_score("数学", 90)
s1.add_score("英语", 88)
s2 = Student("李四", "002")
s2.add_score("语文", 75)
s2.add_score("数学", 80)
s2.add_score("英语", 78)
s3 = Student("王五", "003")
s3.add_score("语文", 92)
s3.add_score("数学", 95)
s3.add_score("英语", 90)
students = [s1, s2, s3]
high_achievers = [s for s in students if s.get_average() > 80]
print("平均分大于80的学生：")
for student in high_achievers:
    print(student)
```

解析

57 • 填空 / 简答

使用 pathlib 编写一个函数 process_directory(path) ，要求：
- 接收一个目录路径
- 使用生成器返回该目录下所有 .txt 文件的完整路径
- 统计文件总数
- 打印每个文件的名称和大小

答案
```
from pathlib import Path
def process_directory(path):
    directory = Path(path)
    if not directory.exists() or not directory.is_dir():
        print("目录不存在")
        return
    txt_files = list(directory.glob('*.txt'))
    print(f"找到{len(txt_files)} 个txt文件")
    for file in txt_files:
        yield file
def file_generator(path):
    for file_path in process_directory(path):
        size = file_path.stat().st_size
        print(f"文件：{file_path.name}，大小：{size} 字节")
        yield file_path
# 使用示例
# for file in file_generator('/path/to/directory'):
# print(file)
```

解析

58 • 填空 / 简答

使用Pandas完成以下任务：
- 创建一个包含学生信息的DataFrame（姓名、语文、数学、英语）
- 计算每个学生的总分和平均分
- 找出总分最高的学生
- 使用 groupby 按平均分段（优秀>=90，良好>=80，及格>=60）统计人数
- 将结果保存到CSV文件

答案
```
import pandas as pd
df = pd.DataFrame({
    '姓名': ['张三', '李四', '王五', '赵六', '钱七'],
    '语文': [85, 92, 78, 88, 95],
    '数学': [90, 88, 85, 92, 87],
    '英语': [88, 95, 82, 90, 93]
})
df['总分'] = df[['语文', '数学', '英语']].sum(axis=1)
df['平均分'] = df[['语文', '数学', '英语']].mean(axis=1)
top_student = df.loc[df['总分'].idxmax()]
print(f"总分最高的学生：{top_student['姓名']}，总分：{top_student['总分']}")
def grade_level(avg):
    if avg >= 90:
        return '优秀'
```

```
    elif avg >= 80:
        return '良好'
    elif avg >= 60:
        return '及格'
    else:
        return '不及格'
df['等级'] = df['平均分'].apply(grade_level)
grade_stats = df.groupby('等级').size()
print("\n各等级人数统计：")
print(grade_stats)
df.to_csv('student_scores.csv', index=False, encoding='utf-8-sig')
print("\n数据已保存到 student_scores.csv")
```

解析

59 • 填空 / 简答

编写一个装饰器 timer ，要求：

- 测量函数执行时间
- 打印函数名和执行时间
- 使用该装饰器装饰一个计算斐波那契数列的函数
- 对比递归和迭代两种实现的性能

答案

```
import time
def timer(func):
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        print(f"函数{func.__name__} 执行时间：{end_time - start_time:.6f} 秒")
        return result
    return wrapper
@timer
def fibonacci_recursive(n):
    if n <= 1:
        return n
    return fibonacci_recursive(n-1) + fibonacci_recursive(n-2)
@timer
def fibonacci_iterative(n):
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n+1):
        a, b = b, a + b
    return b
print("递归实现：")
print(f"结果：{fibonacci_recursive(30)}")
print("\n迭代实现：")
print(f"结果：{fibonacci_iterative(30)}")
```

解析

60 • 填空 / 简答

实现一个自定义迭代器类 EvenNumbers ，要求：

- 接收起始值和结束值
- 迭代返回范围内的所有偶数
- 实现 __iter__() 和 __next__() 方法
- 支持 for 循环遍历
- 演示使用该迭代器

答案

```
class EvenNumbers:
    def __init__(self, start, end):
        self.start = start if start % 2 == 0 else start + 1
        self.end = end
        self.current = self.start
    def __iter__(self):
        return self
    def __next__(self):
        if self.current > self.end:
            raise StopIteration
        result = self.current
        self.current += 2
        return result
even_nums = EvenNumbers(1, 10)
print("1到10之间的偶数：")
for num in even_nums:
    print(num, end=' ')
print()
even_nums2 = EvenNumbers(0, 20)
print("\n0到20之间的偶数：")
print(list(even_nums2))
程序设计题详细参考答案
题目1：学生成绩管理系统（完整实现）
"""
学生成绩管理系统
功能：添加学生、查询成绩、计算统计信息、生成报告
"""
class Student:
    """学生类 - 存储单个学生的信息"""
    def __init__(self, student_id, name, scores):
        """
        初始化学生对象
        :param student_id: 学号
        :param name: 姓名
        :param scores: 成绩字典，如 {'语文': 90, '数学': 85}
        """
        self.student_id = student_id
        self.name = name
        self.scores = scores
```

```
    def average_score(self):
        """计算平均分"""
        if not self.scores:
            return 0
        return sum(self.scores.values()) / len(self.scores)
    def total_score(self):
        """计算总分"""
        return sum(self.scores.values())
    def __str__(self):
        """字符串表示"""
        avg = self.average_score()
        return f"学号: {self.student_id}, 姓名: {self.name}, 平均分: {avg:.2f}"
    def __repr__(self):
        return f"Student({self.student_id}, {self.name}, {self.scores})"
class GradeManager:
"""成绩管理类 - 管理所有学生"""
def __init__(self):
    """初始化管理器"""
    self.students = {}
# 使用字典存储，key为学号
def add_student(self, student):
    """
    添加学生
    :param student: Student对象
    """
    if student.student_id in self.students:
        print(f"警告: 学号{student.student_id} 已存在，将覆盖原数据")
    self.students[student.student_id] = student
    print(f"成功添加学生: {student.name}")
def find_student(self, student_id):
    """
    查找学生
    :param student_id: 学号
    :return: Student对象或None
    """
    return self.students.get(student_id)
def get_subject_average(self, subject):
    """
    计算某科目的平均分
    :param subject: 科目名称
    :return: 平均分
    """
    scores = [s.scores.get(subject, 0) for s in self.students.values()
            if subject in s.scores]
    if not scores:
        return 0
    return sum(scores) / len(scores)
def get_top_students(self, n=3):
```

```
    """
    获取成绩前N名的学生
    :param n: 返回前几名
        :return: 学生列表
        """
        sorted_students = sorted(self.students.values(),
                key=lambda s: s.average_score(),
                reverse=True)
        return sorted_students[:n]
    def generate_report(self):
        """生成成绩报告"""
        print("\n" + "="*50)
        print("成绩统计报告".center(50))
        print("="*50)
        print(f"\n总学生数: {len(self.students)}")
        # 各科平均分
        if self.students:
            all_subjects = set()
            for student in self.students.values():
                all_subjects.update(student.scores.keys())
            print("\n各科平均分:")
            for subject in sorted(all_subjects):
                avg = self.get_subject_average(subject)
                print(f"
                {subject}: {avg:.2f}")
        # 前三名学生
        print("\n成绩前三名:")
        for i, student in enumerate(self.get_top_students(3), 1):
            print(f"
                {i}. {student}")
        print("="*50 + "\n")
# 使用示例
if __name__ == "__main__":
    # 创建管理器
    manager = GradeManager()
    # 添加学生
s1 = Student("001", "张三", {"语文": 90, "数学": 85, "英语": 88})
s2 = Student("002", "李四", {"语文": 85, "数学": 92, "英语": 90})
s3 = Student("003", "王五", {"语文": 88, "数学": 87, "英语": 85})
s4 = Student("004", "赵六", {"语文": 92, "数学": 88, "英语": 91})
manager.add_student(s1)
manager.add_student(s2)
manager.add_student(s3)
manager.add_student(s4)
# 查询学生
student = manager.find_student("001")
if student:
    print(f"\n查询结果: {student}")
```

```
# 生成报告
manager.generate_report()
设计要点：
1. 面向对象设计
面向对象设计：Student类负责单个学生数据，GradeManager负责整体管理
2. 数据结构选择
数据结构选择：使用字典存储学生，便于快速查找
3. 错误处理
错误处理：检查学号是否重复，处理空数据情况
4. 代码复用
代码复用：average_score等方法可被多处调用
5. 可扩展性
可扩展性：易于添加新功能（如删除学生、修改成绩等）
题目2：装饰器实现（完整实现）
"""
装饰器实现：日志记录、性能测试、缓存
"""
import time
import functools
from datetime import datetime
def log_function_call(func):
    """
    日志装饰器 - 记录函数调用信息
    功能：记录函数名、参数、返回值、调用时间
    """
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        # 记录调用时间
        call_time = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        # 记录参数
        args_str = ', '.join(repr(arg) for arg in args)
        kwargs_str = ', '.join(f"{k}={v!r}" for k, v in kwargs.items())
        all_args = ', '.join(filter(None, [args_str, kwargs_str]))
        print(f"[{call_time}] 调用函数: {func.__name__}({all_args})")
        # 执行函数
        try:
            result = func(*args, **kwargs)
            print(f"[{call_time}] 返回值: {result!r}")
            return result
        except Exception as e:
            print(f"[{call_time}] 异常: {type(e).__name__}: {e}")
            raise
    return wrapper
def timer(func):
    """
    性能测试装饰器 - 测量函数执行时间
    """
    @functools.wraps(func)
```

```
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        elapsed_time = end_time - start_time
        print(f"函数{func.__name__} 执行时间: {elapsed_time:.6f} 秒")
        return result
    return wrapper
def cache(func):
    """
    缓存装饰器 - 缓存函数结果
    使用字典存储已计算的结果，避免重复计算
    """
    cached_results = {}
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        # 创建缓存键（注意：kwargs需要转换为可哈希类型）
        cache_key = (args, tuple(sorted(kwargs.items())))
        if cache_key in cached_results:
            print(f"缓存命中: {func.__name__}{args}")
            return cached_results[cache_key]
        # 计算结果并缓存
        result = func(*args, **kwargs)
        cached_results[cache_key] = result
        print(f"缓存存储: {func.__name__}{args} = {result}")
        return result
    # 添加缓存管理方法
    wrapper.cache_info = lambda: {
        'size': len(cached_results),
        'hits': sum(1 for _ in cached_results)
    }
    wrapper.cache_clear = lambda: cached_results.clear()
    return wrapper
# 组合使用多个装饰器
@log_function_call
@timer
@cache
def fibonacci(n):
    """计算斐波那契数列第n项"""
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)
@log_function_call
def add(a, b):
    """加法函数"""
    return a + b
# 使用示例
if __name__ == "__main__":
```

```
    print("=" * 60)
    print("测试日志装饰器")
    print("=" * 60)
    result = add(3, 5)
    print("\n" + "=" * 60)
    print("测试缓存装饰器")
    print("=" * 60)
    fibonacci(5)
                # 第二次调用会使用缓存
print("\n" + "=" * 60)
print("测试性能装饰器")
print("=" * 60)
@timer
def slow_function():
    time.sleep(0.1)
    return "完成"
slow_function()
设计要点：
1. functools.wraps：保留原函数的元数据（名称、文档等）
2. 闭包
闭包：装饰器内部函数可以访问外部变量（如cached_results）
3. 可组合性
可组合性：多个装饰器可以叠加使用
4. 缓存键设计
缓存键设计：将args和kwargs转换为可哈希的元组
5. 扩展性
扩展性：为cache装饰器添加cache_info和cache_clear方法
题目3：生成器实现（完整实现）
"""
生成器实现：文件读取、数据过滤、无限序列
"""
from pathlib import Path
import itertools
def read_large_file(filepath, chunk_size=1024):
    """
    生成器：逐块读取大文件
    :param filepath: 文件路径
    :param chunk_size: 每次读取的字节数
    :yield: 文件内容块
    """
    try:
        with open(filepath, 'r', encoding='utf-8') as f:
            while True:
                chunk = f.read(chunk_size)
                if not chunk:
                break
                yield chunk
    except FileNotFoundError:
```

```
        print(f"文件不存在: {filepath}")
    except Exception as e:
        print(f"读取文件出错: {e}")
def read_lines(filepath):
    """
    生成器：逐行读取文件
    :param filepath: 文件路径
    :yield: 文件的每一行
    """
    try:
        with open(filepath, 'r', encoding='utf-8') as f:
            for line in f:
                yield line.rstrip('\n')
    except Exception as e:
        print(f"读取文件出错: {e}")
def filter_data(data_iter, condition):
    """
    生成器：过滤数据
    :param data_iter: 数据迭代器
    :param condition: 过滤条件函数
    :yield: 满足条件的数据
    """
    for item in data_iter:
        if condition(item):
            yield item
def fibonacci_generator():
    """
    生成器：无限斐波那契数列
    :yield: 斐波那契数
    """
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
def prime_generator():
    """
    生成器：无限素数序列
    :yield: 素数
    """
    def is_prime(n):
        if n < 2:
            return False
        for i in range(2, int(n ** 0.5) + 1):
            if n % i == 0:
                return False
        return True
    n = 2
    while True:
```

```
        if is_prime(n):
            yield n
        n += 1
def batch_generator(iterable, batch_size):
    """
    生成器：将数据分批
    :param iterable: 可迭代对象
    :param batch_size: 批次大小
    :yield: 数据批次
    """
    iterator = iter(iterable)
    while True:
        batch = list(itertools.islice(iterator, batch_size))
        if not batch:
            break
        yield batch
# 使用示例
if __name__ == "__main__":
    print("=" * 60)
    print("示例1: 斐波那契数列生成器")
    print("=" * 60)
    fib = fibonacci_generator()
    print("前10个斐波那契数:")
    for i, num in enumerate(fib):
        if i >= 10:
            break
        print(num, end=' ')
    print("\n")
    print("=" * 60)
    print("示例2: 素数生成器")
    print("=" * 60)
    primes = prime_generator()
    print("前10个素数:")
    for i, prime in enumerate(primes):
    if i >= 10:
        break
    print(prime, end=' ')
print("\n")
print("=" * 60)
print("示例3: 数据过滤")
print("=" * 60)
numbers = range(1, 21)
even_numbers = filter_data(numbers, lambda x: x % 2 == 0)
print("1-20的偶数:")
print(list(even_numbers))
print("\n" + "=" * 60)
print("示例4: 批次生成器")
print("=" * 60)
```

```
data = range(1, 11)
batches = batch_generator(data, 3)
print("将1-10分成批次(每批3个):")
for i, batch in enumerate(batches, 1):
    print(f"批次{i}: {batch}")
# 文件操作示例（需要实际文件）
print("\n" + "=" * 60)
print("示例5: 文件读取生成器")
print("=" * 60)
# 创建测试文件
test_file = Path("test_data.txt")
test_file.write_text("第一行\n第二行\n第三行\n第四行\n第五行\n")
print("逐行读取文件:")
for line in read_lines(test_file):
    print(f"
            {line}")
# 清理测试文件
test_file.unlink()
设计要点：
1. 内存效率
内存效率：生成器逐个产生数据，不需要一次性加载所有数据
2. 惰性计算
惰性计算：只在需要时才计算下一个值
3. 无限序列
无限序列：可以生成无限序列（如斐波那契、素数）
4. 组合使用
组合使用：生成器可以串联使用（如filter_data）
5. 资源管理
资源管理：使用with语句确保文件正确关闭
题目4：Pandas数据分析（完整实现）
"""
Pandas数据分析：数据清洗、统计分析、可视化
"""
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from datetime import datetime, timedelta
# 设置中文显示
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
class DataAnalyzer:
    """数据分析类"""
    def __init__(self, data=None):
        """初始化分析器"""
        self.df = data if data is not None else pd.DataFrame()
    def load_data(self, filepath):
        """加载数据"""
        try:
```

```
            self.df = pd.read_csv(filepath)
            print(f"成功加载数据: {len(self.df)} 行")
            return self.df
        except Exception as e:
            print(f"加载数据失败: {e}")
            return None
    def clean_data(self):
        """数据清洗"""
        print("\n开始数据清洗...")
        original_size = len(self.df)
        # 1. 删除完全重复的行
        self.df.drop_duplicates(inplace=True)
        print(f"删除重复行: {original_size - len(self.df)} 行")
    # 2. 处理缺失值
    missing_before = self.df.isnull().sum().sum()
    # 数值列用中位数填充
    numeric_cols = self.df.select_dtypes(include=[np.number]).columns
    for col in numeric_cols:
        self.df[col].fillna(self.df[col].median(), inplace=True)
    # 分类列用众数填充
    categorical_cols = self.df.select_dtypes(include=['object']).columns
    for col in categorical_cols:
        mode_value = self.df[col].mode()
        if len(mode_value) > 0:
            self.df[col].fillna(mode_value[0], inplace=True)
    missing_after = self.df.isnull().sum().sum()
    print(f"处理缺失值: {missing_before - missing_after} 个")
    # 3. 删除异常值（使用IQR方法）
    for col in numeric_cols:
        Q1 = self.df[col].quantile(0.25)
        Q3 = self.df[col].quantile(0.75)
        IQR = Q3 - Q1
        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR
        outliers = ((self.df[col] < lower_bound) |
                (self.df[col] > upper_bound)).sum()
        if outliers > 0:
            print(f"列{col} 发现{outliers} 个异常值")
            self.df = self.df[
                (self.df[col] >= lower_bound) &
                (self.df[col] <= upper_bound)
            ]
    print(f"清洗完成，剩余{len(self.df)} 行\n")
    return self.df
def basic_statistics(self):
    """基础统计分析"""
    print("=" * 60)
    print("基础统计信息")
```

```
    print("=" * 60)
    print(self.df.describe())
    print("\n数据类型:")
    print(self.df.dtypes)
    print(f"\n数据形状: {self.df.shape}")
    return self.df.describe()
def group_analysis(self, group_by, agg_col, agg_func='mean'):
    """分组分析"""
    print(f"\n按{group_by} 分组，对{agg_col} 进行{agg_func} 统计:")
    result = self.df.groupby(group_by)[agg_col].agg(agg_func)
    print(result)
    return result
def visualize(self, save_path='analysis_report.png'):
    """数据可视化"""
    numeric_cols = self.df.select_dtypes(include=[np.number]).columns
    if len(numeric_cols) == 0:
        print("没有数值列可以可视化")
        return
    n_cols = min(len(numeric_cols), 4)
    fig, axes = plt.subplots(2, 2, figsize=(12, 10))
    axes = axes.flatten()
    for i, col in enumerate(numeric_cols[:4]):
        # 直方图
        axes[i].hist(self.df[col].dropna(), bins=30, edgecolor='black')
        axes[i].set_title(f'{col} 分布')
        axes[i].set_xlabel(col)
        axes[i].set_ylabel('频数')
    plt.tight_layout()
    plt.savefig(save_path)
    print(f"\n可视化图表已保存: {save_path}")
    plt.close()
# 使用示例
if __name__ == "__main__":
    # 创建示例数据
    np.random.seed(42)
    dates = pd.date_range('2024-01-01', periods=100, freq='D')
    df = pd.DataFrame({
        '日期': dates,
        '销售额': np.random.normal(10000, 2000, 100),
        '访客数': np.random.randint(100, 500, 100),
        '转化率': np.random.uniform(0.01, 0.1, 100),
        '地区': np.random.choice(['北京', '上海', '广州', '深圳'], 100)
    })
    # 添加一些缺失值和异常值
    df.loc[np.random.choice(df.index, 5), '销售额'] = np.nan
    df.loc[np.random.choice(df.index, 3), '销售额'] = 50000
    # 异常值
    # 创建分析器
```

```
    analyzer = DataAnalyzer(df)
    # 数据清洗
    analyzer.clean_data()
    # 基础统计
    analyzer.basic_statistics()
    # 分组分析
    analyzer.group_analysis('地区', '销售额', 'mean')
    analyzer.group_analysis('地区', '访客数', 'sum')
    # 可视化
    analyzer.visualize()
    print("\n分析完成！")
```

设计要点：

1. 数据清洗流程

数据清洗流程：去重→处理缺失值→删除异常值

2. 统计方法

统计方法：使用IQR方法检测异常值

3. 分组聚合

分组聚合：灵活的groupby操作

4. 可视化

可视化：使用matplotlib生成图表

5. 面向对象

面向对象：封装成类便于复用

题目5：Pathlib文件操作（完整实现）

```
"""
Pathlib文件操作：文件管理、目录遍历、批量处理
"""
from pathlib import Path
import shutil
from datetime import datetime
class FileManager:
    """文件管理器类"""
    def __init__(self, base_path='.'):
        """初始化文件管理器"""
        self.base_path = Path(base_path)
        if not self.base_path.exists():
            self.base_path.mkdir(parents=True)
    def create_directory_structure(self, structure):
        """
        创建目录结构
        :param structure: 目录结构字典
        """
        for dir_name in structure:
            dir_path = self.base_path / dir_name
            dir_path.mkdir(parents=True, exist_ok=True)
            print(f"创建目录: {dir_path}")
    def find_files(self, pattern='*', recursive=True):
        """
        查找文件
```

```
        :param pattern: 文件模式（如 '*.txt'）
        :param recursive: 是否递归查找
        :return: 文件路径列表
        """
        if recursive:
            files = list(self.base_path.rglob(pattern))
        else:
            files = list(self.base_path.glob(pattern))
    print(f"找到{len(files)} 个匹配的文件")
    return files
def organize_by_extension(self, source_dir=None):
    """
    按扩展名组织文件
    将文件移动到以扩展名命名的子目录中
    """
    source = Path(source_dir) if source_dir else self.base_path
    for file_path in source.iterdir():
        if file_path.is_file():
            # 获取扩展名（去掉点号）
            ext = file_path.suffix[1:] if file_path.suffix else 'no_extension'
            # 创建目标目录
            target_dir = source / ext
            target_dir.mkdir(exist_ok=True)
            # 移动文件
            target_path = target_dir / file_path.name
            shutil.move(str(file_path), str(target_path))
            print(f"移动: {file_path.name} -> {ext}/")
def batch_rename(self, pattern, prefix='', suffix=''):
    """
    批量重命名文件
    :param pattern: 文件模式
    :param prefix: 前缀
    :param suffix: 后缀
    """
    files = self.find_files(pattern, recursive=False)
    for i, file_path in enumerate(files, 1):
        new_name = f"{prefix}{i:03d}{suffix}{file_path.suffix}"
        new_path = file_path.parent / new_name
        file_path.rename(new_path)
        print(f"重命名: {file_path.name} -> {new_name}")
def get_file_info(self, filepath):
    """
    获取文件信息
    :param filepath: 文件路径
    :return: 文件信息字典
    """
    path = Path(filepath)
    if not path.exists():
```

```
        return None
    stat = path.stat()
    info = {
        '文件名': path.name,
        '完整路径': str(path.absolute()),
        '大小(字节)': stat.st_size,
        '创建时间': datetime.fromtimestamp(stat.st_ctime),
        '修改时间': datetime.fromtimestamp(stat.st_mtime),
        '是否文件': path.is_file(),
        '是否目录': path.is_dir(),
        '扩展名': path.suffix
    }
    return info
def search_content(self, keyword, file_pattern='*.txt'):
    """
    在文件中搜索关键词
    :param keyword: 搜索关键词
    :param file_pattern: 文件模式
    :return: 包含关键词的文件列表
    """
    results = []
    files = self.find_files(file_pattern)
    for file_path in files:
        try:
            content = file_path.read_text(encoding='utf-8')
            if keyword in content:
                results.append(file_path)
                print(f"找到: {file_path}")
            except Exception as e:
                print(f"读取文件失败{file_path}: {e}")
        return results
    def backup_files(self, backup_dir='backup'):
        """
        备份文件
        :param backup_dir: 备份目录名
        """
        backup_path = self.base_path / backup_dir
        backup_path.mkdir(exist_ok=True)
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
        for file_path in self.base_path.iterdir():
            if file_path.is_file():
                backup_name = f"{file_path.stem}_{timestamp}{file_path.suffix}"
                backup_file = backup_path / backup_name
                shutil.copy2(str(file_path), str(backup_file))
                print(f"备份: {file_path.name} -> {backup_name}")
# 使用示例
if __name__ == "__main__":
    # 创建测试环境
```

```
    test_dir = Path("test_file_manager")
    test_dir.mkdir(exist_ok=True)
    # 创建文件管理器
    fm = FileManager(test_dir)
    print("=" * 60)
    print("示例1: 创建目录结构")
    print("=" * 60)
    structure = ['documents', 'images', 'videos', 'backup']
    fm.create_directory_structure(structure)
    print("\n" + "=" * 60)
print("示例2: 创建测试文件")
print("=" * 60)
# 创建一些测试文件
(test_dir / 'file1.txt').write_text('这是测试文件1')
(test_dir / 'file2.txt').write_text('这是测试文件2，包含关键词Python')
(test_dir / 'file3.md').write_text('# Markdown文件')
(test_dir / 'image.jpg').touch()
print("\n" + "=" * 60)
print("示例3: 查找文件")
print("=" * 60)
txt_files = fm.find_files('*.txt', recursive=False)
print("\n" + "=" * 60)
print("示例4: 获取文件信息")
print("=" * 60)
if txt_files:
    info = fm.get_file_info(txt_files[0])
    for key, value in info.items():
        print(f"{key}: {value}")
print("\n" + "=" * 60)
print("示例5: 搜索内容")
print("=" * 60)
fm.search_content('Python', '*.txt')
print("\n" + "=" * 60)
print("示例6: 备份文件")
print("=" * 60)
fm.backup_files()
# 清理测试环境
print("\n清理测试环境...")
shutil.rmtree(test_dir)
print("完成！")
```

设计要点：

1. Path对象

对象：使用pathlib.Path代替字符串路径操作

2. 跨平台

跨平台：Path自动处理不同操作系统的路径分隔符

3. 链式操作

链式操作：Path支持/操作符连接路径

4. 异常处理

异常处理：处理文件不存在、权限等问题
5. 实用功能
实用功能：文件组织、批量重命名、内容搜索、备份等
评分说明：
- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析

第 2 套

60 题

一、单项选择题

01 • 单项选择

Python中，以下哪个方法会在对象被删除时自动调用？

A. __init__()
B. __del__()
C. __delete__()
D. __remove__()

答案 B. __del__()

解析 __del__() 是Python中的析构方法（destructor），当对象被垃圾回收机制删除时自动调用。

示例：

```
class MyClass:
    def __init__(self, name):
        self.name = name
        print(f"{self.name} 对象被创建")
    def __del__(self):
        print(f"{self.name} 对象被删除")
obj = MyClass("测试")
del obj
# 显式删除，触发__del__()
# 输出：
# 测试 对象被创建
# 测试 对象被删除
```

知识拓展：

- __del__() 不是严格意义的析构函数，因为Python的垃圾回收机制是不确定的
- 不应该依赖 __del__() 来释放关键资源，应使用上下文管理器（ with 语句）
- 循环引用可能导致 __del__() 不被调用

02 • 单项选择

关于类方法和静态方法，以下说法正确的是：

A. 类方法的第一个参数是 self
B. 静态方法的第一个参数是 cls
C. 类方法可以访问类变量
D. 静态方法可以访问实例变量

答案 C. 类方法可以访问类变量

解析 类方法和静态方法是Python中两种特殊的方法类型。

类方法：

- 使用 @classmethod 装饰器
- 第一个参数是 cls （代表类本身）
- 可以访问和修改类变量

• 可以通过类或实例调用
静态方法：
• 使用 @staticmethod 装饰器
• 没有默认的第一个参数
• 不能访问类变量或实例变量
• 只是逻辑上属于类的普通函数
示例：

```
class MyClass:
    count = 0
    @classmethod
    def class_method(cls):
        cls.count += 1
        return f"类变量count: {cls.count}"
    @staticmethod
    def static_method(x, y):
        return x + y
# 不能访问count
print(MyClass.class_method())
# 类变量count: 1
print(MyClass.static_method(3, 5))
# 8
```

知识拓展：类方法常用于实现工厂模式或提供替代构造器，静态方法用于与类相关但不需要访问类/实例状态的工具函数。

03 • 单项选择

以下代码的输出是什么？

```
class MyClass:
    x = 10
    def __init__(self):
        self.x = 20
obj = MyClass()
print(obj.x, MyClass.x)
```

A. 10 10
B. 20 20
C. 20 10
D. 10 20

答案 C. 20 10

解析 Python中实例变量和类变量的查找遵循特定规则。
变量查找顺序：
1. 先在实例的 __dict__ 中查找
2. 如果找不到，再到类的 __dict__ 中查找
3. 如果还找不到，继续到父类中查找
示例：

```
class MyClass:
    x = 10
    # 类变量
    def __init__(self):
        self.x = 20
# 实例变量，覆盖类变量
```

```
obj = MyClass()
print(obj.x)
# 20 - 访问实例变量
print(MyClass.x)
# 10 - 类变量未被修改
print(obj.__class__.x)
# 10 - 通过类访问
# 删除实例变量后
del obj.x
print(obj.x)
# 10 - 现在访问到类变量
```

知识拓展：

- 实例变量存储在 obj.__dict__ 中
- 类变量存储在 MyClass.__dict__ 中
- 实例变量只对特定实例有效，类变量被所有实例共享

04 • 单项选择

Python中的 __call__() 方法的作用是：

A. 初始化对象

B. 使对象可以像函数一样被调用

C. 删除对象

D. 比较对象

答案 B. 使对象可以像函数一样被调用

解析 __call__() 魔法方法使类的实例可以像函数一样被调用。

示例：

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor
    def __call__(self, x):
        return x * self.factor
# 创建实例
double = Multiplier(2)
triple = Multiplier(3)
# 像函数一样调用实例
print(double(5))
# 10
print(triple(5))
# 15
# 检查是否可调用
print(callable(double))
# True
```

实际应用：

```
class Counter:
    def __init__(self):
        self.count = 0
    def __call__(self):
        self.count += 1
        return self.count
counter = Counter()
print(counter())
```

```
# 1
print(counter())
# 2
print(counter())
# 3
```

知识拓展：

- 装饰器本质上就是利用 `__call__()` 实现的可调用对象
- 函数式编程中的闭包可以用实现了 `__call__()` 的类来替代
- 检查对象是否可调用： `callable(obj)`

05 • 单项选择

以下哪个不是有效的继承语法？

A. `class Child(Parent):`

B. `class Child(Parent1, Parent2):`

C. `class Child extends Parent:`

D. `class Child():`

答案 C. `class Child extends Parent:`

解析 Python使用独特的继承语法，与Java、C++等语言不同。

正确的Python继承语法：

```
# 单继承
class Child(Parent):
    pass
# 多重继承
class Child(Parent1, Parent2, Parent3):
    pass
# 不继承任何类（默认继承object）
class Child:
    pass
# 显式继承object
class Child(object):
    pass
```

错误语法对比：

```
#    Java/C++风格（Python不支持）
class Child extends Parent:
    # SyntaxError
    pass
#    C#风格
class Child : Parent
    # SyntaxError
    pass
```

知识拓展：

- Python的多重继承使用C3线性化算法确定方法解析顺序（MRO）
- 可以使用 `ClassName.__mro__` 或 `ClassName.mro()` 查看继承顺序
- Python 3中所有类都隐式继承自 `object`

06 • 单项选择

关于 `super()` 函数，以下说法错误的是：

A. 用于调用父类的方法

B. 在多重继承中按照MRO顺序调用

C. 只能在 __init__() 方法中使用

D. 可以避免直接使用父类名

答案 C. 只能在 __init__() 方法中使用

解析 super() 函数用于调用父类的方法，可以在类的任何方法中使用。

示例：

```
class Parent:
    def __init__(self):
        self.value = 10
    def method(self):
        print("Parent method")
class Child(Parent):
    def __init__(self):
        super().__init__()
        # 在__init__中使用
        self.value += 5
    def method(self):
        super().method()
        # 在普通方法中使用
        print("Child method")
obj = Child()
obj.method()
# 输出：
# Parent method
# Child method
```

知识拓展：

- Python 2中的用法： super(ClassName, self).method()
- Python 3简化为： super().method()
- 在多重继承中， super() 按照MRO顺序调用下一个类的方法
- 可以通过 ClassName.__mro__ 查看方法解析顺序

7-8. 答案：

答案：C

生成器表达式是一种简洁的创建生成器的方式。

示例：

```
# 列表推导式 - 返回列表
list_comp = [x**2 for x in range(5)]
print(type(list_comp))
# <class 'list'>
print(list_comp)
# [0, 1, 4, 9, 16]
# 生成器表达式 - 返回生成器对象
gen_expr = (x**2 for x in range(5))
print(type(gen_expr))
# <class 'generator'>
print(list(gen_expr))
# [0, 1, 4, 9, 16]
```

区别总结：

```
# 列表推导式：使用[]
list_comp = [x for x in range(1000000)]
# 立即占用大量内存
# 生成器表达式：使用()
```

```
gen_expr = (x for x in range(1000000))
# 惰性计算，节省内存
```

知识拓展：

- 生成器表达式是一次性的，迭代后就耗尽了
- 适合处理大量数据，因为是惰性生成的
- 可以用 next() 函数手动获取下一个值

07 • 单项选择

生成器表达式 (x**2 for x in range(4)) 返回的是：

A. 列表
B. 元组
C. 生成器对象
D. 集合

答案 C. 生成器对象

解析 （原卷合并作答）

08 • 单项选择

以下代码的输出是什么？

```
gen = (x for x in range(3))
print(type(gen))
```

A. <class 'list'>
B. <class 'tuple'>
C. <class 'generator'>
D. <class 'iterator'>

答案 C. <class 'generator'>

解析 （原卷合并作答）

09 • 单项选择

关于 yield from 语句，以下说法正确的是：

A. 用于从生成器中返回值
B. 用于委托给另一个生成器
C. 用于终止生成器
D. 与 yield 完全相同

答案 B. 用于委托给另一个生成器

解析 yield from 用于将生成器的控制权委托给另一个生成器或可迭代对象。

示例：

```
# 使用yield from简化代码
def generator1():
    yield 1
    yield 2
def generator2():
    yield 3
    yield 4
def combined():
    yield from generator1()
    # 委托给generator1
```

```
    yield from generator2()
# 委托给generator2
for value in combined():
    print(value)
# 输出 1, 2, 3, 4
```

不使用yield from的等价代码：

```
def combined():
    for value in generator1():
        yield value
    for value in generator2():
        yield value
```

知识拓展：

- yield from 不仅仅是语法糖，还能正确处理 send() 和异常
- 常用于递归生成器和树遍历
- Python 3.3+才支持此语法

10 • 单项选择

以下哪个函数可以将迭代器转换为列表？

A. list()

B. iter()

C. next()

D. range()

答案 A. list()

解析 list() 函数可以将任何可迭代对象（包括迭代器）转换为列表。

示例：

```
# 迭代器转列表
iter_obj = iter([1, 2, 3, 4, 5])
list_obj = list(iter_obj)
print(list_obj)
# [1, 2, 3, 4, 5]
# 生成器转列表
gen = (x**2 for x in range(5))
print(list(gen))
# [0, 1, 4, 9, 16]
# range对象转列表
print(list(range(5)))
# [0, 1, 2, 3, 4]
# 字符串转列表
print(list("hello"))
# ['h', 'e', 'l', 'l', 'o']
```

知识拓展：

- 迭代器是一次性的，转换为list后原迭代器就耗尽了
- 其他转换方式： tuple() , set() , dict()
- 可以使用列表推导式： [x for x in iterator]

11 • 单项选择

装饰器的执行时机是：

A. 函数被调用时

B. 函数被定义时

C. 程序结束时

D. 模块被导入时

答案 B. 函数被定义时

解析 装饰器在函数定义时执行，而不是在函数调用时。

示例：

```
def my_decorator(func):
    print(f"装饰器执行：装饰{func.__name__}")
    def wrapper():
        print("wrapper执行")
        return func()
    return wrapper
@my_decorator
# 这里执行装饰器
def my_function():
    print("函数执行")
print("---分隔线---")
my_function()
# 这里调用函数
# 输出：
# 装饰器执行：装饰 my_function
# ---分隔线---
# wrapper执行
# 函数执行
```

知识拓展：

- 装饰器的本质是函数转换： `my_function = my_decorator(my_function)`
- 可以使用多个装饰器，执行顺序从下到上
- 应使用 `@functools.wraps` 保留原函数的元数据

12 • 单项选择

以下代码的输出是什么？

```
def outer(x):
    def inner(y):
        return x + y
    return inner
f = outer(10)
print(f(5))
```

A. 10

B. 5

C. 15

D. 报错

答案 C. 15

解析 闭包会保留对外部变量的引用，即使外部函数已经返回。

示例：

```
def outer():
    x = 10
    def inner(y):
        return x + y
    # 闭包：保留了x的引用
```

```
    return inner
func = outer()
print(func(5))
# 15
# 查看闭包变量
print(func.__closure__)
# (<cell at 0x...: int object at 0x...>,)
print(func.__closure__[0].cell_contents)
# 10
```

多个闭包共享变量：

```
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment
counter1 = make_counter()
counter2 = make_counter()
print(counter1())
# 1
print(counter1())
# 2
print(counter2())
# 1 - 独立的闭包
```

知识拓展：

- 闭包常用于数据封装和装饰器实现
- 使用 nonlocal 关键字可以修改外部变量
- 闭包可能导致内存不能被释放，需注意内存泄漏

13 • 单项选择

关于闭包，以下说法正确的是：

A. 内部函数可以访问外部函数的变量

B. 闭包会保存外部函数的环境

C. 闭包可以实现数据隐藏

D. 以上都正确

答案 D. 以上都正确

解析 *args 和 **kwargs 的多种用途。

示例：

```
# 1. 接收任意数量的位置参数
def func1(*args):
    print(args)
func1(1, 2, 3)
# (1, 2, 3)
# 2. 接收任意数量的关键字参数
def func2(**kwargs):
    print(kwargs)
func2(a=1, b=2)
# {'a': 1, 'b': 2}
# 3. 传递参数给其他函数
```

```
def wrapper(*args, **kwargs):
    return other_func(*args, **kwargs)
# 4. 组合使用
def func3(a, b, *args, x=10, **kwargs):
    print(f"a={a}, b={b}, args={args}")
    print(f"x={x}, kwargs={kwargs}")
func3(1, 2, 3, 4, x=5, y=6, z=7)
# a=1, b=2, args=(3, 4)
# x=5, kwargs={'y': 6, 'z': 7}
```

知识拓展：

- *args 必须在 **kwargs 之前
- Python 3支持在 *args 之后使用关键字参数
- 可以用于函数定义和函数调用

14 • 单项选择

以下代码的输出是什么？

```
def func(a, b, *args, **kwargs):
    print(len(args), len(kwargs))
func(1, 2, 3, 4, x=5, y=6)
```

A. 2 2

B. 4 2

C. 2 4

D. 6 0

答案 A. 2 2

解析 参数解包和打包的区别。

示例：

```
def func(a, b, *args, x, **kwargs):
    print(f"a={a}, b={b}")
    print(f"args={args}, type={type(args)}")
    print(f"x={x}")
    print(f"kwargs={kwargs}, type={type(kwargs)}")
func(1, 2, 3, 4, x=5, y=6, z=7)
# a=1, b=2
# args=(3, 4), type=<class 'tuple'>
# x=5
# kwargs={'y': 6, 'z': 7}, type=<class 'dict'>
```

调用时解包：

```
def func(a, b, c):
    print(a, b, c)
args_tuple = (1, 2, 3)
func(*args_tuple)
# 解包元组
kwargs_dict = {'a': 1, 'b': 2, 'c': 3}
func(**kwargs_dict)
# 解包字典
```

知识拓展：

- 在函数定义中： *args 是打包， **kwargs 是打包
- 在函数调用中： *args 是解包， **kwargs 是解包
- Python 3.5+支持多次解包： {**dict1, **dict2}

15 • 单项选择

关于参数解包，以下说法错误的是：

A. * 用于解包序列

B. ** 用于解包字典

C. 只能在函数定义时使用

D. 可以在函数调用时使用

答案 C. 只能在函数定义时使用

解析 * 和 ** 可以在函数定义和调用时使用。

函数定义时解包（打包）：

```
def func(*args, **kwargs):
    # args是元组，kwargs是字典
    pass
```

函数调用时解包：

```
args = (1, 2, 3)
kwargs = {'a': 10, 'b': 20}
func(*args, **kwargs)
```

其他解包场景：

```
# 列表解包
first, *rest = [1, 2, 3, 4, 5]
print(first, rest)
# 1 [2, 3, 4, 5]
# 字典合并
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, 'd': 4}
merged = {**dict1, **dict2}
print(merged)
# {'a': 1, 'b': 2, 'c': 3, 'd': 4}
```

知识拓展：

- Python 3.5+支持字典解包： {**dict1, **dict2}
- Python 3.0+支持扩展解包： a, *b, c = [1,2,3,4,5]
- 解包可以用于函数调用、列表赋值、字典合并等

16 • 单项选择

以下代码的输出是什么？

```
*a, b = [1, 2, 3, 4]
print(a)
```

A. [1, 2, 3]

B. [1]

C. 4

D. [2, 3, 4]

答案 A. [1, 2, 3]

解析 扩展解包语法允许在赋值时捕获多个值。

示例：

```
# 基本用法
*a, b = [1, 2, 3, 4]
print(a)
# [1, 2, 3]
print(b)
```

```
# 4
# 中间位置
a, *b, c = [1, 2, 3, 4, 5]
print(a)
# 1
print(b)
# [2, 3, 4]
print(c)
# 5
# 开头位置
first, *rest = [1, 2, 3, 4, 5]
print(first)
# 1
print(rest)
# [2, 3, 4, 5]
# 只能有一个*
# a, *b, *c = [1,2,3] # SyntaxError
```

知识拓展：

- Python 3.0+才支持扩展解包
- * 只能出现一次
- 如果没有足够的值， * 捕获的是空列表
- 常用于处理不定长度的序列

17 • 单项选择

Pandas中，Series和DataFrame的主要区别是：

A. Series是一维的，DataFrame是二维的

B. Series是二维的，DataFrame是一维的

C. 没有区别

D. Series不能有索引

答案 A. Series是一维的，DataFrame是二维的

解析 Pandas的两个核心数据结构：Series和DataFrame。

示例：

```
import pandas as pd
# Series - 一维带标签的数组
s = pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])
print(s)
# a 1
# b 2
# c 3
# d 4
# dtype: int64
# DataFrame - 二维表格数据
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
})
print(df)
# A B C
# 0 1 4 7
```

```
# 1 2 5 8
# 2 3 6 9
# 关系
print(type(df['A']))
# <class 'pandas.core.series.Series'>
# DataFrame的每一列都是一个Series
```

知识拓展：

- Series可以看作是带索引的一维数组
- DataFrame可以看作是Series的字典
- Pandas还有Panel（三维），但已被弃用

18 • 单项选择

以下哪个方法用于查看DataFrame的前几行？

A. head()

B. top()

C. first()

D. show()

答案 A. head()

解析 Pandas查看数据的常用方法。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': range(100),
    'B': range(100, 200)
})
# 查看前几行
print(df.head())
# 默认前5行
print(df.head(10))
# 前10行
# 查看后几行
print(df.tail())
# 默认后5行
print(df.tail(10))
# 后10行
# 查看数据信息
print(df.info())
# 数据类型、内存使用
# 查看统计信息
print(df.describe()) # 基本统计指标
```

知识拓展：

- head() 和 tail() 常用于快速查看数据样本
- info() 显示每列的数据类型和缺失值情况
- describe() 只显示数值列的统计信息

19 • 单项选择

Pandas中，以下哪个方法用于数据排序？

A. sort()

B. order()

C. sort_values()

D. arrange()

答案 C. sort_values()

解析 Pandas数据排序的方法。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [3, 1, 2],
    'B': [6, 4, 5]
})
# 按单列排序
df_sorted = df.sort_values('A')
print(df_sorted)
# A B
# 1 1 4
# 2 2 5
# 0 3 6
# 按多列排序
df_sorted = df.sort_values(['A', 'B'], ascending=[True, False])
# 按索引排序
df_sorted = df.sort_index()
# 就地排序
df.sort_values('A', inplace=True)
```

知识拓展：

- sort_values() 按值排序， sort_index() 按索引排序
- ascending 控制升序/降序
- inplace=True 直接修改原数据
- 可以指定 na_position 处理缺失值的位置

20 • 单项选择

关于Pandas的 apply() 方法，以下说法正确的是：

A. 只能应用于DataFrame

B. 只能应用于Series

C. 可以应用自定义函数

D. 不能使用lambda函数

答案 C. 可以应用自定义函数

解析 Pandas的 apply() 方法用于应用自定义函数。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
# 对列应用函数
df['C'] = df['A'].apply(lambda x: x ** 2)
print(df)
# A B C
# 0 1 4 1
# 1 2 5 4
```

```
# 2 3 6 9
# 对整个DataFrame应用
result = df.apply(lambda col: col.sum(), axis=0)
# 按列
print(result)
# A 6
# B 15
# C 14
result = df.apply(lambda row: row.sum(), axis=1)
# 按行
print(result)
# 0 6
# 1 11
# 2 18
# 使用自定义函数
def custom_func(x):
    if x > 2:
        return x * 2
df['D'] = df['A'].apply(custom_func)
```

知识拓展：

- apply() 比循环更高效，但比向量化操作慢
- axis=0 按列， axis=1 按行
- 可以使用 applymap() 对每个元素应用函数
- 优先使用向量化操作，其次才是 apply()

21-25. 答案：

答案：B B B A B - Pandas和Pathlib相关题目，已在第1套中有类似详细解释

21 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
s = pd.Series([1, 2, 3, 4])
print(s[s > 2].sum())
```

A. 3
B. 7
C. 10
D. 4

答案 B. 7

解析 （原卷合并作答）

22 • 单项选择

pathlib.Path 对象的 parent 属性返回的是：

A. 文件名
B. 父目录路径
C. 文件扩展名
D. 绝对路径

答案 B. 父目录路径

解析 （原卷合并作答）

23 • 单项选择

以下哪个方法用于创建目录？

A. path.create()
B. path.mkdir()
C. path.makedir()
D. path.new()

答案 B. path.mkdir()

解析 （原卷合并作答）

24 • 单项选择

关于 Path.glob() 方法，以下说法正确的是：

A. 用于搜索匹配模式的文件
B. 返回列表
C. 不支持通配符
D. 只能搜索文件不能搜索目录

答案 B. 返回列表

解析 （原卷合并作答）

25 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('folder/subfolder/file.txt')
print(p.parts)
```

A. 'folder/subfolder/file.txt'
B. ('folder', 'subfolder', 'file.txt')
C. ['folder', 'subfolder', 'file.txt']
D. 'file.txt'

答案 B. ('folder', 'subfolder', 'file.txt')

解析 （原卷合并作答）

26 • 单项选择

关于 enumerate() 函数，以下说法正确的是：

A. 返回索引和值的元组
B. 只能用于列表
C. 不能指定起始索引
D. 返回字典

答案 A. 返回索引和值的元组

解析 enumerate() 函数用于在迭代时同时获取索引和值。

示例：

```
# 基本用法
fruits = ['apple', 'banana', 'cherry']
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")
# 0: apple
```

```
# 1: banana
# 2: cherry
# 指定起始索引
for index, fruit in enumerate(fruits, start=1):
    print(f"{index}: {fruit}")
# 1: apple
# 2: banana
# 3: cherry
# 转换为列表
indexed_list = list(enumerate(fruits))
print(indexed_list)
# [(0, 'apple'), (1, 'banana'), (2, 'cherry')]
```

知识拓展：

- enumerate() 返回的是迭代器，不是列表
- 可以通过 start 参数指定起始索引
- 常用于需要同时使用索引和值的场景

27 • 单项选择

以下代码的输出是什么？

```
result = [x+y for x in [1,2] for y in [3,4]]
print(result)
```

A. [4, 5, 5, 6]

B. [4, 6]

C. [1, 2, 3, 4]

D. [[4, 5], [5, 6]]

答案 A. [4, 5, 5, 6]

解析 嵌套列表推导式的执行顺序。

示例：

```
# 题目代码
result = [x + y for x in [1, 2] for y in [3, 4]]
print(result)
# [4, 5, 5, 6]
# 等价的普通循环
result = []
for x in [1, 2]:
    for y in [3, 4]:
        result.append(x + y)
# x=1: 1+3=4, 1+4=5
# x=2: 2+3=5, 2+4=6
# 三重嵌套
result = [x + y + z
        for x in [1, 2]
        for y in [10, 20]
        for z in [100, 200]]
print(len(result))
# 2*2*2 = 8
```

知识拓展：

- 嵌套循环的顺序：左侧是外层，右侧是内层
- 可以添加条件过滤： [x+y for x in [1,2] for y in [3,4] if x != y]

- 过多嵌套会降低可读性，建议不超过两层

28 • 单项选择

集合推导式的语法是：

A. `[x for x in iterable]`

B. `(x for x in iterable)`

C. `{x for x in iterable}`

D. `<x for x in iterable>`

答案 C. `{x for x in iterable}`

解析 Python中的四种推导式语法。

示例：

```
# 列表推导式 - []
list_comp = [x**2 for x in range(5)]
print(list_comp)
# [0, 1, 4, 9, 16]
print(type(list_comp))
# <class 'list'>
# 集合推导式 - {}
set_comp = {x**2 for x in range(5)}
print(set_comp)
# {0, 1, 4, 9, 16}
print(type(set_comp))
# <class 'set'>
# 字典推导式 - {k: v}
dict_comp = {x: x**2 for x in range(5)}
print(dict_comp)
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
print(type(dict_comp))
# <class 'dict'>
# 生成器表达式 - ()
gen_expr = (x**2 for x in range(5))
print(list(gen_expr))
# [0, 1, 4, 9, 16]
print(type(gen_expr))
# <class 'generator'>
```

知识拓展：

- 集合推导式会自动去重
- 字典推导式需要同时指定键和值
- 生成器表达式不会立即计算，节省内存

29 • 单项选择

以下代码的输出是什么？

```
a = [1, 2, 3]
b = [4, 5]
result = list(zip(a, b))
print(len(result))
```

A. 2

B. 3

C. 5

D. 报错

答案 A. 2

解析 zip() 函数将多个可迭代对象打包在一起。

示例：

```
# 基本用法
names = ['Alice', 'Bob', 'Charlie']
ages = [25, 30]
scores = [85, 90, 95, 100]
result = list(zip(names, ages, scores))
print(result)
# [('Alice', 25, 85), ('Bob', 30, 90)]
# 以最短序列为准，长度为2
# 解包
for name, age, score in zip(names, ages, scores):
    print(f"{name}: {age}岁, {score}分")
# 转置矩阵
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
transposed = list(zip(*matrix))
print(transposed)
# [(1, 4, 7), (2, 5, 8), (3, 6, 9)]
# zip_longest从 itertools模块
from itertools import zip_longest
result = list(zip_longest(names, ages, fillvalue=0))
print(result)
# [('Alice', 25), ('Bob', 30), ('Charlie', 0)]
```

知识拓展：

- zip() 返回迭代器，需要 list() 转换
- 长度以最短序列为准
- 使用 itertools.zip_longest() 可以以最长序列为准
- * 操作符可以解压 zip 结果

30 • 单项选择

关于 filter() 函数，以下说法正确的是：

A. 返回列表

B. 返回迭代器

C. 不能使用lambda函数

D. 只能过滤数字

答案 B. 返回迭代器

解析 filter() 函数用于过滤序列。

示例：

```
# 过滤偶数
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(type(even_numbers))
# <class 'filter'> - 迭代器
print(list(even_numbers))
# [2, 4, 6, 8, 10]
# 使用自定义函数
```

```
def is_positive(x):
    return x > 0
numbers = [-2, -1, 0, 1, 2, 3]
positive = list(filter(is_positive, numbers))
print(positive)
# [1, 2, 3]
# 过滤None值
values = [0, 1, None, 2, False, 3, '', 4]
filtered = list(filter(None, values))
# None作为过滤器
print(filtered)
# [1, 2, 3, 4] - 过滤掉假值
对比列表推导式：
# filter
result = list(filter(lambda x: x > 0, numbers))
# 列表推导式 - 更常用
result = [x for x in numbers if x > 0]
知识拓展：
• filter() 返回迭代器，需要 list() 转换
• 第一个参数是判断函数，第二个参数是可迭代对象
• 现代Python更推荐使用列表推导式
• filter(None, iterable) 可以过滤掉所有假值
```

二、不定项选择题

31 • 不定项选择

关于Python的魔法方法，以下说法正确的有：

A. __eq__() 用于定义相等比较
B. __lt__() 用于定义小于比较
C. __add__() 用于定义加法运算
D. __getitem__() 用于定义索引访问

答案 A. __eq__() 用于定义相等比较；B. __lt__() 用于定义小于比较；C. __add__() 用于定义加法运算；D. __getitem__() 用于定义索引访问

解析 Python常用魔法方法概览。

示例：

```
class MyClass:
    # A. __str__ - 定义print()和str()的行为
    def __str__(self):
        return "MyClass对象"
    # B. __repr__ - 定义repr()的行为，用于调试
    def __repr__(self):
        return "MyClass()"
    # C. __len__ - 定义len()的行为
    def __len__(self):
        return 10
    # D. __getitem__ - 定义索引访问 obj[key]
    def __getitem__(self, key):
        return key * 2
```

```
obj = MyClass()
print(str(obj))
# MyClass对象
print(repr(obj))
# MyClass()
print(len(obj))
# 10
print(obj[5])
# 10
```

知识拓展：其他重要魔法方法： __init__ ， __new__ ， __del__ ， __call__ ， __enter__ ， __exit__ ， __iter__ ， __next__ ， __eq__ ， __lt__ ， __add__ ， __mul__ 等。

32 • 不定项选择

以下哪些是Python的抽象基类相关的模块：

A. abc

B. collections.abc

C. typing

D. abstract

答案 A. abc；B. collections.abc

解析 Python中的抽象基类模块。

示例：

```
from abc import ABC, abstractmethod
# 使用abc模块
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass
class Dog(Animal):
    def make_sound(self):
        return "汪汪"
# 使用collections.abc
from collections.abc import Iterable, Iterator
class MyRange:
    def __init__(self, n):
        self.n = n
        self.i = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.i < self.n:
            result = self.i
            self.i += 1
            return result
        raise StopIteration
print(isinstance(MyRange(5), Iterable))
# True
print(isinstance(MyRange(5), Iterator))
# True
```

知识拓展：

• abc 模块：提供 ABC 基类和 @abstractmethod 装饰器

- collections.abc ：提供容器相关的抽象基类（Iterable, Iterator, Sequence等）

33 • 不定项选择

关于属性装饰器，以下说法正确的有：

A. @property 定义getter方法

B. @属性名.setter 定义setter方法

C. @属性名.deleter 定义deleter方法

D. 可以实现只读属性

答案 A. @property 定义getter方法；B. @属性名.setter 定义setter方法；C. @属性名.deleter 定义deleter方法；D. 可以实现只读属性

解析 生成器的多种特性。

示例：

```
# A. 惰性计算，节省内存
def large_range(n):
    i = 0
    while i < n:
        yield i
        i += 1
gen = large_range(1000000)
# 不会立即分配内存
# B. 只能迭代一次
gen = (x for x in range(3))
print(list(gen))
# [0, 1, 2]
print(list(gen))
# [] - 已耗尽
# C. 可以使用yield表达式接收值
def coroutine():
    total = 0
    while True:
        x = yield total
        if x is not None:
            total += x
coro = coroutine()
next(coro)
# 启动协程
print(coro.send(10))
# 10
print(coro.send(20))
# 30
# D. 支持异常处理
def gen_with_exception():
    try:
        yield 1
        yield 2
    except ValueError:
        yield "caught error"
g = gen_with_exception()
print(next(g))
# 1
```

```
print(g.throw(ValueError))
                # caught error
```

知识拓展：生成器的三个主要方法： `__next__()` 、 `send()` 、 `throw()` 、 `close()` 。

34 • 不定项选择

关于生成器的优势，以下说法正确的有：

A. 节省内存
B. 支持惰性计算
C. 可以表示无限序列
D. 执行速度总是更快

答案 A. 节省内存；B. 支持惰性计算；C. 可以表示无限序列

解析 生成器的优劣势分析。

示例：

```
import sys
# A. 节省内存
list_data = [x for x in range(1000000)]
gen_data = (x for x in range(1000000))
print(sys.getsizeof(list_data))
# ~8MB
print(sys.getsizeof(gen_data))
# ~200 bytes
# B. 惰性计算
def process_large_file():
    with open('large.txt') as f:
        for line in f:
            # 文件对象是迭代器
            yield line.strip()
# C. 可以表示无限序列
def fibonacci():
    a, b = 0, 1
    while True:
        # 无限循环
        yield a
        a, b = b, a + b
# D. 不总是更快
# 如果需要多次访问数据，列表可能更快
data = [x**2 for x in range(1000)]
# 只计算一次
for _ in range(100):
    for x in data:
        # 快速重复访问
        pass
```

知识拓展：生成器适合一次性处理大量数据，但如果需要多次访问或随机访问，列表更合适。

35 • 不定项选择

以下哪些操作会创建迭代器：

A. `iter([1, 2, 3])`
B. `(x for x in range(10))`
C. `open('file.txt')`

D. range(10)

答案 A. iter([1, 2, 3])；B. (x for x in range(10))；C. open('file.txt')；D. range(10)

解析 创建迭代器的多种方式。

示例：

```
# A. iter()函数
my_list = [1, 2, 3]
iterator = iter(my_list)
print(next(iterator))
# 1
# B. 生成器函数
def my_generator():
    yield 1
    yield 2
    yield 3
gen = my_generator()
print(next(gen))
# 1
# C. 生成器表达式
gen_expr = (x for x in range(3))
print(next(gen_expr))
# 0
# D. 自定义迭代器类
class MyIterator:
    def __init__(self, n):
        self.n = n
        self.i = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.i < self.n:
            result = self.i
            self.i += 1
            return result
        raise StopIteration
my_iter = MyIterator(3)
print(next(my_iter))
# 0
```

知识拓展：Python中的迭代器协议需要实现 __iter__() 和 __next__() 方法。

36 • 不定项选择

关于装饰器的特性，以下说法正确的有：

A. 可以嵌套使用

B. 可以带参数

C. 可以装饰类

D. 会修改原函数的 __name__ 属性

答案 A. 可以嵌套使用；C. 可以装饰类；D. 会修改原函数的 __name__ 属性

解析 装饰器对函数元数据的影响。

示例：

```
from functools import wraps
```

```
# 没有使用wraps
def my_decorator1(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
@my_decorator1
def original_func1():
    """This is the original function"""
    pass
print(original_func1.__name__)
# wrapper - 被修改了
print(original_func1.__doc__)
# None - 丢失了
# 使用wraps保留元数据
def my_decorator2(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
@my_decorator2
def original_func2():
    """This is the original function"""
    pass
print(original_func2.__name__)
# original_func2 - 保留了
print(original_func2.__doc__)
# This is the original function
```

知识拓展： functools.wraps 会保留 __name__ , __doc__ , __module__ , __qualname__ , __annotations__ 等元数据。

7-15. 答案：

答案：ABCD, ABC, ABCD, ABCD, ABCD, ABC, ABCD, ABCD, ABC

这些题目涉及闭包、Pandas、pathlib、函数式编程等高级概念，已在第1套中有类似详细详解。

核心要点包括：

- 闭包的多种应用场景
- Pandas数据结构和聚合方法
- pathlib的文件操作能力
- 高阶函数和函数式编程特性

37 • 不定项选择

关于函数的默认参数，以下说法正确的有：

A. 默认参数在函数定义时计算

B. 可变对象作为默认参数可能导致问题

C. 默认参数必须在位置参数之后

D. 可以使用 None 作为默认值

答案 A. 默认参数在函数定义时计算；B. 可变对象作为默认参数可能导致问题；C. 默认参数必须在位置参数之后；D. 可以使用 None 作为默认值

解析 （原卷合并作答）

38 • 不定项选择

以下哪些是Pandas的数据结构：

A. Series
B. DataFrame
C. Panel
D. Array

答案 A. Series；B. DataFrame；C. Panel；D. Array

解析 （原卷合并作答）

39 • 不定项选择

关于Pandas的索引操作，以下说法正确的有：

A. loc 支持切片操作
B. iloc 只接受整数索引
C. at 用于快速标量访问
D. iat 用于快速整数位置访问

答案 A. loc 支持切片操作；B. iloc 只接受整数索引；C. at 用于快速标量访问；D. iat 用于快速整数位置访问

解析 （原卷合并作答）

40 • 不定项选择

Pandas中，以下哪些方法用于数据聚合：

A. sum()
B. mean()
C. count()
D. agg()

答案 A. sum()；B. mean()；C. count()；D. agg()

解析 （原卷合并作答）

41 • 不定项选择

关于 pathlib 的优势，以下说法正确的有：

A. 面向对象的API
B. 跨平台路径处理
C. 链式调用
D. 比os.path更直观

答案 A. 面向对象的API；B. 跨平台路径处理；C. 链式调用；D. 比os.path更直观

解析 （原卷合并作答）

42 • 不定项选择

Path 对象的以下哪些方法会修改文件系统：

A. mkdir()
B. touch()
C. unlink()
D. exists()

答案 A. mkdir()；B. touch()；C. unlink()；D. exists()

解析 （原卷合并作答）

43 • 不定项选择

关于列表推导式的性能，以下说法正确的有：

A. 通常比for循环快
B. 代码更简洁
C. 可读性更好
D. 适合简单的转换操作

答案 A. 通常比for循环快；B. 代码更简洁；C. 可读性更好；D. 适合简单的转换操作

解析 （原卷合并作答）

44 • 不定项选择

以下哪些是Python的内置高阶函数：

A. map()
B. filter()
C. reduce()
D. zip()

答案 A. map()；B. filter()；C. reduce()；D. zip()

解析 （原卷合并作答）

45 • 不定项选择

关于 *args 和 **kwargs ，以下说法正确的有：

A. 名称可以自定义
B. *args 是元组类型
C. **kwargs 是字典类型
D. 必须同时使用

答案 A. 名称可以自定义；B. *args 是元组类型；C. **kwargs 是字典类型；D. 必须同时使用

解析 （原卷合并作答）

三、填空题

46 • 填空 / 简答

Python中，使用 _____ 方法可以使对象支持 len() 函数，使用 _____ 方法可以使对象支持索引访问。

答案 __len__() ， __getitem__()

解析 实现序列协议需要这两个魔法方法。

示例：

```
class MySequence:
    def __init__(self, data):
        self._data = data
    def __len__(self):
        return len(self._data)
    def __getitem__(self, index):
        return self._data[index]
seq = MySequence([1, 2, 3, 4, 5])
print(len(seq))
```

```
# 5
print(seq[2])
# 3
print(seq[1:4])
# [2, 3, 4]
# 可以使用for循环
for item in seq:
    print(item)
```

知识拓展：实现这两个方法后，对象就可以支持 len() 、索引、切片和迭代。

47 • 填空 / 简答

迭代器必须实现 _____ 和 _____ 两个方法才能符合迭代器协议。

答案 __iter__() ， __next__()

解析 解析：迭代器协议的两个核心方法。

示例：

```
class Countdown:
    def __init__(self, start):
        self.current = start
    def __iter__(self):
        return self
    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1
# 使用
for num in Countdown(5):
    print(num)
# 5, 4, 3, 2, 1
```

知识拓展： __iter__() 返回迭代器对象本身， __next__() 返回下一个值或抛出 StopIteration。

48 • 填空 / 简答

在Pandas中，_____ 方法用于删除重复行，_____ 方法用于重置索引。

答案 drop_duplicates() ， reset_index()

解析 解析：Pandas数据清洗的两个关键方法。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 2, 3, 3, 3],
    'B': [4, 5, 5, 6, 7, 7]
})
# 去重
df_unique = df.drop_duplicates()
print(df_unique)
# A B
# 0 1 4
# 1 2 5
# 3 3 6
```

```
# 4 3 7
# 重置索引
df_reset = df_unique.reset_index(drop=True)
print(df_reset)
# A B
# 0 1 4
# 1 2 5
# 2 3 6
# 3 3 7
```

知识拓展： drop_duplicates() 可指定列去重， reset_index(drop=True) 丢弃原索引。

49 • 填空 / 简答

使用 _____ 函数可以同时获取索引和值，使用 _____ 函数可以反转序列。

答案 enumerate() ， reversed()

解析 触析：内置函数，用于序列操作。

示例：

```
data = ['a', 'b', 'c']
# enumerate - 同时获取索引和值
for i, val in enumerate(data, start=1):
    print(f"{i}: {val}")
# 1: a
# 2: b
# 3: c
# reversed - 反向迭代
for val in reversed(data):
    print(val)
# c, b, a
```

知识拓展： enumerate() 返回迭代器， reversed() 也返回迭代器，不修改原序列。

50 • 填空 / 简答

pathlib.Path 对象的 _____ 方法返回所有父目录，_____ 方法用于解析相对路径为绝对路径。

答案 parents ， resolve()

解析 羞析：pathlib路径操作方法。

示例：

```
from pathlib import Path
p = Path('/home/user/docs/file.txt')
# parents - 获取所有父目录
for parent in p.parents:
    print(parent)
# /home/user/docs
# resolve() - 解析为绝对路径
rel_path = Path('docs/../file.txt')
abs_path = rel_path.resolve()
print(abs_path)
# /home/user/file.txt
```

知识拓展： parents 返回所有祖先目录， resolve() 解析符号链接和 .. 。

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class Rectangle:
    def __init__(self, width, height):
        self.width = width
        self.height = height
    @property
    def area(self):
        return self.width * self.height
    @property
    def perimeter(self):
        return 2 * (self.width + self.height)
    def __str__(self):
        return f"Rectangle({self.width}x{self.height})"
rect = Rectangle(5, 3)
print(rect.area)
print(rect.perimeter)
print(rect)
```

问题：

- (1) 三个 print 语句的输出分别是什么？
- (2) 为什么 area 和 perimeter 调用时不需要括号？
- (3) 如果要添加一个setter使 area 可以被修改，应该如何实现？

答案 • (1) 输出：

```
15
16
Rectangle(5x3)
```

- (2) 因为使用了 @property 装饰器，将方法转换为属性
- (3) 实现setter需要计算并设置width和height，但这在数学上不唯一，通常area作为只读属性更合理

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
def countdown(n):
    print(f"开始倒计时: {n}")
    while n > 0:
        yield n
        n -= 1
    print("倒计时结束")
gen = countdown(3)
print(next(gen))
for i in gen:
    print(i)
```

问题：

- (1) 完整的输出是什么？
- (2) "倒计时结束"会在什么时候打印？
- (3) 如果再次调用 next(gen) 会发生什么？

答案 • (1) 输出：

开始倒计时: 3
3
2
1
倒计时结束

- (2) 当生成器耗尽时（所有值都被yield后）打印
- (3) 会抛出 StopIteration 异常

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'product': ['A', 'B', 'A', 'B', 'A'],
    'sales': [100, 150, 200, 120, 180],
    'region': ['East', 'West', 'East', 'West', 'East']
})
result = df.groupby('product')['sales'].sum()
print(result)
print(type(result))
```

问题：

- (1) result 的输出是什么？
- (2) result 的类型是什么？
- (3) 如何获取产品A的总销售额？

答案 分组聚合

- (1) 输出：

product
A 480
B 270
Name: sales, dtype: int64

- (2) pandas.Series
- (3) result['A'] 或 result.loc['A']

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
from pathlib import Path
def find_files(directory, pattern):
    path = Path(directory)
    for file in path.rglob(pattern):
        yield file.name
```

```
# 假设目录结构：
# /data
# ├── file1.txt
# ├── subdir
# │   └── file2.txt
# └── file3.csv
files = list(find_files('/data', '*.txt'))
print(files)
```

问题：

- (1) files 的内容是什么？
- (2) rglob() 和 glob() 的区别是什么？
- (3) 如果要获取完整路径而不是文件名，应该如何修改？

答案 • (1) ['file1.txt', 'file2.txt']
• (2) rglob() 递归搜索所有子目录， glob() 只搜索当前目录
• (3) 将 yield file.name 改为 yield file 或 yield str(file)

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
def memoize(func):
    cache = {}
    def wrapper(n):
        if n not in cache:
            cache[n] = func(n)
        return cache[n]
    return wrapper
@memoize
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n-1)
print(factorial(5))
```

问题：

- (1) 两次调用 factorial(5) 的输出是什么？
- (2) 这个装饰器实现了什么功能？
- (3) 第二次调用 factorial(5) 为什么更快？

答案 • (1) 两次都输出：120
• (2) 实现了记忆化（缓存），避免重复计算
• (3) 第二次直接从缓存中获取结果，不需要重新计算

解析

五、程序设计题

56 • 填空 / 简答

设计一个 Book 类和 Library 类：
Book类要求：
类要求：
• 属性：书名、作者、ISBN、是否借出
• 方法： __str__() 返回书籍信息
Library类要求：
类要求：
• 属性：图书列表
• 方法：
add_book(book) ：添加图书
borrow_book(isbn) ：借书
return_book(isbn) ：还书
available_books ：使用 @property 返回可借图书列表
__len__() ：返回图书总数
创建图书馆实例，添加至少3本书，演示借书和还书功能。

答案
```
class Book:
    def __init__(self, title, author, isbn):
        self.title = title
        self.author = author
        self.isbn = isbn
        self.is_borrowed = False
    def __str__(self):
        status = "已借出" if self.is_borrowed else "可借"
        return f"《{self.title}》- {self.author} (ISBN: {self.isbn}) [{status}]"
class Library:
    def __init__(self):
        self.books = []
    def add_book(self, book):
        self.books.append(book)
        print(f"已添加图书：{book.title}")
    def borrow_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                if not book.is_borrowed:
                book.is_borrowed = True
                print(f"成功借出：{book.title}")
                return True
                else:
                print(f"《{book.title}》已被借出")
                return False
        print("未找到该图书")
        return False
    def return_book(self, isbn):
        for book in self.books:
            if book.isbn == isbn:
                if book.is_borrowed:
                book.is_borrowed = False
```

```
                print(f"成功归还：{book.title}")
                return True
                else:
                print(f"《{book.title}》未被借出")
                return False
        print("未找到该图书")
        return False
    @property
    def available_books(self):
        return [book for book in self.books if not book.is_borrowed]
    def __len__(self):
        return len(self.books)
# 使用示例
library = Library()
library.add_book(Book("Python编程", "张三", "001"))
library.add_book(Book("数据分析", "李四", "002"))
library.add_book(Book("机器学习", "王五", "003"))
print(f"\n图书馆共有{len(library)} 本书")
print("\n可借图书：")
for book in library.available_books:
    print(book)
library.borrow_book("001")
library.return_book("001")
```

解析

57 • 填空 / 简答

编写以下函数：

1. read_numbers(filename) ：生成器函数，逐行读取文件中的数字
2. running_average(numbers) ：生成器函数，计算移动平均值
3. 使用装饰器 @timer 测量函数执行时间

要求：

- 使用 yield 实现惰性计算
- 处理文件不存在的异常
- 使用 pathlib 处理文件路径

答案

```
from pathlib import Path
import time
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} 执行时间: {end-start:.4f}秒")
        return result
    return wrapper
def read_numbers(filename):
    path = Path(filename)
    try:
```

```
        if not path.exists():
            raise FileNotFoundError(f"文件{filename} 不存在")
        with path.open('r') as f:
            for line in f:
                try:
                yield float(line.strip())
                except ValueError:
                continue
    except Exception as e:
        print(f"读取文件错误: {e}")
def running_average(numbers):
    total = 0
    count = 0
    for num in numbers:
        total += num
        count += 1
        yield total / count
@timer
def process_data(filename):
    numbers = read_numbers(filename)
    averages = running_average(numbers)
    return list(averages)
# 使用示例
# 创建测试文件
test_file = Path('numbers.txt')
test_file.write_text('10\n20\n30\n40\n50')
result = process_data('numbers.txt')
print(f"移动平均值: {result}")
```

解析

58 • 填空 / 简答

使用Pandas完成以下任务：

创建销售数据DataFrame（日期、产品、销量、金额）：

- 计算每个产品的总销量和总金额
- 找出销量最高的产品
- 按月份统计销售额
- 使用 apply() 计算折扣后金额（9折）
- 使用列表推导式筛选销量大于100的记录
- 将结果保存为Excel文件

答案

```
import pandas as pd
from datetime import datetime, timedelta
# 创建销售数据
dates = [datetime(2024, 1, 1) + timedelta(days=i) for i in range(10)]
data = {
    '日期': dates,
    '产品': ['A', 'B', 'A', 'C', 'B', 'A', 'C', 'B', 'A', 'C'],
    '销量': [120, 80, 150, 90, 110, 130, 95, 100, 140, 85],
```

```
    '金额': [1200, 1600, 1500, 1800, 2200, 1300, 1900, 2000, 1400, 1700]
}
df = pd.DataFrame(data)
# 计算每个产品的总销量和总金额
product_stats = df.groupby('产品').agg({
    '销量': 'sum',
    '金额': 'sum'
})
print("产品统计：")
print(product_stats)
# 找出销量最高的产品
top_product = df.groupby('产品')['销量'].sum().idxmax()
print(f"\n销量最高的产品: {top_product}")
# 按月份统计销售额
df['月份'] = df['日期'].dt.to_period('M')
monthly_sales = df.groupby('月份')['金额'].sum()
print(f"\n月度销售额:\n{monthly_sales}")
# 计算折扣后金额
df['折扣后金额'] = df['金额'].apply(lambda x: x * 0.9)
# 筛选销量大于100的记录
high_sales = [row for _, row in df.iterrows() if row['销量'] > 100]
print(f"\n销量大于100的记录数: {len(high_sales)}")
# 保存为Excel
df.to_excel('sales_analysis.xlsx', index=False)
print("\n数据已保存到 sales_analysis.xlsx")
```

解析

59 • 填空 / 简答

使用 pathlib 实现以下功能：

1. create_project_structure(base_path, structure) ：
- 根据字典结构创建项目目录
- 例如： {'src': ['main.py', 'utils.py'], 'tests': []}
2. find_large_files(directory, size_mb) ：
- 使用生成器返回大于指定大小的文件
- 返回文件路径和大小
3. backup_files(source, destination, pattern) ：
- 复制匹配模式的文件到目标目录
- 使用 Path 对象操作

答案

```
from pathlib import Path
import shutil
def create_project_structure(base_path, structure):
    base = Path(base_path)
    base.mkdir(parents=True, exist_ok=True)
    for folder, files in structure.items():
        folder_path = base / folder
        folder_path.mkdir(exist_ok=True)
        for file in files:
```

```
            file_path = folder_path / file
            file_path.touch()
    print(f"项目结构已创建在: {base}")
def find_large_files(directory, size_mb):
    path = Path(directory)
    size_bytes = size_mb * 1024 * 1024
    for file in path.rglob('*'):
        if file.is_file():
            file_size = file.stat().st_size
            if file_size > size_bytes:
                yield (file, file_size / 1024 / 1024)
def backup_files(source, destination, pattern):
    source_path = Path(source)
    dest_path = Path(destination)
    dest_path.mkdir(parents=True, exist_ok=True)
    count = 0
    for file in source_path.glob(pattern):
        if file.is_file():
            shutil.copy2(file, dest_path / file.name)
            count += 1
    print(f"已备份{count} 个文件到{destination}")
# 使用示例
structure = {
'src': ['main.py', 'utils.py'],
'tests': ['test_main.py'],
'docs': []
}
create_project_structure('my_project', structure)
# 查找大文件
# for file, size in find_large_files('.', 1):
# print(f"{file}: {size:.2f} MB")
# 备份文件
# backup_files('src', 'backup', '*.py')
```

解析

60 • 填空 / 简答

实现以下高级函数功能：

1. compose(*functions) ：函数组合装饰器
• 将多个函数组合成一个
• 从右到左执行
2. retry(times, exceptions) ：重试装饰器
• 函数失败时自动重试
• 指定重试次数和捕获的异常
3. curry(func) ：柯里化装饰器
• 将多参数函数转换为单参数函数链

演示这些装饰器的使用。

答案

```
from functools import wraps
import time
def compose(*functions):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            result = func(*args, **kwargs)
            for f in reversed(functions):
                result = f(result)
            return result
        return wrapper
    return decorator
def retry(times=3, exceptions=(Exception,)):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                return func(*args, **kwargs)
                except exceptions as e:
                if attempt == times - 1:
                raise
                print(f"尝试{attempt + 1} 失败: {e}")
                time.sleep(1)
        return wrapper
    return decorator
def curry(func):
    @wraps(func)
    def wrapper(x):
        def inner(y):
            return func(x, y)
        return inner
    return wrapper
# 使用示例
@compose(lambda x: x * 2, lambda x: x + 10)
def base_value():
    return 5
print(f"组合函数结果: {base_value()}")
# (5 + 10) * 2 = 30
@retry(times=3, exceptions=(ValueError,))
def unstable_function():
    import random
    if random.random() < 0.7:
        raise ValueError("随机错误")
    return "成功"
try:
    result = unstable_function()
    print(f"重试函数结果: {result}")
```

```
except ValueError:
    print("所有重试都失败了")
@curry
def add(x, y):
    return x + y
add_5 = add(5)
print(f"柯里化结果: {add_5(3)}")
                # 8
```

程序设计题详细参考答案

题目1：图书管理系统（完整实现）

```
"""
图书管理系统
功能：图书管理、借阅管理、统计查询
使用面向对象设计和装饰器模式
"""
from datetime import datetime, timedelta
from functools import wraps
import json
def log_operation(func):
    """日志装饰器 - 记录所有操作"""
    @wraps(func)
    def wrapper(self, *args, **kwargs):
        operation = func.__name__
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        result = func(self, *args, **kwargs)
        print(f"[{timestamp}] 操作: {operation}, 参数: {args}")
        return result
    return wrapper
class Book:
    """图书类"""
    def __init__(self, book_id, title, author, isbn, total_copies=1):
        self.book_id = book_id
        self.title = title
        self.author = author
        self.isbn = isbn
        self.total_copies = total_copies
        self.available_copies = total_copies
        self.borrowed_by = []
    # 借阅记录
    def is_available(self):
        """检查是否有可借阅的副本"""
        return self.available_copies > 0
    def __str__(self):
        return f"《{self.title}》- {self.author} (可借: {self.available_copies}/{self.total
    def __repr__(self):
        return f"Book({self.book_id}, {self.title}, {self.author})"
class Member:
    """会员类"""
```

```
    def __init__(self, member_id, name, max_books=3):
        self.member_id = member_id
        self.name = name
        self.max_books = max_books
        self.borrowed_books = []
        # 当前借阅的图书
        self.borrow_history = []
    # 借阅历史
    def can_borrow(self):
        """检查是否可以借书"""
        return len(self.borrowed_books) < self.max_books
    def __str__(self):
        return f"会员: {self.name} (已借: {len(self.borrowed_books)}/{self.max_books})"
class BorrowRecord:
    """借阅记录类"""
    def __init__(self, book, member, borrow_date=None):
        self.book = book
        self.member = member
        self.borrow_date = borrow_date or datetime.now()
        self.due_date = self.borrow_date + timedelta(days=30)
        self.return_date = None
    def is_overdue(self):
        """检查是否逾期"""
        if self.return_date:
            return False
        return datetime.now() > self.due_date
    def days_overdue(self):
        """计算逾期天数"""
        if not self.is_overdue():
            return 0
        return (datetime.now() - self.due_date).days
    def __str__(self):
        status = "已归还" if self.return_date else ("逾期" if self.is_overdue() else "借阅中
        return f"{self.member.name} 借阅 《{self.book.title}》 - {status}"
class Library:
    """图书馆管理系统"""
    def __init__(self, name):
        self.name = name
        self.books = {}
        # book_id -> Book
        self.members = {}
        # member_id -> Member
        self.borrow_records = []
    @log_operation
    def add_book(self, book):
        """添加图书"""
        if book.book_id in self.books:
            # 如果图书已存在，增加副本数
```

```
            existing_book = self.books[book.book_id]
            existing_book.total_copies += book.total_copies
            existing_book.available_copies += book.total_copies
            print(f"增加《{book.title}》副本，现有{existing_book.total_copies} 本")
        else:
            self.books[book.book_id] = book
            print(f"成功添加图书: {book}")
    @log_operation
    def add_member(self, member):
        """添加会员"""
        if member.member_id in self.members:
            print(f"会员{member.name} 已存在")
            return False
    self.members[member.member_id] = member
    print(f"成功添加会员: {member}")
    return True
@log_operation
def borrow_book(self, member_id, book_id):
    """借书"""
    # 检查会员和图书是否存在
    if member_id not in self.members:
        print("会员不存在")
        return False
    if book_id not in self.books:
        print("图书不存在")
        return False
    member = self.members[member_id]
    book = self.books[book_id]
    # 检查借阅条件
    if not member.can_borrow():
        print(f"{member.name} 已达到借阅上限")
        return False
    if not book.is_available():
        print(f"《{book.title}》暂无可借副本")
        return False
    # 执行借阅
    record = BorrowRecord(book, member)
    self.borrow_records.append(record)
    book.available_copies -= 1
    book.borrowed_by.append(member_id)
    member.borrowed_books.append(book_id)
    member.borrow_history.append(record)
    print(f"借阅成功: {member.name} 借阅《{book.title}》")
    print(f"应还日期: {record.due_date.strftime('%Y-%m-%d')}")
    return True
@log_operation
def return_book(self, member_id, book_id):
    """还书"""
```

```
    if member_id not in self.members or book_id not in self.books:
        print("会员或图书不存在")
        return False
    member = self.members[member_id]
    book = self.books[book_id]
    if book_id not in member.borrowed_books:
        print(f"{member.name} 未借阅《{book.title}》")
        return False
    # 查找借阅记录
    record = None
    for r in reversed(self.borrow_records):
        if r.member.member_id == member_id and r.book.book_id == book_id and not r.ret
            record = r
            break
    if not record:
        print("未找到借阅记录")
        return False
    # 执行归还
    record.return_date = datetime.now()
    book.available_copies += 1
    book.borrowed_by.remove(member_id)
    member.borrowed_books.remove(book_id)
    # 检查是否逾期
    if record.is_overdue():
        days = record.days_overdue()
        fine = days * 0.5
        # 每天罚款0.5元
        print(f"归还成功（逾期{days} 天，罚款{fine:.2f} 元）")
    else:
        print(f"归还成功: {member.name} 归还《{book.title}》")
    return True
def search_books(self, keyword):
    """搜索图书"""
    results = []
    keyword_lower = keyword.lower()
    for book in self.books.values():
        if (keyword_lower in book.title.lower() or
            keyword_lower in book.author.lower() or
            keyword_lower in book.isbn):
            results.append(book)
    return results
def get_overdue_books(self):
    """获取逾期图书列表"""
    overdue = []
    for record in self.borrow_records:
        if record.is_overdue():
            overdue.append(record)
    return overdue
```

```
def generate_report(self):
    """生成统计报告"""
    print("\n" + "="*60)
    print(f"{self.name} - 统计报告".center(60))
    print("="*60)
    print(f"\n图书总数: {len(self.books)} 种")
    total_copies = sum(book.total_copies for book in self.books.values())
    available_copies = sum(book.available_copies for book in self.books.values())
    print(f"藏书总量: {total_copies} 本 (可借: {available_copies} 本)")
    print(f"\n会员总数: {len(self.members)} 人")
    active_members = sum(1 for m in self.members.values() if m.borrowed_books)
    print(f"活跃会员: {active_members} 人")
    print(f"\n借阅记录: {len(self.borrow_records)} 条")
    active_borrows = sum(1 for r in self.borrow_records if not r.return_date)
    print(f"当前借阅: {active_borrows} 本")
        overdue = self.get_overdue_books()
        if overdue:
            print(f"\n逾期图书: {len(overdue)} 本")
            for record in overdue[:5]:
                # 显示前5条
                print(f" - {record}")
        print("="*60 + "\n")
# 使用示例
if __name__ == "__main__":
    # 创建图书馆
    library = Library("市图书馆")
    # 添加图书
    library.add_book(Book("B001", "Python编程", "张三", "978-1234567890", 3))
    library.add_book(Book("B002", "数据结构", "李四", "978-0987654321", 2))
    library.add_book(Book("B003", "算法导论", "王五", "978-1111111111", 1))
    # 添加会员
    library.add_member(Member("M001", "小明"))
    library.add_member(Member("M002", "小红"))
    library.add_member(Member("M003", "小刚"))
    # 借书
    print("\n" + "="*60)
    print("借阅操作")
    print("="*60)
    library.borrow_book("M001", "B001")
    library.borrow_book("M001", "B002")
    library.borrow_book("M002", "B001")
    # 搜索图书
    print("\n" + "="*60)
    print("搜索图书")
    print("="*60)
    results = library.search_books("Python")
    print(f"搜索 'Python' 的结果:")
    for book in results:
```

```
    print(f"
            {book}")
# 还书
print("\n" + "="*60)
print("归还操作")
print("="*60)
library.return_book("M001", "B001")
# 生成报告
library.generate_report()
设计要点：
1. 多类协作
多类协作：Book、Member、BorrowRecord、Library四个类各司其职
2. 装饰器应用
装饰器应用：使用@log_operation记录所有操作
3. 业务逻辑
业务逻辑：完整的借还书流程，包括逾期检查
4. 数据管理
数据管理：使用字典存储，便于快速查找
5. 扩展性
扩展性：易于添加新功能（如预约、续借等）
题目2：日志分析工具（完整实现）
"""
日志分析工具
功能：解析日志文件、统计分析、生成报告
使用生成器处理大文件，Pandas进行数据分析
"""
import re
from pathlib import Path
from datetime import datetime
from collections import Counter, defaultdict
import pandas as pd
class LogParser:
    """日志解析器"""
    # 常见日志格式的正则表达式
    PATTERNS = {
        'apache': r'(\S+) \S+ \S+ \[([\w:/]+\s[+\-]\d{4})\] "(\S+) (\S+) (\S+)" (\d{3}) (\
        'nginx': r'(\S+) - - \[(.*?)\] "(\w+) (.*?) HTTP/.*?" (\d+) (\d+)',
        'python': r'(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}),\d+ - (\w+) - (.+)',
        'custom': r'\[(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\] (\w+): (.+)'
    }
    def __init__(self, log_format='custom'):
        self.log_format = log_format
        self.pattern = re.compile(self.PATTERNS.get(log_format, self.PATTERNS['custom']))
    def parse_line(self, line):
        """解析单行日志"""
        match = self.pattern.match(line.strip())
        if not match:
            return None
```

```
        if self.log_format == 'apache':
            return {
                'ip': match.group(1),
                'timestamp': match.group(2),
                'method': match.group(3),
                'url': match.group(4),
                'protocol': match.group(5),
                'status': int(match.group(6)),
                'size': int(match.group(7))
            }
        elif self.log_format == 'python':
            return {
                'timestamp': match.group(1),
                'level': match.group(2),
                'message': match.group(3)
            }
        elif self.log_format == 'custom':
            return {
                'timestamp': match.group(1),
                'level': match.group(2),
                'message': match.group(3)
            }
        return None
    def parse_file(self, filepath):
        """
        生成器：逐行解析日志文件
        :param filepath: 日志文件路径
        :yield: 解析后的日志字典
        """
        try:
            with open(filepath, 'r', encoding='utf-8') as f:
                for line_num, line in enumerate(f, 1):
                parsed = self.parse_line(line)
                if parsed:
                parsed['line_num'] = line_num
                yield parsed
        except Exception as e:
            print(f"解析文件出错: {e}")
class LogAnalyzer:
    """日志分析器"""
def __init__(self, parser):
    self.parser = parser
    self.logs = []
def load_logs(self, filepath):
    """加载日志"""
    print(f"正在加载日志: {filepath}")
    self.logs = list(self.parser.parse_file(filepath))
    print(f"成功加载{len(self.logs)} 条日志")
```

```
    return self.logs
def filter_by_level(self, level):
    """按日志级别过滤"""
    return [log for log in self.logs if log.get('level') == level]
def filter_by_time_range(self, start_time, end_time):
    """按时间范围过滤"""
    filtered = []
    for log in self.logs:
        try:
            log_time = datetime.strptime(log['timestamp'], '%Y-%m-%d %H:%M:%S')
            if start_time <= log_time <= end_time:
                filtered.append(log)
        except:
            continue
    return filtered
def count_by_level(self):
    """统计各级别日志数量"""
    levels = [log.get('level', 'UNKNOWN') for log in self.logs]
    return Counter(levels)
def count_by_hour(self):
    """按小时统计日志数量"""
    hours = defaultdict(int)
    for log in self.logs:
        try:
            timestamp = log['timestamp']
            hour = timestamp.split()[1].split(':')[0]
            hours[hour] += 1
        except:
            continue
    return dict(sorted(hours.items()))
def find_errors(self):
    """查找错误日志"""
    errors = []
    for log in self.logs:
        level = log.get('level', '').upper()
        if level in ['ERROR', 'CRITICAL', 'FATAL']:
            errors.append(log)
    return errors
def search_keyword(self, keyword):
    """搜索包含关键词的日志"""
    results = []
    keyword_lower = keyword.lower()
    for log in self.logs:
        message = log.get('message', '').lower()
        if keyword_lower in message:
            results.append(log)
    return results
def to_dataframe(self):
```

```
    """转换为Pandas DataFrame"""
    if not self.logs:
        return pd.DataFrame()
    return pd.DataFrame(self.logs)
def generate_report(self, output_file='log_report.txt'):
    """生成分析报告"""
    report = []
    report.append("="*70)
    report.append("日志分析报告".center(70))
    report.append("="*70)
    report.append(f"\n总日志条数: {len(self.logs)}")
    # 按级别统计
    level_counts = self.count_by_level()
    report.append("\n按级别统计:")
        for level, count in sorted(level_counts.items()):
            percentage = (count / len(self.logs)) * 100
            report.append(f"
                {level}: {count} ({percentage:.2f}%)")
        # 按小时统计
        hour_counts = self.count_by_hour()
        report.append("\n按小时统计:")
        for hour, count in hour_counts.items():
            report.append(f"
                {hour}:00 - {count} 条")
        # 错误日志
        errors = self.find_errors()
        report.append(f"\n错误日志: {len(errors)} 条")
        if errors:
            report.append("最近5条错误:")
            for error in errors[:5]:
                report.append(f" [{error['timestamp']}] {error.get('message', '')[:60]}")
        report.append("="*70)
        # 输出到文件
        report_text = '\n'.join(report)
        Path(output_file).write_text(report_text, encoding='utf-8')
        print(f"\n报告已保存到: {output_file}")
        print(report_text)
        return report_text
# 使用示例
if __name__ == "__main__":
    # 创建测试日志文件
    test_log = Path("test_app.log")
    log_content = """[2024-01-01 10:15:23] INFO: 应用启动成功
[2024-01-01 10:15:24] DEBUG: 加载配置文件
[2024-01-01 10:15:25] INFO: 数据库连接成功
[2024-01-01 10:20:30] WARNING: 内存使用率达到80%
[2024-01-01 10:25:45] ERROR: 数据库查询超时
[2024-01-01 10:30:12] INFO: 处理请求 /api/users
```

```
[2024-01-01 10:35:20] ERROR: 文件未找到: config.json
[2024-01-01 10:40:15] INFO: 用户登录: user123
[2024-01-01 11:10:30] WARNING: API调用频率过高
[2024-01-01 11:15:45] INFO: 定时任务执行完成
[2024-01-01 11:20:50] CRITICAL: 系统内存不足
[2024-01-01 11:25:10] INFO: 应用正常运行
"""
test_log.write_text(log_content, encoding='utf-8')
# 创建解析器和分析器
parser = LogParser(log_format='custom')
analyzer = LogAnalyzer(parser)
# 加载日志
analyzer.load_logs(test_log)
# 统计分析
print("\n" + "="*70)
print("日志级别统计")
print("="*70)
level_counts = analyzer.count_by_level()
for level, count in sorted(level_counts.items()):
    print(f"{level}: {count}")
# 查找错误
print("\n" + "="*70)
print("错误日志")
print("="*70)
errors = analyzer.find_errors()
for error in errors:
    print(f"[{error['timestamp']}] {error['level']}: {error['message']}")
# 搜索关键词
print("\n" + "="*70)
print("搜索关键词: '数据库'")
print("="*70)
results = analyzer.search_keyword('数据库')
for log in results:
    print(f"[{log['timestamp']}] {log['message']}")
# 生成报告
analyzer.generate_report()
# 转换为DataFrame进行高级分析
df = analyzer.to_dataframe()
print("\n" + "="*70)
print("DataFrame分析")
print("="*70)
print(df[['timestamp', 'level', 'message']].head())
# 清理测试文件
test_log.unlink()
```

设计要点：

1. 正则表达式

正则表达式：支持多种日志格式的解析

2. 生成器模式

生成器模式：逐行读取大文件，节省内存
3. 统计分析
统计分析：使用Counter和defaultdict进行高效统计
4. Pandas集成
集成：转换为DataFrame便于高级分析
5. 报告生成
报告生成：自动生成详细的分析报告
由于篇幅限制，其余题目的详细答案请参考第1套试卷的答案模式。核心要点：
• 题目
题目3：使用Pandas进行销售数据分析，包括数据清洗、分组聚合、时间序列分析
• 题目
题目4：实现配置管理系统，使用装饰器和上下文管理器
• 题目
题目5：高级装饰器实现（compose、retry、curry等函数式编程技巧）
评分说明：
• 单选题：每题1分，共30分
• 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
• 填空题：每空1分，共10分
• 代码阅读题：每题3分，共15分
• 程序设计题：5选3，每题5分，共15分
• 总分：
总分：100分

解析

第 3 套

59 题

一、单项选择题

01 • 单项选择

关于Python的私有属性，以下说法正确的是：

A. 使用单下划线开头定义私有属性
B. 使用双下划线开头定义私有属性
C. Python有严格的私有属性限制
D. 私有属性完全无法访问

答案 B. 使用双下划线开头定义私有属性

解析 Python中的名称改编（Name Mangling）机制。
示例：

```
class MyClass:
    def __init__(self):
        self._single = "protected"
        # 约定为保护
        self.__double = "private"
    # 名称改编
    def get_private(self):
        return self.__double
obj = MyClass()
print(obj._single)
# protected
# print(obj.__double) # AttributeError
print(obj._MyClass__double)
# private - 名称改编后的名称
print(obj.get_private())
# private
```

知识拓展：

- 单下划线 _ ：约定为内部使用，但不强制
- 双下划线 __ ：触发名称改编，变为 _ClassName__attribute
- 名称改编主要用于避免子类意外覆盖父类属性

02 • 单项选择

以下代码的输出是什么？

```
class A:
    def __init__(self):
        self.__x = 10
a = A()
print(a._A__x)
```

A. 报错
B. 10
C. None

D. __x

答案 B. 10

解析 __new__() 与 __init__() 的区别：

- __new__() ：真正的构造函数，负责创建并返回实例对象
- __init__() ：初始化函数，对已创建的实例进行初始化
- __new__() 先执行， __init__() 后执行

示例：

```
class MyClass:
    def __new__(cls, *args, **kwargs):
        print("1. __new__被调用，创建实例")
        instance = super().__new__(cls)
        return instance
    def __init__(self, value):
        print("2. __init__被调用，初始化实例")
        self.value = value
obj = MyClass(42)
# 输出：
# 1. __new__被调用，创建实例
# 2. __init__被调用，初始化实例
```

知识拓展： __new__() 的主要应用场景：

1. 实现单例模式
2. 继承不可变类型（如int、str、tuple）
3. 元类编程

```
# 单例模式示例
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
s1 = Singleton()
s2 = Singleton()
print(s1 is s2)
# True - 同一个实例
```

03 • 单项选择

关于类的 __new__() 方法，以下说法正确的是：

A. 在 __init__() 之后调用

B. 在 __init__() 之前调用

C. 用于初始化实例

D. 不能被重写

答案 B. 在 __init__() 之前调用

解析 上下文管理器协议包含两个核心方法：

- __enter__() ：进入with块时调用，返回值赋给as后的变量
- __exit__(exc_type, exc_val, exc_tb) ：退出with块时调用，处理异常

示例：

```
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
```

```
        self.mode = mode
        self.file = None
    def __enter__(self):
        print(f"打开文件: {self.filename}")
        self.file = open(self.filename, self.mode)
        return self.file
    def __exit__(self, exc_type, exc_val, exc_tb):
        print(f"关闭文件: {self.filename}")
        if self.file:
            self.file.close()
        if exc_type:
            print(f"异常类型: {exc_type}")
        return False
# 不抑制异常
with FileManager('test.txt', 'w') as f:
    f.write('Hello, World!')
# 输出:
# 打开文件: test.txt
# 关闭文件: test.txt
```

知识拓展： __exit__() 返回True可以抑制异常，False则继续抛出异常。

04 • 单项选择

以下哪个魔法方法用于实现对象的上下文管理？

A. __enter__() 和 __exit__()

B. __open__() 和 __close__()

C. __start__() 和 __end__()

D. __begin__() 和 __finish__()

答案 A. __enter__() 和 __exit__()

解析 Python使用C3线性化算法（MRO - Method Resolution Order）解决多重继承的方法查找顺序。

示例：

```
class A:
    def method(self):
        print("A")
class B(A):
    def method(self):
        print("B")
class C(A):
    def method(self):
        print("C")
class D(B, C):
    # 多重继承
    pass
d = D()
d.method()
# 输出: B
# 查看MRO
print(D.__mro__)
# (<class '__main__.D'>, <class '__main__.B'>,
# <class '__main__.C'>, <class '__main__.A'>,
```

```
# <class 'object'>)
```

知识拓展：MRO遵循三个原则：

1. 子类优先于父类
2. 多个父类按照定义顺序
3. 保持单调性（如果C在A之前，则所有子类中C都在A之前）

05 • 单项选择

关于Python的MRO（方法解析顺序），以下说法正确的是：

A. 使用深度优先搜索
B. 使用广度优先搜索
C. 使用C3线性化算法
D. 随机选择

答案 C. 使用C3线性化算法

解析 super() 用于调用父类方法，在多重继承中遵循MRO顺序。

示例：

```
class A:
    def method(self):
        print("A.method")
        return "A"
class B(A):
    def method(self):
        print("B.method")
        result = super().method()
        # 调用A.method
        return "B" + result
class C(A):
    def method(self):
        print("C.method")
        result = super().method()
        # 调用A.method
        return "C" + result
class D(B, C):
    def method(self):
        print("D.method")
        result = super().method()
        # 按MRO调用B.method
        return "D" + result
d = D()
result = d.method()
# 输出:
# D.method
# B.method
# C.method
# A.method
print(result)
# DBCA
```

知识拓展： super() 的优点：

1. 避免硬编码父类名
2. 支持协作式多重继承
3. 遵循MRO顺序

06 • 单项选择

以下代码的输出是什么？

```
class A:
    x = 1
class B(A):
    pass
class C(A):
    x = 2
class D(B, C):
    pass
print(D.x)
```

A. 1
B. 2
C. 报错
D. None

答案 B. 2

解析 根据MRO查找顺序，D的方法解析顺序是D->B->C->A。
示例：

```
class A:
    x = 1
class B(A):
    pass
class C(A):
    x = 2
class D(B, C):
    pass
print(D.x)
# 2
print(D.__mro__)
# (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>)
# 按MRO顺序查找x：D没有 -> B没有 -> C有x=2 -> 返回2
```

知识拓展：类属性查找遵循MRO顺序，实例属性查找先查实例字典，再查类属性。

07 • 单项选择

关于生成器的 send() 方法，以下说法正确的是：

A. 只能发送None
B. 可以向生成器发送值
C. 与 next() 完全相同
D. 不能使用

答案 B. 可以向生成器发送值

解析 send() 方法可以向生成器发送值，并恢复生成器执行。
示例：

```
def generator():
    print("生成器启动")
    while True:
        value = yield
```

```
        # 接收send()发送的值
        print(f"收到值: {value}")
gen = generator()
next(gen)
# 启动生成器，输出：生成器启动
gen.send(10)
# 发送10，输出：收到值: 10
gen.send(20)
# 发送20，输出：收到值: 20
```

知识拓展：生成器的三个主要方法：

1. `__next__()` 或 `next()` ：获取下一个值
2. `send(value)` ：发送值到生成器
3. `close()` ：关闭生成器

08 • 单项选择

以下代码的输出是什么？

```
def gen():
    x = yield 1
    yield x
g = gen()
print(next(g))
print(g.send(10))
```

A. 1 10
B. 1 None
C. None 10
D. 报错

答案 A. 1 10

解析 生成器中 yield 表达式的返回值来自 send() 方法。

示例：

```
def gen():
    x = yield 1
    # 第1次yield返回1，等待接收值
    print(f"x = {x}")
    y = yield x + 2
    # 第2次yield返回x+2
    print(f"y = {y}")
g = gen()
print(next(g))
# 1 - 第1次yield返回1
print(g.send(10))
# x = 10，输出12 - send(10)赋值给x，第2次yield返回12
```

知识拓展：协程的工作原理：

- yield 暂停函数执行并返回值
- send() 恢复执行并传入值
- 第一次必须用 next() 或 send(None) 启动

09 • 单项选择

关于迭代器的 `__iter__()` 方法，以下说法正确的是：

A. 必须返回self
B. 必须返回新的迭代器
C. 可以返回任何可迭代对象
D. 不需要返回值

答案 A. 必须返回self

解析 迭代器协议要求 __iter__() 返回迭代器对象本身。
示例：

```
class MyIterator:
    def __init__(self, max_num):
        self.max_num = max_num
        self.current = 0
    def __iter__(self):
        return self
    # 返回自身
    def __next__(self):
        if self.current < self.max_num:
            self.current += 1
            return self.current
        raise StopIteration
my_iter = MyIterator(3)
for num in my_iter:
    print(num)
# 1, 2, 3
```

知识拓展：可迭代对象vs迭代器：
• 可迭代对象：实现 __iter__() 方法
• 迭代器：实现 __iter__() 和 __next__() 方法

10 • 单项选择

以下哪个不是有效的装饰器语法？

A. @decorator
B. @decorator()
C. @decorator(arg)
D. decorator@

答案 D. decorator@

解析 装饰器的正确语法是 @decorator_name ， decorator@ 不是有效语法。
示例：

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("函数执行前")
        result = func(*args, **kwargs)
        print("函数执行后")
        return result
    return wrapper
@my_decorator
# 正确语法
def say_hello():
    print("Hello!")
say_hello()
# 输出：
```

```
# 函数执行前
# Hello!
# 函数执行后
```

知识拓展：装饰器本质是语法糖：

```
@decorator
def func():
    pass
# 等价于
def func():
    pass
func = decorator(func)
```

11 • 单项选择

关于装饰器的执行顺序，以下说法正确的是：

```
@decorator1
@decorator2
def func():
    pass
```

A. 先执行decorator1，再执行decorator2

B. 先执行decorator2，再执行decorator1

C. 同时执行

D. 不确定

答案 B. 先执行decorator2，再执行decorator1

解析 多个装饰器的执行顺序是从下到上（靠近函数定义的先执行），但调用顺序是从上到下。

示例：

```
def decorator1(func):
    print("装饰器1执行")
    def wrapper(*args, **kwargs):
        print("装饰器1调用")
        return func(*args, **kwargs)
    return wrapper
def decorator2(func):
    print("装饰器2执行")
    def wrapper(*args, **kwargs):
        print("装饰器2调用")
        return func(*args, **kwargs)
    return wrapper
@decorator1
@decorator2
def my_func():
    print("原函数")
# 输出：
# 装饰器2执行 <- 从下到上执行
# 装饰器1执行
my_func()
# 输出：
# 装饰器1调用 <- 从上到下调用
# 装饰器2调用
```

```
# 原函数
知识拓展：装饰器等价形式：
@decorator1
@decorator2
def func():
    pass
# 等价于
func = decorator1(decorator2(func))
```

12 • 单项选择

以下代码的输出是什么？

```
def outer():
    x = []
    def inner(y):
        x.append(y)
        return x
    return inner
f = outer()
print(f(1))
print(f(2))
```

A. [1] [2]

B. [1] [1, 2]

C. [1, 2] [1, 2]

D. 报错

答案 B. [1] [1, 2]

解析 闭包中的自由变量（外层函数的变量）会被保存，所有闭包共享同一个变量对象。

```
示例：
def create_multipliers():
    multipliers = []
    for i in range(3):
        def multiplier(x):
            return x * i
        # i是自由变量
        multipliers.append(multiplier)
    return multipliers
funcs = create_multipliers()
print(funcs[0](2))
# 4 - 不是0，因为i=2
print(funcs[1](2))
# 4 - 所有闭包共享同一个i
print(funcs[2](2))
# 4
# 正确写法：使用默认参数捕获当前值
def create_multipliers_correct():
    multipliers = []
    for i in range(3):
        def multiplier(x, i=i):
            # i=i捕获当前值
            return x * i
```

```
            multipliers.append(multiplier)
        return multipliers
funcs = create_multipliers_correct()
print(funcs[0](2))
# 0
print(funcs[1](2))
# 2
print(funcs[2](2))
# 4
```

知识拓展：闭包的三个特性：

1. 内部函数引用外部函数的变量
2. 外部函数返回内部函数
3. 变量在外部函数结束后仍然存在

13 • 单项选择

关于 nonlocal 关键字，以下说法正确的是：

A. 用于声明全局变量

B. 用于声明外层函数的变量

C. 用于声明局部变量

D. 与 global 完全相同

答案 B. 用于声明外层函数的变量

解析 nonlocal 用于在内层函数中修改外层函数的变量。

示例：

```
def outer():
    count = 0
    def increment_wrong():
        count = count + 1
    # UnboundLocalError
    def increment_correct():
        nonlocal count
        # 声明使用外层变量
        count = count + 1
    increment_correct()
    print(count)
    # 1
    increment_correct()
    print(count)
# 2
outer()
```

对比global和nonlocal：

```
x = 0
# 全局变量
def outer():
    x = 10
    # 外层函数变量
    def use_global():
        global x
        x = 100
    # 修改全局变量
    def use_nonlocal():
```

```
            nonlocal x
            x = 20
        # 修改外层函数变量
        use_nonlocal()
        print(f"外层x: {x}")
    # 20
    outer()
    print(f"全局x: {x}")
    # 0
```

知识拓展： nonlocal 只能用于嵌套函数， global 用于全局变量。

14 • 单项选择

以下代码的输出是什么？

```
def func(**kwargs):
    return len(kwargs)
print(func(a=1, b=2, c=3))
```

A. 1

B. 2

C. 3

D. 6

答案 C. 3

解析 **kwargs 接收关键字参数，返回字典， len(kwargs) 是键值对的数量。

示例：

```
def func(**kwargs):
    print(f"kwargs类型: {type(kwargs)}")
    print(f"kwargs内容: {kwargs}")
    return len(kwargs)
result = func(a=1, b=2, c=3)
print(f"长度: {result}")
# 输出：
# kwargs类型: <class 'dict'>
# kwargs内容: {'a': 1, 'b': 2, 'c': 3}
# 长度: 3
```

完整的参数类型示例：

```
def full_params(pos1, pos2, *args, kw1, kw2=None, **kwargs):
    print(f"位置参数: {pos1}, {pos2}")
    print(f"可变位置参数: {args}")
    print(f"关键字参数: {kw1}, {kw2}")
    print(f"可变关键字参数: {kwargs}")
full_params(1, 2, 3, 4, kw1='a', kw2='b', extra1='c', extra2='d')
# 输出：
# 位置参数: 1, 2
# 可变位置参数: (3, 4)
# 关键字参数: a, b
# 可变关键字参数: {'extra1': 'c', 'extra2': 'd'}
```

知识拓展：参数顺序：位置参数 > *args > 关键字参数 > **kwargs

15 • 单项选择

关于参数解包，以下代码的输出是什么？

```
def func(a, b, c):
    return a + b + c
args = [1, 2, 3]
print(func(*args))
```

A. [1, 2, 3]

B. 6

C. 报错

D. 123

答案 B. 6

解析 *args 可以解包列表，将列表元素作为独立参数传递。

示例：

```
def sum_numbers(*args):
    print(f"args: {args}")
    return sum(args)
# 直接传递多个参数
result1 = sum_numbers(1, 2, 3)
print(result1)
# 6
# 使用*解包列表
numbers = [1, 2, 3]
result2 = sum_numbers(*numbers)
# 等价于sum_numbers(1, 2, 3)
print(result2)
# 6
# 对比：不解包
result3 = sum_numbers(numbers)
# 传递一个列表对象
print(result3)
# TypeError: unsupported operand type(s)
```

解包的各种用法：

```
# 函数调用时解包
def func(a, b, c):
    return a + b + c
values = [1, 2, 3]
print(func(*values))
# 6
# 列表解包
a, *b, c = [1, 2, 3, 4, 5]
print(a, b, c)
# 1 [2, 3, 4] 5
# 字典解包
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, **dict1}
print(dict2)
# {'c': 3, 'a': 1, 'b': 2}
```

知识拓展： * 用于解包序列， ** 用于解包字典。

16 • 单项选择

Pandas中，以下哪个方法用于数据透视？

A. pivot()
B. rotate()
C. transform()
D. reshape()

答案 A. pivot()

解析 pivot() 方法用于数据透视，将长格式数据转换为宽格式。

示例：

```
import pandas as pd
# 原始数据（长格式）
df = pd.DataFrame({
    'date': ['2023-01', '2023-01', '2023-02', '2023-02'],
    'product': ['A', 'B', 'A', 'B'],
    'sales': [100, 150, 120, 180]
})
# 数据透视
pivot_df = df.pivot(index='date', columns='product', values='sales')
print(pivot_df)
# product A B
# date
# 2023-01 100 150
# 2023-02 120 180
pivot vs pivot_table:
# pivot - 不支持聚合，要求索引唯一
pivot_df = df.pivot(index='date', columns='product', values='sales')
# pivot_table - 支持聚合函数
pivot_table_df = df.pivot_table(
    index='date',
    columns='product',
    values='sales',
    aggfunc='mean'
# 可以使用聚合函数
)
```

知识拓展：常用的数据重塑方法：

- pivot() ：简单透视
- pivot_table() ：带聚合的透视
- melt() ：宽格式转长格式
- stack()/unstack() ：索引级别转换

17 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
s = pd.Series([1, 2, 3, 4, 5])
print(s[s > 2].index.tolist())
```

A. [3, 4, 5]
B. [2, 3, 4]
C. [0, 1, 2]
D. [1, 2]

答案 B. [2, 3, 4]

解析 loc[2:4] 在Pandas中是闭区间，包含起始和结束标签。

示例：

```
import pandas as pd
df = pd.DataFrame(
    {'A': [10, 20, 30, 40, 50, 60]},
    index=[0, 1, 2, 3, 4, 5]
)
# loc使用标签，闭区间
print(df.loc[2:4])
# A
# 2 30
# 3 40
# 4 50 <- 包含4
# iloc使用位置，左闭右开
print(df.iloc[2:4])
# A
# 2 30
# 3 40 <- 不包含4
loc vs iloc对比：
        对比：
df = pd.DataFrame(
    {'A': [10, 20, 30]},
    index=['a', 'b', 'c']
)
# loc - 基于标签
print(df.loc['a':'c'])
# 包含'c'
# iloc - 基于位置
print(df.iloc[0:2])
# 不包含位置2
```

知识拓展：Pandas的索引规则：

- loc ：基于标签，闭区间
- iloc ：基于位置，左闭右开
- at ：快速标签访问单个值
- iat ：快速位置访问单个值

18 • 单项选择

Pandas中， inplace=True 参数的作用是：

A. 创建新的DataFrame

B. 在原DataFrame上修改

C. 不做任何修改

D. 删除DataFrame

答案 B. 在原DataFrame上修改

解析 inplace=True 直接在原DataFrame上修改，不返回新对象。

示例：

```
import pandas as pd
df = pd.DataFrame({'A': [3, 1, 2]})
# inplace=False（默认）：返回新对象
```

```
df_new = df.sort_values('A')
print("原df:")
print(df)
# A列仍是[3, 1, 2]
print("新df:")
print(df_new)
# A列是[1, 2, 3]
# inplace=True：修改原对象
df.sort_values('A', inplace=True)
print("修改后的df:")
print(df)
# A列变为[1, 2, 3]
支持inplace的常用方法：
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
# 删除列
df.drop('B', axis=1, inplace=True)
# 填充缺失值
df.fillna(0, inplace=True)
# 排序
df.sort_values('A', inplace=True)
# 重置索引
df.reset_index(drop=True, inplace=True)
知识拓展：使用 inplace=True 的注意事项：
1. 节省内存（不创建副本）
2. 不能链式调用（返回None）
3. Pandas 2.0+推荐避免使用inplace
```

19 • 单项选择

关于Pandas的 merge() 方法，以下说法正确的是：

A. 只能按索引合并

B. 只能按列名合并

C. 可以指定合并方式（inner、outer等）

D. 不能合并多个DataFrame

答案 C. 可以指定合并方式（inner、outer等）

解析 merge() 方法可以通过 how 参数指定不同的合并方式。

```
示例：
import pandas as pd
df1 = pd.DataFrame({'key': ['A', 'B', 'C'], 'val1': [1, 2, 3]})
df2 = pd.DataFrame({'key': ['B', 'C', 'D'], 'val2': [4, 5, 6]})
# inner join（默认）：保留两个表都有的键
inner = pd.merge(df1, df2, on='key', how='inner')
print("Inner:")
print(inner)
# 只有B和C
# outer join：保留所有键
outer = pd.merge(df1, df2, on='key', how='outer')
print("Outer:")
print(outer)
# A, B, C, D，缺失值为NaN
```

```
# left join：保留左表所有键
left = pd.merge(df1, df2, on='key', how='left')
print("Left:")
print(left)
# A, B, C
# right join：保留右表所有键
right = pd.merge(df1, df2, on='key', how='right')
print("Right:")
print(right)
# B, C, D
merge的高级用法：
# 多列合并
df1 = pd.DataFrame({'key1': ['A', 'B'], 'key2': [1, 2], 'val1': [10, 20]})
df2 = pd.DataFrame({'key1': ['A', 'B'], 'key2': [1, 2], 'val2': [30, 40]})
result = pd.merge(df1, df2, on=['key1', 'key2'])
# 不同列名
df1 = pd.DataFrame({'left_key': ['A', 'B'], 'val1': [1, 2]})
df2 = pd.DataFrame({'right_key': ['A', 'B'], 'val2': [3, 4]})
result = pd.merge(df1, df2, left_on='left_key', right_on='right_key')
知识拓展：合并方法对比：
• merge() ：SQL风格的连接
• join() ：基于索引的连接
• concat() ：简单的拼接
```

20 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
print(df.shape)
```

A. (2, 2)

B. [2, 2]

C. 4

D. (4,)

答案 A. (2, 2)

解析 shape 属性返回(行数, 列数)的元组。

```
示例：
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
})
print(df.shape)
# (3, 3) - 3行3列
print(type(df.shape))
# <class 'tuple'>
# 获取行数和列数
rows, cols = df.shape
print(f"行数: {rows}, 列数: {cols}")
```

```
# 也可以单独获取
print(f"行数: {len(df)}")
print(f"列数: {len(df.columns)}")
DataFrame的常用属性:
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4.0, 5.0, 6.0],
    'C': ['x', 'y', 'z']
})
print(df.shape)
# (3, 3) - 形状
print(df.size)
# 9 - 总元素数
print(df.ndim)
# 2 - 维度
print(df.columns)
# Index(['A', 'B', 'C']) - 列名
print(df.index)
# RangeIndex(start=0, stop=3) - 索引
print(df.dtypes)
# 各列数据类型
print(df.values)
# numpy数组形式
知识拓展：DataFrame的维度信息：
• shape ：返回(行, 列)元组
• size ：返回总元素数（行×列）
• ndim ：返回维度数（DataFrame总是2）
• len(df) ：返回行数
```

21 • 单项选择

pathlib.Path 对象的 with_suffix() 方法的作用是：

A. 删除文件扩展名
B. 替换文件扩展名
C. 添加文件扩展名
D. 获取文件扩展名

答案 B. 替换文件扩展名

```
解析 with_suffix() 用于更改文件扩展名，返回新的Path对象。
示例:
from pathlib import Path
p = Path('/home/user/docs/file.txt')
# 更改扩展名
new_p = p.with_suffix('.md')
print(new_p)
# /home/user/docs/file.md
# 删除扩展名
no_suffix = p.with_suffix('')
print(no_suffix)
# /home/user/docs/file
# 更改多重扩展名
```

```
p2 = Path('archive.tar.gz')
new_p2 = p2.with_suffix('.zip')
# 只更改最后一个
print(new_p2)
# archive.tar.zip
```

知识拓展：Path对象的修改方法：

- with_suffix() ：更改扩展名
- with_name() ：更改文件名
- with_stem() ：Python 3.9+，更改主文件名

22 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('file.txt')
print(p.with_suffix('.md'))
```

A. file.txt.md

B. file.md

C. .md

D. file

答案 B. file.md

解析 with_suffix() 可以将.txt扩展名替换为.md。

示例：

```
from pathlib import Path
# 单个文件
original = Path('README.txt')
markdown = original.with_suffix('.md')
print(markdown)
# README.md
# 批量转换
import os
for file in Path('.').glob('*.txt'):
    new_file = file.with_suffix('.md')
    # os.rename(file, new_file) # 实际重命名
    print(f"{file} -> {new_file}")
```

知识拓展：文件名组件：

```
from pathlib import Path
p = Path('/home/user/docs/file.txt')
print(p.name)
# file.txt - 完整文件名
print(p.stem)
# file - 主文件名
print(p.suffix)
# .txt - 扩展名
print(p.suffixes) # ['.txt'] - 所有扩展名
```

23 • 单项选择

Path.iterdir() 方法返回的是：

A. 列表

B. 元组

C. 生成器

D. 字典

答案 C. 生成器

解析 iterdir() 返回生成器，需要迭代或转换为列表。

示例：

```
from pathlib import Path
project_dir = Path('/home/user/project')
# iterdir()返回生成器
entries = project_dir.iterdir()
print(type(entries))
# <class 'generator'>
# 迭代使用
for entry in project_dir.iterdir():
    if entry.is_file():
        print(f"文件: {entry.name}")
    elif entry.is_dir():
        print(f"目录: {entry.name}")
# 转换为列表
all_entries = list(project_dir.iterdir())
print(f"总共{len(all_entries)} 个条目")
# 筛选文件
files = [f for f in project_dir.iterdir() if f.is_file()]
dirs = [d for d in project_dir.iterdir() if d.is_dir()]
```

知识拓展：目录遍历方法对比：

- iterdir() ：返回当前目录的直接子项
- glob(pattern) ：模式匹配，非递归
- rglob(pattern) ：模式匹配，递归
- walk() ：Python 3.12+，类似os.walk()

24 • 单项选择

关于 Path.resolve() 方法，以下说法正确的是：

A. 删除路径

B. 解析为绝对路径

C. 创建路径

D. 检查路径

答案 B. 解析为绝对路径

解析 resolve() 解析符号链接并返回绝对路径。

示例：

```
from pathlib import Path
import os
# 相对路径
relative = Path('docs/readme.md')
print(relative)
# docs/readme.md
# 转为绝对路径
absolute = relative.resolve()
print(absolute)
# /home/user/project/docs/readme.md
# 解析符号链接
```

```
os.symlink('/original/path', 'symlink')
symlink_path = Path('symlink')
resolved = symlink_path.resolve()
print(resolved)
# /original/path - 真实路径
# 简化路径
complex_path = Path('/home/user/../user/./docs')
simplified = complex_path.resolve()
print(simplified)
# /home/user/docs
```

知识拓展：路径解析方法：

- resolve() ：完全解析，返回绝对路径
- absolute() ：转为绝对路径，不解析符号链接
- expanduser() ：展开~为用户主目录

25 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('a/b/../c')
print(p)
```

A. a/c

B. a/b/../c

C. c

D. 报错

答案 B. a/b/../c

解析 Path对象不会自动简化路径，需要调用 resolve() 。

示例：

```
from pathlib import Path
# 不自动简化
p1 = Path('/home/user/../user/docs')
print(p1)
# /home/user/../user/docs - 保持原样
# 使用resolve()简化
p2 = p1.resolve()
print(p2)
# /home/user/docs - 简化后
# ./也不会自动移除
p3 = Path('./docs/./file.txt')
print(p3)
# docs/./file.txt
print(p3.resolve())
# /home/user/project/docs/file.txt
# 多个..
p4 = Path('/a/b/c/../../d')
print(p4)
# /a/b/c/../../d
print(p4.resolve())
# /a/d
```

知识拓展：路径规范化的重要性：

1. 避免路径比较错误
2. 解析符号链接
3. 保证路径唯一性

26 • 单项选择

关于 map() 函数，以下说法正确的是：

A. 返回列表
B. 返回map对象（迭代器）
C. 只能接受一个可迭代对象
D. 不能使用lambda函数

答案 B. 返回map对象（迭代器）

解析 map() 返回map对象（迭代器），需要转换为列表或迭代。

示例：

```
numbers = [1, 2, 3, 4, 5]
# map()返回迭代器
result = map(lambda x: x * 2, numbers)
print(type(result))
# <class 'map'>
print(result)
# <map object at 0x...>
# 转换为列表
result_list = list(map(lambda x: x * 2, numbers))
print(result_list)
# [2, 4, 6, 8, 10]
# 直接迭代
for item in map(lambda x: x * 2, numbers):
    print(item)
# 2, 4, 6, 8, 10
# 使用函数名
def square(x):
    return x ** 2
squares = list(map(square, numbers))
print(squares)
# [1, 4, 9, 16, 25]
map vs 列表推导式：
numbers = [1, 2, 3, 4, 5]
# 使用map
result1 = list(map(lambda x: x * 2, numbers))
# 使用列表推导式
result2 = [x * 2 for x in numbers]
print(result1 == result2)
# True
# 列表推导式通常更Pythonic
知识拓展：函数式编程工具：
• map(func, iterable) ：映射
• filter(func, iterable) ：过滤
• reduce(func, iterable) ：聚合（functools模块）
```

27 • 单项选择

以下代码的输出是什么？

```
result = list(map(str, [1, 2, 3]))
print(result)
```

A. [1, 2, 3]

B. ['1', '2', '3']

C. '123'

D. 报错

答案 B. ['1', '2', '3']

解析 map(str, iterable) 将迭代器中的整数转换为字符串。

示例：

```
numbers = [1, 2, 3, 4, 5]
# 转换为字符串
str_numbers = list(map(str, numbers))
print(str_numbers)
# ['1', '2', '3', '4', '5']
# 常用场景：拼接字符串
result = ', '.join(map(str, numbers))
print(result)
# '1, 2, 3, 4, 5'
# 其他类型转换
str_list = ['1', '2', '3']
int_list = list(map(int, str_list))
print(int_list)
# [1, 2, 3]
float_list = list(map(float, ['1.5', '2.7', '3.9']))
print(float_list)
# [1.5, 2.7, 3.9]
常用的类型转换：
# str - 转字符串
list(map(str, [1, 2, 3]))
# ['1', '2', '3']
# int - 转整数
list(map(int, ['1', '2', '3']))
# [1, 2, 3]
# float - 转浮点数
list(map(float, [1, 2, 3]))
# [1.0, 2.0, 3.0]
# bool - 转布尔值
list(map(bool, [0, 1, '', 'a']))
# [False, True, False, True]
知识拓展： map() 可以接受多个可迭代对象：
list1 = [1, 2, 3]
list2 = [4, 5, 6]
# 多个参数
result = list(map(lambda x, y: x + y, list1, list2))
print(result)
# [5, 7, 9]
```

28 • 单项选择

关于 reduce() 函数，以下说法正确的是：

A. 在内置函数中

B. 在 functools 模块中

C. 在 itertools 模块中

D. 已被废弃

答案 B. 在 functools 模块中

解析 reduce() 函数在 functools 模块中，不是内置函数。

示例：

```
from functools import reduce
numbers = [1, 2, 3, 4, 5]
# 计算乘积
product = reduce(lambda x, y: x * y, numbers)
print(product)
# 120 - 1*2*3*4*5
# 计算和
sum_result = reduce(lambda x, y: x + y, numbers)
print(sum_result)
# 15
# 指定初始值
result = reduce(lambda x, y: x + y, numbers, 100)
print(result)
# 115 - 100+1+2+3+4+5
# 找最大值
max_value = reduce(lambda x, y: x if x > y else y, numbers)
print(max_value)
# 5
```

reduce的工作原理：

```
from functools import reduce
# reduce的执行过程
numbers = [1, 2, 3, 4]
result = reduce(lambda x, y: x + y, numbers)
# 等价于：
# step1: 1 + 2 = 3
# step2: 3 + 3 = 6
# step3: 6 + 4 = 10
print(result)
# 10
```

常用场景：

```
from functools import reduce
# 字符串拼接
words = ['Hello', ' ', 'World']
sentence = reduce(lambda x, y: x + y, words)
print(sentence)
# 'Hello World'
# 嵌套列表展开
lists = [[1, 2], [3, 4], [5, 6]]
flat = reduce(lambda x, y: x + y, lists)
print(flat)
# [1, 2, 3, 4, 5, 6]
# 阶乘
```

```
import operator
factorial_5 = reduce(operator.mul, range(1, 6))
print(factorial_5)
# 120
```

知识拓展：Python 2中 reduce() 是内置函数，Python 3中移到了 functools 模块。

29 • 单项选择

以下代码的输出是什么？

```
from functools import reduce
result = reduce(lambda x, y: x * y, [1, 2, 3, 4])
print(result)
```

A. 10
B. 24
C. [1, 2, 3, 4]
D. 报错

答案 B. 24

解析 reduce() 计算乘积：123*4=24。

示例：

```
from functools import reduce
import operator
# 使用lambda
result1 = reduce(lambda x, y: x * y, [1, 2, 3, 4])
print(result1)
# 24
# 使用operator.mul
result2 = reduce(operator.mul, [1, 2, 3, 4])
print(result2)
# 24
# 阶乘函数
def factorial(n):
    return reduce(operator.mul, range(1, n + 1))
print(factorial(4))
# 24
print(factorial(5))
# 120
print(factorial(10)) # 3628800
reduce vs 循环：
        循环：
from functools import reduce
numbers = [1, 2, 3, 4]
# 使用reduce
result1 = reduce(lambda x, y: x * y, numbers)
# 使用循环
result2 = 1
for num in numbers:
    result2 *= num
print(result1 == result2)
# True
# reduce更简洁，但可读性略差
```

知识拓展：Python内置了一些常用聚合函数：

```
numbers = [1, 2, 3, 4, 5]
print(sum(numbers))
# 15 - 求和
print(max(numbers))
# 5 - 最大值
print(min(numbers))
# 1 - 最小值
print(all([True, True, False]))
# False - 全为True
print(any([False, False, True])) # True - 任一True
```

30 • 单项选择

关于 all() 和 any() 函数，以下说法正确的是：

A. all() 要求所有元素为True

B. any() 要求至少一个元素为True

C. 两者都接受可迭代对象

D. 以上都正确

答案 D. 以上都正确

解析 所有选项都是Python函数式编程的特性。

示例：

```
# A. 高阶函数：函数作为参数或返回值
def apply_twice(func, value):
    return func(func(value))
result = apply_twice(lambda x: x * 2, 3)
print(result)
# 12 - (3*2)*2
# B. Lambda表达式：匿名函数
square = lambda x: x ** 2
print(square(5))
# 25
# C. 闭包：内部函数引用外部变量
def multiplier(n):
    def multiply(x):
        return x * n
    return multiply
times_3 = multiplier(3)
print(times_3(4))
# 12
# D. 不可变性：避免副作用
original = [1, 2, 3]
# 不修改原列表，返回新列表
new_list = [x * 2 for x in original]
print(original)
# [1, 2, 3] - 未变
print(new_list)
# [2, 4, 6]
```

函数式编程的核心原则：

```
# 1. 纯函数：相同输入总是相同输出，无副作用
def pure_function(x, y):
```

```
    return x + y
# 不修改外部状态
# 2. 不可变性：不修改原数据
def add_element(lst, element):
    return lst + [element]
# 返回新列表
# 3. 高阶函数：函数作为一等公民
def compose(f, g):
    return lambda x: f(g(x))
# 4. 递归：函数调用自身
def factorial(n):
    return 1 if n <= 1 else n * factorial(n - 1)
```

知识拓展：Python支持函数式编程风格，但不是纯函数式语言：

- 支持：高阶函数、lambda、闭包、map/filter/reduce
- 不强制：不可变性、纯函数
- 混合范式：同时支持面向对象、过程式编程

二、不定项选择题

31 • 不定项选择

关于Python的元类（metaclass），以下说法正确的有：

A. 元类是创建类的类
B. 所有类的默认元类是 type
C. 可以自定义元类
D. 元类用于控制类的创建过程

答案 A. 元类是创建类的类；B. 所有类的默认元类是 type；C. 可以自定义元类；D. 元类用于控制类的创建过程

解析 所有选项都是封装的正确目的。

示例：

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance
    # A. 隐藏内部实现
    def deposit(self, amount):
        if amount > 0:
            # B. 提供访问控制
            self.__balance += amount
        else:
            raise ValueError("金额必须为正")
    def get_balance(self):
        # C. 提供简洁接口
        return self.__balance
    def transfer(self, other, amount):
        # D. 便于维护和修改
        if amount <= self.__balance:
            self.__balance -= amount
            other.deposit(amount)
account = BankAccount(1000)
account.deposit(500)
```

```
print(account.get_balance())
# 1500
```

知识拓展：封装的三要素：

1. 数据隐藏：使用私有属性（ __ 前缀）
2. 接口提供：公共方法访问数据
3. 访问控制：验证和保护数据完整性

32 • 不定项选择

以下哪些是Python的描述符协议方法：

A. __get__()

B. __set__()

C. __delete__()

D. __desc__()

答案 A. __get__()；B. __set__()；C. __delete__()

解析 描述符协议包含 __get__ 、 __set__ 、 __delete__ 三个方法。

示例：

```
class Descriptor:
    def __init__(self, name):
        self.name = name
    def __get__(self, instance, owner):
        if instance is None:
            return self
        print(f"Getting {self.name}")
        return instance.__dict__.get(self.name, None)
    def __set__(self, instance, value):
        print(f"Setting {self.name} = {value}")
        if value < 0:
            raise ValueError("值不能为负")
        instance.__dict__[self.name] = value
    def __delete__(self, instance):
        print(f"Deleting {self.name}")
        del instance.__dict__[self.name]
class MyClass:
    value = Descriptor('value')
    def __init__(self, value):
        self.value = value
obj = MyClass(10)
print(obj.value)
# Getting value -> 10
obj.value = 20
# Setting value = 20
del obj.value
# Deleting value
```

知识拓展：描述符的应用场景：

- 属性验证和类型检查
- 延迟计算（lazy evaluation）
- 实现property、classmethod、staticmethod

33 • 不定项选择

关于Python的单例模式实现，以下方法正确的有：

A. 使用 __new__() 方法
B. 使用装饰器
C. 使用元类
D. 使用模块

答案 A. 使用 __new__() 方法；B. 使用装饰器；C. 使用元类；D. 使用模块

解析 所有方法都可以实现单例模式。

示例：

```
# A. 使用__new__方法
class Singleton1:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
# B. 使用装饰器
def singleton(cls):
    instances = {}
    def get_instance(*args, **kwargs):
        if cls not in instances:
            instances[cls] = cls(*args, **kwargs)
        return instances[cls]
    return get_instance
@singleton
class Singleton2:
    pass
# C. 使用元类
class SingletonMeta(type):
    _instances = {}
    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]
class Singleton3(metaclass=SingletonMeta):
    pass
# D. 使用模块
# singleton_module.py
class Singleton4:
    pass
instance = Singleton4()
                # 模块只加载一次
# 验证
s1 = Singleton1()
s2 = Singleton1()
print(s1 is s2)
# True
```

知识拓展：单例模式的使用场景：

- 配置管理器
- 日志记录器
- 数据库连接池

- 线程池

34 • 不定项选择

关于生成器表达式和列表推导式，以下说法正确的有：

A. 生成器表达式使用 ()

B. 列表推导式使用 []

C. 生成器表达式更节省内存

D. 两者语法完全相同

答案 A. 生成器表达式使用 ()；B. 列表推导式使用 []；C. 生成器表达式更节省内存；D. 两者语法完全相同

解析 所有选项都是异常处理的正确做法。

示例：

```
# A. 捕获特定异常
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"除零错误: {e}")
# B. 使用finally清理资源
try:
    f = open('file.txt', 'r')
    data = f.read()
except FileNotFoundError:
    print("文件不存在")
finally:
    if 'f' in locals():
        f.close()
# 确保文件关闭
# C. 自定义异常
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        self.balance = balance
        self.amount = amount
        super().__init__(f"余额{balance}不足，需要{amount}")
def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(balance, amount)
    return balance - amount
try:
    withdraw(100, 150)
except InsufficientFundsError as e:
    print(e)
# D. 避免捕获所有异常（除非必要）
try:
    risky_operation()
except Exception as e:
    # 不推荐，太宽泛
    print(f"错误: {e}")
# 更好的做法：捕获具体异常
try:
    risky_operation()
except (ValueError, TypeError, KeyError) as e:
```

```
    print(f"预期的错误: {e}")
```

知识拓展：异常处理的最佳实践：

1. 捕获特定异常，避免裸except
2. 使用finally清理资源
3. 自定义异常提供更多上下文
4. 不要吞没异常（silent failure）

35 • 不定项选择

以下哪些操作会消耗迭代器：

A. list(iterator)

B. for item in iterator

C. next(iterator)

D. iter(iterator)

答案 A. list(iterator)；B. for item in iterator；C. next(iterator)

解析 iter() 返回迭代器本身，不消耗元素。

示例：

```
my_list = [1, 2, 3]
# A. iter()返回迭代器
iterator = iter(my_list)
print(type(iterator))
# <class 'list_iterator'>
# B. next()消耗元素
print(next(iterator))
# 1
print(next(iterator))
# 2
# C. 可以多次调用iter()，每次返回新迭代器
iter1 = iter(my_list)
iter2 = iter(my_list)
print(next(iter1))
# 1
print(next(iter2))
# 1 - 独立的迭代器
# D错误：iter()不会消耗元素
original = [1, 2, 3]
iterator = iter(original)
print(original)
# [1, 2, 3] - 原列表未变
```

迭代器vs可迭代对象：

```
# 可迭代对象：有__iter__方法
class Iterable:
    def __iter__(self):
        return iter([1, 2, 3])
# 迭代器：有__iter__和__next__方法
class Iterator:
    def __init__(self):
        self.data = [1, 2, 3]
        self.index = 0
    def __iter__(self):
        return self
```

```
    def __next__(self):
        if self.index < len(self.data):
            result = self.data[self.index]
            self.index += 1
            return result
        raise StopIteration
it = Iterator()
for item in it:
    print(item)
# 1, 2, 3
```

知识拓展：迭代器的优势：

- 节省内存（惰性求值）
- 支持无限序列
- 流式处理大数据

36 · 不定项选择

关于装饰器的应用，以下说法正确的有：

A. 可以用于函数
B. 可以用于类
C. 可以用于方法
D. 可以嵌套使用

答案 A. 可以用于函数；B. 可以用于类；C. 可以用于方法；D. 可以嵌套使用

解析 装饰器可以应用于函数、方法、类和生成器。

示例：

```
import time
# 通用装饰器
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__}耗时: {end-start:.4f}秒")
        return result
    return wrapper
# A. 装饰普通函数
@timer
def slow_function():
    time.sleep(0.1)
    return "完成"
slow_function()
# B. 装饰类方法
class MyClass:
    @timer
    def method(self):
        time.sleep(0.1)
        return "方法完成"
obj = MyClass()
obj.method()
# C. 装饰类
def add_str_method(cls):
```

```
    cls.__str__ = lambda self: f"{cls.__name__}实例"
    return cls
@add_str_method
class Person:
    pass
print(Person())
# Person实例
# D. 装饰生成器
@timer
def slow_generator():
    for i in range(3):
        time.sleep(0.01)
        yield i
for val in slow_generator():
    print(val)
```

知识拓展：装饰器的高级应用：

- 带参数的装饰器
- 类装饰器（使用 __call__ ）
- 多个装饰器叠加
- functools.wraps保留元数据

37 · 不定项选择

关于闭包的特性，以下说法正确的有：

A. 内部函数引用外部函数的变量

B. 外部函数返回内部函数

C. 可以保存状态

D. 不能修改外部变量

答案 A. 内部函数引用外部函数的变量；B. 外部函数返回内部函数；C. 可以保存状态

解析 使用 nonlocal 可以在闭包中修改外层变量。

示例：

```
# A. 闭包可以读取外层变量
def outer():
    count = 0
    def inner():
        print(count)
    # 可以读取
    inner()
outer()
# 0
# B. 使用nonlocal修改外层变量
def counter():
    count = 0
    def increment():
        nonlocal count
        # 声明修改外层变量
        count += 1
        return count
    return increment
count_func = counter()
print(count_func())
```

```
# 1
print(count_func())
# 2
# C. 闭包可以访问多层外部作用域
def level1():
    x = 1
    def level2():
        y = 2
        def level3():
            nonlocal x, y
            x += 1
            y += 1
            return x, y
        return level3
    return level2
func = level1()()
print(func())
                # (2, 3)
# D错误：没有nonlocal会创建局部变量
def wrong_example():
    count = 0
    def increment():
        count = count + 1
    # UnboundLocalError
    increment()
知识拓展：闭包的实际应用：
# 装饰器中的闭包
def repeat(times):
    def decorator(func):
        def wrapper(*args, **kwargs):
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator
@repeat(3)
def say_hello():
    print("Hello!")
say_hello()
# 打印3次Hello!
```

38 • 不定项选择

以下哪些是Pandas的索引类型：

A. RangeIndex

B. Int64Index

C. DatetimeIndex

D. MultiIndex

答案 A. RangeIndex；B. Int64Index；C. DatetimeIndex；D. MultiIndex

解析 所有选项都是Pandas的索引类型。

示例：

```
import pandas as pd
# A. RangeIndex - 默认整数索引
df1 = pd.DataFrame({'A': [1, 2, 3]})
print(type(df1.index))
# RangeIndex
print(df1.index)
# RangeIndex(start=0, stop=3, step=1)
# B. Index - 通用索引
df2 = pd.DataFrame({'A': [1, 2, 3]}, index=['a', 'b', 'c'])
print(type(df2.index))
# Index
print(df2.index)
# Index(['a', 'b', 'c'], dtype='object')
# C. DatetimeIndex - 时间索引
dates = pd.date_range('2023-01-01', periods=3)
df3 = pd.DataFrame({'A': [1, 2, 3]}, index=dates)
print(type(df3.index))
# DatetimeIndex
print(df3.index)
# D. MultiIndex - 多级索引
arrays = [['A', 'A', 'B', 'B'], [1, 2, 1, 2]]
index = pd.MultiIndex.from_arrays(arrays, names=['letter', 'number'])
df4 = pd.DataFrame({'value': [10, 20, 30, 40]}, index=index)
print(type(df4.index))
# MultiIndex
print(df4)
```

更多索引类型：

```
import pandas as pd
# CategoricalIndex - 分类索引
df5 = pd.DataFrame(
    {'A': [1, 2, 3]},
    index=pd.CategoricalIndex(['low', 'medium', 'high'])
)
# IntervalIndex - 区间索引
intervals = pd.interval_range(start=0, end=3)
df6 = pd.DataFrame({'A': [1, 2, 3]}, index=intervals)
# PeriodIndex - 时期索引
periods = pd.period_range('2023-01', periods=3, freq='M')
df7 = pd.DataFrame({'A': [1, 2, 3]}, index=periods)
```

知识拓展：索引的作用：

- 快速数据访问和选择
- 自动对齐操作
- 支持时间序列分析
- 层次化数据组织

39 • 不定项选择

关于Pandas的 groupby() 操作，以下说法正确的有：

A. 可以按单列分组

B. 可以按多列分组

C. 返回GroupBy对象

D. 可以应用多种聚合函数

答案 A. 可以按单列分组；B. 可以按多列分组；C. 返回GroupBy对象；D. 可以应用多种聚合函数

解析 所有选项都Pandas DataFrame的正确操作方式。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
})
# A. 使用loc选择行和列
print(df.loc[0:1, ['A', 'B']])
# A B
# 0 1 4
# 1 2 5
# B. 使用iloc选择按位置
print(df.iloc[0:2, 0:2])
# A B
# 0 1 4
# 1 2 5
# C. 使用布尔索引
print(df[df['A'] > 1])
# A B C
# 1 2 5 8
# 2 3 6 9
# D. 使用query()方法
print(df.query('A > 1 and B < 7'))
# A B C
# 1 2 5 8
```

知识拓展：DataFrame选择方法对比：

- loc ：基于标签的选择
- iloc ：基于位置的选择
- [] ：布尔索引、列名选择
- query() ：使用字符串表达式

40 • 不定项选择

Pandas中，以下哪些方法用于数据重塑：

A. pivot()

B. melt()

C. stack()

D. unstack()

答案 A. pivot()；B. melt()；C. stack()；D. unstack()

解析 所有选项都是Pandas数据重塑方法。

示例：

```
import pandas as pd
# A. melt() - 宽转长
df_wide = pd.DataFrame({
    'id': [1, 2],
    '2023': [100, 200],
```

```
    '2024': [150, 250]
})
df_long = df_wide.melt(id_vars=['id'], var_name='year', value_name='sales')
print(df_long)
# id year sales
# 0 1 2023 100
# 1 2 2023 200
# 2 1 2024 150
# 3 2 2024 250
# B. pivot() - 长转宽
df_pivoted = df_long.pivot(index='id', columns='year', values='sales')
print(df_pivoted)
# C. stack() - 列转行
df = pd.DataFrame([[1, 2], [3, 4]], columns=['A', 'B'])
stacked = df.stack()
print(stacked)
# 0 A 1
# B 2
# 1 A 3
# B 4
# D. unstack() - 行转列
unstacked = stacked.unstack()
print(unstacked)
# A B
# 0 1 2
# 1 3 4
```

知识拓展：数据重塑场景：

- melt ：制作时间序列数据
- pivot ：数据汇总分析
- stack/unstack ：处理多级索引

41 • 不定项选择

关于 pathlib 的路径操作，以下说法正确的有：

A. / 运算符用于路径拼接

B. joinpath() 方法用于拼接路径

C. 自动处理不同操作系统的路径分隔符

D. 支持相对路径和绝对路径

答案 A. / 运算符用于路径拼接；B. joinpath() 方法用于拼接路径；C. 自动处理不同操作系统的路径分隔符；D. 支持相对路径和绝对路径

解析 所有选项都是Pandas数据清洗的正确方法。

示例：

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    'A': [1, 2, 2, np.nan, 5],
    'B': [10, 20, 20, 40, 50],
    'C': ['a', 'b', 'b', 'd', 'e']
})
# A. 删除重复行
df_no_dup = df.drop_duplicates()
```

```
print(df_no_dup)
# B. 处理缺失值
df_filled = df.fillna(0)
# 填充
df_dropped = df.dropna()
# 删除
print(df_filled)
# C. 替换值
df_replaced = df.replace({'a': 'A', 'b': 'B'})
print(df_replaced)
# D. 重命名列
df_renamed = df.rename(columns={'A': 'col_a', 'B': 'col_b'})
print(df_renamed.columns)
```

常用清洗操作：

```
import pandas as pd
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Charlie'],
    'age': ['25', '30', 'unknown'],
    'score': [85.5, 90.0, 88.3]
})
# 类型转换
df['age'] = pd.to_numeric(df['age'], errors='coerce')
# 删除空白
df['name'] = df['name'].str.strip()
# 统一大小写
df['name'] = df['name'].str.lower()
```

知识拓展：数据质量检查：

```
print(df.info())
# 数据类型和缺失值
print(df.describe())
# 描述性统计
print(df.isnull().sum())
# 缺失值计数
print(df.duplicated().sum())
# 重复行计数
```

42 • 不定项选择

Path 对象的以下哪些属性返回路径信息：

A. name

B. stem

C. suffix

D. parent

答案 A. name；B. stem；C. suffix；D. parent

解析 所有选项都返回Pathlib的路径信息。

示例：

```
from pathlib import Path
p = Path('/home/user/docs/project/file.txt')
# A. parent - 父目录
print(p.parent)
# /home/user/docs/project
```

```
print(p.parent.parent)
# /home/user/docs
# B. parents - 所有父目录
for parent in p.parents:
    print(parent)
# /home/user/docs/project
# C. name - 文件名
print(p.name)
# file.txt
# D. parts - 路径分量
print(p.parts)
# ('/', 'home', 'user', 'docs', 'project', 'file.txt')
更多路径信息属性：
from pathlib import Path
p = Path('/home/user/archive.tar.gz')
print(p.stem)
# archive.tar - 主文件名
print(p.suffix)
# .gz - 扩展名
print(p.suffixes)
# ['.tar', '.gz'] - 所有扩展名
print(p.anchor)
# / - 锚点（根目录）
print(p.drive)
# '' - 驱动器（Windows为C:）
知识拓展：路径操作与os.path的对比：
import os
from pathlib import Path
path_str = '/home/user/file.txt'
# os.path
print(os.path.dirname(path_str))
# /home/user
print(os.path.basename(path_str))
# file.txt
# pathlib (更面向对象)
p = Path(path_str)
print(p.parent)
# /home/user
print(p.name)
# file.txt
```

43 • 不定项选择

关于列表推导式的嵌套，以下说法正确的有：

A. 可以嵌套多层

B. 外层循环在前

C. 可以包含多个if条件

D. 可读性可能降低

答案 A. 可以嵌套多层；B. 外层循环在前；C. 可以包含多个if条件；D. 可读性可能降低

解析 所有选项都是列表推导式的正确特性。

示例：

```
# A. 更简洁 - 与for循环对比
# for循环
result = []
for i in range(5):
    if i % 2 == 0:
        result.append(i * 2)
# 列表推导式
result = [i * 2 for i in range(5) if i % 2 == 0]
print(result)
# [0, 4, 8]
# B. 支持条件筛选
squares = [x**2 for x in range(10) if x % 2 == 0]
print(squares)
# [0, 4, 16, 36, 64]
# C. 可以嵌套
matrix = [[i*j for j in range(3)] for i in range(3)]
print(matrix)
# [[0, 0, 0],
# [0, 1, 2],
# [0, 2, 4]]
# D. 一般比for循环快
import time
start = time.time()
result1 = [i**2 for i in range(10000)]
time1 = time.time() - start
start = time.time()
result2 = []
for i in range(10000):
    result2.append(i**2)
time2 = time.time() - start
print(f"推导式: {time1:.4f}s, 循环: {time2:.4f}s")
```

各种推导式：

```
# 列表推导式
list_comp = [x**2 for x in range(5)]
# 集合推导式
set_comp = {x**2 for x in range(5)}
# 字典推导式
dict_comp = {x: x**2 for x in range(5)}
# 生成器表达式
gen_exp = (x**2 for x in range(5))
```

知识拓展：推导式的复杂用法：

```
# 多重条件
result = [x for x in range(20) if x % 2 == 0 if x % 3 == 0]
print(result)
# [0, 6, 12, 18]
# 多重循环
result = [(x, y) for x in range(3) for y in range(3)]
print(result)
# [(0,0), (0,1), ..., (2,2)]
# if-else表达式
result = [x if x % 2 == 0 else -x for x in range(5)]
```

```
print(result)
# [0, -1, 2, -3, 4]
```

44 • 不定项选择

以下哪些是Python的函数式编程特性：

A. 高阶函数
B. lambda表达式
C. map/filter/reduce
D. 列表推导式

答案 A. 高阶函数；B. lambda表达式；C. map/filter/reduce；D. 列表推导式

解析 所有选项都是Python函数式编程的特性。

示例：

```
from functools import reduce
# A. map - 映射变换
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x**2, numbers))
print(squares)
# [1, 4, 9, 16, 25]
# B. filter - 过滤
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(even_numbers)
# [2, 4]
# C. reduce - 聚合
sum_all = reduce(lambda x, y: x + y, numbers)
print(sum_all)
# 15
# D. lambda - 匿名函数
add = lambda x, y: x + y
print(add(3, 5))
# 8
# 组合使用
result = reduce(
    lambda x, y: x + y,
    map(lambda x: x**2,
        filter(lambda x: x % 2 == 0, numbers))
)
print(result)
# 4 + 16 = 20
```

函数式编程风格对比：

```
numbers = [1, 2, 3, 4, 5]
# 命令式（过程式）
result = 0
for num in numbers:
    if num % 2 == 0:
        result += num ** 2
print(result)
# 20
# 函数式
result = sum(map(lambda x: x**2, filter(lambda x: x % 2 == 0, numbers)))
print(result)
```

```
# 20
# Pythonic（推导式）
result = sum(x**2 for x in numbers if x % 2 == 0)
print(result)
# 20
```

知识拓展：函数式编程的优缺点：

优点：

- 代码简洁、易读
- 更容易测试和调试
- 更容易并行化

缺点：

- 可能性能较差
- 学习曲线较陡
- 过度使用可能降低可读性

45 • 不定项选择

关于 zip() 函数的特性，以下说法正确的有：

A. 可以接受多个可迭代对象

B. 返回元组的迭代器

C. 以最短序列为准

D. 可以使用 zip(*zipped) 解压

答案 A. 可以接受多个可迭代对象；B. 返回元组的迭代器；C. 以最短序列为准

解析 Python是多范式语言，支持面向对象、过程式和函数式编程。

示例：

```
# A. 面向对象编程
class Calculator:
    def __init__(self):
        self.result = 0
    def add(self, x):
        self.result += x
        return self
    def get_result(self):
        return self.result
calc = Calculator()
result = calc.add(5).add(3).get_result()
print(result)
# 8
# B. 过程式编程
def calculate_total(numbers):
    total = 0
    for num in numbers:
        total += num
    return total
print(calculate_total([1, 2, 3]))
# 6
# C. 函数式编程
from functools import reduce
numbers = [1, 2, 3]
result = reduce(lambda x, y: x + y, numbers)
print(result)
```

```
# 6
三种范式对比：
# 同一任务：计算偶数平方和
data = [1, 2, 3, 4, 5]
# 1. 面向对象
class EvenSquareSum:
    def __init__(self, numbers):
        self.numbers = numbers
    def calculate(self):
        result = 0
        for num in self.numbers:
            if num % 2 == 0:
                result += num ** 2
        return result
obj = EvenSquareSum(data)
print(obj.calculate())
# 20
# 2. 过程式
def even_square_sum(numbers):
    result = 0
    for num in numbers:
        if num % 2 == 0:
            result += num ** 2
    return result
print(even_square_sum(data))
# 20
# 3. 函数式
from functools import reduce
result = reduce(
    lambda x, y: x + y,
    map(lambda x: x**2, filter(lambda x: x % 2 == 0, data))
)
print(result)
                # 20
# 4. Pythonic
result = sum(x**2 for x in data if x % 2 == 0)
print(result)
# 20
```

知识拓展：Python不是纯函数式语言的原因：

- 不强制不可变性
- 支持变量重赋值
- 有副作用（print、I/O操作）
- 不是惰性求值（默认是严格求值）

三、填空题

46 • 填空 / 简答

Python中，_____ 方法用于定义对象的字符串表示（面向用户），_____ 方法用于定义对象的官方字符串表示（面向开发者）。

答案 super() ， __mro__

解析 多重继承中的关键工具。

示例：

```
class A:
    def method(self):
        print("A")
class B(A):
    def method(self):
        super().method()
        # 调用父类方法
        print("B")
class C(A):
    def method(self):
        super().method()
        print("C")
class D(B, C):
    def method(self):
        super().method()
        print("D")
# 查看MRO
print(D.__mro__)
# (<class 'D'>, <class 'B'>, <class 'C'>, <class 'A'>, <class 'object'>)
D().method()
# A
# C
# B
# D
```

知识拓展：MRO使用C3线性化算法，保证继承的一致性。

47 • 填空 / 简答

在装饰器中，使用 _____ 模块的 _____ 装饰器可以保留原函数的元数据。

答案 with ， finally

解析 幂析：资源管理的两种方式。

示例：

```
# 使用with（上下文管理器）
with open('file.txt', 'r') as f:
    content = f.read()
# 文件自动关闭
# 使用finally
try:
    f = open('file.txt', 'r')
    content = f.read()
finally:
    f.close()
# 总是执行
```

知识拓展： with 更简洁，自动处理异常； finally 更灵活，但需要手动管理资源。

48 • 填空 / 简答

Pandas中，_____ 方法用于数据透视，_____ 方法用于将宽格式数据转换为长格式。

答案 fillna() ， dropna()

解析 幂析：Pandas处理缺失值的两种方法。

示例：

```
import pandas as pd
import numpy as np
df = pd.DataFrame({
    'A': [1, 2, np.nan, 4],
    'B': [np.nan, 2, 3, 4],
    'C': [1, 2, 3, 4]
})
# fillna - 填充缺失值
df_filled = df.fillna(0)
df_filled = df.fillna(method='ffill')
# 前向填充
df_filled = df.fillna(df.mean())
# 用均值填充
# dropna - 删除缺失值
df_dropped = df.dropna()
# 删除包含缺失值的行
df_dropped = df.dropna(axis=1)
# 删除包含缺失值的列
```

知识拓展：可以使用 isnull() 、 notnull() 检查缺失值。

49 • 填空 / 简答

使用 _____ 函数可以创建计数器，使用 _____ 函数可以将多个迭代器组合。

答案 filter() ， reduce()

解析 幂析：函数式编程的两个核心函数。

示例：

```
from functools import reduce
# filter - 过滤元素
numbers = [1, 2, 3, 4, 5, 6]
even = list(filter(lambda x: x % 2 == 0, numbers))
print(even)
# [2, 4, 6]
# reduce - 累积计算
product = reduce(lambda x, y: x * y, numbers)
print(product)
                # 720 (1*2*3*4*5*6)
```

知识拓展： reduce() 在Python 3中移至 functools 模块。

50 • 填空 / 简答

pathlib.Path 对象的 _____ 方法用于读取文本文件，_____ 方法用于写入文本文件。

答案 cwd() ， iterdir()

解析 幂析：Path对象的目录操作方法。

示例：

```
from pathlib import Path
# cwd() - 获取当前工作目录
current = Path.cwd()
print(current)
# /home/user/project
# iterdir() - 迭代目录内容
for item in current.iterdir():
    if item.is_file():
        print(f"文件: {item.name}")
    elif item.is_dir():
        print(f"目录: {item.name}")
```

知识拓展： iterdir() 返回生成器，不递归。递归搜索使用 rglob() 。

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class Singleton:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
s1 = Singleton()
s2 = Singleton()
print(s1 is s2)
print(id(s1) == id(s2))
```

问题：

- (1) 两个 print 语句的输出分别是什么？
- (2) 这段代码实现了什么设计模式？
- (3) __new__() 方法的作用是什么？

答案 单例模式

- (1) 两个都输出： True
- (2) 单例模式（Singleton Pattern）
- (3) __new__() 用于创建实例，在 __init__() 之前调用，控制实例的创建过程

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
def counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment
```

```
c1 = counter()
c2 = counter()
print(c1())
print(c2())
```

问题：

- (1) 三个 print 语句的输出分别是什么？
- (2) nonlocal 关键字的作用是什么？
- (3) 为什么 c1 和 c2 的计数是独立的？

答案 • (1) 输出：1, 2, 1
• (2) nonlocal 声明要修改外层函数的变量，而不是创建新的局部变量
• (3) 每次调用 counter() 都创建新的闭包环境，有独立的 count 变量

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'category': ['A', 'B', 'A', 'B', 'A'],
    'value': [10, 20, 30, 40, 50]
})
result = df.groupby('category')['value'].agg(['sum', 'mean', 'count'])
print(result)
print(type(result))
```

问题：

- (1) result 的输出是什么样的？
- (2) agg() 方法可以接受哪些类型的参数？
- (3) 如何只获取A类别的统计信息？

答案 聚合
• (1) 输出DataFrame，包含sum、mean、count三列，索引为A和B
• (2) 可以接受函数名字符串、函数对象、函数列表、字典等
• (3) result.loc['A']

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
from pathlib import Path
def process_files(directory):
    path = Path(directory)
    python_files = list(path.glob('**/*.py'))
    for file in python_files:
        size = file.stat().st_size
        yield {
            'name': file.name,
            'path': str(file),
            'size': size
```

```
        }
files = list(process_files('.'))
print(len(files))
```

问题：

- (1) glob('**/*.py') 中的 ** 表示什么？
- (2) stat().st_size 返回的是什么？
- (3) 这个函数为什么使用生成器？

答案 • (1) ** 表示递归搜索所有子目录
- (2) 返回文件大小（字节数）
- (3) 使用生成器可以节省内存，特别是处理大量文件时

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
from functools import wraps
def validate(func):
    @wraps(func)
    def wrapper(x):
        if x < 0:
            raise ValueError("参数必须非负")
        return func(x)
    return wrapper
@validate
def square_root(x):
    return x ** 0.5
try:
    print(square_root(16))
    print(square_root(-4))
except ValueError as e:
    print(e)
```

问题：

- (1) 完整的输出是什么？
- (2) @wraps(func) 的作用是什么？
- (3) 如果不使用装饰器，如何实现相同功能？

答案 • (1) 输出：
4.0
参数必须非负
- (2) @wraps(func) 保留原函数的元数据（如 __name__ 、 __doc__ 等）
- (3) 可以在函数内部添加验证逻辑，或者手动调用验证函数

解析

五、程序设计题

56 • 填空 / 简答

设计一个员工管理系统，包含以下类：
Employee类（基类）：
• 属性：姓名、工号、基本工资
• 方法： calculate_salary() （抽象方法）
• 魔法方法： __str__() 、 __eq__() （按工号比较）
Manager类（继承
类（继承Employee）：
）：
• 额外属性：奖金
• 重写 calculate_salary() ：基本工资 + 奖金
Developer类（继承
类（继承Employee）：
）：
• 额外属性：项目数量
• 重写 calculate_salary() ：基本工资 + 项目数量 * 1000
要求：
• 创建多个员工实例
• 使用列表推导式筛选工资大于10000的员工
• 实现 __repr__() 方法
• 使用 @property 实现只读属性

答案
```
from abc import ABC, abstractmethod
class Employee(ABC):
    def __init__(self, name, emp_id, base_salary):
        self._name = name
        self._emp_id = emp_id
        self._base_salary = base_salary
    @property
    def name(self):
        return self._name
    @property
    def emp_id(self):
        return self._emp_id
    @abstractmethod
    def calculate_salary(self):
        pass
    def __str__(self):
        return f"{self.name} (ID: {self.emp_id})"
    def __repr__(self):
        return f"{self.__class__.__name__}('{self.name}', '{self.emp_id}', {self._base_sal
    def __eq__(self, other):
        if isinstance(other, Employee):
            return self.emp_id == other.emp_id
        return False
class Manager(Employee):
    def __init__(self, name, emp_id, base_salary, bonus):
        super().__init__(name, emp_id, base_salary)
        self.bonus = bonus
```

```
    def calculate_salary(self):
        return self._base_salary + self.bonus
class Developer(Employee):
    def __init__(self, name, emp_id, base_salary, project_count):
        super().__init__(name, emp_id, base_salary)
        self.project_count = project_count
    def calculate_salary(self):
        return self._base_salary + self.project_count * 1000
# 使用示例
employees = [
    Manager("张三", "M001", 8000, 5000),
    Developer("李四", "D001", 7000, 3),
    Manager("王五", "M002", 9000, 3000),
    Developer("赵六", "D002", 6000, 5)
]
print("所有员工：")
for emp in employees:
    print(f"{emp} - 工资: {emp.calculate_salary()}")
high_salary = [emp for emp in employees if emp.calculate_salary() > 10000]
print(f"\n工资大于10000的员工：")
for emp in high_salary:
    print(f"{emp} - 工资: {emp.calculate_salary()}")
print(f"\n员工对象表示：")
print(repr(employees[0]))
```

解析

57 • 填空 / 简答

编写一个日志分析工具，要求：

1. read_log_file(filename) ：生成器函数，逐行读取日志文件
2. parse_log_line(line) ：解析日志行，提取时间、级别、消息
3. filter_by_level(logs, level) ：生成器函数，按级别过滤日志
4. count_by_level(logs) ：统计各级别日志数量

要求：

- 使用 pathlib 处理文件路径
- 使用生成器实现惰性加载
- 使用装饰器记录函数执行时间
- 处理文件不存在等异常

答案

```
from pathlib import Path
import re
from collections import Counter
import time
from functools import wraps
def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
```

```
        end = time.time()
        print(f"{func.__name__} 执行时间: {end-start:.4f}秒")
        return result
    return wrapper
def read_log_file(filename):
    path = Path(filename)
    try:
        if not path.exists():
            raise FileNotFoundError(f"文件{filename} 不存在")
        with path.open('r', encoding='utf-8') as f:
            for line in f:
                yield line.strip()
    except Exception as e:
        print(f"读取文件错误: {e}")
def parse_log_line(line):
    pattern = r'(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}) \[(\w+)\] (.+)'
    match = re.match(pattern, line)
    if match:
        return {
            'time': match.group(1),
            'level': match.group(2),
            'message': match.group(3)
        }
    return None
def filter_by_level(logs, level):
    for log in logs:
        parsed = parse_log_line(log)
        if parsed and parsed['level'] == level:
            yield parsed
@timer
def count_by_level(logs):
    counter = Counter()
    for log in logs:
        parsed = parse_log_line(log)
        if parsed:
            counter[parsed['level']] += 1
    return counter
# 使用示例
log_content = """2024-01-01 10:00:00 [INFO] 系统启动
2024-01-01 10:01:00 [ERROR] 连接失败
2024-01-01 10:02:00 [INFO] 重试连接
2024-01-01 10:03:00 [WARNING] 内存使用过高
2024-01-01 10:04:00 [ERROR] 数据库错误"""
# 创建测试日志文件
log_file = Path('test.log')
log_file.write_text(log_content)
# 分析日志
logs = list(read_log_file('test.log'))
```

```
print("所有日志：")
for log in logs:
    print(log)
print("\nERROR级别日志：")
for log in filter_by_level(logs, 'ERROR'):
    print(log)
print("\n日志级别统计：")
stats = count_by_level(logs)
for level, count in stats.items():
    print(f"{level}: {count}")
```

解析

58 • 填空 / 简答

使用Pandas完成以下数据清洗和分析任务：
给定学生成绩数据（包含缺失值和重复数据）：

- 删除重复记录
- 填充缺失值（使用平均分）
- 添加总分和平均分列
- 按平均分排序
- 使用 groupby() 按班级统计
- 使用 apply() 添加等级列（优秀/良好/及格/不及格）
- 使用 pivot_table() 创建数据透视表
- 导出清洗后的数据

答案

```
import pandas as pd
import numpy as np
# 创建包含缺失值和重复数据的DataFrame
data = {
    '姓名': ['张三', '李四', '王五', '张三', '赵六', '钱七'],
    '班级': ['1班', '2班', '1班', '1班', '2班', '1班'],
    '语文': [85, 90, np.nan, 85, 88, 92],
    '数学': [90, 85, 88, 90, np.nan, 95],
    '英语': [88, np.nan, 90, 88, 92, 87]
}
df = pd.DataFrame(data)
print("原始数据：")
print(df)
# 删除重复记录
df_clean = df.drop_duplicates()
print(f"\n删除重复后: {len(df_clean)} 条记录")
# 填充缺失值（使用各科平均分）
for col in ['语文', '数学', '英语']:
    df_clean[col].fillna(df_clean[col].mean(), inplace=True)
# 添加总分和平均分
df_clean['总分'] = df_clean[['语文', '数学', '英语']].sum(axis=1)
df_clean['平均分'] = df_clean[['语文', '数学', '英语']].mean(axis=1)
# 按平均分排序
df_clean = df_clean.sort_values('平均分', ascending=False)
```

```
print("\n清洗后的数据：")
print(df_clean)
# 添加等级列
def grade_level(score):
    if score >= 90:
        return '优秀'
    elif score >= 80:
        return '良好'
    elif score >= 60:
        return '及格'
    else:
        return '不及格'
df_clean['等级'] = df_clean['平均分'].apply(grade_level)
# 按班级统计
class_stats = df_clean.groupby('班级').agg({
    '平均分': ['mean', 'max', 'min'],
    '姓名': 'count'
})
print("\n班级统计：")
print(class_stats)
# 创建数据透视表
pivot = df_clean.pivot_table(
    values='平均分',
    index='班级',
    columns='等级',
    aggfunc='count',
    fill_value=0
)
print("\n数据透视表：")
print(pivot)
# 导出数据
df_clean.to_csv('cleaned_data.csv', index=False, encoding='utf-8-sig')
df_clean.to_excel('cleaned_data.xlsx', index=False)
print("\n数据已导出")
```

解析

59 • 填空 / 简答

编写一个文件组织工具，要求：

1. organize_by_extension(source_dir, target_dir) ：
- 按文件扩展名分类文件
- 创建对应的子目录
- 移动文件到对应目录
2. find_duplicates(directory) ：
- 使用生成器查找重复文件（基于文件大小和内容）
- 返回重复文件列表
3. clean_empty_dirs(directory) ：
- 递归删除空目录

要求：

- 使用 pathlib 进行所有路径操作
- 实现错误处理
- 使用生成器优化内存使用
- 添加日志输出

答案

```
from pathlib import Path
import shutil
import hashlib
def organize_by_extension(source_dir, target_dir):
    source = Path(source_dir)
    target = Path(target_dir)
    target.mkdir(parents=True, exist_ok=True)
    file_count = 0
    for file in source.rglob('*'):
        if file.is_file():
            ext = file.suffix[1:] if file.suffix else 'no_extension'
            ext_dir = target / ext
            ext_dir.mkdir(exist_ok=True)
            dest_file = ext_dir / file.name
            try:
                shutil.copy2(file, dest_file)
                file_count += 1
                print(f"已复制: {file.name} -> {ext}/")
            except Exception as e:
                print(f"复制失败{file.name}: {e}")
    print(f"\n共组织{file_count} 个文件")
def get_file_hash(file_path):
    hash_md5 = hashlib.md5()
    with open(file_path, 'rb') as f:
        for chunk in iter(lambda: f.read(4096), b""):
            hash_md5.update(chunk)
    return hash_md5.hexdigest()
def find_duplicates(directory):
    path = Path(directory)
    size_map = {}
    # 按大小分组
    for file in path.rglob('*'):
        if file.is_file():
            size = file.stat().st_size
            if size not in size_map:
                size_map[size] = []
            size_map[size].append(file)
    # 检查相同大小的文件
    for size, files in size_map.items():
        if len(files) > 1:
            hash_map = {}
            for file in files:
```

```
                file_hash = get_file_hash(file)
                if file_hash not in hash_map:
                hash_map[file_hash] = []
                hash_map[file_hash].append(file)
            for file_hash, dup_files in hash_map.items():
                if len(dup_files) > 1:
                yield dup_files
def clean_empty_dirs(directory):
    path = Path(directory)
    removed_count = 0
    for dir_path in sorted(path.rglob('*'), reverse=True):
        if dir_path.is_dir():
            try:
                if not any(dir_path.iterdir()):
                dir_path.rmdir()
                removed_count += 1
                print(f"已删除空目录: {dir_path}")
            except Exception as e:
                print(f"删除失败{dir_path}: {e}")
    print(f"\n共删除{removed_count} 个空目录")
# 使用示例
# organize_by_extension('source', 'organized')
# print("\n查找重复文件：")
# for duplicates in find_duplicates('.'):
# print("重复文件组：")
# for file in duplicates:
# print(f" {file}")
# clean_empty_dirs('organized')
```

解析

第 4 套

60 题

一、单项选择题

01 • 单项选择

关于Python的多态性，以下说法正确的是：

A. 多态只能通过继承实现
B. Python通过鸭子类型实现多态
C. 多态需要显式声明
D. Python不支持多态

答案 B. Python通过鸭子类型实现多态

解析 Python通过鸭子类型（duck typing）实现多态，不需要继承或显式声明。
示例：

```
# 鸭子类型：如果它走起来像鸭子，叫起来像鸭子，那它就是鸭子
class Duck:
    def quack(self):
        return "Quack!"
    def fly(self):
        return "Duck flying"
class Person:
    def quack(self):
        return "I'm imitating a duck!"
    def fly(self):
        return "Person can't fly"
def make_it_quack(duck_like):
    # 不检查类型，只要有quack方法就行
    print(duck_like.quack())
duck = Duck()
person = Person()
make_it_quack(duck)
# Quack!
make_it_quack(person)
# I'm imitating a duck!
知识拓展：鸭子类型vs传统多态：
# 传统面向对象（Java风格）
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass
class Dog(Animal):
    def make_sound(self):
        return "Woof"
class Cat(Animal):
    def make_sound(self):
```

```
        return "Meow"
def animal_sound(animal: Animal):
    print(animal.make_sound())
# Python鸭子类型（更灵活）
class Robot:
    def make_sound(self):
        return "Beep"
# Robot不继承Animal，但也能使用
animal_sound(Dog())
# Woof
animal_sound(Robot())
# Beep - 鸭子类型允许
```

02 • 单项选择

以下代码的输出是什么？

```
class A:
    def method(self):
        return "A"
class B(A):
    def method(self):
        return "B"
obj = B()
print(obj.method())
```

A. A
B. B
C. AB
D. 报错

答案 B. B

解析 子类重写（override）了父类方法，调用时执行子类的实现。

示例：

```
class Animal:
    def speak(self):
        return "Some sound"
class Dog(Animal):
    def speak(self):
        # 重写父类方法
        return "Woof"
class Cat(Animal):
    def speak(self):
        # 重写父类方法
        return "Meow"
dog = Dog()
cat = Cat()
print(dog.speak())
# Woof
print(cat.speak())
# Meow
# 多态性：同一方法，不同行为
animals = [Dog(), Cat(), Animal()]
```

```
for animal in animals:
    print(animal.speak())
# 输出：
# Woof
# Meow
# Some sound
知识拓展：调用父类方法：
class Parent:
    def method(self):
        return "Parent"
class Child(Parent):
    def method(self):
        parent_result = super().method()
        # 调用父类方法
        return f"{parent_result} + Child"
obj = Child()
print(obj.method())
# Parent + Child
```

03 • 单项选择

关于 @classmethod 装饰器，以下说法正确的是：

A. 第一个参数是 self

B. 第一个参数是 cls

C. 不能访问类变量

D. 只能通过实例调用

答案 B. 第一个参数是 cls

解析 @classmethod 装饰器的第一个参数是 cls ，代表类本身。

```
示例：
class MyClass:
    count = 0
    def __init__(self, name):
        self.name = name
        MyClass.count += 1
    @classmethod
    def get_count(cls):
        # cls代表类本身
        return cls.count
    @classmethod
    def create_default(cls):
        # 可以用cls创建实例
        return cls("default")
obj1 = MyClass("A")
obj2 = MyClass("B")
print(MyClass.get_count())
# 2 - 类方法可以通过类调用
print(obj1.get_count())
# 2 - 也可以通过实例调用
obj3 = MyClass.create_default()
print(obj3.name)
# default
```

三种方法类型对比：

```
class Example:
    class_var = "class variable"
    def instance_method(self):
        # 实例方法：第一个参数是self
        return f"Instance: {self}"
    @classmethod
    def class_method(cls):
        # 类方法：第一个参数是cls
        return f"Class: {cls.__name__}, {cls.class_var}"
    @staticmethod
    def static_method(x, y):
        # 静态方法：没有特殊的第一个参数
        return x + y
obj = Example()
print(obj.instance_method())
# Instance: <__main__.Example object>
print(Example.class_method())
# Class: Example, class variable
print(Example.static_method(1, 2))
# 3
```

知识拓展：类方法的应用场景：

- 工厂方法（alternative constructors）
- 访问/修改类变量
- 实现单例模式

04 • 单项选择

以下哪个魔法方法用于实现对象的比较运算？

A. `__cmp__()`

B. `__compare__()`

C. `__eq__()`

D. `__is__()`

答案 C. `__eq__()`

解析 `__eq__()` 魔法方法用于实现相等比较（ == 运算符）。

示例：

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __eq__(self, other):
        # 实现 == 运算符
        if not isinstance(other, Point):
            return False
        return self.x == other.x and self.y == other.y
    def __ne__(self, other):
        # 实现 != 运算符
        return not self.__eq__(other)
p1 = Point(1, 2)
p2 = Point(1, 2)
p3 = Point(3, 4)
```

```
print(p1 == p2)
# True
print(p1 == p3)
# False
print(p1 != p3)
# True
```

所有比较魔法方法：

```
class Number:
    def __init__(self, value):
        self.value = value
    def __eq__(self, other):
        # ==
        return self.value == other.value
    def __ne__(self, other):
        # !=
        return self.value != other.value
    def __lt__(self, other):
        # <
        return self.value < other.value
    def __le__(self, other):
        # <=
        return self.value <= other.value
    def __gt__(self, other):
        # >
        return self.value > other.value
    def __ge__(self, other):
        # >=
        return self.value >= other.value
n1 = Number(5)
n2 = Number(10)
print(n1 < n2)
# True
print(n1 <= n2)
# True
print(n1 == n2)
# False
print(n1 > n2)
# False
```

知识拓展：使用 functools.total_ordering 简化：

```
from functools import total_ordering
@total_ordering
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade
    def __eq__(self, other):
        return self.grade == other.grade
    def __lt__(self, other):
        return self.grade < other.grade
# 只需定义__eq__和__lt__，其他比较方法自动生成
s1 = Student("Alice", 85)
```

```
s2 = Student("Bob", 90)
print(s1 < s2)
# True
print(s1 <= s2)
# True - 自动生成
print(s1 > s2)
# False - 自动生成
```

05 • 单项选择

关于Python的抽象基类，以下说法正确的是：

A. 使用 abc 模块
B. 使用 @abstractmethod 装饰器
C. 抽象类不能被实例化
D. 以上都正确

答案 D. 以上都正确

解析 所有选项都正确，抽象基类使用 abc 模块、 @abstractmethod 装饰器，且不能被实例化。

示例：

```
from abc import ABC, abstractmethod
# A. 使用abc模块
class Shape(ABC):
    # 继承ABC
    # B. 使用@abstractmethod装饰器
    @abstractmethod
    def area(self):
        pass
    @abstractmethod
    def perimeter(self):
        pass
    def describe(self):
        # 可以有具体方法
        return f"Area: {self.area()}, Perimeter: {self.perimeter()}"
# C. 抽象类不能被实例化
try:
    shape = Shape()
except TypeError as e:
    print(f"错误: {e}")
# 错误: Can't instantiate abstract class Shape with abstract methods area, perimeter
# 子类必须实现所有抽象方法
class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height
    def area(self):
        return self.width * self.height
    def perimeter(self):
        return 2 * (self.width + self.height)
rect = Rectangle(5, 3)
print(rect.describe())
                # Area: 15, Perimeter: 16
```

```
知识拓展：抽象属性和类方法：
from abc import ABC, abstractmethod
class Animal(ABC):
    @property
    @abstractmethod
    def name(self):
        # 抽象属性
        pass
    @classmethod
    @abstractmethod
    def category(cls):
        # 抽象类方法
        pass
    @staticmethod
    @abstractmethod
    def description():
        # 抽象静态方法
        pass
class Dog(Animal):
    @property
    def name(self):
        return "Dog"
    @classmethod
    def category(cls):
        return "Mammal"
    @staticmethod
    def description():
        return "A domestic animal"
dog = Dog()
print(dog.name)
# Dog
print(Dog.category())
# Mammal
print(Dog.description())
                # A domestic animal
```

06 • 单项选择

以下代码的输出是什么？

```
class MyClass:
    count = 0
    def __init__(self):
        MyClass.count += 1
    @classmethod
    def get_count(cls):
        return cls.count
a = MyClass()
b = MyClass()
print(MyClass.get_count())
```

A. 0

B. 1

C. 2

D. 报错

答案 C. 2

解析 每次创建实例时， __init__ 执行， count 递增，创建两个实例后count为2。

示例：

```
class Counter:
    count = 0
    # 类变量
    def __init__(self):
        Counter.count += 1
        # 每次实例化时递增
        self.instance_id = Counter.count
c1 = Counter()
print(f"c1 ID: {c1.instance_id}, Total: {Counter.count}")
# c1 ID: 1, Total: 1
c2 = Counter()
print(f"c2 ID: {c2.instance_id}, Total: {Counter.count}")
# c2 ID: 2, Total: 2
c3 = Counter()
print(f"c3 ID: {c3.instance_id}, Total: {Counter.count}")
# c3 ID: 3, Total: 3
```

类变量vs实例变量：

```
class Example:
    class_var = 0
    # 类变量，所有实例共享
    def __init__(self, value):
        self.instance_var = value
        # 实例变量，每个实例独立
        Example.class_var += 1
e1 = Example(10)
e2 = Example(20)
print(f"e1.instance_var: {e1.instance_var}")
# 10
print(f"e2.instance_var: {e2.instance_var}")
# 20
print(f"class_var: {Example.class_var}")
# 2
# 修改类变量影响所有实例
Example.class_var = 100
print(f"e1通过类访问: {Example.class_var}")
# 100
print(f"e2通过类访问: {Example.class_var}")
# 100
```

知识拓展：类变量的陷阱：

```
class Trap:
    data = []
    # 类变量，共享！
    def add(self, item):
        self.data.append(item)
```

```
t1 = Trap()
t2 = Trap()
t1.add(1)
t2.add(2)
print(t1.data)
# [1, 2] - 意外!
print(t2.data)
# [1, 2] - 共享同一个列表
# 正确做法：使用实例变量
class Correct:
    def __init__(self):
        self.data = []
    # 每个实例独立的列表
    def add(self, item):
        self.data.append(item)
c1 = Correct()
c2 = Correct()
c1.add(1)
c2.add(2)
print(c1.data)
# [1]
print(c2.data)
# [2] - 正确
```

07 • 单项选择

关于生成器的 close() 方法，以下说法正确的是：

A. 用于关闭生成器
B. 会触发 GeneratorExit 异常
C. 之后不能再使用该生成器
D. 以上都正确

答案 D. 以上都正确

解析 所有选项都是生成器的正确特性。

示例：

```
# A. 使用yield关键字
def simple_generator():
    yield 1
    yield 2
    yield 3
gen = simple_generator()
print(next(gen))
# 1
print(next(gen))
# 2
# B. 惰性求值
def infinite_sequence():
    num = 0
    while True:
        yield num
        num += 1
gen = infinite_sequence()
```

```
print(next(gen))
# 0
print(next(gen))
# 1
# 不会无限循环，因为是惰性求值
# C. 节省内存
import sys
# 列表推导式：立即创建所有元素
list_comp = [x**2 for x in range(1000000)]
print(f"列表大小: {sys.getsizeof(list_comp)} bytes")
# 生成器表达式：按需生成
gen_exp = (x**2 for x in range(1000000))
print(f"生成器大小: {sys.getsizeof(gen_exp)} bytes")
# 生成器远小于列表
# D. 只能遍历一次
gen = (x for x in range(3))
print(list(gen))
# [0, 1, 2]
print(list(gen))
                # [] - 已耗尽
知识拓展：生成器的高级用法：
# 生成器表达式
gen = (x**2 for x in range(5))
# 使用yield from委托
def delegate_generator():
    yield from range(3)
    yield from ['a', 'b', 'c']
for item in delegate_generator():
    print(item)
# 0, 1, 2, a, b, c
# 生成器的send()方法
def echo_generator():
    while True:
        received = yield
        print(f"Received: {received}")
gen = echo_generator()
next(gen)
# 启动生成器
gen.send("Hello")
# Received: Hello
gen.send("World")
# Received: World
```

08 • 单项选择

以下代码的输出是什么？

```
def gen():
    try:
        yield 1
        yield 2
    finally:
```

```
        print("清理")
g = gen()
next(g)
g.close()
```

A. 只输出1

B. 输出1和"清理"

C. 输出1、2和"清理"

D. 报错

答案 B. 输出1和"清理"

解析 try块执行并返回1，finally块总是执行并打印"清理"。

示例：

```
def test_finally():
    try:
        print("try块")
        return 1
    finally:
        print("清理")
# 即使有return，finally也会执行
result = test_finally()
print(f"返回值: {result}")
# 输出：
# try块
# 清理
# 返回值: 1
finally的各种情况：
# 1. 正常执行
def case1():
    try:
        return "normal"
    finally:
        print("cleanup")
print(case1())
# cleanup
# normal
# 2. 异常发生
def case2():
    try:
        raise ValueError("error")
    except ValueError:
        return "caught"
    finally:
        print("cleanup")
print(case2())
# cleanup
# caught
# 3. finally中的return会覆盖try中的return
def case3():
    try:
        return "from try"
    finally:
```

```
        return "from finally"
# 不推荐!
print(case3())
# from finally
# 4. finally总是执行，即使break/continue
for i in range(3):
    try:
        if i == 1:
            continue
        print(f"i = {i}")
    finally:
        print(f"cleanup for {i}")
# 输出:
# i = 0
# cleanup for 0
# cleanup for 1
# i = 2
# cleanup for 2
知识拓展：try-except-else-finally的完整用法:
def complete_example(x):
    try:
        result = 10 / x
    except ZeroDivisionError:
        print("除零错误")
        return None
    else:
        # 只在没有异常时执行
        print("计算成功")
        return result
    finally:
        # 总是执行
        print("清理资源")
print(complete_example(2))
# 计算成功
# 清理资源
# 5.0
print(complete_example(0))
# 除零错误
# 清理资源
# None
```

09 • 单项选择

关于迭代器和生成器的关系，以下说法正确的是：

A. 生成器是特殊的迭代器

B. 迭代器是特殊的生成器

C. 两者没有关系

D. 完全相同

答案 A. 生成器是特殊的迭代器

解析 生成器是实现了迭代器协议（ `__iter__` 和 `__next__` ）的特殊迭代器。

示例：

```
# 生成器自动实现迭代器协议
def my_generator():
    yield 1
    yield 2
    yield 3
gen = my_generator()
# 生成器有__iter__和__next__方法
print(hasattr(gen, '__iter__'))
# True
print(hasattr(gen, '__next__'))
# True
# 可以手动调用__next__
print(gen.__next__())
# 1
print(gen.__next__())
# 2
# 也可以用next()函数
print(next(gen))
# 3
# 耗尽后抛出StopIteration
try:
    next(gen)
except StopIteration:
    print("生成器耗尽")
```

生成器vs手动实现的迭代器：

```
# 手动实现迭代器
class ManualIterator:
    def __init__(self, max_num):
        self.max_num = max_num
        self.current = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.current < self.max_num:
            self.current += 1
            return self.current
        raise StopIteration
# 使用生成器（更简洁）
def generator_version(max_num):
    current = 0
    while current < max_num:
        current += 1
        yield current
# 两者功能相同
for i in ManualIterator(3):
    print(i)
# 1, 2, 3
for i in generator_version(3):
    print(i)
# 1, 2, 3
```

知识拓展：生成器的内部状态：

```
import inspect
def stateful_generator():
    print("开始")
    yield 1
    print("中间")
    yield 2
    print("结束")
    yield 3
gen = stateful_generator()
# 查看生成器状态
print(inspect.getgeneratorstate(gen))
# GEN_CREATED
next(gen)
# 开始
print(inspect.getgeneratorstate(gen))
# GEN_SUSPENDED
next(gen)
# 中间
next(gen)
# 结束
try:
    next(gen)
except StopIteration:
    pass
print(inspect.getgeneratorstate(gen))
# GEN_CLOSED
```

10 • 单项选择

以下代码的输出是什么？

```
it = iter([1, 2, 3])
print(next(it))
```

A. 1 2 3 None

B. 1 2 3 StopIteration

C. 1 2 3 然后抛出StopIteration异常

D. 1 2 3 0

答案 C. 1 2 3 然后抛出StopIteration异常

解析 生成器前三次调用正常输出0、1、2，第四次调用抛出 StopIteration 异常。

示例：

```
def count_to_three():
    for i in range(3):
        yield i
gen = count_to_three()
print(next(gen))
# 0
print(next(gen))
# 1
print(next(gen))
# 2
try:
```

```
    print(next(gen))
# 第4次调用
except StopIteration:
    print("生成器已耗尽")
# 输出：生成器已耗尽
for循环自动处理
循环自动处理StopIteration：
def my_range(n):
    i = 0
    while i < n:
        yield i
        i += 1
# for循环自动捕获StopIteration
for num in my_range(3):
    print(num)
# 0, 1, 2 - 自动停止，不会抛出异常
# 手动迭代需要处理异常
gen = my_range(3)
while True:
    try:
        print(next(gen))
    except StopIteration:
        print("结束")
        break
知识拓展：StopIteration的值：
def generator_with_return():
    yield 1
    yield 2
    return "Done"
# return的值会传递给StopIteration
gen = generator_with_return()
print(next(gen))
# 1
print(next(gen))
# 2
try:
    next(gen)
except StopIteration as e:
    print(f"StopIteration值: {e.value}")
# Done
# 在协程中使用
def delegating_generator():
    result = yield from generator_with_return()
    print(f"收到返回值: {result}")
    return result
gen = delegating_generator()
print(next(gen))
# 1
print(next(gen))
# 2
try:
```

```
    next(gen)
except StopIteration as e:
    print(f"最终返回: {e.value}")
# 收到返回值: Done
# 最终返回: Done
```

11 • 单项选择

关于装饰器的参数，以下说法正确的是：

A. 装饰器不能有参数
B. 带参数的装饰器需要多层嵌套
C. 装饰器参数只能是字符串
D. 装饰器参数必须有默认值

答案 B. 带参数的装饰器需要多层嵌套

解析 带参数的装饰器需要三层函数：最外层接收参数，中间层是真正的装饰器，最内层是包装函数。

示例：

```
# 带参数的装饰器：三层结构
def repeat(times):
    # 第1层：接收装饰器参数
    def decorator(func):
        # 第2层：真正的装饰器
        def wrapper(*args, **kwargs):
            # 第3层：包装函数
            for _ in range(times):
                result = func(*args, **kwargs)
            return result
        return wrapper
    return decorator
@repeat(3)
# 传入参数
def say_hello():
    print("Hello!")
say_hello()
# 打印3次Hello!
# Hello!
```

无参数vs有参数装饰器对比：

```
# 无参数装饰器：两层结构
def simple_decorator(func):
    def wrapper(*args, **kwargs):
        print("函数执行前")
        result = func(*args, **kwargs)
        print("函数执行后")
        return result
    return wrapper
@simple_decorator
def func1():
    print("函数1")
# 有参数装饰器：三层结构
def parameterized_decorator(prefix):
    def decorator(func):
```

```
        def wrapper(*args, **kwargs):
            print(f"{prefix}: 函数执行前")
            result = func(*args, **kwargs)
            print(f"{prefix}: 函数执行后")
            return result
        return wrapper
    return decorator
@parameterized_decorator("LOG")
def func2():
    print("函数2")
func2()
# LOG: 函数执行前
# 函数2
# LOG: 函数执行后
```

知识拓展：可选参数的装饰器：

```
import functools
def smart_decorator(arg=None):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            if arg:
                print(f"Arg: {arg}")
            return func(*args, **kwargs)
        return wrapper
    # 判断是否直接作为装饰器使用
    if callable(arg):
        return decorator(arg)
# 无参数使用
@smart_decorator
def func1():
    print("func1")
# 有参数使用
@smart_decorator(arg="custom")
def func2():
    print("func2")
func1()
# func1
func2()
# Arg: custom\nfunc2
```

12 • 单项选择

以下代码的输出是什么？

```
def repeat(n):
    def decorator(func):
        def wrapper(*args):
            for _ in range(n):
                func(*args)
        return wrapper
    return decorator
@repeat(3)
```

```
def say(msg):
    print(msg)
say("Hello")
```

A. Hello

B. Hello Hello Hello

C. 3

D. 报错

答案 B. Hello Hello Hello

解析 带参数的装饰器 @repeat(3) 使函数执行3次，每次调用都打印"Hello"。

示例：

```
def repeat(n):
    def decorator(func):
        def wrapper(*args):
            for _ in range(n):
                func(*args)
        return wrapper
    return decorator
@repeat(3)
def say(msg):
    print(msg)
say("Hello")
# 输出：
# Hello
```

知识拓展：带返回值的重复装饰器

```
def repeat_with_return(n):
    def decorator(func):
        def wrapper(*args, **kwargs):
            result = None
            for i in range(n):
                result = func(*args, **kwargs)
                if i < n - 1:
                print(f"第{i+1}次执行")
            return result
        # 返回最后一次的结果
        return wrapper
    return decorator
@repeat_with_return(3)
def calculate(x, y):
    result = x + y
    print(f"计算结果: {result}")
    return result
final = calculate(5, 3)
print(f"最终返回: {final}")
# 第1次执行
# 计算结果: 8
# 第2次执行
# 计算结果: 8
# 最终返回: 8
```

13 • 单项选择

关于 functools.wraps ，以下说法正确的是：

A. 用于创建装饰器
B. 用于保留原函数的元数据
C. 用于包装函数
D. 用于执行函数

答案 B. 用于保留原函数的元数据

解析 functools.wraps 用于保留原函数的元数据（如函数名、文档字符串等）。

示例：

```
import functools
# 不使用wraps
def decorator_without_wraps(func):
    def wrapper(*args, **kwargs):
        """wrapper的文档字符串"""
        print("装饰器执行")
        return func(*args, **kwargs)
    return wrapper
# 使用wraps
def decorator_with_wraps(func):
    @functools.wraps(func)
    # 保留原函数元数据
    def wrapper(*args, **kwargs):
        """wrapper的文档字符串"""
        print("装饰器执行")
        return func(*args, **kwargs)
    return wrapper
@decorator_without_wraps
def func1():
    """func1的文档字符串"""
    pass
@decorator_with_wraps
def func2():
    """func2的文档字符串"""
    pass
print(f"func1.__name__: {func1.__name__}")
# wrapper
print(f"func1.__doc__: {func1.__doc__}")
# wrapper的文档字符串
print(f"func2.__name__: {func2.__name__}")
# func2
print(f"func2.__doc__: {func2.__doc__}")
# func2的文档字符串
知识拓展：wraps保留的元数据
import functools
def my_decorator(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper
@my_decorator
def example_func(x, y):
```

```
    """这是一个示例函数"""
    return x + y
# wraps保留了这些属性：
print(f"__name__: {example_func.__name__}")
# example_func
print(f"__doc__: {example_func.__doc__}")
# 这是一个示例函数
print(f"__module__: {example_func.__module__}")
# __main__
print(f"__annotations__: {example_func.__annotations__}")
# {}
print(f"__qualname__: {example_func.__qualname__}")
# example_func
```

14 • 单项选择

以下代码的输出是什么？

```
def func(a, b=2, *args, **kwargs):
    return (a, b, args, kwargs)
result = func(1, 3, 4, 5, x=6, y=7)
print(len(result[2]))
```

A. 2

B. 3

C. 4

D. 5

答案 A. 2

解析 *args 接收位置参数 (4, 5) ，因此 args 是一个包含2个元素的元组，长度为2。

示例：

```
def func(a, b=2, *args, **kwargs):
    return (a, b, args, kwargs)
result = func(1, 3, 4, 5, x=6, y=7)
print(result)
# (1, 3, (4, 5), {'x': 6, 'y': 7})
print(f"a = {result[0]}")
# 1
print(f"b = {result[1]}")
# 3
print(f"args = {result[2]}")
# (4, 5)
print(f"kwargs = {result[3]}")
# {'x': 6, 'y': 7}
print(f"len(args) = {len(result[2])}")
# 2
```

知识拓展：参数解包详解

```
# 各种参数类型
def full_params(a, b=2, *args, c, d=4, **kwargs):
    print(f"必需参数 a: {a}")
    print(f"默认参数 b: {b}")
    print(f"可变位置参数 *args: {args}")
    print(f"keyword-only参数 c: {c}")
```

```
    print(f"keyword-only默认参数 d: {d}")
    print(f"可变关键字参数 **kwargs: {kwargs}")
# 调用示例
full_params(1, 2, 3, 4, c=5, d=6, x=7, y=8)
# 必需参数 a: 1
# 默认参数 b: 2
# 可变位置参数 *args: (3, 4)
# keyword-only参数 c: 5
# keyword-only默认参数 d: 6
# 可变关键字参数 **kwargs: {'x': 7, 'y': 8}
# 参数解包
args_tuple = (10, 20, 30)
kwargs_dict = {'c': 50, 'e': 60}
full_params(*args_tuple, **kwargs_dict)
# 必需参数 a: 10
# 默认参数 b: 20
# 可变位置参数 *args: (30,)
# keyword-only参数 c: 50
# keyword-only默认参数 d: 4
# 可变关键字参数 **kwargs: {'e': 60}
```

15 • 单项选择

关于关键字参数，以下说法错误的是：

A. 可以指定参数名传递

B. 必须在位置参数之后

C. 可以有默认值

D. 必须按顺序传递

答案 D. 必须按顺序传递

解析 关键字参数不必按顺序传递，可以按任意顺序指定参数名。

示例：

```
def greet(name, age, city):
    return f"{name}, {age}岁, 来自{city}"
# 按顺序传递
result1 = greet("Alice", 25, "北京")
print(result1)
# Alice, 25岁, 来自北京
# 使用关键字参数，不按顺序
result2 = greet(city="上海", name="Bob", age=30)
print(result2)
# Bob, 30岁, 来自上海
# 混合使用（位置参数必须在关键字参数前）
result3 = greet("Charlie", city="广州", age=28)
print(result3)
# Charlie, 28岁, 来自广州
知识拓展：仅限关键字参数
# 使用*强制后面的参数必须用关键字传递
def create_user(name, *, age, email, active=True):
    return {
        'name': name,
        'age': age,
```

```
        'email': email,
        'active': active
    }
# 正确调用
user1 = create_user("Alice", age=25, email="alice@example.com")
print(user1)
# 错误调用（会报错）
try:
    user2 = create_user("Bob", 30, "bob@example.com")
except TypeError as e:
    print(f"错误: {e}")
# 错误: create_user() takes 1 positional argument but 3 were given
# Python 3.8+ 仅限位置参数
def divide(a, b, /):
    """/ 前面的参数只能按位置传递"""
    return a / b
print(divide(10, 2))
# 5.0
# 错误调用
try:
    print(divide(a=10, b=2))
except TypeError as e:
    print(f"错误: {e}")
```

16 • 单项选择

Pandas中，以下哪个方法用于数据去重？

A. unique()
B. drop_duplicates()
C. remove_duplicates()
D. distinct()

答案 B. drop_duplicates()

解析 drop_duplicates() 方法用于删除DataFrame中的重复行。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 2, 3, 3, 3],
    'B': [10, 20, 20, 30, 30, 40]
})
print("原始数据：")
print(df)
# A B
# 0 1 10
# 1 2 20
# 2 2 20
# 3 3 30
# 4 3 30
# 5 3 40
# 删除完全重复的行
df_no_dup = df.drop_duplicates()
print("\n删除重复行后：")
```

```
print(df_no_dup)
# A B
# 0 1 10
# 1 2 20
# 3 3 30
# 5 3 40
# 基于特定列去重
df_no_dup_A = df.drop_duplicates(subset=['A'])
print("\n基于列A去重：")
print(df_no_dup_A)
# A B
# 0 1 10
# 1 2 20
# 3 3 30
# 保留最后一个重复项
df_keep_last = df.drop_duplicates(keep='last')
print("\n保留最后一个重复项：")
print(df_keep_last)
知识拓展：去重的高级用法
import pandas as pd
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Alice', 'Charlie'],
    'age': [25, 30, 25, 35],
    'score': [85, 90, 85, 88]
})
# keep参数：'first'（默认）、'last'、False
print(df.drop_duplicates(keep='first'))
# 保留第一个
print(df.drop_duplicates(keep='last'))
# 保留最后一个
print(df.drop_duplicates(keep=False))
# 删除所有重复
# inplace参数：直接修改原DataFrame
df.drop_duplicates(inplace=True)
# 查找重复行
duplicated = df.duplicated()
print(f"重复行：\n{duplicated}")
# 统计重复次数
from collections import Counter
dup_count = df['name'].value_counts()
print(f"\n各值出现次数：\n{dup_count}")
```

17 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 2, 3]})
print(len(df['A'].unique()))
```

A. 2

B. 3

C. 4

D. 1

答案 B. 3

解析 unique() 返回Series中的唯一值数组，[1, 2, 3] 的长度为3。

示例：

```
import pandas as pd
s = pd.Series([1, 2, 2, 3, 3, 3])
# 获取唯一值
unique_values = s.unique()
print(f"唯一值: {unique_values}")
# [1 2 3]
print(f"长度: {len(unique_values)}")
# 3
# nunique()直接返回唯一值的数量
print(f"唯一值数量: {s.nunique()}")
# 3
# value_counts()统计每个值的出现次数
print("\n值计数：")
print(s.value_counts())
# 3 3
# 2 2
# 1 1
知识拓展：unique vs nunique vs value_counts
import pandas as pd
import numpy as np
data = pd.Series([1, 2, 2, 3, 3, 3, np.nan, np.nan])
# unique(): 返回唯一值数组（包含NaN）
print(f"unique: {data.unique()}")
# [1.0 2.0 3.0 nan]
# nunique(): 返回唯一值数量（默认不计NaN）
print(f"nunique (不计NaN): {data.nunique()}")
# 3
print(f"nunique (计NaN): {data.nunique(dropna=False)}")
# 4
# value_counts(): 返回每个值的计数（默认不计NaN）
print("\nvalue_counts:")
print(data.value_counts())
# 3.0 3
# 2.0 2
# 1.0 1
print("\nvalue_counts (计NaN):")
print(data.value_counts(dropna=False))
# 3.0 3
# NaN 2
# 2.0 2
# 1.0 1
# DataFrame的unique操作
df = pd.DataFrame({
    'A': [1, 2, 2, 3],
    'B': [4, 5, 5, 6]
})
print("\nDataFrame唯一值：")
```

```
for col in df.columns:
    print(f"{col}: {df[col].unique()}")
```

18 • 单项选择

Pandas中， axis=0 和 axis=1 分别表示：

A. 行和列
B. 列和行
C. 横向和纵向
D. 上下和左右

答案 A. 行和列

解析 在Pandas中， axis=0 表示沿着行的方向（列操作）， axis=1 表示沿着列的方向（行操作）。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
})
print("原始DataFrame：")
print(df)
# A B C
# 0 1 4 7
# 1 2 5 8
# 2 3 6 9
# axis=0: 沿着行方向，对每列进行操作
print("\naxis=0 (对每列求和):")
print(df.sum(axis=0))
# A 6
# B 15
# C 24
# axis=1: 沿着列方向，对每行进行操作
print("\naxis=1 (对每行求和):")
print(df.sum(axis=1))
# 0 12
# 1 15
# 2 18
知识拓展：axis参数的记忆技巧和应用
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})
# 记忆技巧：axis=0是"行索引"方向，操作结果是列
# axis=1是"列索引"方向，操作结果是行
# 各种操作的axis参数
print("mean (axis=0):", df.mean(axis=0))
# 每列的平均值
print("mean (axis=1):", df.mean(axis=1))
# 每行的平均值
```

```
print("\nmax (axis=0):", df.max(axis=0))
# 每列的最大值
print("max (axis=1):", df.max(axis=1))
# 每行的最大值
# 删除操作
df2 = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6],
    'C': [7, 8, 9]
})
# axis=0: 删除行
df_drop_row = df2.drop(0, axis=0)
print("\n删除第0行：")
print(df_drop_row)
# axis=1: 删除列
df_drop_col = df2.drop('B', axis=1)
print("\n删除B列：")
print(df_drop_col)
# 拼接操作
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
print("\naxis=0拼接（垂直）：")
print(pd.concat([df1, df2], axis=0))
print("\naxis=1拼接（水平）：")
print(pd.concat([df1, df2], axis=1))
```

19 • 单项选择

关于Pandas的 concat() 方法，以下说法正确的是：

A. 只能纵向拼接

B. 只能横向拼接

C. 可以指定拼接方向

D. 不能拼接多个DataFrame

答案 C. 可以指定拼接方向

解析 concat() 函数可以通过 axis 参数控制拼接方向： axis=0 垂直拼接， axis=1 水平拼接。

示例：

```
import pandas as pd
df1 = pd.DataFrame({
    'A': [1, 2],
    'B': [3, 4]
})
df2 = pd.DataFrame({
    'A': [5, 6],
    'B': [7, 8]
})
# axis=0: 垂直拼接（默认）
result_vertical = pd.concat([df1, df2], axis=0)
print("垂直拼接 (axis=0):")
print(result_vertical)
# A B
```

```
# 0 1 3
# 1 2 4
# 0 5 7
# 1 6 8
# axis=1: 水平拼接
result_horizontal = pd.concat([df1, df2], axis=1)
print("\n水平拼接 (axis=1):")
print(result_horizontal)
# A B A B
# 0 1 3 5 7
# 1 2 4 6 8
# 重置索引
result_reset = pd.concat([df1, df2], axis=0, ignore_index=True)
print("\n重置索引后:")
print(result_reset)
# A B
# 0 1 3
# 1 2 4
# 2 5 7
# 3 6 8
知识拓展：concat的高级用法
import pandas as pd
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'B': [5, 6], 'C': [7, 8]})
# join参数：inner（交集）或outer（并集，默认）
print("inner join:")
print(pd.concat([df1, df2], join='inner'))
# B
# 0 3
# 1 4
# 0 5
# 1 6
print("\nouter join (默认):")
print(pd.concat([df1, df2], join='outer'))
# A B C
# 0 1.0 3.0 NaN
# 1 2.0 4.0 NaN
# 0 NaN 5.0 7.0
# 1 NaN 6.0 8.0
# keys参数：添加层次化索引
result = pd.concat([df1, df2], keys=['df1', 'df2'])
print("\n带keys的拼接:")
print(result)
# Series的拼接
s1 = pd.Series([1, 2, 3])
s2 = pd.Series([4, 5, 6])
print("\nSeries拼接:")
print(pd.concat([s1, s2]))
# 0 1
# 1 2
# 2 3
```

```
# 0 4
# 1 5
# 2 6
```

20 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
s1 = pd.Series([1, 2, 3])
s2 = pd.Series([4, 5, 6])
result = pd.concat([s1, s2])
print(len(result))
```

A. 2

B. 3

C. 6

D. 9

答案 C. 6

解析 两个长度为3的Series拼接后，总长度为6。

示例：

```
import pandas as pd
s1 = pd.Series([1, 2, 3])
s2 = pd.Series([4, 5, 6])
# 拼接Series
result = pd.concat([s1, s2])
print(result)
# 0 1
# 1 2
# 2 3
# 0 4
# 1 5
# 2 6
print(f"\n拼接后长度: {len(result)}")
# 6
# 重置索引
result_reset = pd.concat([s1, s2], ignore_index=True)
print("\n重置索引后:")
print(result_reset)
# 0 1
# 1 2
# 2 3
# 3 4
# 4 5
# 5 6
```

知识拓展：Series拼接的多种方式

```
import pandas as pd
s1 = pd.Series([1, 2, 3], index=['a', 'b', 'c'])
s2 = pd.Series([4, 5, 6], index=['d', 'e', 'f'])
# 方法1: pd.concat()
result1 = pd.concat([s1, s2])
print("concat方式:")
```

```
print(result1)
# 方法2: append() (已弃用，但可能在旧代码中见到)
# result2 = s1.append(s2) # Deprecated
# 方法3: 使用list合并再创建
result3 = pd.Series(list(s1) + list(s2))
print("\nlist合并方式:")
print(result3)
# 多个Series拼接
s3 = pd.Series([7, 8, 9])
result_multi = pd.concat([s1, s2, s3], ignore_index=True)
print("\n多个Series拼接:")
print(result_multi)
# 0 1
# 1 2
# 2 3
# 3 4
# 4 5
# 5 6
# 6 7
# 7 8
# 8 9
```

21 • 单项选择

pathlib.Path 对象的 absolute() 和 resolve() 的区别是：

A. 完全相同
B. resolve() 会解析符号链接
C. absolute() 会解析符号链接
D. 两者都不解析符号链接

答案 B. resolve() 会解析符号链接

解析 resolve() 会解析符号链接并返回绝对路径，而 absolute() 只是返回绝对路径但不解析符号链接。

示例：

```
from pathlib import Path
# 相对路径
p = Path('docs/../config.ini')
print(f"原始路径: {p}")
print(f"absolute(): {p.absolute()}")
print(f"resolve(): {p.resolve()}")
```

知识拓展：路径解析的各种方法详见第1-3套相关题目。

22 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('.')
print(p.is_dir())
```

A. False
B. True
C. None

D. 报错

答案 B. True

解析 Path('.') 表示当前目录， is_dir() 返回True。

23 • 单项选择

Path.home() 方法返回的是：

A. 当前目录
B. 用户主目录
C. 根目录
D. 临时目录

答案 B. 用户主目录

解析 Path.home() 返回当前用户的主目录。

24 • 单项选择

关于 Path.match() 方法，以下说法正确的是：

A. 用于正则表达式匹配
B. 用于glob模式匹配
C. 用于精确匹配
D. 用于模糊匹配

答案 B. 用于glob模式匹配

解析 match() 方法用于glob模式匹配。

25 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('/a/b/c.txt')
print(len(p.parents))
```

A. 1
B. 2
C. 3
D. 4

答案 C. 3

解析 /a/b/c.txt 有3个父目录： /a/b 、 /a 和 / 。

26 • 单项选择

关于 sorted() 函数，以下说法正确的是：

A. 会修改原列表
B. 返回新列表
C. 不能指定排序键
D. 只能升序排序

答案 B. 返回新列表

解析 sorted() 函数返回一个新的排序列表，不修改原列表。

27 • 单项选择

以下代码的输出是什么？

```
data = [('a', 3), ('b', 1), ('c', 2)]
result = sorted(data, key=lambda x: x[1])
print(result[0])
```

A. ('a', 3)
B. ('b', 1)
C. ('c', 2)
D. 1

答案 B. ('b', 1)

解析 按照每个元素的第二个值排序，最小的是 ('b', 1) 。

28 • 单项选择

关于集合推导式，以下说法正确的是：

A. 使用 []
B. 使用 {}
C. 使用 ()
D. 使用 <>

答案 B. 使用 {}

解析 集合推导式使用大括号 {} 。

29 • 单项选择

以下代码的输出是什么？

```
result = {x % 3 for x in range(10)}
print(len(result))
```

A. 3
B. 10
C. 5
D. 7

答案 A. 3

解析 x % 3 对于0-9的结果只能是0、1、2，集合去重后有3个元素。

30 • 单项选择

关于 itertools 模块，以下说法正确的是：

A. 提供迭代器工具
B. 包含 chain() 、 cycle() 等函数
C. 用于高效循环
D. 以上都正确

答案 D. 以上都正确

解析 itertools 模块提供了各种迭代器工具，所有选项都正确。
（题21-30的详细示例代码和知识拓展可参考第1-3套相关题目的详细解析）
（这些题目的概念在第1-3套中已有详细解释，可参考相关套题的详细示例）

二、不定项选择题

31 • 不定项选择

关于Python的组合和继承，以下说法正确的有：

A. 组合是"has-a"关系
B. 继承是"is-a"关系
C. 组合比继承更灵活
D. 应该优先使用继承

答案 A. 组合是"has-a"关系；B. 继承是"is-a"关系；C. 组合比继承更灵活

解析 应该优先使用组合而非继承（Composition over Inheritance）是面向对象设计的重要原则。

示例：

```
# 继承方式（不推荐）
class Vehicle:
    def move(self):
        pass
class FlyingVehicle(Vehicle):
    def fly(self):
        return "飞行"
class SwimmingVehicle(Vehicle):
    def swim(self):
        return "游泳"
class FlyingSwimmingVehicle(FlyingVehicle, SwimmingVehicle):
    # 多重继承复杂，难以维护
    pass
# 组合方式（推荐）
class FlyingAbility:
    def fly(self):
        return "飞行"
class SwimmingAbility:
    def swim(self):
        return "游泳"
class Vehicle:
    def __init__(self):
        self.abilities = []
    def add_ability(self, ability):
        self.abilities.append(ability)
    def perform_abilities(self):
        for ability in self.abilities:
            if hasattr(ability, 'fly'):
                print(ability.fly())
            if hasattr(ability, 'swim'):
                print(ability.swim())
# 灵活组合
vehicle = Vehicle()
vehicle.add_ability(FlyingAbility())
vehicle.add_ability(SwimmingAbility())
vehicle.perform_abilities()
```

知识拓展：组合vs继承的优缺点：

- 组合优点：更灵活、更好的封装、避免类层次爆炸
- 继承优点：代码复用、is-a关系明确

2-5. 答案：

答案：ABCD ABCD ABCD ABCD

这些题目涉及魔法方法、生成器特性、迭代器创建方式等，在第1-3套中已有详细解释，可参考相关套题。

核心要点：

- 魔法方法：__str__ 、 __repr__ 、 __len__ 、 __getitem__ 等
- 生成器：惰性求值、节省内存、只能遍历一次
- 迭代器：实现 __iter__ 和 __next__ 、生成器表达式、iter()函数

32 • 不定项选择

以下哪些是Python的数据模型方法（魔法方法）：

A. __add__()
B. __mul__()
C. __contains__()
D. __missing__()

答案 A. __add__()；B. __mul__()；C. __contains__()；D. __missing__()

解析 （原卷合并作答）

33 • 不定项选择

关于Python的属性访问，以下说法正确的有：

A. __getattr__() 在属性不存在时调用
B. __setattr__() 在设置属性时调用
C. __delattr__() 在删除属性时调用
D. __getattribute__() 在访问任何属性时调用

答案 A. __getattr__() 在属性不存在时调用；B. __setattr__() 在设置属性时调用；C. __delattr__() 在删除属性时调用；D. __getattribute__() 在访问任何属性时调用

解析 （原卷合并作答）

34 • 不定项选择

关于生成器的特性，以下说法正确的有：

A. 惰性计算
B. 只能迭代一次
C. 节省内存
D. 可以表示无限序列

答案 A. 惰性计算；B. 只能迭代一次；C. 节省内存；D. 可以表示无限序列

解析 （原卷合并作答）

35 • 不定项选择

以下哪些是创建迭代器的方法：

A. 实现 __iter__() 和 __next__()
B. 使用生成器函数
C. 使用生成器表达式
D. 使用 iter() 函数

答案 A. 实现 __iter__() 和 __next__()；B. 使用生成器函数；C. 使用生成器表达式；D. 使用 iter() 函数

解析 （原卷合并作答）

36 • 不定项选择

关于装饰器的高级用法，以下说法正确的有：

A. 可以装饰类
B. 可以叠加使用
C. 可以带参数
D. 可以访问被装饰函数的属性

答案 A. 可以装饰类；B. 可以叠加使用；C. 可以带参数；D. 可以访问被装饰函数的属性

解析 装饰器的所有选项都是正确的应用场景。
（详细示例可参考第1-3套装饰器相关题目）

37 • 不定项选择

关于闭包的应用场景，以下说法正确的有：

A. 实现装饰器
B. 实现回调函数
C. 保存状态
D. 实现工厂函数

答案 A. 实现装饰器；B. 实现回调函数；C. 保存状态；D. 实现工厂函数

解析 所有选项都是闭包的常见应用场景。
（详细示例可参考第1-3套闭包相关题目）

38 • 不定项选择

以下哪些是Pandas的数据类型：

A. int64
B. float64
C. object
D. datetime64

答案 A. int64；B. float64；C. object；D. datetime64

解析 所有选项都是Pandas中的数据类型。
（详细示例可参考第1-3套Pandas相关题目）

39 • 不定项选择

关于Pandas的数据选择，以下说法正确的有：

A. df['col'] 选择列
B. df[0] 选择第一行
C. df[0:3] 选择前三行
D. df.col 选择列

答案 A. df['col'] 选择列；C. df[0:3] 选择前三行；D. df.col 选择列

解析 df[0] 会报错，应该使用 df.iloc[0] 或 df.loc[0] 。
（详细示例可参考第1-3套Pandas索引相关题目）

40 • 不定项选择

Pandas中，以下哪些方法用于数据转换：

A. astype()
B. apply()
C. map()
D. transform()

答案 A. astype()；B. apply()；C. map()；D. transform()

解析 所有选项都是数据转换的常用方法。
（详细示例可参考第1-3套Pandas数据转换相关题目）

41 • 不定项选择

关于 pathlib 的文件操作，以下说法正确的有：

A. read_text() 读取文本文件
B. write_text() 写入文本文件
C. read_bytes() 读取二进制文件
D. write_bytes() 写入二进制文件

答案 A. read_text() 读取文本文件；B. write_text() 写入文本文件；C. read_bytes() 读取二进制文件；D. write_bytes() 写入二进制文件

解析 pathlib 的所有选项都是正确的优势。
（详细示例可参考第1-3套pathlib相关题目）

42 • 不定项选择

Path 对象的以下哪些方法会创建文件或目录：

A. mkdir()
B. touch()
C. exists()
D. rename()

答案 A. mkdir()；B. touch()

解析 exists() 用于检查路径是否存在， mkdir() 用于创建目录； rename() 是重命名而非移动文件。
（详细示例可参考第1-3套pathlib相关题目）

43 • 不定项选择

关于推导式的性能，以下说法正确的有：

A. 通常比等价的for循环快
B. 代码更简洁
C. 可读性更好
D. 适合简单的转换操作

答案 A. 通常比等价的for循环快；B. 代码更简洁；D. 适合简单的转换操作

解析 推导式的可读性不一定更好，取决于复杂度。
（详细示例可参考第1-3套推导式相关题目）

44 • 不定项选择

以下哪些是Python的内置迭代工具：

A. enumerate()

B. zip()
C. reversed()
D. range()

答案 A. enumerate()；B. zip()；C. reversed()；D. range()

解析 所有选项都是Python内置的迭代工具。
（详细示例可参考第1-3套函数式编程相关题目）

45 • 不定项选择

关于函数式编程，以下说法正确的有：

A. 避免副作用
B. 使用纯函数
C. 数据不可变
D. Python完全支持函数式编程

答案 A. 避免副作用；B. 使用纯函数；C. 数据不可变

解析 Python支持函数式编程特性，但不是纯函数式语言。
（详细示例可参考第1-3套函数式编程相关题目）

三、填空题

46 • 填空 / 简答

Python中，使用 _____ 可以创建抽象基类，使用 _____ 装饰器标记抽象方法。

答案 abc.ABC （或

解析 （或 ABC ），
），@abstractmethod
抽象基类用于定义接口。
示例：

```
from abc import ABC, abstractmethod
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass
    @abstractmethod
    def move(self):
        pass
class Dog(Animal):
    def make_sound(self):
        return "汪汪"
    def move(self):
        return "跑"
# 无法实例化抽象类
# animal = Animal() # TypeError
dog = Dog()
print(dog.make_sound())
# 汪汪
```

知识拓展：抽象方法强制子类必须实现，否则无法实例化。

47 • 填空 / 简答

生成器的三个主要方法是、和 _____。

答案 __next__() ， send() ， close()

解析 生成器的三个核心方法。

示例：

```
def counter():
    total = 0
    while True:
        increment = yield total
        if increment:
            total += increment
gen = counter()
print(next(gen))
# 0 - __next__()
print(gen.send(5))
# 5 - send()
print(gen.send(10))
# 15
gen.close()
# close()
# print(next(gen)) # StopIteration
```

知识拓展： send() 用于协程， close() 提前终止生成器。

48 • 填空 / 简答

Pandas中，_____ 方法用于数据透视表，_____ 方法用于交叉表。

答案 pivot_table() ， crosstab()

解析 Pandas数据透视表方法。

示例：

```
import pandas as pd
df = pd.DataFrame({
    'date': ['2023-01', '2023-01', '2023-02', '2023-02'],
    'product': ['A', 'B', 'A', 'B'],
    'sales': [100, 200, 150, 250]
})
# pivot_table
pt = df.pivot_table(values='sales',
                    index='date',
                    columns='product')
print(pt)
# crosstab
ct = pd.crosstab(df['date'], df['product'],
                 values=df['sales'], aggfunc='sum')
print(ct)
```

知识拓展： pivot_table() 功能更强， crosstab() 专用于频数统计。

49 • 填空 / 简答

使用 _____ 函数可以对可迭代对象进行排序，使用 _____ 参数可以指定排序键。

答案 sorted() ， key

解析 排序函数和关键字参数。

示例：

```
# 基本排序
data = [3, 1, 4, 1, 5, 9, 2, 6]
print(sorted(data))
# [1, 1, 2, 3, 4, 5, 6, 9]
# 使用key指定排序规则
words = ['apple', 'pie', 'zoo', 'a']
print(sorted(words, key=len))
# ['a', 'pie', 'zoo', 'apple']
# 复杂排序
students = [
    {'name': 'Alice', 'age': 25, 'score': 85},
    {'name': 'Bob', 'age': 23, 'score': 90},
    {'name': 'Charlie', 'age': 25, 'score': 80}
]
# 按分数排序
print(sorted(students, key=lambda x: x['score']))
# 按多个条件排序
print(sorted(students, key=lambda x: (x['age'], -x['score'])))
```

知识拓展： key 参数接受一个函数，用于提取比较键。

50 • 填空 / 简答

pathlib.Path 对象的 _____ 方法返回所有匹配的文件，_____ 方法递归搜索。

答案 glob() ， rglob()

解析 Path对象的文件搜索方法。

示例：

```
from pathlib import Path
project = Path('/home/user/project')
# glob - 非递归搜索
for file in project.glob('*.py'):
    print(file)
# 只搜索当前目录
# rglob - 递归搜索
for file in project.rglob('*.py'):
    print(file)
# 搜索所有子目录
# 复杂模式
for file in project.rglob('**/test_*.py'):
    print(file)
# 找到所有test_开头的py文件
```

知识拓展： rglob('pattern') 等价于 glob('**/pattern') 。

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class Descriptor:
    def __init__(self, name):
        self.name = name
    def __get__(self, instance, owner):
        if instance is None:
            return self
        return instance.__dict__.get(self.name, None)
    def __set__(self, instance, value):
        if value < 0:
            raise ValueError("值不能为负")
        instance.__dict__[self.name] = value
class Product:
    price = Descriptor('price')
    def __init__(self, price):
        self.price = price
p = Product(100)
print(p.price)
p.price = 200
print(p.price)
```

问题：

- (1) 两个 print 语句的输出分别是什么？
- (2) 这段代码实现了什么功能？
- (3) 如果执行 p.price = -10 会发生什么？

答案 描述符

- (1) 输出：100, 200
- (2) 实现了描述符协议，用于属性访问控制和验证
- (3) 会抛出 ValueError 异常，提示"值不能为负"

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
from itertools import islice
def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
fib = fibonacci()
result = list(islice(fib, 10))
print(result)
print(next(fib))
```

问题：

- (1) result 的内容是什么？
- (2) 最后一个 print 输出什么？
- (3) 为什么这个生成器可以表示无限序列？

答案 • (1) [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
• (2) 输出：55（第11个斐波那契数）
• (3) 使用 while True 无限循环，通过 yield 按需生成值

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'date': pd.date_range('2024-01-01', periods=5),
    'value': [10, 20, 30, 40, 50]
})
df['date'] = pd.to_datetime(df['date'])
df.set_index('date', inplace=True)
result = df.resample('2D').sum()
print(result)
```

问题：

• (1) resample('2D') 的作用是什么？
• (2) result 会有几行数据？
• (3) 为什么需要先设置索引为日期？

答案 • (1) resample('2D') 按2天为间隔重新采样数据
• (2) 3行（5天按2天分组：第1-2天、第3-4天、第5天）
• (3) 时间序列操作需要DatetimeIndex

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
from pathlib import Path
def backup_files(source, backup_dir, extensions):
    source_path = Path(source)
    backup_path = Path(backup_dir)
    backup_path.mkdir(parents=True, exist_ok=True)
    for ext in extensions:
        for file in source_path.glob(f'*.{ext}'):
            dest = backup_path / file.name
            dest.write_bytes(file.read_bytes())
            yield file.name
files = list(backup_files('.', 'backup', ['py', 'txt']))
print(f"备份了{len(files)} 个文件")
```

问题：

• (1) 这个函数备份了哪些文件？
• (2) 为什么使用 read_bytes() 和 write_bytes() ？
• (3) 使用生成器的好处是什么？

答案 • (1) 备份当前目录下所有.py和.txt文件
• (2) 使用字节模式可以处理任何文件类型，包括二进制文件

- (3) 生成器可以逐个处理文件，节省内存，并提供进度反馈

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
from functools import lru_cache
@lru_cache(maxsize=None)
def expensive_function(n):
    print(f"计算{n}")
    return n ** 2
print(expensive_function(5))
print(expensive_function(10))
print(expensive_function.cache_info())
```

问题：

- (1) "计算 5"会打印几次？
- (2) lru_cache 的作用是什么？
- (3) maxsize=None 表示什么？

答案 缓存

- (1) 只打印1次（第二次从缓存获取）
- (2) lru_cache 实现最近最少使用缓存，避免重复计算
- (3) maxsize=None 表示缓存大小无限制

解析

五、程序设计题

56 • 填空 / 简答

设计一个购物车系统，包含以下类：

Product类：

类：

- 属性：名称、价格、库存
- 方法： __str__() 、 __eq__() （按名称比较）

ShoppingCart类：

类：

- 属性：商品字典（商品：数量）
- 方法：

add_item(product, quantity) ：添加商品

remove_item(product) ：移除商品

update_quantity(product, quantity) ：更新数量

total_price ：使用 @property 计算总价

__len__() ：返回商品种类数

__iter__() ：使购物车可迭代

要求：

- 检查库存是否足够
- 使用列表推导式筛选价格大于某值的商品
- 实现上下文管理器（ __enter__ 和 __exit__ ）

答案 class Product:
    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
        self.stock = stock
    def __str__(self):
        return f"{self.name} - ¥{self.price} (库存: {self.stock})"
    def __eq__(self, other):
        if isinstance(other, Product):
            return self.name == other.name
        return False
    def __hash__(self):
        return hash(self.name)
class ShoppingCart:
    def __init__(self):
        self.items = {}
    def __enter__(self):
        print("开始购物")
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        print("结束购物")
        return False
    def add_item(self, product, quantity):
        if quantity > product.stock:
            raise ValueError(f"{product.name} 库存不足")
        if product in self.items:
            self.items[product] += quantity
        else:
            self.items[product] = quantity
        product.stock -= quantity
    print(f"已添加{quantity} 个{product.name}")
def remove_item(self, product):
    if product in self.items:
        quantity = self.items[product]
        product.stock += quantity
        del self.items[product]
        print(f"已移除{product.name}")
def update_quantity(self, product, quantity):
    if product in self.items:
        old_quantity = self.items[product]
        diff = quantity - old_quantity
        if diff > product.stock:
            raise ValueError(f"{product.name} 库存不足")
        self.items[product] = quantity
        product.stock -= diff
@property
def total_price(self):
    return sum(product.price * quantity

```
            for product, quantity in self.items.items())
def __len__(self):
    return len(self.items)
def __iter__(self):
    return iter(self.items.items())
def filter_by_price(self, min_price):
    return [product for product, qty in self.items.items()
            if product.price > min_price]
# 使用示例
p1 = Product("Python书籍", 89, 10)
p2 = Product("数据分析", 79, 5)
p3 = Product("机器学习", 99, 8)
with ShoppingCart() as cart:
    cart.add_item(p1, 2)
    cart.add_item(p2, 1)
    cart.add_item(p3, 1)
    print(f"\n购物车商品数: {len(cart)}")
    print(f"总价: ¥{cart.total_price}")
    print("\n购物车内容:")
    for product, quantity in cart:
        print(f"{product.name} x {quantity}")
    expensive = cart.filter_by_price(80)
    print(f"\n价格大于80的商品: {[p.name for p in expensive]}")
```

解析

57 • 填空 / 简答

编写一个配置文件管理器，要求：

1. ConfigManager 类：

- 使用单例模式
- 从JSON/YAML文件加载配置
- 支持嵌套配置访问（如 config.database.host ）
- 使用 @property 实现配置项访问

2. 装饰器 @require_config(key) ：

- 检查配置项是否存在
- 不存在则抛出异常

3. 使用 pathlib 处理配置文件路径

4. 实现配置热加载（文件修改时自动重新加载）

答案

```
import json
from pathlib import Path
from functools import wraps
import time
class ConfigManager:
    _instance = None
    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

```
    def __init__(self, config_file=None):
        if not hasattr(self, '_initialized'):
            self._config = {}
            self._config_file = None
            self._last_modified = None
            if config_file:
                self.load_config(config_file)
            self._initialized = True
    def load_config(self, config_file):
        path = Path(config_file)
        if not path.exists():
            raise FileNotFoundError(f"配置文件{config_file} 不存在")
        with path.open('r', encoding='utf-8') as f:
            self._config = json.load(f)
        self._config_file = path
        self._last_modified = path.stat().st_mtime
        print(f"已加载配置: {config_file}")
    def reload_if_modified(self):
        if self._config_file:
            current_mtime = self._config_file.stat().st_mtime
            if current_mtime > self._last_modified:
                print("检测到配置文件变化，重新加载...")
                self.load_config(self._config_file)
    def get(self, key, default=None):
        self.reload_if_modified()
        keys = key.split('.')
        value = self._config
        for k in keys:
            if isinstance(value, dict) and k in value:
                value = value[k]
            else:
                return default
        return value
    @property
    def config(self):
        self.reload_if_modified()
        return self._config
def require_config(key):
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            config = ConfigManager()
            if config.get(key) is None:
                raise ValueError(f"缺少配置项: {key}")
            return func(*args, **kwargs)
        return wrapper
    return decorator
# 使用示例
```

```
config_data = {
    "database": {
        "host": "localhost",
        "port": 5432,
        "name": "mydb"
    },
    "app": {
        "debug": True,
        "version": "1.0"
    }
# 创建配置文件
config_file = Path('config.json')
config_file.write_text(json.dumps(config_data, indent=2))
# 使用配置管理器
config = ConfigManager('config.json')
print(f"数据库主机: {config.get('database.host')}")
print(f"应用版本: {config.get('app.version')}")
@require_config('database.host')
def connect_database():
    config = ConfigManager()
    host = config.get('database.host')
    print(f"连接到数据库: {host}")
connect_database()
```

解析

58 • 填空 / 简答

使用Pandas分析时间序列数据：

创建一个月的销售数据（日期、产品、销量、金额）：

- 按周统计销售额
- 计算移动平均（7天窗口）
- 找出销售高峰日
- 按产品分组计算增长率
- 使用 pivot_table() 创建产品-日期透视表
- 处理缺失日期（填充为0）
- 绘制销售趋势图（使用matplotlib）
- 导出分析报告

答案

```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import matplotlib.pyplot as plt
# 创建一个月的销售数据
np.random.seed(42)
dates = pd.date_range('2024-01-01', periods=30, freq='D')
products = ['产品A', '产品B', '产品C']
data = []
for date in dates:
    for product in products:
```

```
        sales = np.random.randint(50, 200)
        amount = sales * np.random.uniform(10, 50)
        data.append({
            '日期': date,
            '产品': product,
            '销量': sales,
            '金额': amount
        })
df = pd.DataFrame(data)
df.set_index('日期', inplace=True)
print("原始数据预览:")
print(df.head(10))
# 按周统计销售额
weekly_sales = df.groupby([pd.Grouper(freq='W'), '产品'])['金额'].sum()
print("\n按周统计销售额:")
print(weekly_sales)
# 计算移动平均（7天窗口）
df_sorted = df.sort_index()
df_sorted['移动平均'] = df_sorted.groupby('产品')['金额'].transform(
    lambda x: x.rolling(window=7, min_periods=1).mean()
)
print("\n移动平均数据:")
print(df_sorted.head(10))
# 找出销售高峰日
daily_total = df.groupby(df.index)['金额'].sum()
peak_day = daily_total.idxmax()
peak_amount = daily_total.max()
print(f"\n销售高峰日: {peak_day.date()}, 金额: {peak_amount:.2f}")
# 按产品计算增长率
product_growth = df.groupby('产品')['金额'].apply(
    lambda x: (x.iloc[-7:].mean() - x.iloc[:7].mean()) / x.iloc[:7].mean() * 100
)
print("\n产品增长率（最后7天 vs 前7天）:")
print(product_growth)
# 创建数据透视表
pivot = df.pivot_table(
    values='金额',
    index=df.index.date,
    columns='产品',
    aggfunc='sum',
    fill_value=0
)
print("\n产品-日期透视表:")
print(pivot.head())
# 绘制销售趋势图
plt.figure(figsize=(12, 6))
for product in products:
    product_data = df[df['产品'] == product].groupby(df.index)['金额'].sum()
```

```
    plt.plot(product_data.index, product_data.values, label=product, marker='o')
plt.title('销售趋势图')
plt.xlabel('日期')
plt.ylabel('销售金额')
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig('sales_trend.png')
print("\n趋势图已保存为 sales_trend.png")
# 导出分析报告
with pd.ExcelWriter('sales_report.xlsx') as writer:
    df.to_excel(writer, sheet_name='原始数据')
    weekly_sales.to_excel(writer, sheet_name='周度统计')
    pivot.to_excel(writer, sheet_name='透视表')
print("分析报告已导出为 sales_report.xlsx")
```

解析

59 • 填空 / 简答

编写一个Python代码质量检查工具：

1. analyze_file(filepath) ：

• 统计代码行数、注释行数、空行数

• 检测函数和类的数量

• 计算代码复杂度（简单统计if/for/while数量）

2. find_long_functions(directory, max_lines) ：

• 使用生成器查找超过指定行数的函数

• 返回函数名和行数

3. check_naming_convention(filepath) ：

• 检查变量和函数命名是否符合PEP8

要求：

• 使用 pathlib 遍历目录

• 使用正则表达式解析代码

• 使用装饰器记录分析时间

• 生成HTML格式的报告

答案

```
from pathlib import Path
import re
from functools import wraps
import time
def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} 执行时间: {end-start:.4f}秒")
        return result
    return wrapper
```

```
@timer
def analyze_file(filepath):
    path = Path(filepath)
    if not path.exists():
        return None
    with path.open('r', encoding='utf-8') as f:
        lines = f.readlines()
    total_lines = len(lines)
    code_lines = 0
    comment_lines = 0
    blank_lines = 0
    for line in lines:
        stripped = line.strip()
        if not stripped:
            blank_lines += 1
        elif stripped.startswith('#'):
            comment_lines += 1
        else:
            code_lines += 1
    # 统计函数和类
    content = ''.join(lines)
    functions = len(re.findall(r'^\s*def\s+\w+', content, re.MULTILINE))
    classes = len(re.findall(r'^\s*class\s+\w+', content, re.MULTILINE))
    # 简单复杂度（if/for/while数量）
    complexity = (
        len(re.findall(r'\bif\b', content)) +
        len(re.findall(r'\bfor\b', content)) +
        len(re.findall(r'\bwhile\b', content))
    )
    return {
        'file': filepath,
        'total_lines': total_lines,
        'code_lines': code_lines,
        'comment_lines': comment_lines,
        'blank_lines': blank_lines,
        'functions': functions,
        'classes': classes,
        'complexity': complexity
    }
def find_long_functions(directory, max_lines=50):
    path = Path(directory)
    for py_file in path.rglob('*.py'):
        with py_file.open('r', encoding='utf-8') as f:
            content = f.read()
        # 简单查找函数定义
        pattern = r'def\s+(\w+)\s*\([^)]*\):'
        matches = re.finditer(pattern, content)
        for match in matches:
```

```
            func_name = match.group(1)
            start_pos = match.start()
            # 估算函数行数（简化版）
            func_content = content[start_pos:]
            lines = func_content.split('\n')
            func_lines = 0
            indent_level = None
            for line in lines[1:]:
                if line.strip():
                current_indent = len(line) - len(line.lstrip())
                if indent_level is None:
                indent_level = current_indent
                elif current_indent <= indent_level and line.strip():
                break
                func_lines += 1
            if func_lines > max_lines:
                yield {
                'file': str(py_file),
                'function': func_name,
                'lines': func_lines
                }
def check_naming_convention(filepath):
    path = Path(filepath)
    with path.open('r', encoding='utf-8') as f:
        content = f.read()
    issues = []
    # 检查函数名（应该是snake_case）
    func_pattern = r'def\s+([A-Z]\w+)\s*\('
    for match in re.finditer(func_pattern, content):
        issues.append(f"函数名{match.group(1)} 应使用snake_case")
    # 检查类名（应该是PascalCase）
    class_pattern = r'class\s+([a-z]\w+)\s*[:(]'
    for match in re.finditer(class_pattern, content):
        issues.append(f"类名{match.group(1)} 应使用PascalCase")
    return issues
def generate_html_report(analysis_results, output_file='report.html'):
html = """
<html>
<head>
    <title>代码质量报告</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 20px; }
        table { border-collapse: collapse; width: 100%; }
        th, td { border: 1px solid #ddd; padding: 8px; text-align: left; }
        th { background-color: #4CAF50; color: white; }
        tr:nth-child(even) { background-color: #f2f2f2; }
    </style>
</head>
```

```
<body>
    <h1>代码质量分析报告</h1>
    <table>
        <tr>
            <th>文件</th>
            <th>总行数</th>
            <th>代码行</th>
            <th>注释行</th>
            <th>空行</th>
            <th>函数数</th>
            <th>类数</th>
            <th>复杂度</th>
        </tr>
"""
for result in analysis_results:
    if result:
        html += f"""
        <tr>
            <td>{result['file']}</td>
            <td>{result['total_lines']}</td>
            <td>{result['code_lines']}</td>
            <td>{result['comment_lines']}</td>
            <td>{result['blank_lines']}</td>
            <td>{result['functions']}</td>
            <td>{result['classes']}</td>
            <td>{result['complexity']}</td>
        </tr>
        """
html += """
    </table>
</body>
</html>
"""
Path(output_file).write_text(html, encoding='utf-8')
print(f"HTML报告已生成: {output_file}")
# 使用示例
# results = []
# for py_file in Path('.').rglob('*.py'):
# result = analyze_file(py_file)
# if result:
# results.append(result)
# print(f"\n分析 {py_file}:")
# print(f" 代码行: {result['code_lines']}")
# print(f" 函数数: {result['functions']}")
# print(f" 复杂度: {result['complexity']}")
# generate_html_report(results)
# print("\n查找长函数:")
# for func_info in find_long_functions('.', max_lines=30):
```

```
# print(f" {func_info['function']} in {func_info['file']}: {func_info['lines']} 行")
```

解析

60 • 填空 / 简答

实现一个简单的数据处理管道框架：

1. Pipeline 类：

答案

```
from functools import wraps
import time
def pipeline_step(func):
    @wraps(func)
    def wrapper(data):
        start = time.time()
        input_size = len(list(data)) if hasattr(data, '__iter__') else 1
        result = func(data)
        end = time.time()
        output_size = len(list(result)) if hasattr(result, '__iter__') else 1
        print(f"步骤{func.__name__}:")
        print(f" 输入: {input_size} 项")
        print(f" 输出: {output_size} 项")
        print(f" 耗时: {end-start:.4f}秒")
        return result
    return wrapper
class Pipeline:
    def __init__(self):
        self.steps = []
    def add_step(self, step_func):
        self.steps.append(step_func)
        return self
    def execute(self, data):
        result = data
        for step in self.steps:
            result = step(result)
        return result
def filter_data(condition):
    @pipeline_step
    def filter_step(data):
        for item in data:
            if condition(item):
                yield item
    return filter_step
def transform_data(func):
    @pipeline_step
    def transform_step(data):
        for item in data:
            yield func(item)
    return transform_step
def aggregate_data(func):
```

```
    @pipeline_step
    def aggregate_step(data):
        return func(list(data))
    return aggregate_step
# 使用示例
print("=== 数据管道示例 ===\n")
# 原始数据
data = range(-10, 11)
print(f"原始数据: {list(data)}\n")
# 创建管道
pipeline = Pipeline()
result = (pipeline
    .add_step(filter_data(lambda x: x > 0))
    .add_step(transform_data(lambda x: x * 2))
    .add_step(transform_data(lambda x: x ** 2))
    .execute(data))
print(f"\n最终结果: {result}")
# 另一个示例：数据聚合
print("\n=== 聚合示例 ===\n")
data2 = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
pipeline2 = Pipeline()
total = (pipeline2
    .add_step(filter_data(lambda x: x % 2 == 0))
    .add_step(transform_data(lambda x: x * 3))
    .add_step(aggregate_data(sum))
    .execute(data2))
print(f"\n总和: {total}")
```

评分说明：

- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：
    总分：100分

解析

第 5 套

60 题

一、单项选择题

01 • 单项选择

关于Python的组合（Composition），以下说法正确的是：

A. 组合是通过继承实现的
B. 组合是通过包含其他类的实例实现的
C. 组合不如继承灵活
D. Python不支持组合

答案 B. 组合是通过包含其他类的实例实现的

解析 组合（Composition）是通过在一个类中包含其他类的实例来实现功能复用，而不是通过继承。

示例：

```
# 组合方式
class Engine:
    def start(self):
        return "引擎启动"
    def stop(self):
        return "引擎停止"
class Wheels:
    def rotate(self):
        return "轮子转动"
class Car:
    def __init__(self):
        # 组合：包含其他类的实例
        self.engine = Engine()
        self.wheels = Wheels()
    def start(self):
        return self.engine.start()
    def drive(self):
        return f"{self.engine.start()}, {self.wheels.rotate()}"
car = Car()
print(car.start())
# 引擎启动
print(car.drive())
# 引擎启动, 轮子转动
知识拓展：组合vs继承
# 继承方式（不够灵活）
class Vehicle:
    def move(self):
        return "移动"
class Car(Vehicle):
    def drive(self):
        return "开车"
```

```
# 组合方式（更灵活）
class Engine:
    def run(self):
        return "引擎运行"
class ElectricMotor:
    def run(self):
        return "电机运行"
class ModernCar:
    def __init__(self, power_source):
        self.power = power_source
    # 可以灵活替换
    def drive(self):
        return self.power.run()
# 灵活组合
gas_car = ModernCar(Engine())
electric_car = ModernCar(ElectricMotor())
print(gas_car.drive())
# 引擎运行
print(electric_car.drive()) # 电机运行
```

02 • 单项选择

以下代码的输出是什么？

```
class Engine:
    def start(self):
        return "引擎启动"
class Car:
    def __init__(self):
        self.engine = Engine()
    def start(self):
        return self.engine.start()
car = Car()
print(car.start())
```

A. Car启动
B. 引擎启动
C. None
D. 报错

答案 B. 引擎启动

解析 Car类通过组合包含Engine实例，调用 engine.start() 返回"引擎启动"。

示例：

```
class Engine:
    def start(self):
        return "引擎启动"
class Car:
    def __init__(self):
        self.engine = Engine()
    # 组合
    def start(self):
        return self.engine.start()
# 委托给engine
```

```
car = Car()
print(car.start())
# 引擎启动
知识拓展：组合的优势
# 多个组件组合
class Engine:
    def start(self):
        return "引擎启动"
class Radio:
    def play(self):
        return "播放音乐"
class GPS:
    def navigate(self):
        return "导航中"
class ModernCar:
    def __init__(self):
        self.engine = Engine()
        self.radio = Radio()
        self.gps = GPS()
    def start_journey(self):
        actions = [
            self.engine.start(),
            self.radio.play(),
            self.gps.navigate()
        ]
        return ", ".join(actions)
car = ModernCar()
print(car.start_journey())
# 引擎启动, 播放音乐, 导航中
```

03 • 单项选择

关于 __getattr__ 和 __getattribute__ 的区别，以下说法正确的是：

A. 完全相同

B. __getattr__ 在属性不存在时调用， __getattribute__ 总是调用

C. __getattribute__ 在属性不存在时调用

D. 两者都不常用

答案 B. __getattr__ 在属性不存在时调用， __getattribute__ 总是调用

解析 __getattr__ 只在属性不存在时调用， __getattribute__ 在访问任何属性时都会调用。

示例：

```
class GetAttrExample:
    def __init__(self):
        self.x = 10
    def __getattr__(self, name):
        # 只在属性不存在时调用
        print(f"__getattr__ called for: {name}")
        return f"默认值 for {name}"
obj1 = GetAttrExample()
print(obj1.x)
# 10 - 直接访问，不调用__getattr__
```

```
print(obj1.y)
# __getattr__ called for: y\n默认值 for y
class GetAttributeExample:
    def __init__(self):
        # 必须用object.__setattr__避免无限递归
        object.__setattr__(self, 'x', 10)
    def __getattribute__(self, name):
        # 访问任何属性都调用
        print(f"__getattribute__ called for: {name}")
        return object.__getattribute__(self, name)
obj2 = GetAttributeExample()
print(obj2.x)
# __getattribute__ called for: x\n10
print(obj2.y)
# __getattribute__ called for: y\nAttributeError
知识拓展：避免无限递归
class SafeExample:
    def __init__(self):
        self.data = {}
    def __getattr__(self, name):
        # 安全：只处理不存在的属性
        return self.data.get(name, None)
    def __setattr__(self, name, value):
        # 安全：使用__dict__
        if name == 'data':
            object.__setattr__(self, name, value)
        else:
            self.data[name] = value
obj = SafeExample()
obj.x = 100
print(obj.x)
# 100
print(obj.y)
# None
```

04 • 单项选择

以下哪个魔法方法用于实现对象的哈希值？

A. `__hash__()`

B. `__id__()`

C. `__key__()`

D. `__digest__()`

答案 A. `__hash__()`

解析 `__hash__()` 魔法方法用于计算对象的哈希值，使对象可以用作字典键或集合元素。

示例：

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def __hash__(self):
        # 计算哈希值
```

```
        return hash((self.x, self.y))
    def __eq__(self, other):
        # 必须同时实现__eq__
        return self.x == other.x and self.y == other.y
p1 = Point(1, 2)
p2 = Point(1, 2)
p3 = Point(3, 4)
# 可以用作字典键
point_dict = {p1: "第一个点", p3: "第三个点"}
print(point_dict[p2])
# 第一个点 - p1和p2相等
# 可以用于集合
point_set = {p1, p2, p3}
print(len(point_set))
# 2 - p1和p2被认为相同
知识拓展：哈希的规则
# 规则：如果a == b，则hash(a) == hash(b)
class BadHash:
    def __init__(self, value):
        self.value = value
    def __hash__(self):
        return 1
    # 总是返回相同的哈希值（低效但合法）
    def __eq__(self, other):
        return self.value == other.value
# 不可变对象才应该可哈希
class ImmutablePoint:
    def __init__(self, x, y):
        self._x = x
        self._y = y
    @property
    def x(self):
        return self._x
    @property
    def y(self):
        return self._y
    def __hash__(self):
        return hash((self._x, self._y))
    def __eq__(self, other):
        return self._x == other.x and self._y == other.y
```

05 • 单项选择

关于Python的 `__slots__` ，以下说法正确的是：

A. 用于限制实例属性

B. 可以节省内存

C. 使用后不能动态添加属性

D. 以上都正确

答案 D. 以上都正确

解析 所有选项都是上下文管理器的正确特性。

示例：

```
import time
# A. 使用with语句
class Timer:
    def __enter__(self):
        self.start = time.time()
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.end = time.time()
        print(f"耗时: {self.end - self.start:.2f}秒")
        return False
# 不抑制异常
with Timer():
    sum([i**2 for i in range(1000000)])
# 输出：耗时: 0.XX秒
# B. __enter__和__exit__
class FileManager:
    def __init__(self, filename, mode):
        self.filename = filename
        self.mode = mode
        self.file = None
    def __enter__(self):
        self.file = open(self.filename, self.mode)
        return self.file
    def __exit__(self, exc_type, exc_val, exc_tb):
        if self.file:
            self.file.close()
        return False
# C. 自动资源管理
with FileManager('test.txt', 'w') as f:
    f.write('Hello')
# 文件自动关闭
# D. 可以处理异常
class ErrorHandler:
    def __enter__(self):
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        if exc_type is not None:
            print(f"捕获异常: {exc_type.__name__}: {exc_val}")
            return True
        # 抑制异常
        return False
with ErrorHandler():
    raise ValueError("测试错误")
print("继续执行")
# 异常被抑制，继续执行
知识拓展：contextlib简化上下文管理器
from contextlib import contextmanager
@contextmanager
def timer():
    start = time.time()
    try:
```

```
        yield
    finally:
        end = time.time()
        print(f"耗时: {end - start:.2f}秒")
with timer():
    sum([i**2 for i in range(1000000)])
```

06 • 单项选择

以下代码的输出是什么？

```
class Point:
    __slots__ = ['x', 'y']
    def __init__(self, x, y):
        self.x = x
        self.y = y
p = Point(1, 2)
p.z = 3
```

A. 正常执行
B. 抛出AttributeError
C. z被成功添加
D. 输出3

答案 B. 抛出AttributeError

解析 使用 __slots__ 后，类实例不能动态添加新属性，会抛出 AttributeError 。

示例：

```
# 普通类：可以动态添加属性
class NormalClass:
    def __init__(self, x):
        self.x = x
obj1 = NormalClass(10)
obj1.y = 20
# 可以添加新属性
print(obj1.y)
# 20
# 使用__slots__：不能动态添加属性
class SlotClass:
    __slots__ = ['x']
    # 只允许x属性
    def __init__(self, x):
        self.x = x
obj2 = SlotClass(10)
try:
    obj2.y = 20
# 尝试添加新属性
except AttributeError as e:
    print(f"错误: {e}")
# 错误: 'SlotClass' object has no attribute 'y'
知识拓展： __slots__ 的优势
import sys
class WithoutSlots:
    def __init__(self, x, y):
```

```
        self.x = x
        self.y = y
class WithSlots:
    __slots__ = ['x', 'y']
    def __init__(self, x, y):
        self.x = x
        self.y = y
obj1 = WithoutSlots(1, 2)
obj2 = WithSlots(1, 2)
print(f"Without slots: {sys.getsizeof(obj1.__dict__)} bytes")
print(f"With slots: {sys.getsizeof(obj2)} bytes")
# __slots__节省内存，访问速度更快
```

07 • 单项选择

关于生成器的 throw() 方法，以下说法正确的是：

A. 用于向生成器抛出异常

B. 用于终止生成器

C. 用于发送值

D. 与 send() 相同

答案 A. 用于向生成器抛出异常

解析 throw() 方法用于向生成器抛出异常。

示例：

```
def generator_with_exception():
    try:
        yield 1
        yield 2
        yield 3
    except ValueError as e:
        print(f"捕获异常: {e}")
        yield "异常处理后"
gen = generator_with_exception()
print(next(gen))
# 1
print(gen.throw(ValueError, "测试异常"))
# 捕获异常: 测试异常\n异常处理后
知识拓展：生成器的三个控制方法
def controllable_gen():
    try:
        while True:
            value = yield
            print(f"接收到: {value}")
    except GeneratorExit:
        print("生成器关闭")
gen = controllable_gen()
next(gen)
# 启动
gen.send(10)
# send(): 发送值
gen.throw(ValueError)
# throw(): 抛出异常
```

```
gen.close()
# close(): 关闭生成器
```

08 • 单项选择

以下代码的输出是什么？

```
def gen():
    try:
        yield 1
    except ValueError:
        yield 2
    yield 3
g = gen()
print(next(g))
print(g.throw(ValueError))
print(next(g))
```

A. 1 2 3

B. 1 2 StopIteration

C. 1 3 StopIteration

D. 报错

答案 A. 1 2 3

解析 输出1，抛出ValueError被except捕获返回2，最后finally块返回3。

示例：

```
def test_function():
    try:
        print(1)
        raise ValueError("错误")
        print("不会执行")
    # 不会执行
    except ValueError:
        print(2)
        return "from except"
    finally:
        print(3)
        return "from finally"
# 覆盖except的return
result = test_function()
print(f"返回值: {result}")
# 输出:
# 1
# 2
# 3
# 返回值: from finally
知识拓展：finally中return的风险
# 不推荐在finally中return
def risky_function():
    try:
        return "try value"
    finally:
        return "finally value"
```

```
# 覆盖try的返回值
print(risky_function())
# finally value - 可能导致混淆
```

09 • 单项选择

关于协程（Coroutine），以下说法正确的是：

A. 协程是特殊的生成器
B. 使用 async def 定义
C. 使用 await 调用
D. 以上都正确

答案 D. 以上都正确

解析 所有选项都是协程的正确特性（尽管协程是生成器的高级应用）。

示例：

```
# A. 使用yield表达式
def simple_coroutine():
    while True:
        x = yield
        # 接收值
        print(f"接收到: {x}")
coro = simple_coroutine()
next(coro)
# 启动
coro.send(10)
# 接收到: 10
# B. 可以发送值
def accumulator():
    total = 0
    while True:
        value = yield total
        if value is not None:
            total += value
acc = accumulator()
next(acc)
# 启动
print(acc.send(10))
# 10
print(acc.send(20))
# 30
# C. 可以接收异常
def error_handling_coro():
    while True:
        try:
            x = yield
            print(f"处理: {x}")
        except ValueError:
            print("处理异常")
coro = error_handling_coro()
next(coro)
coro.send(10)
# 处理: 10
```

```
coro.throw(ValueError)
# 处理异常
# D. 实现状态机
def state_machine():
    state = "初始"
    while True:
        event = yield state
        if state == "初始" and event == "开始":
            state = "运行中"
        elif state == "运行中" and event == "停止":
            state = "已停止"
sm = state_machine()
print(next(sm))
# 初始
print(sm.send("开始"))
# 运行中
print(sm.send("停止"))
# 已停止
```

10 • 单项选择

以下代码的输出是什么？

```
class MyIter:
    def __init__(self, n):
        self.n = n
        self.i = 0
    def __iter__(self):
        return self
    def __next__(self):
        if self.i < self.n:
            self.i += 1
            return self.i
        raise StopIteration
it = MyIter(3)
print(list(it))
```

A. [1, 2, 3] [1, 2, 3]
B. [1, 2, 3] []
C. [] []
D. 报错

答案 B. [1, 2, 3] []

解析 迭代器只能遍历一次，第一次迭代后耗尽，第二次返回空列表。

```
示例：
data = [1, 2, 3]
iter_data = iter(data)
# 第一次迭代
result1 = list(iter_data)
print(result1)
# [1, 2, 3]
# 第二次迭代（迭代器已耗尽）
```

```
result2 = list(iter_data)
print(result2)
# []
知识拓展：可重复迭代vs一次性迭代
# 列表：可重复迭代
data_list = [1, 2, 3]
print(list(data_list))
# [1, 2, 3]
print(list(data_list))
# [1, 2, 3] - 可以重复
# 迭代器：一次性
data_iter = iter([1, 2, 3])
print(list(data_iter))
# [1, 2, 3]
print(list(data_iter))
# [] - 已耗尽
# 生成器：一次性
data_gen = (x for x in [1, 2, 3])
print(list(data_gen))
# [1, 2, 3]
print(list(data_gen))
# [] - 已耗尽
# 创建可重复迭代的类
class ReusableIterator:
    def __init__(self, data):
        self.data = data
    def __iter__(self):
        return iter(self.data)
# 每次返回新迭代器
reusable = ReusableIterator([1, 2, 3])
print(list(reusable))
# [1, 2, 3]
print(list(reusable))
# [1, 2, 3] - 可以重复
```

11 • 单项选择

关于装饰器类，以下说法正确的是：

A. 装饰器只能是函数

B. 类也可以作为装饰器

C. 类装饰器需要实现 __call__()

D. B和C都正确

答案 D. B和C都正确

解析 类也可以作为装饰器，需要实现 __call__() 方法。B和C都正确。

示例：

```
class Counter:
    def __init__(self, func):
        self.func = func
        self.count = 0
    def __call__(self, *args, **kwargs):
        self.count += 1
```

```
        print(f"调用次数: {self.count}")
        return self.func(*args, **kwargs)
@Counter
def greet(name):
    return f"Hello, {name}"
print(greet("Alice"))
print(greet("Bob"))
print(f"总调用次数: {greet.count}")
知识拓展：类装饰器 vs 函数装饰器
# 函数装饰器
def func_decorator(func):
    def wrapper(*args):
        print("函数装饰器")
        return func(*args)
    return wrapper
# 类装饰器
class ClassDecorator:
    def __init__(self, func):
        self.func = func
    def __call__(self, *args):
        print("类装饰器")
        return self.func(*args)
@func_decorator
def func1():
    return "Function"
@ClassDecorator
def func2():
    return "Class"
print(func1())
print(func2())
```

12 • 单项选择

以下代码的输出是什么？

```
class Counter:
    def __init__(self, func):
        self.func = func
        self.count = 0
    def __call__(self, *args):
        self.count += 1
        return self.func(*args)
@Counter
def say_hello():
    return "Hello"
say_hello()
print(say_hello.count)
```

A. 0
B. 1
C. 2
D. 3

答案 C. 2

解析 @Counter 装饰器记录函数调用次数，调用两次后 count 为2。

示例：

```
class Counter:
    def __init__(self, func):
        self.func = func
        self.count = 0
    def __call__(self, *args):
        self.count += 1
        return self.func(*args)
@Counter
def say_hello():
    return "Hello"
say_hello()
# count = 1
say_hello()
# count = 2
print(say_hello.count)
# 2
```

知识拓展：带状态的装饰器

```
class CallLogger:
    def __init__(self, func):
        self.func = func
        self.calls = []
    def __call__(self, *args, **kwargs):
        import time
        start = time.time()
        result = self.func(*args, **kwargs)
        end = time.time()
        self.calls.append({
            'args': args,
            'kwargs': kwargs,
            'result': result,
            'duration': end - start
        })
        return result
    def get_stats(self):
        return {
            'total_calls': len(self.calls),
            'avg_duration': sum(c['duration'] for c in self.calls) / len(self.calls) if se
        }
@CallLogger
def add(a, b):
    return a + b
add(1, 2)
add(3, 4)
print(add.get_stats())
```

13 • 单项选择

关于 contextlib 模块，以下说法正确的是：

A. 提供上下文管理器工具
B. @contextmanager 装饰器可以简化上下文管理器的创建
C. 使用 yield 分隔 __enter__ 和 __exit__
D. 以上都正确

答案 D. 以上都正确

解析 contextlib 模块提供上下文管理器工具，所有选项都正确。

示例：

```
from contextlib import contextmanager
@contextmanager
def file_manager(filename, mode):
    print(f"打开文件: {filename}")
    f = open(filename, mode)
    try:
        yield f
    # __enter__和__exit__的分隔点
    finally:
        print(f"关闭文件: {filename}")
        f.close()
with file_manager('test.txt', 'w') as f:
    f.write("Hello, World!")
```

知识拓展：多种上下文管理器实现方式

```
from contextlib import contextmanager, closing
import sqlite3
# 1. 类实现
class DatabaseConnection:
    def __init__(self, db_name):
        self.db_name = db_name
    def __enter__(self):
        self.conn = sqlite3.connect(self.db_name)
        return self.conn
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.conn.close()
# 2. @contextmanager装饰器
@contextmanager
def database_connection(db_name):
    conn = sqlite3.connect(db_name)
    try:
        yield conn
    finally:
        conn.close()
# 3. closing()辅助函数
with closing(sqlite3.connect(':memory:')) as conn:
    cursor = conn.cursor()
```

14 • 单项选择

以下代码的输出是什么？

```
def func(x, y, z=3):
    return x + y + z
args = (1, 2)
```

```
kwargs = {'z': 4}
print(func(*args, **kwargs))
```

A. 6

B. 7

C. 10

D. 报错

答案 B. 7

解析 *args 解包为 (1, 2) ， **kwargs 解包为 {'z': 4} ，结果为1+2+4=7。

示例：

```
def func(x, y, z=3):
    return x + y + z
args = (1, 2)
kwargs = {'z': 4}
result = func(*args, **kwargs)
print(result)
# 7
知识拓展：参数解包的各种场景
# 场景1: 位置参数和关键字参数混合
def process(a, b, c=10, d=20):
    return a + b + c + d
args = (1, 2)
kwargs = {'c': 30, 'd': 40}
print(process(*args, **kwargs))
# 73
# 场景2: 字典和列表解包
data = {'name': 'Alice', 'age': 25}
info = ['Bob', 30]
def show_info(name, age):
    print(f"{name}: {age}岁")
show_info(**data)
# Alice: 25岁
show_info(*info)
# Bob: 30岁
# 场景3: 合并字典
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, 'd': 4}
merged = {**dict1, **dict2}
print(merged)
# {'a': 1, 'b': 2, 'c': 3, 'd': 4}
# 场景4: 合并列表
list1 = [1, 2, 3]
list2 = [4, 5, 6]
merged = [*list1, *list2]
print(merged)
# [1, 2, 3, 4, 5, 6]
```

15 • 单项选择

关于Python的参数注解（Type Hints），以下说法正确的是：

A. 强制类型检查

B. 只是提示，不强制
C. 运行时会检查类型
D. Python 2就支持

答案 B. 只是提示，不强制

解析 Python的类型注解只是提示，不强制类型检查。
示例：

```
def add(x: int, y: int) -> int:
    return x + y
# 即使传入错误类型，也能正常运行
result = add("Hello", "World")
print(result)
# HelloWorld
print(type(result))
# <class 'str'>
知识拓展：类型注解的使用
from typing import List, Dict, Optional, Union, Callable
# 基本类型注解
def greet(name: str) -> str:
    return f"Hello, {name}"
# 容器类型注解
def process_list(items: List[int]) -> int:
    return sum(items)
# 可选类型
def find_user(user_id: int) -> Optional[Dict[str, str]]:
    return {'name': 'Alice'} if user_id == 1 else None
# 联合类型
def process(data: Union[int, str]) -> str:
    return str(data)
# 函数类型
def apply(func: Callable[[int, int], int], x: int, y: int) -> int:
    return func(x, y)
# 使用mypy进行静态类型检查
# pip install mypy
# mypy script.py
```

16 • 单项选择

Pandas中，以下哪个方法用于数据分箱？

A. cut()
B. qcut()
C. bin()
D. A和B都对

答案 D. A和B都对

解析 Pandas中 cut() 和 qcut() 都用于数据分箱，前者等宽分箱，后者等频分箱。
示例：

```
import pandas as pd
import numpy as np
data = pd.Series([1, 7, 5, 4, 6, 8, 3, 2, 9, 10])
# cut(): 等宽分箱
bins_cut = pd.cut(data, bins=3)
```

```
print("等宽分箱:")
print(bins_cut.value_counts().sort_index())
# qcut(): 等频分箱
bins_qcut = pd.qcut(data, q=3)
print("\n等频分箱:")
print(bins_qcut.value_counts().sort_index())
知识拓展：分箱的高级用法
import pandas as pd
data = pd.Series([18, 25, 35, 45, 55, 65, 75])
# 自定义分箱边界
bins = [0, 30, 60, 100]
labels = ['青年', '中年', '老年']
result = pd.cut(data, bins=bins, labels=labels)
print(result)
# 包含右边界
result = pd.cut(data, bins=3, right=True)
print(result)
# 包含左边界
result = pd.cut(data, bins=3, right=False)
print(result)
# 返回分箱索引
result = pd.cut(data, bins=3, labels=False)
print(result)
```

17 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
s = pd.Series([1, 2, 3, 4, 5])
result = pd.cut(s, bins=2)
print(len(result.cat.categories))
```

A. 1

B. 2

C. 3

D. 5

答案 B. 2

解析 pd.cut(s, bins=2) 将数据分成2个区间。

```
示例：
import pandas as pd
s = pd.Series([1, 2, 3, 4, 5])
result = pd.cut(s, bins=2)
print(result)
print(f"\n区间数量: {len(result.cat.categories)}")
知识拓展：分类数据操作
import pandas as pd
s = pd.Series([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
cut_result = pd.cut(s, bins=3)
print("类别:", cut_result.cat.categories)
print("编码:", cut_result.cat.codes)
print("值计数:", cut_result.value_counts())
```

18 • 单项选择

Pandas中， MultiIndex 的作用是：

A. 多列索引
B. 多级索引
C. 多个DataFrame
D. 多种数据类型

答案 B. 多级索引

解析 MultiIndex 用于创建多级索引（hierarchical indexing）。
示例：

```
import pandas as pd
# 创建多级索引
arrays = [
    ['A', 'A', 'B', 'B'],
    [1, 2, 1, 2]
]
index = pd.MultiIndex.from_arrays(arrays, names=['字母', '数字'])
df = pd.DataFrame({'值': [10, 20, 30, 40]}, index=index)
print(df)
# 多级索引访问
print("\n访问A:")
print(df.loc['A'])
知识拓展：多级索引的高级操作
import pandas as pd
# 方式1: from_tuples
index = pd.MultiIndex.from_tuples([('A', 1), ('A', 2), ('B', 1), ('B', 2)])
# 方式2: from_product
index = pd.MultiIndex.from_product([['A', 'B'], [1, 2]])
# 方式3: from_frame
df_index = pd.DataFrame({'字母': ['A', 'A', 'B'], '数字': [1, 2, 1]})
index = pd.MultiIndex.from_frame(df_index)
# 多级索引的stack和unstack
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
}, index=['x', 'y', 'z'])
stacked = df.stack()
print(stacked)
print(stacked.unstack())
```

19 • 单项选择

关于Pandas的 stack() 和 unstack() ，以下说法正确的是：

A. stack() 将列转为行
B. unstack() 将行转为列
C. 两者互为逆操作
D. 以上都正确

答案 D. 以上都正确

解析 stack() 将列转为行， unstack() 将行转为列，两者互为逆操作。
示例：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2],
    'B': [3, 4]
}, index=['x', 'y'])
print("原始DataFrame:")
print(df)
stacked = df.stack()
print("\nstack()后:")
print(stacked)
unstacked = stacked.unstack()
print("\nunstack()后:")
print(unstacked)
知识拓展：数据重塑详解
import pandas as pd
# 复杂的stack/unstack
df = pd.DataFrame({
    '城市': ['北京', '北京', '上海', '上海'],
    '季度': ['Q1', 'Q2', 'Q1', 'Q2'],
    '销售额': [100, 150, 120, 180]
})
# pivot
pivoted = df.pivot(index='城市', columns='季度', values='销售额')
print(pivoted)
# melt (逆操作)
melted = pivoted.reset_index().melt(id_vars='城市', var_name='季度', value_name='销售额')
print(melted)
```

20 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
result = df.T
print(result.shape)
```

A. (2, 2)

B. (4,)

C. (2, 2)

D. (1, 4)

答案 A. (2, 2)

解析 DataFrame转置后，行列互换，(2, 2)转置后仍是(2, 2)。

示例：

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
print("原始shape:", df.shape)
# (2, 2)
result = df.T
print("转置后shape:", result.shape)
# (2, 2)
print("\n转置后的DataFrame:")
```

```
print(result)
知识拓展：转置的各种场景
import pandas as pd
# 不同形状的转置
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
print(f"原始: {df1.shape}")
# (3, 2)
print(f"转置: {df1.T.shape}")
# (2, 3)
# 带索引的转置
df2 = pd.DataFrame(
    {'Math': [90, 85], 'English': [88, 92]},
    index=['Alice', 'Bob']
)
print("\n原始:")
print(df2)
print("\n转置后:")
print(df2.T)
```

21 • 单项选择

pathlib.Path 对象的 as_posix() 方法的作用是：

A. 转换为POSIX风格路径
B. 转换为Windows路径
C. 检查是否是POSIX路径
D. 创建POSIX路径

答案 A. 转换为POSIX风格路径

解析 as_posix() 将路径转换为POSIX风格（使用正斜杠 / ）。

```
示例：
from pathlib import Path
# Windows风格路径
p = Path('C:\\Users\\test')
print(f"原始: {p}")
print(f"POSIX: {p.as_posix()}")
# C:/Users/test
知识拓展：路径格式转换
from pathlib import Path
import os
# 不同平台的路径处理
if os.name == 'nt':
    # Windows
    p = Path('C:\\Users\\test\\file.txt')
else:
    # Unix/Linux/Mac
    p = Path('/home/test/file.txt')
print(f"原始路径: {p}")
print(f"POSIX格式: {p.as_posix()}")
print(f"字符串格式: {str(p)}")
print(f"URI格式: {p.as_uri()}")
```

22 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p = Path('C:\\Users\\test')
print(p.as_posix())
```

A. C:\Users\test
B. C:/Users/test
C. /Users/test
D. 报错

答案 B. C:/Users/test

解析 as_posix() 将 C:\Users\test 转换为 C:/Users/test 。

示例：

```
from pathlib import Path
p = Path('C:\\Users\\test')
print(p.as_posix())
# C:/Users/test
```

23 • 单项选择

Path.cwd() 方法返回的是：

A. 用户主目录
B. 当前工作目录
C. 根目录
D. 脚本所在目录

答案 B. 当前工作目录

解析 Path.cwd() 返回当前工作目录（Current Working Directory）。

示例：

```
from pathlib import Path
print("当前工作目录:", Path.cwd())
print("用户主目录:", Path.home())
```

24 • 单项选择

关于 Path.relative_to() 方法，以下说法正确的是：

A. 计算相对路径
B. 转换为相对路径
C. 两个路径必须有共同的父目录
D. 以上都正确

答案 D. 以上都正确

解析 relative_to() 用于计算相对路径，所有选项都正确。

示例：

```
from pathlib import Path
p1 = Path('/a/b/c/d')
p2 = Path('/a/b')
relative = p1.relative_to(p2)
print(relative)
# c/d
知识拓展：相对路径计算
from pathlib import Path
```

```
# 计算相对路径
base = Path('/home/user/projects')
target = Path('/home/user/projects/myapp/src/main.py')
relative = target.relative_to(base)
print(relative)
# myapp/src/main.py
# 不共享父目录会报错
try:
    p1 = Path('/home/user')
    p2 = Path('/var/log')
    p1.relative_to(p2)
except ValueError as e:
    print(f"错误: {e}")
```

25 • 单项选择

以下代码的输出是什么？

```
from pathlib import Path
p1 = Path('/a/b/c')
p2 = Path('/a')
print(p1.relative_to(p2))
```

A. /a/b/c

B. b/c

C. c

D. 报错

答案 B. b/c

解析 /a/b/c 相对于 /a 的路径是 b/c 。

示例：

```
from pathlib import Path
p1 = Path('/a/b/c')
p2 = Path('/a')
print(p1.relative_to(p2))
# b/c
```

26 • 单项选择

关于 collections 模块的 defaultdict ，以下说法正确的是：

A. 访问不存在的键会返回默认值

B. 需要指定默认值工厂函数

C. 比普通字典更高效

D. A和B都正确

答案 D. A和B都正确

解析 defaultdict 访问不存在的键会返回默认值，需要指定默认值工厂函数。A和B都正确。

示例：

```
from collections import defaultdict
# int作为默认工厂
d = defaultdict(int)
d['a'] += 1
print(d['b'])
# 0
```

```
# list作为默认工厂
d = defaultdict(list)
d['key'].append(1)
print(d['key'])
# [1]
知识拓展：defaultdict的各种用法
from collections import defaultdict
# 分组
items = [('fruit', 'apple'), ('vegetable', 'carrot'),
        ('fruit', 'banana'), ('vegetable', 'lettuce')]
groups = defaultdict(list)
for category, item in items:
    groups[category].append(item)
print(dict(groups))
# 计数
text = "hello world"
char_count = defaultdict(int)
for char in text:
    char_count[char] += 1
print(dict(char_count))
# 嵌套defaultdict
nested = defaultdict(lambda: defaultdict(int))
nested['a']['x'] = 1
nested['a']['y'] = 2
print(dict(nested))
```

27 • 单项选择

以下代码的输出是什么？

```
from collections import defaultdict
d = defaultdict(int)
d['a'] += 1
d['b'] += 2
print(d['c'])
```

A. 0

B. None

C. KeyError

D. 1

答案 A. 0

解析 defaultdict(int) 的默认值为0，访问不存在的键 'c' 返回0。

示例：

```
from collections import defaultdict
d = defaultdict(int)
d['a'] += 1
d['b'] += 2
print(d['c'])
# 0
print(d)
# defaultdict(<class 'int'>, {'a': 1, 'b': 2, 'c': 0})
```

28 • 单项选择

关于 collections.Counter ，以下说法正确的是：

A. 用于计数
B. 继承自字典
C. 提供 most_common() 方法
D. 以上都正确

答案 D. 以上都正确

解析 Counter 用于计数，继承自字典，提供 most_common() 方法，所有选项都正确。

示例：

```
from collections import Counter
c = Counter('abracadabra')
print(c)
# Counter({'a': 5, 'b': 2, 'r': 2, 'c': 1, 'd': 1})
print(c.most_common(2))
# [('a', 5), ('b', 2)]
知识拓展：Counter的高级用法
from collections import Counter
# 基本计数
words = ['apple', 'banana', 'apple', 'cherry', 'banana', 'apple']
c = Counter(words)
print(c.most_common(2))
# [('apple', 3), ('banana', 2)]
# 算术运算
c1 = Counter(['a', 'b', 'c'])
c2 = Counter(['b', 'c', 'd'])
print(c1 + c2)
# Counter({'b': 2, 'c': 2, 'a': 1, 'd': 1})
print(c1 - c2)
# Counter({'a': 1})
print(c1 & c2)
# Counter({'b': 1, 'c': 1})
print(c1 | c2)
# Counter({'a': 1, 'b': 1, 'c': 1, 'd': 1})
# 更新计数
c = Counter()
c.update('abracadabra')
c.update({'a': 1, 'b': 2})
print(c)
```

29 • 单项选择

以下代码的输出是什么？

```
from collections import Counter
c = Counter('abracadabra')
print(c.most_common(1))
```

A. [('a', 5)]
B. [('b', 2)]
C. 'a'
D. 5

答案 A. [('a', 5)]

解析 'a'在'abracadabra'中出现5次，是最多的，返回 [('a', 5)] 。

示例：

```
from collections import Counter
c = Counter('abracadabra')
print(c.most_common(1))
# [('a', 5)]
print(c.most_common(3))
# [('a', 5), ('b', 2), ('r', 2)]
```

30 • 单项选择

关于 itertools.chain() ，以下说法正确的是：

A. 用于连接多个迭代器

B. 返回新列表

C. 只能连接两个迭代器

D. 会复制数据

答案 A. 用于连接多个迭代器

解析 itertools.chain() 用于连接多个迭代器，返回迭代器不是列表，不复制数据。

示例：

```
import itertools
list1 = [1, 2, 3]
list2 = [4, 5, 6]
list3 = [7, 8, 9]
# 连接多个迭代器
chained = itertools.chain(list1, list2, list3)
print(list(chained))
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
知识拓展：chain的高级用法
import itertools
# chain.from_iterable: 从可迭代对象的可迭代对象中连接
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = list(itertools.chain.from_iterable(matrix))
print(flattened)
# [1, 2, 3, 4, 5, 6, 7, 8, 9]
# 与生成器配合
def get_numbers():
    yield from range(1, 4)
def get_letters():
    yield from 'abc'
combined = itertools.chain(get_numbers(), get_letters())
for item in combined:
    print(item, end=' ')
# 1 2 3 a b c
```

二、不定项选择题

31 • 不定项选择

关于Python的设计模式，以下说法正确的有：

A. 单例模式可以用 __new__() 实现
B. 工厂模式可以用类方法实现
C. 观察者模式可以用装饰器实现
D. 策略模式可以用函数实现

答案 A. 单例模式可以用 __new__() 实现；B. 工厂模式可以用类方法实现；C. 观察者模式可以用装饰器实现；D. 策略模式可以用函数实现

解析 所有选项都是组合优于继承的原因正确

32 • 不定项选择

以下哪些是Python的容器协议方法：

A. __len__()
B. __getitem__()
C. __setitem__()
D. __contains__()

答案 A. __len__()；B. __getitem__()；C. __setitem__()；D. __contains__()

解析 所有选项都是容器协议方法

33 • 不定项选择

关于Python的弱引用，以下说法正确的有：

A. 使用 weakref 模块
B. 不会增加引用计数
C. 可以避免循环引用
D. 所有对象都支持弱引用

答案 A. 使用 weakref 模块；B. 不会增加引用计数；C. 可以避免循环引用

解析 不是所有对象都支持弱引用

34 • 不定项选择

关于生成器的性能优势，以下说法正确的有：

A. 节省内存
B. 惰性计算
C. 适合处理大数据
D. 总是比列表快

答案 A. 节省内存；B. 惰性计算；C. 适合处理大数据

解析 生成器不总是比列表快

35 • 不定项选择

以下哪些是迭代器的应用场景：

A. 读取大文件
B. 无限序列
C. 数据流处理
D. 节省内存

答案 A. 读取大文件；B. 无限序列；C. 数据流处理；D. 节省内存

解析 所有选项都是迭代器的应用场景

36 • 不定项选择

关于装饰器的最佳实践，以下说法正确的有：

A. 使用 functools.wraps 保留元数据
B. 装饰器应该是幂等的
C. 避免在装饰器中修改全局状态
D. 装饰器可以有副作用

答案 A. 使用 functools.wraps 保留元数据；B. 装饰器应该是幂等的；C. 避免在装饰器中修改全局状态

解析 装饰器最佳实践包括使用 functools.wraps 保留元数据、避免修改全局状态，但装饰器可以有副作用（如日志记录）。
（详细示例可参考第1-4套装饰器相关题目）

37 • 不定项选择

关于闭包和装饰器的关系，以下说法正确的有：

A. 装饰器通常使用闭包实现
B. 闭包可以保存装饰器的状态
C. 所有装饰器都是闭包
D. 闭包是装饰器的基础

答案 A. 装饰器通常使用闭包实现；B. 闭包可以保存装饰器的状态；D. 闭包是装饰器的基础

解析 装饰器通常使用闭包实现，闭包是装饰器的基础，但不是所有装饰器都是闭包（类装饰器除外）。
（详细示例可参考第1-4套闭包和装饰器相关题目）

38 • 不定项选择

以下哪些是Pandas的窗口函数：

A. rolling()
B. expanding()
C. ewm()
D. sliding()

答案 A. rolling()；B. expanding()；C. ewm()

解析 Pandas的窗口函数包括 rolling() 、 expanding() 、 ewm() ，没有 sliding() 。
（详细示例可参考第1-4套Pandas相关题目）

39 • 不定项选择

关于Pandas的性能优化，以下说法正确的有：

A. 使用向量化操作
B. 避免循环遍历DataFrame
C. 使用 apply() 比循环快
D. 选择合适的数据类型

答案 A. 使用向量化操作；B. 避免循环遍历DataFrame；C. 使用 apply() 比循环快；D. 选择合适的数据类型

解析 Pandas性能优化的所有选项都正确：使用向量化操作、避免循环、使用apply()、选择合适的数据类型。
（详细示例可参考第1-4套Pandas性能优化相关题目）

40 • 不定项选择

Pandas中，以下哪些方法可以用于数据导出：

A. to_csv()
B. to_excel()
C. to_json()
D. to_sql()

答案 A. to_csv()；B. to_excel()；C. to_json()；D. to_sql()

解析 所有选项都是Pandas的数据导出方法。
（详细示例可参考第1-4套Pandas相关题目）

41 • 不定项选择

关于 pathlib 的优势，以下说法正确的有：

A. 面向对象
B. 跨平台
C. 链式调用
D. 更安全

答案 A. 面向对象；B. 跨平台；C. 链式调用；D. 更安全

解析 pathlib 的所有优势选项都正确：面向对象、跨平台、链式调用、更安全。
（详细示例可参考第1-4套pathlib相关题目）

42 • 不定项选择

Path 对象的以下哪些方法返回布尔值：

A. exists()
B. is_file()
C. is_dir()
D. is_absolute()

答案 A. exists()；B. is_file()；C. is_dir()；D. is_absolute()

解析 所有选项都是返回布尔值的Path方法。
（详细示例可参考第1-4套pathlib相关题目）

43 • 不定项选择

关于Python的推导式，以下说法正确的有：

A. 列表推导式 []
B. 字典推导式 {}
C. 集合推导式 {}
D. 元组推导式 ()

答案 A. 列表推导式 []；B. 字典推导式 {}；C. 集合推导式 {}

解析 元组推导式使用 () 实际上是生成器表达式，不是元组推导式。
（详细示例可参考第1-4套推导式相关题目）

44 • 不定项选择

以下哪些是Python的函数式编程工具：

A. map()
B. filter()
C. reduce()

D. lambda

答案 A. map()；B. filter()；C. reduce()；D. lambda

解析 所有选项都是Python的函数式编程工具。
（详细示例可参考第1-4套函数式编程相关题目）

45 • 不定项选择

关于Python的性能优化，以下说法正确的有：

A. 使用生成器代替列表
B. 使用局部变量代替全局变量
C. 使用 __slots__ 减少内存
D. 使用列表推导式代替循环

答案 A. 使用生成器代替列表；B. 使用局部变量代替全局变量；C. 使用 __slots__ 减少内存；D. 使用列表推导式代替循环

解析 所有选项都是Python性能优化的有效方法。
（详细示例可参考第1-4套性能优化相关题目）

三、填空题

46 • 填空 / 简答

Python中，使用 _____ 可以实现单例模式，使用 _____ 可以实现工厂模式。

答案 yield ， send()

解析 协程的双向通信机制。
示例：

```
def accumulator():
    total = 0
    while True:
        value = yield total
        # 返回total，接收value
        if value is not None:
            total += value
coro = accumulator()
print(next(coro))
# 0 - 启动协程
print(coro.send(10))
# 10
print(coro.send(5))
# 15
print(coro.send(3))
# 18
```

知识拓展：协程是Python异步编程的基础。

47 • 填空 / 简答

迭代器协议要求实现 _____ 和 _____ 两个方法。

答案 __enter__ ， __exit__

解析 上下文管理器协议。

示例：

```
class MyContext:
    def __enter__(self):
        print("进入上下文")
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        print("退出上下文")
        return False
# 不抑制异常
with MyContext() as ctx:
    print("上下文内部")
# 输出：
# 进入上下文
# 上下文内部
# 退出上下文
```

知识拓展： __exit__ 返回True可以抑制异常。

48 • 填空 / 简答

Pandas中，_____ 用于时间序列重采样，_____ 用于时间窗口计算。

答案 merge() ， join()

解析 Pandas数据合并方法。

示例：

```
import pandas as pd
df1 = pd.DataFrame({'key': ['A', 'B'], 'val1': [1, 2]})
df2 = pd.DataFrame({'key': ['A', 'C'], 'val2': [3, 4]})
# merge
result = pd.merge(df1, df2, on='key', how='inner')
print(result)
# join (基于索引)
df1_indexed = df1.set_index('key')
df2_indexed = df2.set_index('key')
result = df1_indexed.join(df2_indexed, how='outer')
print(result)
```

知识拓展： merge 更灵活， join 基于索引。

49 • 填空 / 简答

使用 _____ 可以创建默认字典，使用 _____ 可以创建有序字典。

答案 itertools ， chain()

解析 迭代工具模块。

示例：

```
from itertools import chain, cycle, repeat
# chain - 连接多个迭代器
list1 = [1, 2, 3]
list2 = [4, 5, 6]
for item in chain(list1, list2):
    print(item)
# 1, 2, 3, 4, 5, 6
# cycle - 无限循环
```

```
counter = 0
for item in cycle([1, 2, 3]):
    print(item)
    counter += 1
    if counter > 5:
        break
# repeat - 重复元素
for item in repeat('hello', 3):
    print(item)
# hello hello hello
```

知识拓展：itertools提供高效的迭代工具。

50 • 填空 / 简答

pathlib.Path 对象的 _____ 方法用于创建符号链接，_____ 方法用于读取链接目标。

答案 with_suffix() ， with_name()

解析 Path对象修改文件名方法。

示例：

```
from pathlib import Path
p = Path('/home/user/docs/file.txt')
# with_suffix - 更改扩展名
new_p = p.with_suffix('.md')
print(new_p)
# /home/user/docs/file.md
# with_name - 更改文件名
new_p = p.with_name('newfile.txt')
print(new_p)
# /home/user/docs/newfile.txt
# 链式调用
new_p = p.with_name('report').with_suffix('.pdf')
print(new_p)
# /home/user/docs/report.pdf
```

知识拓展：这些方法返回新Path对象，不修改原对象。

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class Singleton(type):
    _instances = {}
    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]
class Database(metaclass=Singleton):
    def __init__(self):
        self.connection = "Connected"
```

```
db1 = Database()
db2 = Database()
print(db1 is db2)
print(Database._instances)
```

问题：

- (1) 第一个 print 输出什么？
- (2) 这段代码使用了什么设计模式？
- (3) 元类的作用是什么？

答案 元类单例

- (1) 输出： True
- (2) 单例模式，使用元类实现
- (3) 元类控制类的创建过程，这里用于确保只创建一个实例

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
from contextlib import contextmanager
@contextmanager
def file_manager(filename, mode):
    print(f"打开文件: {filename}")
    f = open(filename, mode)
    try:
        yield f
    finally:
        print(f"关闭文件: {filename}")
        f.close()
with file_manager('test.txt', 'w') as f:
    f.write('Hello World')
```

问题：

- (1) 这段代码的输出是什么？
- (2) @contextmanager 装饰器的作用是什么？
- (3) yield 在这里起什么作用？

答案 • (1) 输出：

```
    打开文件: test.txt
    关闭文件: test.txt
```

- (2) @contextmanager 简化上下文管理器的创建，不需要定义类
- (3) yield 分隔 __enter__ 和 __exit__ ，yield的值传给 as 后的变量

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50]
```

```
})
result = df.rolling(window=2).sum()
print(result)
print(result.iloc[-1])
```

问题：

- (1) rolling(window=2) 的作用是什么？
- (2) 第一行的结果是什么？
- (3) 最后一行的结果是什么？

答案 滚动窗口

- (1) rolling(window=2) 创建大小为2的滚动窗口
- (2) 第一行是NaN（窗口不足）
- (3) 最后一行是 A: 9, B: 90 （4+5=9, 40+50=90）

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
from pathlib import Path
from collections import defaultdict
def group_files_by_extension(directory):
    path = Path(directory)
    groups = defaultdict(list)
    for file in path.rglob('*'):
        if file.is_file():
            groups[file.suffix].append(file.name)
    return groups
result = group_files_by_extension('.')
for ext, files in result.items():
    print(f"{ext}: {len(files)} 个文件")
```

问题：

- (1) 这个函数的功能是什么？
- (2) defaultdict(list) 的作用是什么？
- (3) rglob('*') 和 glob('*') 的区别是什么？

答案 文件分组

- (1) 按文件扩展名分组统计文件
- (2) defaultdict(list) 自动为新键创建空列表
- (3) rglob() 递归搜索所有子目录， glob() 只搜索当前目录

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
from functools import reduce
from operator import add
numbers = [1, 2, 3, 4, 5]
# 方法1
result1 = reduce(add, numbers)
```

```
# 方法2
result2 = reduce(lambda x, y: x + y, numbers)
# 方法3
result3 = sum(numbers)
print(result1 == result2 == result3)
```

问题：

- (1) 输出是什么？
- (2) 三种方法有什么区别？
- (3) 哪种方法最推荐使用？为什么？

答案 函数

- (1) 输出： True
- (2) 方法1使用operator.add，方法2使用lambda，方法3使用内置sum
- (3) 方法3最推荐，因为sum是内置函数，最快且最清晰

解析

五、程序设计题

56 • 填空 / 简答

设计一个任务调度系统，要求：

Task类：

类：

- 属性：任务名、优先级、状态（待执行/执行中/已完成）
- 方法： execute() （抽象方法）
- 实现 __lt__() 用于优先级比较

Scheduler类：

类：

- 使用优先队列管理任务
- 方法：

add_task(task) ：添加任务

run_next() ：执行优先级最高的任务

run_all() ：执行所有任务

使用生成器 pending_tasks() 返回待执行任务

装饰器 @retry_on_failure(max_retries) ：

- 任务失败时自动重试
- 记录重试次数

要求：

- 实现至少2种具体任务类
- 使用 heapq 模块实现优先队列
- 添加日志记录

答案

```
import heapq
from abc import ABC, abstractmethod
from enum import Enum
import logging
from functools import wraps
logging.basicConfig(level=logging.INFO)
```

```
logger = logging.getLogger(__name__)
class TaskStatus(Enum):
    PENDING = "待执行"
    RUNNING = "执行中"
    COMPLETED = "已完成"
    FAILED = "失败"
def retry_on_failure(max_retries=3):
    def decorator(func):
        @wraps(func)
        def wrapper(self, *args, **kwargs):
            retries = 0
            while retries < max_retries:
                try:
                return func(self, *args, **kwargs)
                except Exception as e:
                retries += 1
                logger.warning(f"任务{self.name} 执行失败 (尝试{retries}/{max_retries}
                if retries >= max_retries:
                self.status = TaskStatus.FAILED
                raise
            return None
        return wrapper
    return decorator
class Task(ABC):
    def __init__(self, name, priority=0):
        self.name = name
        self.priority = priority
        self.status = TaskStatus.PENDING
    @abstractmethod
    def execute(self):
        pass
    def __lt__(self, other):
        return self.priority > other.priority
    def __repr__(self):
        return f"Task({self.name}, priority={self.priority}, status={self.status.value})"
class DataProcessTask(Task):
    def __init__(self, name, data, priority=0):
        super().__init__(name, priority)
        self.data = data
    @retry_on_failure(max_retries=3)
    def execute(self):
        logger.info(f"执行数据处理任务: {self.name}")
        self.status = TaskStatus.RUNNING
        # 模拟数据处理
        result = sum(self.data)
        self.status = TaskStatus.COMPLETED
        logger.info(f"任务{self.name} 完成，结果: {result}")
        return result
```

```
class EmailTask(Task):
    def __init__(self, name, recipient, message, priority=0):
        super().__init__(name, priority)
        self.recipient = recipient
        self.message = message
    @retry_on_failure(max_retries=2)
    def execute(self):
        logger.info(f"执行邮件任务: {self.name}")
        self.status = TaskStatus.RUNNING
        # 模拟发送邮件
        logger.info(f"发送邮件到{self.recipient}: {self.message}")
        self.status = TaskStatus.COMPLETED
        return True
class Scheduler:
    def __init__(self):
        self.task_queue = []
        self.completed_tasks = []
    def add_task(self, task):
        heapq.heappush(self.task_queue, task)
        logger.info(f"添加任务: {task.name} (优先级: {task.priority})")
    def run_next(self):
        if not self.task_queue:
            logger.info("没有待执行的任务")
            return None
        task = heapq.heappop(self.task_queue)
        try:
            result = task.execute()
            self.completed_tasks.append(task)
            return result
        except Exception as e:
            logger.error(f"任务执行失败: {task.name}, 错误: {e}")
            return None
    def run_all(self):
        logger.info(f"开始执行所有任务，共{len(self.task_queue)} 个")
        while self.task_queue:
            self.run_next()
        logger.info("所有任务执行完成")
    def pending_tasks(self):
        for task in self.task_queue:
            if task.status == TaskStatus.PENDING:
                yield task
    def __len__(self):
        return len(self.task_queue)
# 使用示例
scheduler = Scheduler()
# 添加任务
scheduler.add_task(DataProcessTask("处理数据1", [1, 2, 3, 4, 5], priority=1))
scheduler.add_task(EmailTask("发送报告", "user@example.com", "报告内容", priority=3))
```

```
scheduler.add_task(DataProcessTask("处理数据2", [10, 20, 30], priority=2))
print("\n待执行任务:")
for task in scheduler.pending_tasks():
    print(f"
          {task}")
print(f"\n开始调度{len(scheduler)} 个任务\n")
scheduler.run_all()
print(f"\n已完成{len(scheduler.completed_tasks)} 个任务")
```

解析

57 • 填空 / 简答

实现一个多级缓存系统：

1. LRUCache类：

类：

- 实现LRU（最近最少使用）缓存
- 使用 OrderedDict
- 方法： get(key) 、 put(key, value)
- 实现 __getitem__ 和 __setitem__

2. CacheDecorator：

- 函数结果缓存装饰器
- 支持TTL（过期时间）
- 支持缓存统计（命中率）

3. MultiLevelCache：

- 组合多个缓存层
- L1缓存（内存）、L2缓存（文件）
- 自动降级和升级

要求：

- 使用 pathlib 处理文件缓存
- 使用 pickle 序列化数据
- 实现缓存预热和清理

答案

```
from collections import OrderedDict
from pathlib import Path
import pickle
import time
from functools import wraps
class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.cache = OrderedDict()
    def get(self, key):
        if key not in self.cache:
            return None
        self.cache.move_to_end(key)
        return self.cache[key]
    def put(self, key, value):
        if key in self.cache:
            self.cache.move_to_end(key)
```

```
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)
    def __getitem__(self, key):
        return self.get(key)
    def __setitem__(self, key, value):
        self.put(key, value)
    def __len__(self):
        return len(self.cache)
    def __repr__(self):
        return f"LRUCache(capacity={self.capacity}, size={len(self.cache)})"
class CacheDecorator:
    def __init__(self, ttl=None):
        self.ttl = ttl
        self.cache = {}
        self.timestamps = {}
        self.hits = 0
        self.misses = 0
    def __call__(self, func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            key = str(args) + str(kwargs)
            current_time = time.time()
            # 检查缓存
            if key in self.cache:
                if self.ttl is None or (current_time - self.timestamps[key]) < self.ttl:
                self.hits += 1
                return self.cache[key]
            # 缓存未命中
            self.misses += 1
            result = func(*args, **kwargs)
            self.cache[key] = result
            self.timestamps[key] = current_time
            return result
        wrapper.cache_info = lambda: {
            'hits': self.hits,
            'misses': self.misses,
            'hit_rate': self.hits / (self.hits + self.misses) if (self.hits + self.misses)
        }
        wrapper.cache_clear = lambda: self.cache.clear()
        return wrapper
class MultiLevelCache:
    def __init__(self, l1_capacity=100, cache_dir='cache'):
        self.l1_cache = LRUCache(l1_capacity)
        self.cache_dir = Path(cache_dir)
        self.cache_dir.mkdir(exist_ok=True)
    def get(self, key):
        # 先查L1缓存
```

```
        value = self.l1_cache.get(key)
        if value is not None:
            print(f"L1缓存命中: {key}")
            return value
        # 查L2缓存（文件）
        cache_file = self.cache_dir / f"{key}.pkl"
        if cache_file.exists():
            print(f"L2缓存命中: {key}")
            with cache_file.open('rb') as f:
                value = pickle.load(f)
            # 升级到L1
            self.l1_cache.put(key, value)
            return value
        print(f"缓存未命中: {key}")
        return None
    def put(self, key, value):
        # 存入L1
        self.l1_cache.put(key, value)
        # 存入L2
        cache_file = self.cache_dir / f"{key}.pkl"
        with cache_file.open('wb') as f:
            pickle.dump(value, f)
    def clear(self):
        self.l1_cache.cache.clear()
        for file in self.cache_dir.glob('*.pkl'):
            file.unlink()
# 使用示例
print("=== LRU缓存示例 ===")
lru = LRUCache(3)
lru['a'] = 1
lru['b'] = 2
lru['c'] = 3
print(f"缓存: {dict(lru.cache)}")
lru['d'] = 4
            # 'a'被淘汰
print(f"添加'd'后: {dict(lru.cache)}")
print("\n=== 缓存装饰器示例 ===")
@CacheDecorator(ttl=5)
def expensive_function(n):
    print(f"计算{n}...")
    time.sleep(0.1)
    return n ** 2
print(expensive_function(5))
# 从缓存获取
print(f"缓存统计: {expensive_function.cache_info()}")
print("\n=== 多级缓存示例 ===")
ml_cache = MultiLevelCache(l1_capacity=2)
ml_cache.put('key1', 'value1')
```

```
ml_cache.put('key2', 'value2')
ml_cache.put('key3', 'value3')
print(ml_cache.get('key1'))
# L2命中，升级到L1
print(ml_cache.get('key1'))
# L1命中
```

解析

58 • 填空 / 简答

使用Pandas创建自动化数据分析报告：
给定销售数据（日期、区域、产品、销量、金额）：
1. 数据清洗：
• 处理缺失值和异常值
• 数据类型转换
• 去重
2. 统计分析：
• 描述性统计
• 按维度聚合（区域、产品、时间）
• 同比环比分析
• 趋势分析（移动平均、增长率）
3. 数据可视化：
• 销售趋势图
• 产品占比饼图
• 区域对比柱状图
4. 报告生成：
• 导出Excel（多个sheet）
• 生成HTML报告
• 添加图表和表格
要求：
• 使用Pandas的高级功能
• 使用matplotlib/seaborn绘图
• 代码模块化

答案

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from datetime import datetime, timedelta
class SalesReportGenerator:
    def __init__(self, data):
        self.df = data.copy()
        self.cleaned_df = None
        self.analysis_results = {}
    def clean_data(self):
        print("开始数据清洗...")
        # 处理缺失值
        self.df.dropna(subset=['日期', '产品'], inplace=True)
        self.df['销量'].fillna(self.df['销量'].median(), inplace=True)
```

```
        self.df['金额'].fillna(self.df['金额'].median(), inplace=True)
        # 数据类型转换
        self.df['日期'] = pd.to_datetime(self.df['日期'])
        self.df['销量'] = self.df['销量'].astype(int)
        self.df['金额'] = self.df['金额'].astype(float)
        # 去重
        self.df.drop_duplicates(inplace=True)
        # 处理异常值（使用IQR方法）
        Q1 = self.df['金额'].quantile(0.25)
        Q3 = self.df['金额'].quantile(0.75)
        IQR = Q3 - Q1
        self.df = self.df[
            (self.df['金额'] >= Q1 - 1.5 * IQR) &
            (self.df['金额'] <= Q3 + 1.5 * IQR)
        ]
        self.cleaned_df = self.df.copy()
        print(f"清洗完成，剩余{len(self.cleaned_df)} 条记录")
def descriptive_statistics(self):
    print("\n描述性统计...")
    stats = self.cleaned_df[['销量', '金额']].describe()
    self.analysis_results['descriptive'] = stats
    return stats
def aggregate_analysis(self):
    print("\n聚合分析...")
    # 按区域聚合
    region_agg = self.cleaned_df.groupby('区域').agg({
        '销量': 'sum',
        '金额': 'sum'
    }).sort_values('金额', ascending=False)
    # 按产品聚合
    product_agg = self.cleaned_df.groupby('产品').agg({
        '销量': 'sum',
        '金额': 'sum'
    }).sort_values('金额', ascending=False)
    # 按时间聚合
    self.cleaned_df.set_index('日期', inplace=True)
    time_agg = self.cleaned_df.resample('W').agg({
        '销量': 'sum',
        '金额': 'sum'
    })
    self.analysis_results['region'] = region_agg
    self.analysis_results['product'] = product_agg
    self.analysis_results['time'] = time_agg
    return region_agg, product_agg, time_agg
def trend_analysis(self):
    print("\n趋势分析...")
    # 移动平均
    self.cleaned_df['MA_7'] = self.cleaned_df['金额'].rolling(window=7).mean()
```

```
    # 增长率
    self.cleaned_df['增长率'] = self.cleaned_df['金额'].pct_change() * 100
    self.analysis_results['trend'] = self.cleaned_df[['金额', 'MA_7', '增长率']]
    return self.cleaned_df[['金额', 'MA_7', '增长率']]
def create_visualizations(self):
    print("\n创建可视化...")
    fig, axes = plt.subplots(2, 2, figsize=(15, 10))
    # 销售趋势图
    self.analysis_results['time'].plot(y='金额', ax=axes[0, 0], title='销售趋势')
    axes[0, 0].set_ylabel('金额')
    # 产品占比饼图
    product_sales = self.analysis_results['product']['金额']
    axes[0, 1].pie(product_sales, labels=product_sales.index, autopct='%1.1f%%')
    axes[0, 1].set_title('产品销售占比')
    # 区域对比柱状图
    self.analysis_results['region']['金额'].plot(kind='bar', ax=axes[1, 0], title='区域
    axes[1, 0].set_ylabel('金额')
    axes[1, 0].tick_params(axis='x', rotation=45)
    # 移动平均图
    self.cleaned_df[['金额', 'MA_7']].plot(ax=axes[1, 1], title='销售额与移动平均')
    plt.tight_layout()
    plt.savefig('sales_analysis.png')
    print("图表已保存: sales_analysis.png")
def generate_report(self, filename='sales_report.xlsx'):
    print(f"\n生成报告: {filename}")
    with pd.ExcelWriter(filename, engine='openpyxl') as writer:
        # 原始数据
        self.cleaned_df.to_excel(writer, sheet_name='清洗后数据')
        # 描述性统计
            self.analysis_results['descriptive'].to_excel(writer, sheet_name='描述性统计')
            # 区域分析
            self.analysis_results['region'].to_excel(writer, sheet_name='区域分析')
            # 产品分析
            self.analysis_results['product'].to_excel(writer, sheet_name='产品分析')
            # 时间序列
            self.analysis_results['time'].to_excel(writer, sheet_name='时间序列')
        print(f"报告已生成: {filename}")
# 使用示例
np.random.seed(42)
dates = pd.date_range('2024-01-01', periods=90, freq='D')
regions = ['华东', '华南', '华北', '西南']
products = ['产品A', '产品B', '产品C', '产品D']
data = {
    '日期': np.random.choice(dates, 500),
    '区域': np.random.choice(regions, 500),
    '产品': np.random.choice(products, 500),
    '销量': np.random.randint(10, 200, 500),
    '金额': np.random.uniform(1000, 10000, 500)
```

```
}
df = pd.DataFrame(data)
# 生成报告
generator = SalesReportGenerator(df)
generator.clean_data()
generator.descriptive_statistics()
generator.aggregate_analysis()
generator.trend_analysis()
generator.create_visualizations()
generator.generate_report()
print("\n分析完成！")
```

题目4和题目

和题目5的参考答案由于篇幅限制，这里提供核心实现思路：

解析

59 • 填空 / 简答

编写一个文件同步工具：

1. FileSyncer类：

类：

- 比较源目录和目标目录
- 识别新增、修改、删除的文件
- 执行同步操作

2. 功能要求：

- 支持增量同步
- 支持文件过滤（按扩展名、大小）
- 支持双向同步
- 冲突处理策略

3. 使用生成器：

- scan_directory() ：扫描目录
- find_differences() ：查找差异
- sync_files() ：同步文件

要求：

- 使用 pathlib 进行所有文件操作
- 使用哈希值检测文件变化
- 添加进度显示
- 支持断点续传

答案 ：文件同步工具- 核心是使用 pathlib 遍历目录，计算文件哈希值比较差异，使用生成器逐个处理文件。

解析

60 • 填空 / 简答

实现一个可扩展的插件系统：

1. Plugin基类：

基类：

- 抽象方法： initialize() 、 execute() 、 cleanup()
- 属性：名称、版本、描述

答案 ：插件系统框架- 核心是使用元类或装饰器实现插件自动注册，使用 importlib 动态加载模块，实现钩子系统管理插件生命周期。

评分说明：

- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析

第 6 套

60 题

一、单项选择题

01 • 单项选择

关于列表的可变性，以下说法正确的是：

A. 列表是不可变对象
B. 列表是可变对象，可以修改元素
C. 列表不能添加或删除元素
D. 列表和元组完全相同

答案 B. 列表是可变对象，可以修改元素

解析 列表是Python中的可变对象，可以修改、添加、删除元素。

示例：

```
lst = [1, 2, 3]
print(f"原始列表: {lst}")
# 修改元素
lst[0] = 10
print(f"修改后: {lst}")
# [10, 2, 3]
# 添加元素
lst.append(4)
print(f"添加后: {lst}")
# [10, 2, 3, 4]
# 删除元素
lst.remove(2)
print(f"删除后: {lst}")
# [10, 3, 4]
知识拓展：可变对象 vs 不可变对象
# 可变对象：list, dict, set
mutable_list = [1, 2, 3]
print(id(mutable_list))
mutable_list.append(4)
print(id(mutable_list))
# ID不变
# 不可变对象：int, str, tuple
immutable_str = "hello"
print(id(immutable_str))
immutable_str = immutable_str + " world"
print(id(immutable_str))
# ID改变，创建了新对象
```

02 • 单项选择

以下代码的输出是什么？

```
list1 = [1, 2, 3]
```

```
list2 = list1
list2.append(4)
print(list1)
```

A. [1, 2, 3]
B. [1, 2, 3, 4]
C. [4]
D. 报错

答案 B. [1, 2, 3, 4]

解析 list2 = list1 创建的是引用，两个变量指向同一个对象，修改list2会影响list1。

示例：

```
list1 = [1, 2, 3]
list2 = list1
# 引用赋值
list2.append(4)
print(list1)
# [1, 2, 3, 4]
print(list2)
# [1, 2, 3, 4]
print(id(list1) == id(list2))
# True
```

知识拓展：引用 vs 拷贝

```
# 引用赋值
a = [1, 2, 3]
b = a
print(a is b)
# True
# 浅拷贝
c = a.copy()
print(a is c)
# False
c.append(4)
print(a)
# [1, 2, 3]
print(c)
# [1, 2, 3, 4]
# 切片也是浅拷贝
d = a[:]
print(a is d)
# False
```

03 • 单项选择

关于元组的不可变性，以下说法正确的是：

A. 元组的元素完全不能改变
B. 元组可以添加元素
C. 元组不可变是指元组对象本身不可变
D. 元组可以使用append()方法

答案 C. 元组不可变是指元组对象本身不可变

解析 元组的不可变性是指元组对象本身（元素的引用）不可变，但元素对象本身可以是可变的。

示例：

```
t = (1, 2, 3)
# t[0] = 10 # TypeError: 元组不支持修改
# 但元组中的可变对象可以修改
t_with_list = ([1, 2], 3)
t_with_list[0].append(3)
# 修改列表内容
print(t_with_list)
# ([1, 2, 3], 3)
# 不能修改元组的元素引用
# t_with_list[0] = [4, 5] # TypeError
```

知识拓展：元组的不可变性详解

```
# 元组的元素引用不能改变
t1 = (1, 2, 3)
print(id(t1[0]))
# t1[0] = 10 # 错误：不能改变引用
# 但如果元素是可变对象，对象内容可以改变
t2 = ([1, 2], [3, 4])
print(id(t2[0]))
# 列表的ID
t2[0].append(5)
# 修改列表内容
print(id(t2[0]))
# 列表的ID不变
print(t2)
# ([1, 2, 5], [3, 4])
```

04 • 单项选择

以下代码的输出是什么？

```
t = ([1, 2], [3, 4])
t[0].append(5)
print(t)
```

A. 报错

B. ([1, 2], [3, 4])

C. ([1, 2, 5], [3, 4])

D. ([1, 2], [3, 4, 5])

答案 C. ([1, 2, 5], [3, 4])

解析 元组中的列表是可变对象，可以修改列表的内容，但不能修改元组的元素引用。

示例：

```
t = ([1, 2], [3, 4])
t[0].append(5)
# 修改第一个列表
print(t)
# ([1, 2, 5], [3, 4])
# 不能替换整个列表
# t[0] = [7, 8] # TypeError
```

知识拓展：嵌套可变对象

```
# 元组包含可变对象
t = ([1, 2], {3: 4}, {5, 6})
# 可以修改列表
t[0].append(3)
print(t[0])
# [1, 2, 3]
# 可以修改字典
t[1][7] = 8
print(t[1])
# {3: 4, 7: 8}
# 可以修改集合
t[2].add(7)
print(t[2])
# {5, 6, 7}
# 但不能替换元组的元素
# t[0] = [] # TypeError
```

05 • 单项选择

关于可变对象作为函数默认参数，以下说法正确的是：

A. 完全没有问题
B. 可能导致意外的行为
C. 默认参数只在函数定义时计算一次
D. B和C都正确

答案 D. B和C都正确

解析 可变对象作为默认参数会导致意外行为，因为默认参数只在函数定义时计算一次。

示例：

```
def bad_append(item, lst=[]):
    lst.append(item)
    return lst
print(bad_append(1))
# [1]
print(bad_append(2))
# [1, 2] - 意外!
print(bad_append(3))
# [1, 2, 3] - 意外!
```

知识拓展：默认参数的正确使用

```
# 错误做法
def bad_function(x, lst=[]):
    lst.append(x)
    return lst
# 正确做法
def good_function(x, lst=None):
    if lst is None:
        lst = []
    lst.append(x)
    return lst
print(good_function(1))
# [1]
print(good_function(2))
# [2] - 正确!
```

```
# 查看默认参数对象
print(bad_function.__defaults__)
# ([1, 2, 3],)
```

06 • 单项选择

以下代码的输出是什么？

```
def append_to_list(value, lst=[]):
    lst.append(value)
    return lst
print(append_to_list(1))
print(append_to_list(2))
```

A. [1] [2]

B. [1] [1, 2]

C. [1, 2] [1, 2]

D. 报错

答案 B. [1] [1, 2]

解析 默认参数 [] 在函数定义时创建一次，两次调用共享同一个列表对象。

示例：

```
def append_to_list(value, lst=[]):
    lst.append(value)
    return lst
result1 = append_to_list(1)
print(result1)
# [1]
result2 = append_to_list(2)
print(result2)
# [1, 2]
print(result1 is result2)
                # True - 同一个对象
```

知识拓展：默认参数陷阱详解

```
# 问题根源：默认参数在函数定义时创建
def demo(x, lst=[]):
    lst.append(x)
    return lst
print(id(demo.__defaults__[0]))
# 默认列表的ID
result1 = demo(1)
print(id(result1))
# 相同的ID
result2 = demo(2)
print(id(result2))
# 相同的ID
# 解决方案
def safe_demo(x, lst=None):
    if lst is None:
        lst = []
    # 每次调用创建新列表
    lst.append(x)
    return lst
```

```
result3 = safe_demo(1)
result4 = safe_demo(2)
print(result3)
# [1]
print(result4)
# [2]
print(result3 is result4)
# False
```

07 • 单项选择

如何正确定义带有可变默认参数的函数？

A. 使用空列表[]作为默认值
B. 使用None作为默认值，在函数内部创建列表
C. 使用tuple作为默认值
D. 不能使用可变对象

答案 B. 使用None作为默认值，在函数内部创建列表

解析 使用None作为默认值，在函数内部判断并创建新对象。

示例：

```
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items
print(add_item(1))
# [1]
print(add_item(2))
# [2]
print(add_item(3, [10, 20]))
# [10, 20, 3]
```

知识拓展：其他安全的默认参数模式

```
# 模式1: 使用None
def method1(x, lst=None):
    lst = lst if lst is not None else []
    lst.append(x)
    return lst
# 模式2: 使用不可变对象作为标记
def method2(x, lst=()):
    if not lst:
        lst = []
    else:
        lst = list(lst)
    # 转换为列表
    lst.append(x)
    return lst
# 模式3: 使用*args
def method3(x, *items):
    lst = list(items)
    lst.append(x)
    return lst
print(method3(1, 2, 3))
```

```
# [2, 3, 1]
```

08 • 单项选择

以下代码的输出是什么？

```
def modify_list(lst):
    lst.append(4)
my_list = [1, 2, 3]
modify_list(my_list)
print(my_list)
```

A. [1, 2, 3]
B. [1, 2, 3, 4]
C. [4]
D. 报错

答案 B. [1, 2, 3, 4]

解析 列表是可变对象，函数内部的修改会影响外部的列表。

示例：

```
def modify_list(lst):
    lst.append(4)
my_list = [1, 2, 3]
modify_list(my_list)
print(my_list)
# [1, 2, 3, 4]
知识拓展：可变对象在函数中的行为
# 修改可变对象的内容
def modify_content(lst):
    lst.append(100)
    lst[0] = 999
data = [1, 2, 3]
modify_content(data)
print(data)
# [999, 2, 3, 100]
# 重新赋值不影响外部
def reassign(lst):
    lst = [999]
    # 创建新对象
    print(f"函数内: {lst}")
data = [1, 2, 3]
reassign(data)
# 函数内: [999]
print(data)
# [1, 2, 3] - 外部不变
# 如果需要避免修改外部
def safe_modify(lst):
    lst = lst.copy()
    # 创建副本
    lst.append(100)
    return lst
data = [1, 2, 3]
result = safe_modify(data)
```

```
print(data)
# [1, 2, 3] - 不变
print(result)
# [1, 2, 3, 100]
```

09 • 单项选择

关于列表的浅拷贝和深拷贝，以下说法正确的是：

A. 浅拷贝创建新对象，但元素是引用

B. 深拷贝递归复制所有元素

C. list.copy()是浅拷贝

D. 以上都正确

答案 D. 以上都正确

解析 所有选项都正确：浅拷贝创建新对象但元素是引用，深拷贝递归复制所有元素，list.copy()是浅拷贝。

示例：

```
import copy
original = [[1, 2], [3, 4]]
# 浅拷贝
shallow = original.copy()
shallow[0].append(5)
print(original)
# [[1, 2, 5], [3, 4]] - 受影响
print(shallow)
# [[1, 2, 5], [3, 4]]
# 深拷贝
original = [[1, 2], [3, 4]]
deep = copy.deepcopy(original)
deep[0].append(5)
print(original)
# [[1, 2], [3, 4]] - 不受影响
print(deep)
# [[1, 2, 5], [3, 4]]
```

知识拓展：浅拷贝 vs 深拷贝详解

```
import copy
# 原始对象
original = [[1, 2], [3, 4]]
# 引用赋值
ref = original
ref[0].append(999)
print(original)
# [[1, 2, 999], [3, 4]]
# 浅拷贝的各种方式
original = [[1, 2], [3, 4]]
shallow1 = original.copy()
shallow2 = original[:]
shallow3 = list(original)
shallow4 = copy.copy(original)
# 浅拷贝：外层独立，内层共享
shallow1[0].append(5)
print(original)
```

```
# [[1, 2, 5], [3, 4]]
# 深拷贝：完全独立
original = [[1, 2], [3, 4]]
deep = copy.deepcopy(original)
deep[0].append(5)
print(original)
# [[1, 2], [3, 4]]
print(deep)
# [[1, 2, 5], [3, 4]]
```

10 • 单项选择

以下代码的输出是什么？

```
import copy
list1 = [[1, 2], [3, 4]]
list2 = list1.copy()
list2[0].append(5)
print(list1)
```

A. 1, 2], [3, 4

B. 1, 2, 5], [3, 4

C. 1, 2], [3, 4, 5

D. 报错

答案 B. 1, 2, 5], [3, 4

解析 list1.copy() 是浅拷贝，只复制外层列表，内层列表仍是引用。

示例：

```
import copy
list1 = [[1, 2], [3, 4]]
list2 = list1.copy()
# 浅拷贝
list2[0].append(5)
print(list1)
# [[1, 2, 5], [3, 4]]
print(list2)
# [[1, 2, 5], [3, 4]]
```

知识拓展：浅拷贝的影响范围

```
import copy
# 一层嵌套
list1 = [[1, 2], [3, 4]]
list2 = list1.copy()
print("修改内层列表：")
list2[0].append(999)
print(f"list1: {list1}")
# [[1, 2, 999], [3, 4]]
print(f"list2: {list2}")
# [[1, 2, 999], [3, 4]]
print("\n替换外层元素：")
list1 = [[1, 2], [3, 4]]
list2 = list1.copy()
list2[0] = [999]
print(f"list1: {list1}")
```

```
# [[1, 2], [3, 4]] - 不受影响
print(f"list2: {list2}")
# [[999], [3, 4]]
print("\n使用深拷贝：")
list1 = [[1, 2], [3, 4]]
list2 = copy.deepcopy(list1)
list2[0].append(999)
print(f"list1: {list1}")
# [[1, 2], [3, 4]] - 不受影响
print(f"list2: {list2}")
# [[1, 2, 999], [3, 4]]
```

11 • 单项选择

关于Python的id()函数，以下说法正确的是：

A. 返回对象的值
B. 返回对象的内存地址
C. 返回对象的类型
D. 返回对象的大小

答案 B. 返回对象的内存地址

解析 id() 函数返回对象的唯一标识符，在CPython中是对象的内存地址。

示例：

```
a = [1, 2, 3]
print(id(a))
# 对象的内存地址
b = a
print(id(b))
# 与a相同
c = a.copy()
print(id(c))
# 不同的地址
```

知识拓展：id()与对象标识

```
# 相同对象
x = 100
y = 100
print(x is y)
# True（小整数缓存）
print(id(x) == id(y))
# True
# 不同对象
a = [1, 2]
b = [1, 2]
print(a == b)
# True（值相等）
print(a is b)
# False（不是同一对象）
print(id(a) == id(b))
# False
```

12 • 单项选择

以下代码的输出是什么？

```
a = [1, 2, 3]
b = a
c = a.copy()
print(id(a) == id(b), id(a) == id(c))
```

A. True True
B. True False
C. False True
D. False False

答案 B. True False

解析 b是a的引用，id相同；c是a的副本，id不同。
详细示例请参考前面题目的引用与拷贝部分。

13 • 单项选择

关于字符串的不可变性，以下说法正确的是：

A. 字符串可以修改单个字符
B. 字符串是不可变对象
C. 字符串可以使用append()
D. 字符串和列表相同

答案 B. 字符串是不可变对象

解析 字符串是Python中的不可变对象，不能修改其内容。
详细示例请参考前面题目的不可变对象部分。

14 • 单项选择

以下代码的输出是什么？

```
s = "hello"
s[0] = "H"
```

A. "Hello"
B. "hello"
C. 报错TypeError
D. None

答案 C. 报错TypeError

解析 字符串不可变，尝试修改单个字符会引发TypeError。
详细示例请参考前面题目的字符串不可变性部分。

15 • 单项选择

关于函数参数传递，以下说法正确的是：

A. Python使用值传递
B. Python使用引用传递
C. Python使用对象引用传递
D. 取决于参数类型

答案 C. Python使用对象引用传递

解析 Python使用对象引用传递，传递的是对象的引用。
详细示例请参考前面题目的参数传递部分。

16 • 单项选择

以下代码的输出是什么？

```
def change_value(x):
    x = 10
a = 5
change_value(a)
print(a)
```

A. 5
B. 10
C. None
D. 报错

答案 A. 5

解析 整数是不可变对象，函数内重新赋值不影响外部变量。
详细示例请参考前面题目的不可变对象在函数中的行为部分。

17 • 单项选择

关于列表的extend()和append()方法，以下说法正确的是：

A. 两者完全相同
B. extend()添加多个元素，append()添加单个元素
C. extend()不修改原列表
D. append()返回新列表

答案 B. extend()添加多个元素，append()添加单个元素

解析 extend()添加多个元素，append()添加单个元素（作为整体）。
详细示例请参考第1-5套相关题目。

18 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3]
lst.extend([4, 5])
print(lst)
```

A. [1, 2, 3, [4, 5]]
B. [1, 2, 3, 4, 5]
C. 1, 2, 3], [4, 5
D. 报错

答案 B. [1, 2, 3, 4, 5]

解析 extend()将可迭代对象的元素逐个添加到列表中。
详细示例请参考第1-5套相关题目。

19 • 单项选择

关于元组的创建，以下哪个是正确的单元素元组？

A. (1)
B. (1,)
C. [1]
D. {1}

答案 B. (1,)

解析 单元素元组需要逗号：(1,)。

详细示例请参考第1-5套相关题目。

20 • 单项选择

以下代码的输出是什么？

```
t = (1)
print(type(t))
```

A. <class 'tuple'>

B. <class 'int'>

C. <class 'list'>

D. 报错

答案 B. <class 'int'>

解析 (1)是整数，不是元组。

详细示例请参考第1-5套相关题目。

21 • 单项选择

Pandas中，DataFrame的可变性体现在：

A. 可以修改单元格的值

B. 可以添加或删除行列

C. 可以修改索引

D. 以上都正确

答案 D. 以上都正确

解析 DataFrame是可变对象，所有选项都正确。

详细示例请参考第1-5套Pandas相关题目。

22 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
df2 = df
df2['B'] = [4, 5, 6]
print('B' in df.columns)
```

A. False

B. True

C. None

D. 报错

答案 B. True

解析 df2是df的引用，修改df2会影响df。

详细示例请参考前面题目的引用部分。

23 • 单项选择

关于字典的可变性，以下说法正确的是：

A. 字典是不可变对象

B. 字典是可变对象
C. 字典的键必须是可变对象
D. 字典的值必须是不可变对象

答案 B. 字典是可变对象

解析 字典是可变对象。
详细示例请参考第1-5套相关题目。

24 • 单项选择

以下代码的输出是什么？

```
d = {'a': [1, 2]}
d['a'].append(3)
print(d)
```

A. {'a': [1, 2]}
B. {'a': [1, 2, 3]}
C. {'a': 3}
D. 报错

答案 B. {'a': [1, 2, 3]}

解析 字典的值是可变列表，可以修改列表内容。
详细示例请参考前面题目的嵌套可变对象部分。

25 • 单项选择

关于集合的可变性，以下说法正确的是：

A. set是可变的，frozenset是不可变的
B. set是不可变的
C. 两者都可变
D. 两者都不可变

答案 A. set是可变的，frozenset是不可变的

解析 set是可变的，frozenset是不可变的。
详细示例请参考第1-5套相关题目。

26 • 单项选择

以下代码的输出是什么？

```
s = {1, 2, 3}
s.add(4)
print(s)
```

A. {1, 2, 3}
B. {1, 2, 3, 4}
C. {4}
D. 报错

答案 B. {1, 2, 3, 4}

解析 set.add()方法添加元素。
详细示例请参考第1-5套相关题目。

27 • 单项选择

关于列表推导式中的可变性，以下说法正确的是：

A. 列表推导式创建新列表
B. 列表推导式修改原列表
C. 列表推导式返回元组
D. 列表推导式不能使用

答案 A. 列表推导式创建新列表

解析 列表推导式创建新列表，不修改原列表。
详细示例请参考第1-5套相关题目。

28 • 单项选择

以下代码的输出是什么？

```
original = [1, 2, 3]
new = [x * 2 for x in original]
print(original)
```

A. [2, 4, 6]
B. [1, 2, 3]
C. [1, 2, 3, 2, 4, 6]
D. 报错

答案 B. [1, 2, 3]

解析 列表推导式不修改原列表。
详细示例请参考第1-5套相关题目。

29 • 单项选择

关于函数返回可变对象，以下说法正确的是：

A. 返回的是对象的引用
B. 返回的是对象的副本
C. 返回的是新对象
D. 不能返回可变对象

答案 A. 返回的是对象的引用

解析 函数返回可变对象时，返回的是对象的引用。
详细示例请参考前面题目的引用部分。

30 • 单项选择

以下代码的输出是什么？

```
def get_list():
    return [1, 2, 3]
lst1 = get_list()
lst2 = get_list()
print(id(lst1) == id(lst2))
```

A. True
B. False
C. None
D. 报错

答案 B. False

解析 每次调用函数都创建新列表，id不同。
详细示例请参考前面题目的对象创建部分。

二、不定项选择题

31 • 不定项选择

关于Python的可变对象，以下说法正确的有：

A. 列表是可变对象
B. 字典是可变对象
C. 集合是可变对象
D. 字符串是可变对象

答案 A. 列表是可变对象；B. 字典是可变对象；C. 集合是可变对象

解析 字符串是不可变对象，列表、字典、集合是可变对象。
（详细示例请参考前面单选题的可变对象部分）

32 • 不定项选择

关于Python的不可变对象，以下说法正确的有：

A. 元组是不可变对象
B. 字符串是不可变对象
C. 整数是不可变对象
D. frozenset是不可变对象

答案 A. 元组是不可变对象；B. 字符串是不可变对象；C. 整数是不可变对象；D. frozenset是不可变对象

解析 所有选项都是不可变对象。
（详细示例请参考前面单选题的不可变对象部分）

33 • 不定项选择

以下哪些操作会修改原列表：

A. `lst.append(x)`
B. `lst.extend(iterable)`
C. `lst.sort()`
D. `sorted(lst)`

答案 A. `lst.append(x)`；B. `lst.extend(iterable)`；C. `lst.sort()`

解析 `sorted()` 返回新列表，不修改原列表。
（详细示例请参考第1-5套相关题目）

34 • 不定项选择

关于可变对象作为默认参数的问题，以下说法正确的有：

A. 默认参数在函数定义时创建
B. 每次调用函数都会创建新的默认参数
C. 可变默认参数会在多次调用间共享
D. 应该使用None作为默认值

答案 A. 默认参数在函数定义时创建；C. 可变默认参数会在多次调用间共享；D. 应该使用None作为默认值

解析 默认参数在函数定义时创建一次，每次调用不会创建新的默认参数。
（详细示例请参考前面单选题的默认参数陷阱部分）

35 • 不定项选择

以下哪些方法可以创建列表的副本：

A. `lst.copy()`
B. `lst[:]`
C. `list(lst)`
D. `copy.deepcopy(lst)`

答案 A. `lst.copy()`；B. `lst[:]`；C. `list(lst)`；D. `copy.deepcopy(lst)`

解析 所有方法都可以创建列表的副本。
（详细示例请参考前面单选题的浅拷贝部分）

36 • 不定项选择

关于元组的特性，以下说法正确的有：

A. 元组不可变是指不能修改元组的元素引用
B. 元组中的可变对象可以被修改
C. 元组不能作为字典的键
D. 元组比列表占用更少内存

答案 A. 元组不可变是指不能修改元组的元素引用；B. 元组中的可变对象可以被修改；D. 元组比列表占用更少内存

解析 包含不可变元素的元组可以作为字典的键。
（详细示例请参考前面单选题的元组特性部分）

37 • 不定项选择

关于深拷贝和浅拷贝，以下说法正确的有：

A. 浅拷贝只复制对象本身
B. 深拷贝递归复制所有嵌套对象
C. `copy.copy()` 是浅拷贝
D. `copy.deepcopy()` 是深拷贝

答案 A. 浅拷贝只复制对象本身；B. 深拷贝递归复制所有嵌套对象；C. `copy.copy()` 是浅拷贝；D. `copy.deepcopy()` 是深拷贝

解析 所有关于深拷贝和浅拷贝的选项都正确。
（详细示例请参考前面单选题的拷贝部分）

38 • 不定项选择

以下哪些操作不会修改原字符串：

A. `s.upper()`
B. `s.replace('a', 'b')`
C. `s.split()`
D. 所有字符串方法都不修改原字符串

答案 D. 所有字符串方法都不修改原字符串

解析 所有字符串方法都返回新字符串，不修改原字符串。
（详细示例请参考前面单选题的字符串不可变性部分）

39 • 不定项选择

关于函数参数的可变性，以下说法正确的有：

A. 传递不可变对象时，函数内修改不影响外部
B. 传递可变对象时，函数内修改会影响外部
C. 可以通过重新赋值避免修改外部对象
D. 应该在函数内创建副本来避免修改

答案 A. 传递不可变对象时，函数内修改不影响外部；B. 传递可变对象时，函数内修改会影响外部；C. 可以通过重新赋值避免修改外部对象；D. 应该在函数内创建副本来避免修改

解析 所有关于函数参数可变性的选项都正确。
（详细示例请参考前面单选题的参数传递部分）

40 • 不定项选择

关于列表的+=和extend()，以下说法正确的有：

A. `lst += [1, 2]` 会修改原列表
B. `lst = lst + [1, 2]` 创建新列表
C. `lst.extend([1, 2])` 修改原列表
D. 三者效果完全相同

答案 A. `lst += [1, 2]` 会修改原列表；B. `lst = lst + [1, 2]` 创建新列表；C. `lst.extend([1, 2])` 修改原列表

解析 `+=` 修改原列表， `+` 创建新列表， `extend()` 修改原列表。
（详细示例请参考第1-5套相关题目）

41 • 不定项选择

关于Pandas中的视图和副本，以下说法正确的有：

A. 某些操作返回视图
B. 某些操作返回副本
C. 修改视图会影响原DataFrame
D. 使用 `.copy()` 可以创建副本

答案 A. 某些操作返回视图；B. 某些操作返回副本；C. 修改视图会影响原DataFrame；D. 使用 `.copy()` 可以创建副本

解析 所有关于Pandas视图和副本的选项都正确。
（详细示例请参考第1-5套Pandas相关题目）

42 • 不定项选择

关于字典的可变性，以下说法正确的有：

A. 字典的键必须是不可变对象
B. 字典的值可以是任何对象
C. 可以动态添加键值对
D. 可以删除键值对

答案 A. 字典的键必须是不可变对象；B. 字典的值可以是任何对象；C. 可以动态添加键值对；D. 可以删除键值对

解析 所有选项都正确。
（详细示例请参考第1-5套相关题目）

43 • 不定项选择

关于对象的引用计数，以下说法正确的有：

A. Python使用引用计数管理内存
B. 赋值操作增加引用计数
C. del语句减少引用计数
D. 引用计数为0时对象被回收

答案 A. Python使用引用计数管理内存；B. 赋值操作增加引用计数；C. del语句减少引用计数；D. 引用计数为0时对象被回收

解析 所有选项都正确。
（详细示例请参考第1-5套相关题目）

44 • 不定项选择

以下哪些是避免可变默认参数问题的方法：

A. 使用None作为默认值
B. 使用元组代替列表
C. 在函数内部创建新对象
D. 使用不可变对象作为默认值

答案 A. 使用None作为默认值；B. 使用元组代替列表；C. 在函数内部创建新对象；D. 使用不可变对象作为默认值

解析 所有选项都是正确的方法。
（详细示例请参考第1-5套相关题目）

45 • 不定项选择

关于列表的切片操作，以下说法正确的有：

A. 切片返回新列表
B. 切片不修改原列表
C. lst[:] 创建浅拷贝
D. 切片可以用于赋值

答案 A. 切片返回新列表；B. 切片不修改原列表；C. lst[:] 创建浅拷贝；D. 切片可以用于赋值

解析 所有选项都正确。
（详细示例请参考第1-5套相关题目）

三、填空题

46 • 填空 / 简答

列表是 _____ 对象，元组是 _____ 对象。

答案 可变；不可变

解析 （原卷第6套填空答案缺失/错位，依据题目整理）列表 list 是可变对象，元组 tuple 是不可变对象。

47 • 填空 / 简答

可变对象作为函数默认参数时，默认参数在 _____ 时创建，多次调用会 _____ 同一个对象。

答案 函数定义（def 执行）时；共享（指向）同一个对象

解析 （原卷答案缺失，依据题目整理）默认参数只在函数定义时求值一次，之后每次调用都复用同一个对象。

48 • 填空 / 简答

使用 _____ 模块的 _____ 函数可以创建对象的深拷贝。

答案 copy；deepcopy

解析 （原卷答案错位，依据题目整理）copy.deepcopy() 递归复制所有嵌套对象。

49 • 填空 / 简答

在函数中修改可变参数时，如果不想影响外部对象，应该先创建 _____，或者使用 _____ 方法。

答案 副本（拷贝）；copy()（或 deepcopy()）

解析 （原卷答案缺失，依据题目整理）先复制再修改，避免原地修改外部传入的可变对象。

50 • 填空 / 简答

元组的不可变性是指 _____ 不可变，但元组中的 _____ 对象仍然可以被修改。

答案 元组本身的元素引用；可变

解析 （原卷答案缺失，依据题目整理）元组不可变指元素引用不能换，但元素本身若是可变对象仍可改。

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
def add_item(item, items=[]):
    items.append(item)
    return items
list1 = add_item(1)
list2 = add_item(2)
list3 = add_item(3, [])
print(list1)
print(list2)
print(list3)
```

问题：

- (1) 三个print语句的输出分别是什么？
- (2) 为什么list1和list2的结果不是预期的？
- (3) 如何修改函数避免这个问题？

答案 （1）[1, 2]；[1, 2]；[3]
（2）默认参数 items=[] 在函数定义时只创建一次，list1 和 list2 指向同一个列表，第二次调用继续在同一个列表上 append。
（3）改为 items=None，函数内 if items is None: items = []。

解析 （原卷答案缺失，依据题目整理）

52 • 填空 / 简答

阅读以下代码，回答问题：

```
def modify_data(data):
    data.append(4)
```

```
    data = [1, 2, 3]
    data.append(5)
    return data
original = [0]
result = modify_data(original)
print(original)
print(result)
```

问题：

- (1) 两个print语句的输出分别是什么？
- (2) 为什么original被修改了？
- (3) 重新赋值 data = [1, 2, 3] 后为什么不影响original？

答案 （1）[0, 4]；[1, 2, 3, 5]
（2）data.append(4) 原地修改了 original 指向的列表。
（3）data = [1, 2, 3] 只是让 data 重新指向新列表，不再指向 original。

解析 （原卷答案缺失，依据题目整理）

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import copy
list1 = [[1, 2], [3, 4]]
list2 = list1.copy()
list3 = copy.deepcopy(list1)
list1[0].append(5)
list2[1].append(6)
print(list1)
print(list2)
print(list3)
```

问题：

- (1) 三个print语句的输出分别是什么？
- (2) 浅拷贝和深拷贝的区别是什么？
- (3) 什么时候应该使用深拷贝？

答案 （1）[[1, 2, 5], [3, 4, 6]]；[[1, 2, 5], [3, 4, 6]]；[[1, 2], [3, 4]]
（2）浅拷贝只复制外层，内层仍是同一对象；深拷贝递归复制所有层。
（3）对象含嵌套可变对象、且需要完全独立的副本时用 deepcopy。

解析 （原卷答案缺失，依据题目整理）

54 • 填空 / 简答

阅读以下代码，回答问题：

```
t = ([1, 2], 3)
t[0].append(4)
print(t)
try:
    t[1] = 5
except TypeError as e:
    print("错误:", e)
```

问题：

- (1) 第一个print的输出是什么？
- (2) 为什么可以修改t[0]但不能修改t[1]？
- (3) 这说明了元组的什么特性？

答案 （1）([1, 2, 4], 3)
（2）t[0] 是列表，列表本身可变；t[1] 是元组元素引用，元组不允许重新赋值。
（3）元组的不可变性指元素引用固定，元素对象本身可以是可变的。

解析 （原卷答案缺失，依据题目整理）

55 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df1 = pd.DataFrame({'A': [1, 2, 3]})
df2 = df1
df3 = df1.copy()
df1['B'] = [4, 5, 6]
print('B' in df2.columns)
print('B' in df3.columns)
print(id(df1) == id(df2))
```

问题：

- (1) 三个print语句的输出分别是什么？
- (2) df2和df3有什么区别？
- (3) 在Pandas中什么时候需要使用copy()？

答案 （1）True；False；True
（2）df2 与 df1 是同一个对象（别名）；df3 是独立副本。
（3）需要在修改数据时不污染原 DataFrame（尤其是切片/筛选后）时使用 copy()。

解析 （原卷答案缺失，依据题目整理）

五、程序设计题

56 • 填空 / 简答

设计一个 SafeList 类，要求：

1. 继承自
   继承自list
2. 重写修改方法
   重写修改方法：
   - 所有修改操作（append、extend、insert等）都记录日志
   - 提供 undo() 方法撤销最后一次修改
   - 提供 get_history() 查看修改历史
3. 实现深拷贝方法

实现深拷贝方法：

- deep_copy() 返回完全独立的副本
- shallow_copy() 返回浅拷贝

4. 防御性编程

防御性编程：

- 提供 safe_append(item, max_size) 限制列表大小
- 提供 freeze() 方法使列表变为只读

要求：

- 使用装饰器记录操作
- 处理嵌套列表的情况
- 提供清晰的错误提示

答案

```
import copy
from functools import wraps
def log_operation(func):
    @wraps(func)
    def wrapper(self, *args, **kwargs):
        # 保存操作前的状态
        state = copy.deepcopy(list(self))
        self._history.append(('before', state, func.__name__))
        # 执行操作
        result = func(self, *args, **kwargs)
        # 保存操作后的状态
        state = copy.deepcopy(list(self))
        self._history.append(('after', state, func.__name__))
        print(f"操作: {func.__name__}, 参数: {args}")
        return result
    return wrapper
class SafeList(list):
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self._history = []
        self._frozen = False
    def _check_frozen(self):
        if self._frozen:
            raise RuntimeError("列表已被冻结，不能修改")
    @log_operation
    def append(self, item):
        self._check_frozen()
        super().append(item)
    @log_operation
    def extend(self, iterable):
        self._check_frozen()
    super().extend(iterable)
@log_operation
def insert(self, index, item):
    self._check_frozen()
    super().insert(index, item)
@log_operation
def remove(self, item):
    self._check_frozen()
    super().remove(item)
@log_operation
```

```
def pop(self, index=-1):
    self._check_frozen()
    return super().pop(index)
def undo(self):
    if len(self._history) < 2:
        print("没有可撤销的操作")
        return
    # 找到最后一个'before'状态
    for i in range(len(self._history) - 1, -1, -1):
        if self._history[i][0] == 'before':
            state = self._history[i][1]
            self.clear()
            self.extend(state)
            print(f"撤销操作: {self._history[i][2]}")
            break
def get_history(self):
    return [(op, name) for op, _, name in self._history]
def deep_copy(self):
    return copy.deepcopy(self)
def shallow_copy(self):
    return SafeList(self)
    def safe_append(self, item, max_size=None):
        if max_size and len(self) >= max_size:
            raise ValueError(f"列表大小已达到限制: {max_size}")
        self.append(item)
    def freeze(self):
        self._frozen = True
        print("列表已冻结")
    def unfreeze(self):
        self._frozen = False
        print("列表已解冻")
# 使用示例
sl = SafeList([1, 2, 3])
sl.append(4)
sl.extend([5, 6])
print(f"当前列表: {sl}")
print(f"\n操作历史: {sl.get_history()}")
sl.undo()
print(f"撤销后: {sl}")
# 测试深拷贝
sl2 = sl.deep_copy()
sl2.append(100)
print(f"\n原列表: {sl}")
print(f"深拷贝: {sl2}")
# 测试冻结
sl.freeze()
try:
    sl.append(7)
```

```
except RuntimeError as e:
    print(f"\n错误: {e}")
```

解析

57 • 填空 / 简答

编写装饰器和工具函数处理可变参数问题：

1. 装饰器

装饰器 @safe_mutable_default ：

- 自动处理可变默认参数
- 每次调用创建新的默认对象
- 支持list、dict、set等

2. 装饰器

装饰器 @immutable_params ：

- 确保函数不修改传入的可变参数
- 在函数内部自动创建副本
- 修改尝试时给出警告

3. 工具函数

工具函数：

- safe_copy(obj, deep=False) ：智能拷贝
- is_mutable(obj) ：判断对象是否可变
- freeze_object(obj) ：将可变对象转为不可变

演示这些工具的使用场景。

答案

```
import copy
from functools import wraps
import sys
def safe_mutable_default(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        # 获取函数签名
        import inspect
        sig = inspect.signature(func)
        bound = sig.bind_partial(*args, **kwargs)
        # 处理默认参数
        for param_name, param in sig.parameters.items():
            if param_name not in bound.arguments and param.default is not param.empty:
                if isinstance(param.default, (list, dict, set)):
                # 创建新的可变对象
                if isinstance(param.default, list):
                bound.arguments[param_name] = []
                elif isinstance(param.default, dict):
                bound.arguments[param_name] = {}
                elif isinstance(param.default, set):
                bound.arguments[param_name] = set()
        return func(*bound.args, **bound.kwargs)
    return wrapper
def immutable_params(func):
    @wraps(func)
```

```
    def wrapper(*args, **kwargs):
        # 创建参数的副本
        new_args = []
        for arg in args:
            if is_mutable(arg):
                new_args.append(copy.deepcopy(arg))
            else:
                new_args.append(arg)
        new_kwargs = {}
        for key, value in kwargs.items():
            if is_mutable(value):
                new_kwargs[key] = copy.deepcopy(value)
            else:
                new_kwargs[key] = value
        return func(*new_args, **new_kwargs)
    return wrapper
def safe_copy(obj, deep=False):
    """智能拷贝对象"""
    if not is_mutable(obj):
        return obj
    if deep:
        return copy.deepcopy(obj)
    else:
        if isinstance(obj, list):
            return obj.copy()
        elif isinstance(obj, dict):
            return obj.copy()
        elif isinstance(obj, set):
            return obj.copy()
        else:
            return copy.copy(obj)
def is_mutable(obj):
    """判断对象是否可变"""
    immutable_types = (int, float, str, tuple, frozenset, bytes, bool, type(None))
    return not isinstance(obj, immutable_types)
def freeze_object(obj):
    """将可变对象转为不可变对象"""
    if isinstance(obj, list):
        return tuple(freeze_object(item) for item in obj)
    elif isinstance(obj, dict):
        return tuple((k, freeze_object(v)) for k, v in sorted(obj.items()))
    elif isinstance(obj, set):
        return frozenset(freeze_object(item) for item in obj)
    else:
        return obj
# 使用示例
print("=== safe_mutable_default 示例 ===")
@safe_mutable_default
```

```
def add_to_list(item, lst=[]):
    lst.append(item)
    return lst
print(add_to_list(1))
# [1]
print(add_to_list(2))
# [2] 而不是 [1, 2]
print("\n=== immutable_params 示例 ===")
@immutable_params
def modify_list(lst):
    lst.append(999)
    return lst
original = [1, 2, 3]
result = modify_list(original)
print(f"原列表: {original}")
# [1, 2, 3]
print(f"返回值: {result}")
# [1, 2, 3, 999]
print("\n=== 工具函数示例 ===")
data = {'a': [1, 2], 'b': {'c': 3}}
print(f"是否可变: {is_mutable(data)}")
frozen = freeze_object(data)
print(f"冻结后: {frozen}")
```

解析

58 • 填空 / 简答

使用Pandas和Python内置类型，实现数据容器分析：

1. 分析函数

分析函数：

- analyze_mutability(obj) ：分析对象的可变性
- find_mutable_elements(container) ：查找容器中的可变元素
- calculate_memory_usage(obj) ：计算对象内存占用

2. 对比测试

对比测试：

- 列表vs元组的性能对比
- 浅拷贝vs深拷贝的性能对比
- 不同数据结构的内存占用对比

3. 生成报告

生成报告：

- 使用Pandas创建对比表格
- 可视化内存和性能差异
- 给出优化建议

要求：

- 使用装饰器测量性能
- 使用生成器处理大数据
- 导出分析报告

答案

```
import sys
import time
import pandas as pd
from functools import wraps
import matplotlib.pyplot as plt
def measure_performance(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        return result, end - start
    return wrapper
def analyze_mutability(obj):
    """分析对象的可变性"""
    info = {
        'type': type(obj).__name__,
        'is_mutable': is_mutable(obj),
        'size': sys.getsizeof(obj),
        'id': id(obj)
    }
    if isinstance(obj, (list, tuple)):
        info['length'] = len(obj)
        info['has_mutable_elements'] = any(is_mutable(item) for item in obj)
    elif isinstance(obj, dict):
        info['length'] = len(obj)
        info['has_mutable_values'] = any(is_mutable(v) for v in obj.values())
    return info
def find_mutable_elements(container):
    """查找容器中的可变元素"""
    mutable_items = []
    if isinstance(container, (list, tuple)):
        for i, item in enumerate(container):
            if is_mutable(item):
                mutable_items.append((i, type(item).__name__, item))
    elif isinstance(container, dict):
        for key, value in container.items():
            if is_mutable(value):
                mutable_items.append((key, type(value).__name__, value))
    return mutable_items
def calculate_memory_usage(obj):
    """计算对象内存占用"""
    total = sys.getsizeof(obj)
    if isinstance(obj, (list, tuple)):
        for item in obj:
            total += sys.getsizeof(item)
    elif isinstance(obj, dict):
        for key, value in obj.items():
```

```
            total += sys.getsizeof(key) + sys.getsizeof(value)
    return total
def is_mutable(obj):
    immutable_types = (int, float, str, tuple, frozenset, bytes, bool, type(None))
    return not isinstance(obj, immutable_types)
@measure_performance
def test_list_operations(n):
    lst = []
    for i in range(n):
        lst.append(i)
    return lst
@measure_performance
def test_tuple_creation(n):
    return tuple(range(n))
@measure_performance
def test_shallow_copy(obj):
    return obj.copy()
@measure_performance
def test_deep_copy(obj):
    import copy
    return copy.deepcopy(obj)
# 性能对比测试
def performance_comparison():
    sizes = [100, 1000, 10000]
    results = []
    for size in sizes:
        # 列表 vs 元组
        list_result, list_time = test_list_operations(size)
        tuple_result, tuple_time = test_tuple_creation(size)
        # 浅拷贝 vs 深拷贝
        nested_list = [[i] for i in range(size)]
        _, shallow_time = test_shallow_copy(nested_list)
        _, deep_time = test_deep_copy(nested_list)
        results.append({
            'size': size,
            'list_time': list_time,
            'tuple_time': tuple_time,
            'shallow_copy_time': shallow_time,
            'deep_copy_time': deep_time,
            'list_memory': sys.getsizeof(list_result),
            'tuple_memory': sys.getsizeof(tuple_result)
        })
    return pd.DataFrame(results)
# 生成分析报告
def generate_report():
    print("=== 数据容器分析报告 ===\n")
    # 分析不同对象
    test_objects = {
```

```
        '列表': [1, 2, 3, [4, 5]],
    '元组': (1, 2, 3, (4, 5)),
    '字典': {'a': [1, 2], 'b': 3},
    '集合': {1, 2, 3}
}
print("1. 对象可变性分析:")
for name, obj in test_objects.items():
    info = analyze_mutability(obj)
    print(f"\n{name}:")
    for key, value in info.items():
        print(f"
                {key}: {value}")
# 查找可变元素
print("\n\n2. 可变元素检测:")
container = [1, "hello", [2, 3], {'a': 1}, (4, 5)]
mutable = find_mutable_elements(container)
for idx, type_name, item in mutable:
    print(f" 索引{idx}: {type_name} - {item}")
# 性能对比
print("\n\n3. 性能对比:")
df = performance_comparison()
print(df.to_string(index=False))
# 保存报告
df.to_excel('container_analysis_report.xlsx', index=False)
print("\n报告已保存到 container_analysis_report.xlsx")
# 可视化
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
# 时间对比
df.plot(x='size', y=['list_time', 'tuple_time'], ax=axes[0], marker='o')
axes[0].set_title('列表 vs 元组创建时间')
axes[0].set_xlabel('元素数量')
axes[0].set_ylabel('时间(秒)')
# 拷贝时间对比
df.plot(x='size', y=['shallow_copy_time', 'deep_copy_time'], ax=axes[1], marker='o')
axes[1].set_title('浅拷贝 vs 深拷贝时间')
axes[1].set_xlabel('元素数量')
axes[1].set_ylabel('时间(秒)')
plt.tight_layout()
plt.savefig('performance_comparison.png')
print("性能对比图已保存到 performance_comparison.png")
# 运行分析
generate_report()
```

题目4和5的参考答案

的参考答案由于篇幅限制，核心思路：

解析

59 • 填空 / 简答

实现一个配置管理系统，处理可变性问题：
1. Config类：
• 使用不可变对象存储配置
• 提供 get() 、 set() 方法
• 配置变更时创建新版本（不修改旧版本）
2. ConfigHistory类：
• 记录所有配置变更
• 支持回滚到历史版本
• 使用深拷贝保存每个版本
3. 功能要求
功能要求：
• 支持嵌套配置
• 配置验证（类型检查）
• 配置导入导出（JSON/YAML）
• 配置差异对比
要求：
• 使用 pathlib 处理文件
• 确保线程安全
• 实现配置继承

答案 ：配置管理系统- 使用不可变数据结构（如frozendict或自定义类），每次修改创建新版本，使用深拷贝保存历史。

解析

60 • 填空 / 简答

实现各种数据结构之间的安全转换：
1. 转换函数
转换函数：
• list_to_tuple(lst, deep=False) ：列表转元组
• tuple_to_list(tpl, deep=False) ：元组转列表
• dict_to_frozendict(d) ：字典转不可变字典
• mutable_to_immutable(obj) ：通用转换
2. 验证函数
验证函数：
• validate_immutable(obj) ：验证对象完全不可变
• find_mutable_refs(obj) ：查找可变引用
• ensure_deep_immutable(obj) ：确保深度不可变
3. 应用场景
应用场景：
• 将配置数据转为不可变对象
• 缓存键的安全处理
• 函数参数的防御性复制
要求：
• 处理循环引用
• 支持自定义类型
• 提供详细的转换报告

答案 ：数据结构转换工具- 递归处理嵌套结构，检测循环引用，提供完整的可变/不可变转换功能。

评分说明：

- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析

第 7 套

60 题

一、单项选择题

01 • 单项选择

以下代码的输出是什么？

```
def func(x, lst=[]):
    lst.append(x)
    return lst
a = func(1)
b = func(2, [])
c = func(3)
print(len(a))
```

A. 1
B. 2
C. 3
D. 报错

答案 B. 2

解析

02 • 单项选择

关于列表的+=操作，以下说法正确的是：

```
lst1 = [1, 2]
lst2 = lst1
lst1 += [3, 4]
```

A. lst2不受影响
B. lst2也变成[1, 2, 3, 4]
C. 创建了新列表
D. 报错

答案 B. lst2也变成[1, 2, 3, 4]

解析

03 • 单项选择

以下代码的输出是什么？

```
t = (1, 2, [3, 4])
t[2] += [5]
```

A. 正常执行，t变为(1, 2, [3, 4, 5])
B. 抛出TypeError
C. t不变
D. A和B都对（先修改后报错）

答案 D. A和B都对（先修改后报错）

解析

04 • 单项选择

关于字典的get()方法和直接索引，以下说法正确的是：

A. 两者完全相同
B. get()不存在时返回None，索引会抛出KeyError
C. get()更慢
D. 索引更安全

答案 B. get()不存在时返回None，索引会抛出KeyError

解析

05 • 单项选择

以下代码的输出是什么？

```
d = {'a': 1}
d2 = d
d2['b'] = 2
print(len(d))
```

A. 1
B. 2
C. 0
D. 报错

答案 B. 2

解析

06 • 单项选择

关于列表的sort()和sorted()，以下说法正确的是：

A. sort()返回新列表
B. sorted()修改原列表
C. sort()修改原列表，sorted()返回新列表
D. 两者完全相同

答案 C. sort()修改原列表，sorted()返回新列表

解析

07 • 单项选择

以下代码的输出是什么？

```
lst = [3, 1, 2]
result = sorted(lst)
print(lst)
```

A. [1, 2, 3]
B. [3, 1, 2]
C. None
D. 报错

答案 B. [3, 1, 2]

解析

08 • 单项选择

关于集合的可变性，以下说法正确的是：

A. 集合元素必须是可变对象
B. 集合元素必须是不可变对象
C. 集合元素可以是任何对象
D. 集合不能包含数字

答案 B. 集合元素必须是不可变对象

解析

09 • 单项选择

以下代码会发生什么？

```
s = {[1, 2], [3, 4]}
```

A. 正常创建集合
B. 抛出TypeError
C. 创建空集合
D. 创建列表

答案 B. 抛出TypeError

解析

10 • 单项选择

关于函数参数的传递，以下说法正确的是：

```
def modify(x):
    x = x + [4]
    return x
lst = [1, 2, 3]
result = modify(lst)
```

A. lst被修改
B. lst不被修改
C. result是None
D. 报错

答案 B. lst不被修改

解析

11 • 单项选择

以下代码的输出是什么？

```
def append_and_return(lst):
    lst.append(4)
    return lst
original = [1, 2, 3]
new = append_and_return(original)
print(original is new)
```

A. True
B. False

C. None
D. 报错

答案 A. True

解析

12 • 单项选择

关于字符串的拼接，以下哪种方式效率最高？

A. 使用+循环拼接
B. 使用join()方法
C. 使用+=
D. 都一样

答案 B. 使用join()方法

解析

13 • 单项选择

以下代码的输出是什么？

```
s1 = "hello"
s2 = s1
s1 += " world"
print(s2)
```

A. "hello world"
B. "hello"
C. ""
D. 报错

答案 B. "hello"

解析

14 • 单项选择

关于列表的clear()方法，以下说法正确的是：

A. 返回新的空列表
B. 清空原列表
C. 删除列表对象
D. 不修改列表

答案 B. 清空原列表

解析

15 • 单项选择

以下代码的输出是什么？

```
lst1 = [1, 2, 3]
lst2 = lst1
lst1.clear()
print(lst2)
```

A. [1, 2, 3]
B. []

C. None
D. 报错

答案 B. []

解析

16 • 单项选择

Pandas中，以下哪个操作会创建视图而不是副本？

A. df'col1', 'col2'
B. df.loc[:, 'col1']
C. df.iloc[0:5]
D. 不确定，取决于具体情况

答案 D. 不确定，取决于具体情况

解析

17 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
view = df['A']
view[0] = 100
print(df['A'][0])
```

A. 1
B. 100
C. 可能是1或100
D. 报错

答案 C. 可能是1或100

解析

18 • 单项选择

关于元组的性能优势，以下说法正确的是：

A. 元组比列表占用更少内存
B. 元组的访问速度更快
C. 元组可以作为字典的键
D. 以上都正确

答案 D. 以上都正确

解析

19 • 单项选择

以下代码的输出是什么？

```
t1 = (1, 2, 3)
t2 = (1, 2, 3)
print(t1 is t2)
```

A. True
B. False

C. 可能是True或False

D. 报错

答案 C. 可能是True或False

解析

20 • 单项选择

关于列表的reverse()和reversed()，以下说法正确的是：

A. reverse()返回新列表

B. reversed()修改原列表

C. reverse()修改原列表，reversed()返回迭代器

D. 两者完全相同

答案 C. reverse()修改原列表，reversed()返回迭代器

解析

21 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3]
rev = reversed(lst)
print(type(rev))
```

A. <class 'list'>

B. <class 'list_reverseiterator'>

C. <class 'reversed'>

D. <class 'iterator'>

答案 B. <class 'list_reverseiterator'>

解析

22 • 单项选择

关于字典的keys()、values()、items()，以下说法正确的是：

A. 返回列表

B. 返回视图对象

C. 返回元组

D. 返回集合

答案 B. 返回视图对象

解析

23 • 单项选择

以下代码的输出是什么？

```
d = {'a': 1, 'b': 2}
keys = d.keys()
d['c'] = 3
print(len(keys))
```

A. 2

B. 3

C. 0

D. 报错

答案 B. 3

解析

24 • 单项选择

关于列表的*操作符，以下说法正确的是：

```
lst = [[]] * 3
lst[0].append(1)
```

A. 只有lst[0]被修改
B. 所有子列表都被修改
C. 报错
D. 创建深拷贝

答案 B. 所有子列表都被修改

解析

25 • 单项选择

以下代码的输出是什么？

```
matrix = [[0] * 2] * 3
matrix[0][0] = 1
print(matrix)
```

A. 1, 0], [0, 0], [0, 0
B. 1, 0], [1, 0], [1, 0
C. 0, 0], [0, 0], [0, 0
D. 报错

答案 B. 1, 0], [1, 0], [1, 0

解析

26 • 单项选择

关于函数的默认参数求值时机，以下说法正确的是：

A. 每次调用时求值
B. 函数定义时求值一次
C. 第一次调用时求值
D. 随机求值

答案 B. 函数定义时求值一次

解析

27 • 单项选择

以下代码的输出是什么？

```
import time
def func(t=time.time()):
    return t
a = func()
time.sleep(1)
b = func()
```

```
print(a == b)
```

A. True
B. False
C. 可能是True或False
D. 报错

答案 A. True

解析

28 • 单项选择

关于列表的pop()方法，以下说法正确的是：

A. 不修改原列表
B. 修改原列表并返回删除的元素
C. 只能删除最后一个元素
D. 返回新列表

答案 B. 修改原列表并返回删除的元素

解析

29 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3, 4, 5]
x = lst.pop(2)
print(x, len(lst))
```

A. 3 4
B. 2 4
C. 3 5
D. 报错

答案 A. 3 4

解析

30 • 单项选择

关于Pandas的inplace参数，以下说法正确的是：

A. inplace=True修改原对象
B. inplace=False返回新对象
C. 默认值通常是False
D. 以上都正确

答案 D. 以上都正确

解析

二、不定项选择题

31 • 不定项选择

以下哪些是可变对象？

A. list

B. dict
C. set
D. bytearray

答案 A. list；B. dict；C. set；D. bytearray

解析 题目）

32 • 不定项选择

以下哪些操作会创建列表的副本？

A. lst[:]
B. lst.copy()
C. list(lst)
D. lst

答案 A. lst[:]；B. lst.copy()；C. list(lst)；D. lst

解析 目）

33 • 不定项选择

关于元组包含可变对象，以下说法正确的有：

A. 元组本身不可变
B. 元组中的可变对象可以被修改
C. 这样的元组不能作为字典的键
D. 元组的哈希值会改变

答案 A. 元组本身不可变；B. 元组中的可变对象可以被修改；C. 这样的元组不能作为字典的键

解析

34 • 不定项选择

以下哪些方法会修改原列表？

A. append()
B. extend()
C. insert()
D. remove()

答案 A. append()；B. extend()；C. insert()；D. remove()

解析

35 • 不定项选择

关于字符串的不可变性，以下说法正确的有：

A. 字符串操作总是返回新字符串
B. 字符串不能被修改
C. 字符串拼接效率较低
D. 字符串可以作为字典的键

答案 A. 字符串操作总是返回新字符串；B. 字符串不能被修改；C. 字符串拼接效率较低；D. 字符串可以作为字典的键

解析

36 • 不定项选择

以下哪些是避免可变默认参数问题的方法？

A. 使用None作为默认值
B. 使用元组代替列表
C. 在函数内部创建新对象
D. 使用不可变对象

答案 A. 使用None作为默认值；B. 使用元组代替列表；C. 在函数内部创建新对象；D. 使用不可变对象

解析

37 • 不定项选择

关于深拷贝和浅拷贝的应用场景，以下说法正确的有：

A. 简单对象用浅拷贝
B. 嵌套对象用深拷贝
C. 深拷贝更安全但更慢
D. 浅拷贝更快但可能有副作用

答案 A. 简单对象用浅拷贝；B. 嵌套对象用深拷贝；C. 深拷贝更安全但更慢；D. 浅拷贝更快但可能有副作用

解析

38 • 不定项选择

以下哪些操作不会修改原字典？

A. d.get(key)
B. d.keys()
C. d.pop(key)
D. d.items()

答案 A. d.get(key)；B. d.keys()；D. d.items()

解析

39 • 不定项选择

关于列表的切片赋值，以下说法正确的有：

A. 可以改变列表长度
B. 会修改原列表
C. 可以删除元素
D. 可以插入元素

答案 A. 可以改变列表长度；B. 会修改原列表；C. 可以删除元素；D. 可以插入元素

解析

40 • 不定项选择

关于函数参数的可变性陷阱，以下说法正确的有：

A. 传递可变对象时要小心
B. 函数内部修改会影响外部
C. 可以通过传递副本避免
D. 不可变对象没有这个问题

答案 A. 传递可变对象时要小心；B. 函数内部修改会影响外部；C. 可以通过传递副本避免；D. 不可变对象没有这个问题

解析

41 • 不定项选择

以下哪些是Pandas中创建副本的方法？

A. df.copy()
B. df.copy(deep=True)
C. df[:]
D. df.loc[:, :]

答案 A. df.copy()；B. df.copy(deep=True)

解析

42 • 不定项选择

关于集合的元素要求，以下说法正确的有：

A. 元素必须可哈希
B. 元素必须不可变
C. 不能包含列表
D. 可以包含元组

答案 A. 元素必须可哈希；B. 元素必须不可变；C. 不能包含列表；D. 可以包含元组

解析

43 • 不定项选择

关于列表推导式和可变性，以下说法正确的有：

A. 列表推导式创建新列表
B. 不会修改原数据
C. 可以包含可变对象
D. 比for循环更安全

答案 A. 列表推导式创建新列表；B. 不会修改原数据；C. 可以包含可变对象

解析

44 • 不定项选择

以下哪些操作会改变对象的id？

A. 列表的append()
B. 字符串的拼接
C. 整数的加法
D. 元组的拼接

答案 B. 字符串的拼接；C. 整数的加法；D. 元组的拼接

解析

45 • 不定项选择

关于Pandas的链式索引警告，以下说法正确的有：

A. 可能产生副本

B. 修改可能不生效
C. 应该使用loc或iloc
D. 可以忽略警告

答案 A. 可能产生副本；B. 修改可能不生效；C. 应该使用loc或iloc

解析

三、填空题

46 • 填空 / 简答

列表的 _____ 方法会修改原列表，而 _____ 函数返回新列表。

答案 sort() ， sorted()

解析 列表排序的两种方式：
• sort() ：就地排序
就地排序，修改原列表，返回None
• sorted() ：返回新排序列表
新排序列表，原列表不变

```
lst = [3, 1, 2]
lst.sort()
# 修改lst，lst=[1,2,3]
new_lst = sorted([3,1,2])
# 返回新列表[1,2,3]
```

47 • 填空 / 简答

可变对象作为默认参数时，应该使用 _____ 作为默认值，然后在函数内部判断并创建 _____。

答案 None ， 新对象

解析 新对象
就地修改方法通常返回None。

```
lst = [3, 1, 2]
result = lst.sort()
# 就地排序
print(result)
# None
print(lst)
# [1, 2, 3]
lst2 = [3, 1, 2]
result2 = sorted(lst2)
# 返回新列表
print(result2)
# [1, 2, 3]
print(lst2)
# [3, 1, 2] - 未变
```

48 • 填空 / 简答

元组的不可变性体现在 _____ 不可变，但如果元组包含 _____ 对象，该对象的内容仍可修改。

答案 元素引用

解析 元素引用 ， 可变
可变
幂析：浅拷贝的元素共享问题。

```
import copy
original = [1, [2, 3], 4]
shallow = copy.copy(original)
shallow[1].append(999)
print(original)
# [1, [2, 3, 999], 4] - 受影响
```

49 • 填空 / 简答

在Pandas中，使用 _____ 参数可以确保操作返回副本而不是视图，使用 _____ 方法可以显式创建副本。

答案 copy=True ， copy()

解析 幂析：Pandas复制数据的方式。

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
# copy方法
df_copy = df.copy()
df_copy.loc[0, 'A'] = 999
print(df)
# 原数据未变
```

50 • 填空 / 简答

字符串是 _____ 对象，所有字符串方法都返回 _____ 而不修改原字符串。

答案 不可变

解析 不可变 ， 新字符串
新字符串
幂析：字符串不可变性。

```
s = "hello"
s_upper = s.upper()
# 返回新字符串
print(s)
# hello - 未变
print(s_upper)
# HELLO
```

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class DataHolder:
    def __init__(self, data=[]):
        self.data = data
    def add(self, item):
        self.data.append(item)
```

```
obj1 = DataHolder()
obj2 = DataHolder()
obj1.add(1)
obj2.add(2)
print(obj1.data)
print(obj2.data)
```

问题：

- (1) 两个print语句的输出分别是什么？
- (2) 为什么会出现这种情况？
- (3) 如何修改类定义避免这个问题？

答案 • (1) 输出：

```
[1, 2]
```

- (2) items是类属性，所有实例共享同一个列表对象
- (3) 修改方法：

```
    class DataHolder:
        def __init__(self, data=None):
            self.data = data if data is not None else []
```

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
def process_list(lst, operation='append', value=None):
    if operation == 'append':
        lst.append(value)
    elif operation == 'new':
        lst = [value]
    elif operation == 'extend':
        lst.extend(value)
    return lst
data = [1, 2, 3]
result1 = process_list(data, 'append', 4)
result2 = process_list(data, 'new', 10)
result3 = process_list(data, 'extend', [5, 6])
print(data)
print(result2)
```

问题：

- (1) 两个print语句的输出分别是什么？
- (2) 为什么'new'操作不影响原列表？
- (3) 哪些操作修改了原列表？

答案 • (1) 输出：

```
    [1, 2, 3, 4, 5, 6]
    [10]
```

- (2) 'new'操作中 lst = [value] 创建新对象，不影响原列表
- (3) 'append'和'extend'操作修改了原列表

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
# 方法1
subset1 = df[df['A'] > 1]
subset1['C'] = [10, 20]
# 方法2
subset2 = df[df['A'] > 1].copy()
subset2['D'] = [30, 40]
print('C' in df.columns)
print('D' in df.columns)
```

问题：

- (1) 两个print语句的输出分别是什么？
- (2) 为什么需要使用copy()？
- (3) 什么是SettingWithCopyWarning?

答案 使用

- (1) 输出：

```
False
```

或可能是： True False （取决于Pandas版本）

- (2) 使用copy()创建独立副本，修改副本不影响原DataFrame
- (3) SettingWithCopyWarning是Pandas在链式索引可能修改副本而非原数据时发出的警告

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
def create_matrix(rows, cols, default=0):
    return [[default] * cols for _ in range(rows)]
def create_matrix_wrong(rows, cols, default=0):
    return [[default] * cols] * rows
matrix1 = create_matrix(3, 3, 0)
matrix2 = create_matrix_wrong(3, 3, 0)
matrix1[0][0] = 1
matrix2[0][0] = 1
print(matrix1)
print(matrix2)
```

问题：

- (1) 两个print语句的输出分别是什么？
- (2) 为什么matrix2的所有行都被修改了？
- (3) 正确的矩阵创建方法是什么？

答案 • (1) 输出：

```
[[1, 0, 0], [0, 0, 0], [0, 0, 0]]
[[1, 0, 0], [1, 0, 0], [1, 0, 0]]
```

- (2) [[default] * cols] * rows 创建的是同一个列表的多个引用
- (3) 使用列表推导式： [[default] * cols for _ in range(rows)]

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
def update_config(config, updates):
    for key, value in updates.items():
        if isinstance(value, dict) and key in config:
            update_config(config[key], value)
        else:
            config[key] = value
    return config
original_config = {
    'database': {'host': 'localhost', 'port': 5432},
    'debug': True
}
new_config = update_config(original_config, {
    'database': {'port': 3306},
    'debug': False
})
print(original_config is new_config)
print(original_config['database']['port'])
```

问题：

- (1) 两个print语句的输出分别是什么？
- (2) 这个函数有什么问题？
- (3) 如何修改使其不影响原配置？

答案 • (1) 输出：

```
True
3306
```

- (2) 函数直接修改了传入的config对象
- (3) 修改方法：

```
def update_config(config, updates):
    import copy
    new_config = copy.deepcopy(config)
    for key, value in updates.items():
        if isinstance(value, dict) and key in new_config:
            new_config[key] = update_config(new_config[key], value)
        else:
            new_config[key] = value
    return new_config
```

解析

五、程序设计题

56 • 填空 / 简答

实现一个不可变列表类 ImmutableList ：

1. 基本功能

基本功能：
- 支持索引访问
- 支持切片
- 支持迭代
- 支持len()

2. 不可变性保证

不可变性保证：
- 任何修改操作返回新对象
- 原对象保持不变
- 实现 append() 、 extend() 等方法（返回新对象）

3. 性能优化

性能优化：
- 使用元组内部存储
- 实现结构共享（避免完全复制）
- 提供 to_list() 转换方法

4. 额外功能

额外功能：
- 实现 __hash__() 使其可作为字典键
- 实现 __add__() 支持+操作
- 提供 map() 、 filter() 等函数式方法

答案
```
class ImmutableList:
    """
    不可变列表实现
    设计思路：
    1. 内部使用元组存储数据，保证不可变性
    2. 所有修改操作返回新的ImmutableList对象
    3. 实现__hash__使其可作为字典键
    4. 提供函数式编程接口
    """
    def __init__(self, items=()):
        # 使用元组作为内部存储，确保不可变性
        if isinstance(items, tuple):
            self._data = items
        else:
            self._data = tuple(items)
    # === 基本访问方法 ===
    def __getitem__(self, index):
        """支持索引和切片访问"""
        if isinstance(index, slice):
            return ImmutableList(self._data[index])
        return self._data[index]
    def __len__(self):
        """返回长度"""
        return len(self._data)
    def __iter__(self):
        """支持迭代"""
        return iter(self._data)
```

```
    def __repr__(self):
        """字符串表示"""
        return f"ImmutableList({list(self._data)})"
    # === 不可变性保证 ===
    def __hash__(self):
    """实现哈希，使其可作为字典键"""
    return hash(self._data)
def __eq__(self, other):
    """相等性比较"""
    if isinstance(other, ImmutableList):
        return self._data == other._data
    return False
# === 修改操作（返回新对象）===
def append(self, item):
    """添加元素，返回新对象"""
    return ImmutableList(self._data + (item,))
def extend(self, items):
    """扩展列表，返回新对象"""
    return ImmutableList(self._data + tuple(items))
def insert(self, index, item):
    """插入元素，返回新对象"""
    return ImmutableList(self._data[:index] + (item,) + self._data[index:])
def remove(self, item):
    """移除元素，返回新对象"""
    data_list = list(self._data)
    data_list.remove(item)
    return ImmutableList(data_list)
def pop(self, index=-1):
    """弹出元素，返回(新列表, 弹出的元素)"""
    data_list = list(self._data)
    popped = data_list.pop(index)
    return ImmutableList(data_list), popped
# === 运算符重载 ===
def __add__(self, other):
    """支持+操作"""
    if isinstance(other, ImmutableList):
        return ImmutableList(self._data + other._data)
    return ImmutableList(self._data + tuple(other))
    def __mul__(self, n):
        """支持*操作"""
        return ImmutableList(self._data * n)
    # === 函数式方法 ===
    def map(self, func):
        """映射操作"""
        return ImmutableList(func(item) for item in self._data)
    def filter(self, predicate):
        """过滤操作"""
        return ImmutableList(item for item in self._data if predicate(item))
```

```
    def reduce(self, func, initial=None):
        """归约操作"""
        from functools import reduce as functools_reduce
        if initial is None:
            return functools_reduce(func, self._data)
        return functools_reduce(func, self._data, initial)
    # === 转换方法 ===
    def to_list(self):
        """转换为普通列表"""
        return list(self._data)
    def to_tuple(self):
        """转换为元组"""
        return self._data
# === 使用示例 ===
print("=== ImmutableList 示例 ===\n")
# 创建不可变列表
lst = ImmutableList([1, 2, 3])
print(f"原列表: {lst}")
# 添加元素（返回新对象）
lst2 = lst.append(4)
print(f"添加后: {lst2}")
print(f"原列表不变: {lst}")
# 扩展列表
lst3 = lst2.extend([5, 6])
print(f"扩展后: {lst3}")
# 函数式操作
doubled = lst.map(lambda x: x * 2)
print(f"\nMap操作: {doubled}")
evens = lst3.filter(lambda x: x % 2 == 0)
print(f"Filter操作: {evens}")
total = lst3.reduce(lambda a, b: a + b)
print(f"Reduce操作: {total}")
# 作为字典键
d = {lst: "value1", lst2: "value2"}
print(f"\n作为字典键: {d}")
# 链式操作
result = (ImmutableList([1, 2, 3])
    .append(4)
    .extend([5, 6])
    .map(lambda x: x ** 2)
    .filter(lambda x: x > 10))
print(f"链式操作结果: {result}")
```

解析

57 • 填空 / 简答

实现参数验证和保护装饰器：

1. @validate_types：

- 验证参数类型
- 支持类型注解
- 类型不匹配时抛出异常

2. @protect_mutable：
- 自动复制可变参数
- 防止函数修改外部对象
- 可配置深拷贝或浅拷贝

3. @freeze_return：
- 将返回的可变对象转为不可变
- 列表转元组
- 字典转frozendict

4. 综合示例

综合示例：
- 组合使用多个装饰器
- 处理复杂嵌套结构
- 提供详细的错误信息

答案
```
import copy
from functools import wraps
import inspect
from typing import get_type_hints
def validate_types(func):
    """
    验证参数类型装饰器
    功能说明：
    1. 基于函数的类型注解验证参数类型
    2. 类型不匹配时抛出TypeError
    3. 保留原函数的元数据
    """
    @wraps(func)
    def wrapper(*args, **kwargs):
        # 获取函数签名和类型注解
        sig = inspect.signature(func)
        bound = sig.bind(*args, **kwargs)
        bound.apply_defaults()
        # 获取类型提示
        hints = get_type_hints(func)
        # 验证每个参数
        for param_name, param_value in bound.arguments.items():
            if param_name in hints:
                expected_type = hints[param_name]
                if not isinstance(param_value, expected_type):
                raise TypeError(
                f"参数 '{param_name}' 期望类型{expected_type.__name__}, "
                f"但得到{type(param_value).__name__}"
                )
        return func(*args, **kwargs)
    return wrapper
```

```
def protect_mutable(deep=False):
    """
    保护可变参数装饰器
    参数说明：
    - deep: True使用深拷贝，False使用浅拷贝
    功能说明：
    1. 自动复制可变参数
    2. 防止函数修改外部对象
    3. 可配置拷贝深度
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            # 复制可变参数
            new_args = []
            for arg in args:
                if isinstance(arg, (list, dict, set)):
                if deep:
                new_args.append(copy.deepcopy(arg))
                else:
                new_args.append(copy.copy(arg))
                else:
                new_args.append(arg)
            new_kwargs = {}
            for key, value in kwargs.items():
                if isinstance(value, (list, dict, set)):
                if deep:
                new_kwargs[key] = copy.deepcopy(value)
                else:
                new_kwargs[key] = copy.copy(value)
                else:
                new_kwargs[key] = value
            return func(*new_args, **new_kwargs)
        return wrapper
    return decorator
def freeze_return(func):
    """
    冻结返回值装饰器
    功能说明：
    1. 将返回的可变对象转为不可变
    2. list → tuple
    3. dict → frozendict (使用元组模拟)
    4. set → frozenset
    """
    @wraps(func)
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return _make_immutable(result)
```

```
    return wrapper
def _make_immutable(obj):
    """递归将对象转为不可变"""
    if isinstance(obj, list):
        return tuple(_make_immutable(item) for item in obj)
    elif isinstance(obj, dict):
        # 使用元组模拟frozendict
        return tuple((k, _make_immutable(v)) for k, v in sorted(obj.items()))
    elif isinstance(obj, set):
        return frozenset(_make_immutable(item) for item in obj)
    else:
        return obj
# === 使用示例 ===
print("=== 参数验证装饰器示例 ===\n")
# 示例1：类型验证
@validate_types
def greet(name: str, age: int) -> str:
    return f"Hello {name}, you are {age} years old"
try:
    print(greet("Alice", 25))
    print(greet("Bob", "30"))
# 类型错误
except TypeError as e:
    print(f"类型错误: {e}")
# 示例2：保护可变参数
@protect_mutable(deep=True)
def modify_list(lst):
    """这个函数修改列表，但不会影响外部"""
    lst.append(999)
    return lst
original = [1, 2, 3]
result = modify_list(original)
print(f"\n原列表: {original}")
# [1, 2, 3]
print(f"返回值: {result}")
# [1, 2, 3, 999]
# 示例3：冻结返回值
@freeze_return
def create_config():
    """返回配置字典，但会被冻结"""
    return {
        'name': 'app',
        'settings': {'debug': True, 'port': 8000},
        'features': ['auth', 'api']
    }
config = create_config()
print(f"\n冻结的配置: {config}")
print(f"类型: {type(config)}")
```

```
# 示例4：组合使用装饰器
@freeze_return
@protect_mutable()
@validate_types
def process_data(data: list, multiplier: int) -> list:
    """处理数据：验证类型、保护参数、冻结返回值"""
    return [x * multiplier for x in data]
data = [1, 2, 3]
result = process_data(data, 2)
print(f"\n原数据: {data}")
print(f"处理结果: {result}")
print(f"结果类型: {type(result)}")
```

解析

58 • 填空 / 简答

实现考虑可变性的缓存系统：

1. CacheKey类：

- 将可变对象转为可哈希的键
- 支持列表、字典等
- 处理嵌套结构

2. SmartCache类：

- 使用不可变键存储
- 自动检测参数变化
- 支持TTL过期

3. 缓存策略

缓存策略：

- LRU淘汰策略
- 基于内存限制的淘汰
- 缓存统计和监控

4. 应用示例

应用示例：

- 装饰器形式使用
- 函数结果缓存
- 处理可变参数的函数

答案

```
from collections import OrderedDict
import time
import hashlib
import pickle
class CacheKey:
    """
    将可变对象转为可哈希的缓存键
    设计思路：
    1. 将任意对象序列化为字符串
    2. 计算哈希值作为缓存键
    3. 处理嵌套结构和循环引用
    """
    @staticmethod
```

```
    def make_key(*args, **kwargs):
        """创建缓存键"""
        try:
            # 尝试直接哈希
            key_data = (args, tuple(sorted(kwargs.items())))
            return hash(key_data)
        except TypeError:
            # 对于不可哈希的对象，使用pickle序列化
            serialized = pickle.dumps((args, kwargs))
            return hashlib.md5(serialized).hexdigest()
class SmartCache:
    """
    智能缓存系统
    功能特性：
    1. 自动处理可变参数
    2. 支持TTL过期
    3. LRU淘汰策略
    4. 缓存统计
    """
def __init__(self, maxsize=128, ttl=None):
    """
    参数说明：
    - maxsize: 最大缓存条目数
    - ttl: 过期时间（秒），None表示不过期
    """
    self.maxsize = maxsize
    self.ttl = ttl
    self.cache = OrderedDict()
    self.timestamps = {}
    self.hits = 0
    self.misses = 0
def get(self, *args, **kwargs):
    """获取缓存值"""
    key = CacheKey.make_key(*args, **kwargs)
    # 检查是否存在
    if key not in self.cache:
        self.misses += 1
        return None
    # 检查是否过期
    if self.ttl is not None:
        if time.time() - self.timestamps[key] > self.ttl:
            del self.cache[key]
            del self.timestamps[key]
            self.misses += 1
            return None
    # 命中：移到末尾（LRU）
    self.cache.move_to_end(key)
    self.hits += 1
```

```
    return self.cache[key]
def put(self, value, *args, **kwargs):
    """存入缓存"""
    key = CacheKey.make_key(*args, **kwargs)
        # 如果已存在，更新并移到末尾
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        self.timestamps[key] = time.time()
        # 检查大小限制
        if len(self.cache) > self.maxsize:
            # 删除最旧的条目
            oldest_key = next(iter(self.cache))
            del self.cache[oldest_key]
            del self.timestamps[oldest_key]
    def clear(self):
        """清空缓存"""
        self.cache.clear()
        self.timestamps.clear()
        self.hits = 0
        self.misses = 0
    def stats(self):
        """获取统计信息"""
        total = self.hits + self.misses
        hit_rate = self.hits / total if total > 0 else 0
        return {
            'hits': self.hits,
            'misses': self.misses,
            'hit_rate': hit_rate,
            'size': len(self.cache),
            'maxsize': self.maxsize
        }
def cached(cache_instance):
    """
    缓存装饰器
    使用方法：
    @cached(my_cache)
    def expensive_function(x, y):
        ...
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            # 尝试从缓存获取
            cached_value = cache_instance.get(*args, **kwargs)
            if cached_value is not None:
                return cached_value
            # 计算结果
```

```
            result = func(*args, **kwargs)
            # 存入缓存
            cache_instance.put(result, *args, **kwargs)
            return result
        wrapper.cache = cache_instance
        return wrapper
    return decorator
# === 使用示例 ===
print("=== 智能缓存系统示例 ===\n")
# 创建缓存实例
cache = SmartCache(maxsize=100, ttl=5)
# 示例1：处理可变参数
@cached(cache)
def process_list(data, multiplier):
    """耗时操作"""
    print(f" 计算: {data} * {multiplier}")
    time.sleep(0.1)
    return [x * multiplier for x in data]
print("第1次调用（未命中）:")
result1 = process_list([1, 2, 3], 2)
print(f"结果: {result1}")
print("\n第2次调用（命中）:")
result2 = process_list([1, 2, 3], 2)
print(f"结果: {result2}")
print("\n第3次调用（不同参数，未命中）:")
result3 = process_list([1, 2, 3], 3)
print(f"结果: {result3}")
# 统计信息
print(f"\n缓存统计: {cache.stats()}")
# 示例2：TTL过期测试
print("\n=== TTL过期测试 ===")
cache_ttl = SmartCache(maxsize=10, ttl=2)
@cached(cache_ttl)
def slow_function(x):
    print(f" 计算{x}")
    return x ** 2
print("首次调用:")
print(slow_function(5))
print("\n立即再次调用（缓存命中）:")
print(slow_function(5))
print("\n等待3秒后调用（缓存过期）:")
time.sleep(3)
print(slow_function(5))
print(f"\n统计: {cache_ttl.stats()}")
```

评分说明：

- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分

- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析

59 • 填空 / 简答

实现数据对象的版本控制系统：

1. VersionedData类：

- 记录所有修改历史
- 每次修改创建新版本
- 支持回滚到任意版本

2. 差异追踪

差异追踪：

- 记录每次修改的差异
- 最小化存储空间
- 支持差异查看

3. 分支管理

分支管理：

- 支持创建分支
- 支持合并分支
- 处理冲突

4. 持久化

持久化：

- 保存版本历史到文件
- 使用pickle或JSON
- 支持增量保存

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析

60 • 填空 / 简答

实现Pandas的安全操作包装器：

1. SafeDataFrame类：

- 继承DataFrame
- 所有修改操作自动创建副本
- 提供明确的inplace控制

2. 链式操作支持

链式操作支持：

- 支持方法链
- 每步返回新对象
- 保持操作历史

3. 视图检测

视图检测：

- 自动检测视图操作
- 警告潜在的副作用
- 提供安全的替代方法

4. 性能优化

性能优化：

- 延迟复制（写时复制）
- 智能判断何时需要复制
- 提供性能分析工具

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析

第 8 套

60 题

一、单项选择题

01 • 单项选择

以下代码的输出是什么？

```
a = [1, 2]
b = a
a = a + [3]
print(b)
```

A. [1, 2, 3]
B. [1, 2]
C. [3]
D. 报错

答案 B. [1, 2]

解析 + vs += 的重要区别：

```
a = [1, 2]
b = a
# b和a指向同一个对象
a = a + [3]
# 创建新列表对象并赋值给a
print(b)
# [1, 2] - b仍指向原对象
```

原理：

- a + [3] 创建新列表对象
- a = ... 将a重新绑定到新对象
- b仍然指向原列表 [1, 2]

内存示意：

初始：a, b → [1, 2]
执行后：a → [1, 2, 3]（新对象）
b → [1, 2]（原对象）

02 • 单项选择

以下代码的输出是什么？

```
a = [1, 2]
b = a
a += [3]
print(b)
```

A. [1, 2, 3]
B. [1, 2]
C. [3]
D. 报错

答案 A. [1, 2, 3]

解析 += 就地修改列表：

```
a = [1, 2]
b = a
# b和a指向同一个对象
a += [3]
# 就地修改原列表
print(b)
# [1, 2, 3] - b也看到了修改
```

关键差异：

- += 对可变对象是就地操作
  就地操作（in-place）
- 不创建新对象，直接修改原对象
- 所有引用都能看到变化

对比总结：

```
# 对于列表（可变对象）
a = a + [3]
# 创建新对象
a += [3]
# 就地修改
# 对于元组（不可变对象）
t = t + (3,)
# 创建新对象
t += (3,)
# 也创建新对象（元组不可变）
```

03 • 单项选择

关于列表的is和==比较，以下说法正确的是：

```
a = [1, 2, 3]
b = [1, 2, 3]
```

A. a is b 为True

B. a == b 为True

C. 两者都为True

D. 两者都为False

答案 B. a == b 为True

解析 is vs == 的本质区别：

```
a = [1, 2, 3]
b = [1, 2, 3]
 == 比较值：
    比较值：
```

- 比较两个对象的内容
  内容是否相同
- a == b → True（内容相同）

```
 is 比较身份：
```

- 比较两个变量是否指向同一个对象
- a is b → False（不同对象）
- 等价于 id(a) == id(b)

示例对比：

```
a = [1, 2, 3]
b = [1, 2, 3]
```

```
# 新对象
c = a
# 同一对象
print(a == b)
# True - 内容相同
print(a is b)
# False - 不同对象
print(a is c)
# True - 同一对象
print(id(a), id(b), id(c))
```

04 • 单项选择

以下代码的输出是什么？

```
def modify(lst=[]):
    lst.append(len(lst))
    return lst
print(modify())
```

A. [0] [0] [0]
B. [0] [0, 1] [0, 1, 2]
C. [0] [1] [2]
D. 报错

答案 B. [0] [0, 1] [0, 1, 2]

解析 可变默认参数陷阱：

```
def modify(lst=[]):
    lst.append(len(lst))
    return lst
print(modify())
# [0]
print(modify())
                # [0, 1] - 共享同一个列表！
```

原因：

- 默认参数在函数定义时
  函数定义时创建，只创建一次
- 所有调用共享同一个默认参数对象
- 可变对象会累积修改

正确写法：

```
def modify(lst=None):
    if lst is None:
        lst = []
    # 每次创建新列表
    lst.append(len(lst))
    return lst
```

知识拓展：默认参数的创建时机：

```
def func(x, y=time.time()):
    # y在函数定义时求值
    return y
func(1)
# 每次返回相同时间戳
```

05 • 单项选择

关于元组的+操作，以下说法正确的是：

```
t1 = (1, 2)
t2 = t1 + (3,)
```

A. t1被修改
B. 创建新元组
C. 报错
D. t1和t2相同

答案 B. 创建新元组

解析 列表切片创建副本：

```
a = [1, 2, 3]
b = a[:]
# 切片创建副本
b.append(4)
print(a)
# [1, 2, 3] - 不受影响
```

切片的复制特性：

- a[:] 创建列表的浅拷贝
  浅拷贝
- 新列表，但元素是原对象的引用

其他复制方式：

```
import copy
# 浅拷贝
b = a.copy()
b = list(a)
b = a[:]
# 深拷贝（嵌套对象）
b = copy.deepcopy(a)
```

06 • 单项选择

以下代码的输出是什么？

```
t = (1, 2, 3)
t_id = id(t)
t = t + (4,)
print(id(t) == t_id)
```

A. True
B. False
C. 可能是True或False
D. 报错

答案 B. False

解析 字典的 get() 方法：

```
d = {'a': 1}
result = d.get('b', 0)
# 键不存在时返回默认值0
```

get() vs 直接索引：

```
d = {'a': 1}
# 直接索引 - 键不存在会报错
value = d['b']
```

```
# KeyError
# get() - 键不存在返回默认值
value = d.get('b', 0)
# 返回0
value = d.get('b')
# 返回None
```

07 • 单项选择

关于字典的update()方法，以下说法正确的是：

A. 返回新字典
B. 修改原字典
C. 不修改任何字典
D. 创建副本

答案 B. 修改原字典

解析 集合的无序性：

```
s = {3, 1, 2}
print(s)
# 输出顺序不确定
```

集合特性：

- 无序
  无序：不保证元素顺序
- 唯一
  唯一：自动去重
- 可哈希元素
  可哈希元素：只能包含不可变对象

注意：Python 3.7+ 字典保持插入顺序，但集合仍然无序

08 • 单项选择

以下代码的输出是什么？

```
d1 = {'a': 1}
d2 = d1
d1.update({'b': 2})
print(len(d2))
```

A. 1
B. 2
C. 0
D. 报错

答案 B. 2

解析 浅拷贝与嵌套对象：

```
import copy
a = [[1, 2], [3, 4]]
b = copy.copy(a)
# 浅拷贝
b[0].append(3)
print(a)
# [[1, 2, 3], [3, 4]] - 内部列表被修改
```

浅拷贝 vs 深拷贝：

深拷贝：

```
# 浅拷贝：只复制第一层
b = copy.copy(a)
b = a.copy()
b = a[:]
# 深拷贝：递归复制所有层
b = copy.deepcopy(a)
```

09 • 单项选择

关于列表的remove()方法，以下说法正确的是：

A. 删除所有匹配的元素
B. 删除第一个匹配的元素
C. 返回删除的元素
D. 不修改原列表

答案 B. 删除第一个匹配的元素

解析 元组的不可变性是浅层的：

```
t = ([1, 2], 3)
t[0].append(3)
# 可以！修改列表内容
print(t)
# ([1, 2, 3], 3)
t[0] = [4, 5]
# 不可以！不能替换元素
```

原理：

- 元组本身不可变（不能修改元组结构）
- 但元组内的可变对象可以被修改

10 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3, 2, 1]
lst.remove(2)
print(lst)
```

A. [1, 3, 1]
B. [1, 3, 2, 1]
C. [1, 2, 3, 1]
D. [1, 1]

答案 B. [1, 3, 2, 1]

解析 字符串的不可变性：

```
s = "hello"
s[0] = 'H'
# TypeError: 字符串不支持修改
# 正确方式：创建新字符串
s = 'H' + s[1:]
# "Hello"
```

不可变对象修改的实质：

```
s = "hello"
s = s.upper()
# 创建新字符串"HELLO"
# 原字符串"hello"仍存在（如果有其他引用）
```

11 • 单项选择

关于集合的add()和update()，以下说法正确的是：

A. add()添加单个元素，update()添加多个元素
B. 两者完全相同
C. add()不修改原集合
D. update()返回新集合

答案 A. add()添加单个元素，update()添加多个元素

解析 字典键的要求：

```
d = {[1, 2]: 'value'}
                # TypeError: 列表不可哈希
```

可作为字典键的类型：

- 不可变类型：int, float, str, tuple, frozenset
- 必须是可哈希的（ hash(obj) 不报错）

示例：

```
# 正确
d = {(1, 2): 'a'}
# 元组可以
d = {'key': 'value'}
# 字符串可以
# 错误
d = {[1, 2]: 'a'}
# 列表不可以
d = {{1}: 'a'}
# 集合不可以
```

12 • 单项选择

以下代码的输出是什么？

```
s = {1, 2, 3}
s.update([3, 4, 5])
print(len(s))
```

A. 3
B. 5
C. 6
D. 8

答案 B. 5

解析 global 关键字的作用：

```
x = 10
def func():
    global x
    x = 20
# 修改全局变量
func()
print(x)
# 20
```

作用域规则（LEGB）：

）：

- Local：局部作用域

- Enclosing：闭包作用域
- Global：全局作用域
- Built-in：内置作用域

13 • 单项选择

关于字符串的replace()方法，以下说法正确的是：

A. 修改原字符串
B. 返回新字符串
C. 只替换第一个匹配
D. 不能使用

答案 B. 返回新字符串

解析 nonlocal 用于闭包：

```
def outer():
    x = 10
    def inner():
        nonlocal x
        x = 20
    # 修改外层函数的x
    inner()
    print(x)
# 20
 global vs nonlocal ：
x = 1
# 全局
def outer():
    x = 2
    # 外层
    def inner():
        nonlocal x
        # 引用外层的x
        x = 3
    inner()
    print(x)
# 3
print(x)
# 1 - 全局不变
```

14 • 单项选择

以下代码的输出是什么？

```
s = "hello"
s_id = id(s)
s = s.replace('l', 'L')
print(id(s) == s_id)
```

A. True
B. False
C. 可能是True或False
D. 报错

答案 B. False

解析 列表推导式中的变量作用域：

```
[x for x in range(3)]
print(x)
# Python 3: NameError
```

Python 2 vs 3 差异：

差异：

- Python 2: 列表推导式变量泄漏到外部
- Python 3: 列表推导式有自己的作用域

15 • 单项选择

关于列表的insert()方法，以下说法正确的是：

A. 返回新列表
B. 修改原列表
C. 不能在开头插入
D. 只能在末尾插入

答案 B. 修改原列表

解析 字典推导式：

```
{k: v for k, v in [('a', 1), ('b', 2)]}
# {'a': 1, 'b': 2}
```

推导式类型：

```
# 列表推导式
[x for x in range(3)]
# [0, 1, 2]
# 集合推导式
{x for x in range(3)}
# {0, 1, 2}
# 字典推导式
{x: x**2 for x in range(3)}
# {0:0, 1:1, 2:4}
# 生成器表达式
(x for x in range(3))
# generator object
```

16-30. 答案：

答案：A A B B B B B C B A B C C B A

这些题目涵盖了Python中可变性、数据结构操作、Pandas等关键概念。

16 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3]
lst.insert(0, 0)
print(lst)
```

A. [0, 1, 2, 3]
B. [1, 2, 3, 0]
C. [1, 0, 2, 3]
D. 报错

答案 A. [0, 1, 2, 3]

解析 （原卷合并作答）

17 • 单项选择

关于Pandas的at和iat，以下说法正确的是：

A. at使用标签，iat使用位置
B. 两者完全相同
C. at更快
D. iat返回副本

答案 A. at使用标签，iat使用位置

解析 （原卷合并作答）

18 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
df.at[0, 'A'] = 10
print(df.at[0, 'A'])
```

A. 1
B. 10
C. None
D. 报错

答案 A. 1

解析 （原卷合并作答）

19 • 单项选择

关于列表的count()方法，以下说法正确的是：

A. 修改列表
B. 返回元素出现次数
C. 删除元素
D. 返回列表长度

答案 A. 修改列表

解析 （原卷合并作答）

20 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 2, 3, 2]
print(lst.count(2))
```

A. 1
B. 2
C. 3
D. 5

答案 A. 1

解析 （原卷合并作答）

21 • 单项选择

关于字典的setdefault()方法，以下说法正确的是：

A. 不修改字典
B. 键存在时返回值，不存在时设置并返回默认值
C. 总是设置新值
D. 删除键

答案 A. 不修改字典

解析 （原卷合并作答）

22 • 单项选择

以下代码的输出是什么？

```
d = {'a': 1}
x = d.setdefault('b', 2)
print(x, len(d))
```

A. None 1
B. 2 2
C. 1 1
D. 报错

答案 A. None 1

解析 （原卷合并作答）

23 • 单项选择

关于列表的index()方法，以下说法正确的是：

A. 修改列表
B. 返回元素第一次出现的索引
C. 元素不存在时返回-1
D. 返回所有索引

答案 A. 修改列表

解析 （原卷合并作答）

24 • 单项选择

以下代码会发生什么？

```
lst = [1, 2, 3]
idx = lst.index(4)
```

A. 返回-1
B. 返回None
C. 抛出ValueError
D. 返回3

答案 A. 返回-1

解析 （原卷合并作答）

25 • 单项选择

关于元组的count()和index()方法，以下说法正确的是：

A. 元组没有这些方法
B. 元组有这些方法且不修改元组
C. 这些方法会修改元组

D. 只有列表有这些方法

答案 A. 元组没有这些方法

解析 （原卷合并作答）

26 • 单项选择

以下代码的输出是什么？

```
t = (1, 2, 2, 3)
print(t.count(2), t.index(2))
```

A. 2 1
B. 2 2
C. 1 1
D. 报错

答案 A. 2 1

解析 （原卷合并作答）

27 • 单项选择

关于集合的discard()和remove()，以下说法正确的是：

A. 完全相同
B. discard()元素不存在时不报错，remove()会报错
C. discard()返回元素
D. remove()不修改集合

答案 A. 完全相同

解析 （原卷合并作答）

28 • 单项选择

以下代码会发生什么？

```
s = {1, 2, 3}
s.discard(4)
s.remove(5)
```

A. 正常执行
B. 第一行报错
C. 第二行报错
D. 都报错

答案 A. 正常执行

解析 （原卷合并作答）

29 • 单项选择

关于字典的pop()方法，以下说法正确的是：

A. 不修改字典
B. 删除并返回指定键的值
C. 只能删除最后一个键
D. 返回新字典

答案 A. 不修改字典

解析 （原卷合并作答）

30 • 单项选择

以下代码的输出是什么？

```
d = {'a': 1, 'b': 2}
x = d.pop('a')
print(x, len(d))
```

A. 1 1
B. 1 2
C. None 1
D. 报错

答案 A. 1 1

解析 （原卷合并作答）

二、不定项选择题

31 • 不定项选择

以下哪些操作会改变列表的id？

A. `lst.append(1)`
B. `lst = lst + [1]`
C. `lst += [1]`
D. `lst.extend([1])`

答案 B. `lst = lst + [1]`

解析 关于 `+=` 操作符，只有B选项正确。 `+=` 对可变对象是就地修改，对不可变对象会创建新对象。

32 • 不定项选择

关于可变对象和不可变对象的性能，以下说法正确的有：

A. 不可变对象更安全
B. 可变对象修改更高效
C. 不可变对象适合作为字典键
D. 可变对象占用更多内存

答案 A. 不可变对象更安全；B. 可变对象修改更高效；C. 不可变对象适合作为字典键

解析 列表的可变性影响：
• A. 修改会影响所有引用
修改会影响所有引用　　所有引用指向同一对象
• B. 作为函数参数可能被修改
作为函数参数可能被修改　　传递引用
• C. 需要注意浅拷贝深拷贝
需要注意浅拷贝深拷贝　　嵌套结构问题
• D. 不能作为字典的值
不能作为字典的值　　可以作为字典的值

33 • 不定项选择

以下哪些方法返回新对象而不修改原对象？

A. str.upper()
B. list.sort()
C. tuple + tuple
D. dict.get()

答案 A. str.upper()；C. tuple + tuple；D. dict.get()

解析 元组的特性：
• A. 元组是不可变的
元组是不可变的　元组本身不可变
• B. 元组内的元素都不可变
元组内的元素都不可变　元组可包含可变对象
• C. 元组可以包含列表
元组可以包含列表　可以包含任何对象
• D. 元组可以作为字典键
元组可以作为字典键　元组是可哈希的
示例：

```
t = ([1, 2], 3)
# 元组包含列表
t[0].append(3)
# 可以修改列表内容
d = {t: 'value'} # TypeError - 包含列表的元组不可哈希
```

34 • 不定项选择

关于列表的切片操作，以下说法正确的有：

A. lst[:]创建浅拷贝
B. lst[1:3]返回新列表
C. 切片赋值会修改原列表
D. 切片不能用于元组

答案 A. lst[:]创建浅拷贝；B. lst[1:3]返回新列表；C. 切片赋值会修改原列表

解析 字典的可变性：
• A. 字典是可变的
字典是可变的　可以添加删除键值对
• B. 字典的键必须是不可变的
字典的键必须是不可变的　键必须可哈希
• C. 字典的值可以是任何类型
字典的值可以是任何类型　没有限制
• D. 字典保证有序
字典保证有序　Python 3.7+保持插入顺序，但这是实现细节

35 • 不定项选择

以下哪些是字典的视图对象？

A. keys()
B. values()
C. items()
D. get()

答案

解析

36 • 不定项选择

关于函数参数的默认值，以下说法正确的有：

A. 默认值在函数定义时计算
B. 可变默认值会被共享
C. 每次调用都会重新计算默认值
D. 应避免使用可变对象作为默认值

答案

解析

37 • 不定项选择

以下哪些操作会创建新的字符串对象？

A. `s.upper()`
B. `s + "hello"`
C. `s * 2`
D. `s[0]`

答案

解析

38 • 不定项选择

关于Pandas的copy参数，以下说法正确的有：

A. copy=True创建副本
B. copy=False可能返回视图
C. 默认值通常是True
D. 影响性能

答案

解析

39 • 不定项选择

以下哪些类型是可哈希的？

A. `int`
B. `str`
C. `tuple`
D. `list`

答案

解析

40 • 不定项选择

关于列表的`extend()`和`append()`，以下说法正确的有：

A. `extend()`添加多个元素
B. `append()`添加单个元素（作为整体）
C. 两者都修改原列表
D. `extend()`更快

答案

解析

41 • 不定项选择

以下哪些操作不会抛出异常？

A. `[1, 2, 3].index(4)`
B. `{1, 2, 3}.discard(4)`
C. `{'a': 1}.get('b')`
D. `(1, 2, 3)[5]`

答案

解析

42 • 不定项选择

关于集合的运算，以下说法正确的有：

A. `s1 | s2` 返回并集
B. `s1 & s2` 返回交集
C. `s1 - s2` 返回差集
D. 这些运算不修改原集合

答案

解析

43 • 不定项选择

以下哪些方法会修改原字典？

A. `update()`
B. `pop()`
C. `setdefault()`
D. `get()`

答案

解析

44 • 不定项选择

关于元组的特性，以下说法正确的有：

A. 元组比列表快
B. 元组占用更少内存
C. 元组可以作为字典键
D. 元组完全不可变

答案

解析

45 • 不定项选择

以下哪些是避免修改外部对象的方法？

A. 传递副本
B. 在函数内创建新对象

C. 使用不可变对象

D. 使用global关键字

答案

解析

三、填空题

46 • 填空 / 简答

列表的 _____ 操作符会修改原列表，而 _____ 操作符会创建新列表。

答案 += ， 就地修改

解析 就地修改 （或
（或 修改原对象
修改原对象 ）
+= 对可变对象和不可变对象的行为不同。

```
# 可变对象 - 就地修改
lst = [1, 2]
id_before = id(lst)
lst += [3]
print(id(lst) == id_before)
# True - 同ID
# 不可变对象 - 创建新对象
tup = (1, 2)
id_before = id(tup)
tup += (3,)
print(id(tup) == id_before)
                # False - 不ID
```

47 • 填空 / 简答

字典的 _____ 方法会修改原字典，而 _____ 方法只是查询不修改。

答案 update() ， setdefault()

解析 字典的两个常用修改方法。

```
# update - 批量更新
dict1 = {'a': 1, 'b': 2}
dict1.update({'b': 3, 'c': 4})
print(dict1)
# {'a': 1, 'b': 3, 'c': 4}
# setdefault - 设置默认值
dict2 = {'a': 1}
val = dict2.setdefault('b', 2)
# 返回2
print(dict2)
# {'a': 1, 'b': 2}
val = dict2.setdefault('a', 999)
# 返回1
print(dict2)
# {'a': 1, 'b': 2} - a未变
```

48 • 填空 / 简答

在函数中，如果要修改全局变量，需要使用 _____ 关键字；如果要修改外层函数的变量，需要使用 _____ 关键字。

答案 global ， nonlocal

解析 修改外部变量的关键字。

```
# global - 修改全局变量
x = 10
def modify_global():
    global x
    x = 20
modify_global()
print(x)
# 20
# nonlocal - 修改外层函数变量
def outer():
    x = 10
    def inner():
        nonlocal x
        x = 20
    inner()
    print(x)
# 20
outer()
```

49 • 填空 / 简答

Pandas中，_____ 会给出SettingWithCopyWarning警告，应该使用 _____ 或 _____ 来避免。

答案 is ， ==

解析 身份比较与值比较。

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a == b)
# True - 值相等
print(a is b)
# False - 不同对象
print(a is c)
# True - 同一对象
# 小整数缓存
x = 256
y = 256
print(x is y)
# True - 缓存
x = 257
y = 257
print(x is y)
# False - 超出缓存范围
```

50 • 填空 / 简答

集合的 _____ 方法删除元素时如果元素不存在会报错，而 _____ 方法不会报错。

答案 copy() ， view

解析 Pandas的副本和视图概念。

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
# 明确创建副本
df_copy = df.copy()
df_copy.loc[0, 'A'] = 999
print(df)
# 原数据未变
# 视图（可能）
df_view = df[['A']]
# 可能是视图
# 注意：Pandas 2.0+使用COW机制
```

1. 答案：

答案： += ， +

两种运算符的差异：

+= 就地修改：

```
a = [1, 2]
b = a
a += [3]
# 就地修改，a和b都变为[1,2,3]
```

+ 创建新对象：

```
a = [1, 2]
b = a
a = a + [3]
# 创建新对象，a=[1,2,3]，b=[1,2]
```

关键点： += 对可变对象是in-place操作， + 总是创建新对象。

2. 答案：

答案： update() （或
（或 pop() 等），
等）， get()

字典的修改方法vs查询方法：

修改原字典的方法：

- update() : 更新多个键值对
- pop() : 删除并返回值
- popitem() : 删除并返回键值对
- setdefault() : 设置默认值
- clear() : 清空字典

不修改原字典的方法：

- get() : 获取值（不存在返回默认值）
- keys() , values() , items() : 返回视图对象
- copy() : 返回浅拷贝

对比示例：

```
d = {'a': 1}
# 修改方法
d.update({'b': 2})
# d变为{'a':1, 'b':2}
d.pop('a')
```

```
# 删除'a'，返回1
# 查询方法
value = d.get('c', 0)
# 返回0，d不变
keys = d.keys()
# 返回视图，d不变
```

3. 答案：

答案： global ， nonlocal

作用域关键字：

global - 声明全局变量：

```
x = 10
# 全局变量
def func():
    global x
    # 声明使用全局x
    x = 20
# 修改全局变量
func()
print(x)
# 20
```

nonlocal - 声明外层变量：

```
def outer():
    x = 10
    # 外层变量
    def inner():
        nonlocal x
        # 声明使用外层x
        x = 20
    # 修改外层变量
    inner()
    print(x)
# 20
outer()
```

使用场景：

- global ：在函数内修改全局变量
- nonlocal ：在嵌套函数中修改外层函数的变量（闭包）

注意：如果只是读取变量，不需要声明；只有修改时才需要。

4. 答案：

答案： 链式索引

链式索引 ， loc ， iloc

Pandas中的安全索引方法：

链式索引的问题：

```
df[df['A'] > 1]['B'] = 100
# 可能在副本上修改，产生警告
```

正确的索引方法：

loc - 基于标签：

```
df.loc[df['A'] > 1, 'B'] = 100
# 正确
df.loc['row_label', 'col_label']
# 使用标签
```

iloc - 基于位置：

```
df.iloc[0:5, 2:4]
# 使用整数位置
```

为什么要避免链式索引：

- 可能在临时副本上操作
- 行为不确定（可能修改原数据，也可能不修改）
- Pandas会发出 SettingWithCopyWarning

最佳实践：

```
# 错误
df[df['A'] > 1]['B'] = 100
# 正确
df.loc[df['A'] > 1, 'B'] = 100
```

5. 答案：

答案： remove() ， discard()

集合删除元素的两种方法：

remove() - 删除元素，不存在则报错：

```
s = {1, 2, 3}
s.remove(2)
# 成功，s变为{1, 3}
s.remove(5)
# KeyError: 5
```

discard() - 删除元素，不存在不报错：

```
s = {1, 2, 3}
s.discard(2)
# 成功，s变为{1, 3}
s.discard(5)
# 不报错，s不变
```

使用建议：

- 确定元素存在时用 remove()
- 不确定元素是否存在时用 discard()

其他删除方法：

```
s.pop()
# 删除并返回任意元素
s.clear()
# 清空集合
```

对比总结：

```
s = {1, 2, 3}
# remove: 严格模式
try:
    s.remove(5)
except KeyError:
    print("元素不存在")
# discard: 宽松模式
s.discard(5)
# 静默忽略
```

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
def mystery_function(n, cache={}):
```

```
    if n in cache:
        return cache[n]
    if n <= 1:
        result = n
    else:
        result = mystery_function(n-1) + mystery_function(n-2)
    cache[n] = result
    return result
print(mystery_function(5))
print(mystery_function(10))
print(len(mystery_function.__defaults__[0]))
```

问题：

- (1) 这个函数计算什么？
- (2) 使用可变默认参数的优缺点是什么？
- (3) 最后一个print输出什么？

答案 - (1) 这个函数计算斐波那契数列
- (2) 优点：利用可变默认参数实现简单的缓存机制，避免重复计算。缺点：缓存是全局的，多次调用会累积，可能导致内
- (3) 输出：11（缓存中有11个键值对：0-10的斐波那契数）

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
class ShoppingCart:
    items = []
    def add_item(self, item):
        self.items.append(item)
    def get_items(self):
        return self.items
cart1 = ShoppingCart()
cart2 = ShoppingCart()
cart1.add_item("Apple")
cart2.add_item("Banana")
print(cart1.get_items())
print(cart2.get_items())
```

问题：

- (1) 两个print的输出是什么？
- (2) 这是什么问题？
- (3) 如何修复？

答案 - (1) 输出：
['Apple', 'Banana']
- (2) items是类属性而非实例属性，所有实例共享同一个列表
- (3) 修复方法：
```python
class ShoppingCart:
    def __init__(self):
```

```
        self.items = [] # 在__init__中创建实例属性
```

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
# 操作1
df['C'] = df['A'] + df['B']
# 操作2
df.loc[:, 'D'] = df['A'] * 2
# 操作3
subset = df[df['A'] > 1]
subset.loc[:, 'E'] = 100
print('E' in df.columns)
```

问题：

- (1) 输出是什么？
- (2) 为什么操作3不会修改df？
- (3) 如何修改操作3使其能修改df？

答案 • (1) 输出：False（或可能产生警告）

- (2) subset是df的一个副本（或视图），对subset的修改不会影响df
- (3) 修改方法：使用loc直接修改： df.loc[df['A'] > 1, 'E'] = 100

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
def process_data(data, transform=lambda x: x * 2):
    result = []
    for item in data:
        result.append(transform(item))
    return result
numbers = [1, 2, 3, 4, 5]
doubled = process_data(numbers)
tripled = process_data(numbers, lambda x: x * 3)
print(numbers)
print(doubled)
print(tripled)
```

问题：

- (1) 三个print的输出分别是什么？
- (2) 为什么numbers没有被修改？
- (3) 这种设计模式的优点是什么？

答案 • (1) 输出：

```
    [1, 2, 3, 4, 5]
    [2, 4, 6, 8, 10]
    [3, 6, 9, 12, 15]
```

- (2) process_data创建新列表，不修改原列表
- (3) 优点：无副作用、可测试性强、可复用性好、易于理解

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
def create_multipliers(n):
    multipliers = []
    for i in range(n):
        multipliers.append(lambda x: x * i)
    return multipliers
funcs = create_multipliers(3)
print([f(10) for f in funcs])
def create_multipliers_fixed(n):
    multipliers = []
    for i in range(n):
        multipliers.append(lambda x, i=i: x * i)
    return multipliers
funcs_fixed = create_multipliers_fixed(3)
print([f(10) for f in funcs_fixed])
```

问题：

- (1) 两个print的输出分别是什么？
- (2) 第一个函数有什么问题？
- (3) 第二个函数如何解决这个问题？

答案 • (1) 输出：

```
[20, 20, 20]
[0, 10, 20]
```

- (2) 第一个函数中所有lambda都引用同一个变量i，循环结束后i=2
- (3) 第二个函数使用默认参数 i=i 在定义时捕获i的值

解析

五、程序设计题

56 • 填空 / 简答

实现一个工具来追踪对象的可变性变化：

1. MutabilityTracker类：
    - 包装任何对象
    - 记录所有修改操作
    - 提供修改历史查询
2. 功能要求

功能要求：

- 拦截所有修改方法
- 记录修改前后的状态
- 支持撤销操作
- 导出修改日志

3. 支持的类型
支持的类型：
- 列表
- 字典
- 集合
- 自定义对象

答案
```
import copy
from functools import wraps
from datetime import datetime
class MutabilityTracker:
    """
    可变性追踪器 - 追踪对象的所有修改操作
    功能特性：
    1. 记录所有修改操作
    2. 保存修改前后的状态
    3. 支持撤销操作
    4. 导出修改日志
    使用示例：
    tracker = MutabilityTracker([1, 2, 3])
    tracker.append(4)
    tracker.undo()
    """
    def __init__(self, obj):
        """
        初始化追踪器
        参数：
        - obj: 要追踪的对象（列表、字典或集合）
        """
        self._obj = obj
        self._history = []
        self._operation_log = []
        # 保存初始状态
        self._save_state("初始化", copy.deepcopy(obj))
    def _save_state(self, operation, state_before=None):
        """保存操作前的状态"""
        timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
        if state_before is None:
            state_before = copy.deepcopy(self._obj)
    self._history.append({
        'timestamp': timestamp,
        'operation': operation,
        'state_before': state_before,
        'state_after': copy.deepcopy(self._obj)
    })
    self._operation_log.append(f"[{timestamp}] {operation}")
def __getattr__(self, name):
    """拦截所有方法调用"""
```

```
    attr = getattr(self._obj, name)
    # 如果是修改方法，包装它
    if callable(attr) and self._is_mutating_method(name):
        @wraps(attr)
        def wrapper(*args, **kwargs):
            # 保存操作前状态
            state_before = copy.deepcopy(self._obj)
            # 执行操作
            result = attr(*args, **kwargs)
            # 记录操作
            args_str = ', '.join(repr(a) for a in args)
            self._save_state(f"{name}({args_str})", state_before)
            return result
        return wrapper
    return attr
def _is_mutating_method(self, method_name):
    """判断是否是修改方法"""
    mutating_methods = {
        # 列表方法
        'append', 'extend', 'insert', 'remove', 'pop',
        'clear', 'sort', 'reverse',
        # 字典方法
        'update', 'setdefault', 'popitem',
        # 集合方法
        'add', 'discard', 'update', 'intersection_update',
        'difference_update', 'symmetric_difference_update'
    }
    return method_name in mutating_methods
def undo(self):
    """撤销最后一次操作"""
    if len(self._history) <= 1:
        print("没有可撤销的操作")
        return
    # 获取上一个状态
    self._history.pop()
    # 移除当前状态
    previous_state = self._history[-1]['state_after']
    # 恢复状态
    if isinstance(self._obj, list):
        self._obj.clear()
        self._obj.extend(previous_state)
    elif isinstance(self._obj, dict):
        self._obj.clear()
        self._obj.update(previous_state)
    elif isinstance(self._obj, set):
        self._obj.clear()
        self._obj.update(previous_state)
    print(f"已撤销到：{previous_state}")
```

```
def get_history(self):
    """获取修改历史"""
    return self._history.copy()
def export_log(self, filename=None):
    """导出修改日志"""
    log = "\n".join(self._operation_log)
    if filename:
        with open(filename, 'w', encoding='utf-8') as f:
            f.write(log)
            print(f"日志已导出到{filename}")
        else:
            print("=== 修改日志 ===")
            print(log)
        return log
    def get_obj(self):
        """获取被追踪的对象"""
        return self._obj
    def __repr__(self):
        return f"MutabilityTracker({self._obj})"
# === 使用示例 === {#-使用示例- }
print("=== 可变性追踪器示例 ===\n")
# 示例1：追踪列表 {#示例1追踪列表 }
print("示例1：追踪列表操作")
tracker = MutabilityTracker([1, 2, 3])
print(f"初始状态: {tracker.get_obj()}")
tracker.append(4)
print(f"添加后: {tracker.get_obj()}")
tracker.extend([5, 6])
print(f"扩展后: {tracker.get_obj()}")
tracker.remove(2)
print(f"删除后: {tracker.get_obj()}")
# 查看历史 {#查看历史 }
print("\n修改历史:")
for record in tracker.get_history()[-3:]:
    print(f"
            {record['operation']}: {record['state_before']} -> {record['state_after']}")
# 撤销操作 {#撤销操作 }
print("\n撤销最后一次操作:")
tracker.undo()
print(f"当前状态: {tracker.get_obj()}")
# 导出日志 {#导出日志 }
print()
tracker.export_log()
# 示例2：追踪字典 {#示例2追踪字典 }
print("\n\n示例2：追踪字典操作")
dict_tracker = MutabilityTracker({'a': 1, 'b': 2})
print(f"初始: {dict_tracker.get_obj()}")
dict_tracker.update({'c': 3, 'd': 4})
```

```
print(f"更新后: {dict_tracker.get_obj()}")
dict_tracker.pop('b')
print(f"删除后: {dict_tracker.get_obj()}")
dict_tracker.undo()
print(f"撤销后: {dict_tracker.get_obj()}")
```

解析

57 • 填空 / 简答

实现自动处理函数参数可变性的工具：

1. 装饰器

装饰器@safe_params：

- 自动检测可变参数
- 创建参数副本
- 可配置深拷贝/浅拷贝

2. 装饰器

装饰器@immutable_return：

- 将返回值转为不可变
- 支持嵌套结构
- 提供转换报告

3. 工具函数

工具函数：

- make_immutable(obj)
- is_deeply_immutable(obj)
- compare_mutability(obj1, obj2)

答案

```
import copy
from functools import wraps
import inspect
def safe_params(deep=False):
    """
    自动安全化参数装饰器
    功能：
    1. 自动检测可变参数
    2. 创建参数副本
    3. 可配置深拷贝/浅拷贝
    使用示例：
    @safe_params(deep=True)
    def my_function(data):
        data.append(1) # 不会影响外部
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            # 处理位置参数
            safe_args = []
            for arg in args:
                if _is_mutable(arg):
                safe_args.append(copy.deepcopy(arg) if deep else copy.copy(arg))
```

```
                else:
                safe_args.append(arg)
            # 处理关键字参数
            safe_kwargs = {}
            for key, value in kwargs.items():
                if _is_mutable(value):
                safe_kwargs[key] = copy.deepcopy(value) if deep else copy.copy(value)
                else:
                safe_kwargs[key] = value
            return func(*safe_args, **safe_kwargs)
        return wrapper
    return decorator
def immutable_return(func):
    """
    将返回值转为不可变装饰器
    功能：
    1. list -> tuple
    2. dict -> frozenset of items
    3. set -> frozenset
    4. 支持嵌套结构
    """
    @wraps(func)
    def wrapper(*args, **kwargs):
        result = func(*args, **kwargs)
        return _make_immutable(result)
    return wrapper
def _is_mutable(obj):
    """判断对象是否可变"""
    immutable_types = (int, float, str, tuple, frozenset, bytes, bool, type(None))
    return not isinstance(obj, immutable_types)
def _make_immutable(obj):
    """递归转换为不可变对象"""
    if isinstance(obj, list):
        return tuple(_make_immutable(item) for item in obj)
    elif isinstance(obj, dict):
        return frozenset((k, _make_immutable(v)) for k, v in obj.items())
    elif isinstance(obj, set):
        return frozenset(_make_immutable(item) for item in obj)
    else:
        return obj
def is_deeply_immutable(obj, visited=None):
    """
    检查对象是否深度不可变
    参数：
    - obj: 要检查的对象
    - visited: 已访问的对象集合（处理循环引用）
    返回：
    - bool: 是否深度不可变
```

```
    """
    if visited is None:
        visited = set()
    # 避免循环引用
    obj_id = id(obj)
    if obj_id in visited:
        return True
    visited.add(obj_id)
    # 检查基本不可变类型
    if isinstance(obj, (int, float, str, bytes, bool, type(None))):
        return True
    # 检查frozenset
    if isinstance(obj, frozenset):
        return all(is_deeply_immutable(item, visited) for item in obj)
    # 检查元组
    if isinstance(obj, tuple):
        return all(is_deeply_immutable(item, visited) for item in obj)
    # 其他都是可变的
    return False
def compare_mutability(obj1, obj2):
    """
    比较两个对象的可变性
    返回：
    - dict: 包含两个对象的可变性信息
    """
    return {
        'obj1': {
            'is_mutable': _is_mutable(obj1),
            'is_deeply_immutable': is_deeply_immutable(obj1),
            'type': type(obj1).__name__
        },
        'obj2': {
            'is_mutable': _is_mutable(obj2),
            'is_deeply_immutable': is_deeply_immutable(obj2),
            'type': type(obj2).__name__
        }
# === 使用示例 === {#-使用示例--1 }
print("=== 函数参数安全化工具示例 ===\n")
# 示例1：安全参数装饰器 {#示例1安全参数装饰器 }
@safe_params(deep=True)
def modify_data(data):
    """这个函数会修改数据，但不影响外部"""
    data.append(999)
    if isinstance(data, list) and len(data) > 1:
        data[0] = {'modified': True}
    return data
original = [{'value': 1}, {'value': 2}]
print(f"原始数据: {original}")
```

```
result = modify_data(original)
print(f"返回值: {result}")
print(f"原始数据未变: {original}\n")
# 示例2：不可变返回值 {#示例2不可变返回值 }
@immutable_return
def create_data():
    """返回可变数据，但会被转换为不可变"""
    return {
        'items': [1, 2, 3],
        'settings': {'debug': True},
        'tags': {'python', 'programming'}
    }
data = create_data()
print(f"返回的数据（已冻结）: {data}")
print(f"数据类型: {type(data)}\n")
# 示例3：深度不可变性检查 {#示例3深度不可变性检查 }
test_cases = [
    (1, 2, 3),
    ([1, 2], 3),
    ((1, 2), (3, 4)),
    frozenset([1, 2, 3])
]
print("深度不可变性检查:")
for obj in test_cases:
    is_immutable = is_deeply_immutable(obj)
    print(f"
            {obj}: {is_immutable}")
# 示例4：比较可变性 {#示例4比较可变性 }
print("\n可变性比较:")
comparison = compare_mutability([1, 2, 3], (1, 2, 3))
for obj_name, info in comparison.items():
    print(f"
            {obj_name}: {info}")
```

解析

58 • 填空 / 简答

实现不可变配置对象：

1. ImmutableConfig类：

- 初始化后不可修改
- 支持嵌套配置
- 提供只读访问

2. ConfigBuilder类：

- 构建器模式
- 验证配置
- 生成不可变配置

3. 功能

功能：

- 从文件加载
- 配置合并
- 配置导出

答案

```
from collections.abc import Mapping
import json
from pathlib import Path
class ImmutableConfig(Mapping):
    """
    不可变配置对象
    特性：
    1. 初始化后完全不可修改
    2. 支持嵌套配置访问
    3. 提供只读的字典接口
    4. 可序列化
    使用示例：
    config = ImmutableConfig({'database': {'host': 'localhost'}})
    print(config['database']['host'])
    """
    def __init__(self, data):
        """
        初始化配置
        参数：
        - data: 配置字典
        """
        # 递归转换嵌套字典为ImmutableConfig
        object.__setattr__(self, '_data', self._make_immutable(data))
        object.__setattr__(self, '_hash', None)
    def _make_immutable(self, obj):
        """递归转换为不可变配置"""
        if isinstance(obj, dict):
            return {k: self._make_immutable(v) for k, v in obj.items()}
        elif isinstance(obj, list):
            return tuple(self._make_immutable(item) for item in obj)
        elif isinstance(obj, set):
            return frozenset(self._make_immutable(item) for item in obj)
    else:
        return obj
# 实现Mapping接口
def __getitem__(self, key):
    """获取配置项"""
    value = self._data[key]
    if isinstance(value, dict):
        return ImmutableConfig(value)
    return value
def __iter__(self):
    """迭代键"""
    return iter(self._data)
```

```
def __len__(self):
    """配置项数量"""
    return len(self._data)
# 防止修改
def __setitem__(self, key, value):
    """禁止设置"""
    raise TypeError("不可修改不可变配置")
def __delitem__(self, key):
    """禁止删除"""
    raise TypeError("不可修改不可变配置")
def __setattr__(self, name, value):
    """禁止设置属性"""
    raise AttributeError("不可修改不可变配置的属性")
# 哈希支持
def __hash__(self):
    """使配置可哈希"""
    if self._hash is None:
        object.__setattr__(self, '_hash', hash(self._to_hashable(self._data)))
    return self._hash
def _to_hashable(self, obj):
        """转换为可哈希对象"""
        if isinstance(obj, dict):
            return tuple(sorted((k, self._to_hashable(v)) for k, v in obj.items()))
        elif isinstance(obj, (list, tuple)):
            return tuple(self._to_hashable(item) for item in obj)
        elif isinstance(obj, set):
            return frozenset(self._to_hashable(item) for item in obj)
        else:
            return obj
    def __repr__(self):
        """字符串表示"""
        return f"ImmutableConfig({self._data})"
    def to_dict(self):
        """转换为普通字典"""
        return self._data.copy()
class ConfigBuilder:
    """
    配置构建器
    功能：
    1. 使用构建器模式创建配置
    2. 支持链式调用
    3. 配置验证
    4. 生成不可变配置
    使用示例：
    config = (ConfigBuilder()
        .set('database', {'host': 'localhost'})
        .set('port', 8000)
        .validate()
```

```
        .build())
    """
    def __init__(self):
        """初始化构建器"""
        self._config = {}
    self._validators = []
def set(self, key, value):
    """设置配置项"""
    keys = key.split('.')
    current = self._config
    for k in keys[:-1]:
        if k not in current:
            current[k] = {}
        current = current[k]
    current[keys[-1]] = value
    return self
def merge(self, other_config):
    """合并其他配置"""
    self._deep_merge(self._config, other_config)
    return self
def _deep_merge(self, base, update):
    """深度合并字典"""
    for key, value in update.items():
        if key in base and isinstance(base[key], dict) and isinstance(value, dict):
            self._deep_merge(base[key], value)
        else:
            base[key] = value
def add_validator(self, validator_func):
    """添加验证器"""
    self._validators.append(validator_func)
    return self
def validate(self):
    """验证配置"""
    for validator in self._validators:
        if not validator(self._config):
            raise ValueError(f"配置验证失败: {validator.__name__}")
    return self
    def load_from_file(self, filepath):
        """从文件加载配置"""
        path = Path(filepath)
        if path.suffix == '.json':
            with path.open('r', encoding='utf-8') as f:
                config = json.load(f)
            self.merge(config)
        return self
    def build(self):
        """构建不可变配置"""
        return ImmutableConfig(self._config)
```

```
# === 使用示例 === {#-使用示例--2 }
print("=== 不可变配置系统示例 ===\n")
# 示例1：创建不可变配置 {#示例1创建不可变配置 }
config_data = {
    'database': {
        'host': 'localhost',
        'port': 5432,
        'credentials': {
            'user': 'admin',
            'password': 'secret'
    },
    'app': {
        'debug': False,
        'workers': 4
    },
    'features': ['auth', 'api', 'cache']
}
config = ImmutableConfig(config_data)
print(f"配置: {config}")
print(f"数据库主机: {config['database']['host']}")
print(f"功能列表: {config['features']}")
# 尝试修改（会失败） {#尝试修改会失败 }
try:
    config['database']['host'] = 'newhost'
except TypeError as e:
    print(f"\n   修改被阻止: {e}")
# 示例2：使用构建器 {#示例2使用构建器 }
print("\n使用ConfigBuilder:")
# 定义验证器 {#定义验证器 }
def validate_port(config):
    """验证端口号"""
    if 'database' in config and 'port' in config['database']:
        port = config['database']['port']
        return 1024 <= port <= 65535
    return True
# 构建配置 {#构建配置 }
built_config = (ConfigBuilder()
    .set('database.host', 'localhost')
    .set('database.port', 5432)
    .set('app.debug', True)
    .add_validator(validate_port)
    .validate()
    .build())
print(f"构建的配置: {built_config}")
print(f"配置哈希值: {hash(built_config)}")
# 配置可以作为字典键 {#配置可以作为字典键 }
config_cache = {built_config: "cached_data"}
print(f"\n配置作为字典键: {len(config_cache)} 个条目")
```

评分说明：
- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析

59 • 填空 / 简答

实现数据对象的快照功能：

1. Snapshot类：
- 保存对象状态
- 支持恢复
- 最小化存储

2. SnapshotManager：
- 管理多个快照
- 自动快照
- 快照比较

3. 应用

应用：
- 数据库事务模拟
- 游戏存档
- 配置回滚

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析

60 • 填空 / 简答

实现Pandas操作的安全检查工具：

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析

第 9 套

60 题

一、单项选择题

01 • 单项选择

以下代码的输出是什么？

```
def add_element(element, container=[]):
    container.append(element)
    print(id(container))
    return container
list1 = add_element(1)
list2 = add_element(2)
print(id(list1) == id(list2))
```

A. True
B. False
C. 可能是True或False
D. 报错

答案 A. True

解析

02 • 单项选择

关于列表的copy()方法，以下说法正确的是：

```
original = [[1, 2], [3, 4]]
copied = original.copy()
copied[0].append(5)
```

A. original不受影响
B. original也被修改
C. 报错
D. copied是深拷贝

答案 A. original不受影响

解析

03 • 单项选择

以下代码的输出是什么？

```
t = ([1, 2], 3)
original_id = id(t[0])
t[0].append(4)
print(id(t[0]) == original_id)
```

A. True
B. False
C. 报错

D. None

答案 A. True

解析

04 • 单项选择

关于字典的fromkeys()方法，以下说法正确的是：

```
d = dict.fromkeys(['a', 'b'], [])
d['a'].append(1)
```

A. 只有d['a']被修改
B. d['a']和d['b']都被修改
C. 报错
D. 不修改任何值

答案 C. 报错

解析

05 • 单项选择

以下代码的输出是什么？

```
def func(x=[]):
    x.append(1)
    return len(x)
print(func(), func(), func())
```

A. 1 1 1
B. 1 2 3
C. 3 3 3
D. 报错

答案 B. 1 2 3

解析

06 • 单项选择

关于列表的*操作符，以下说法正确的是：

```
lst = [0] * 5
lst[0] = 1
```

A. 所有元素都变成1
B. 只有第一个元素是1
C. 报错
D. 创建5个独立的0

答案 A. 所有元素都变成1

解析

07 • 单项选择

以下代码的输出是什么？

```
matrix = [[0] * 3 for _ in range(3)]
matrix[0][0] = 1
print(sum(row[0] for row in matrix))
```

A. 1
B. 3
C. 0
D. 9

答案 B. 3

解析

08 • 单项选择

关于元组的可哈希性，以下说法正确的是：

A. 所有元组都可哈希
B. 只有包含不可变元素的元组可哈希
C. 元组不可哈希
D. 取决于元组长度

答案 A. 所有元组都可哈希

解析

09 • 单项选择

以下代码会发生什么？

```
t = ([1, 2], 3)
d = {t: 'value'}
```

A. 正常执行
B. 抛出TypeError
C. 创建空字典
D. t被转换为字符串

答案 B. 抛出TypeError

解析

10 • 单项选择

关于字符串的join()方法，以下说法正确的是：

A. 修改原字符串
B. 返回新字符串
C. 只能连接字符串列表
D. 不能使用空字符串

答案 B. 返回新字符串

解析

11 • 单项选择

以下代码的输出是什么？

```
words = ['hello', 'world']
result = ' '.join(words)
words.append('!')
print(result)
```

A. "hello world !"
B. "hello world"

C. "helloworld"

D. 报错

答案 B. "hello world"

解析

12 • 单项选择

关于列表的切片赋值，以下说法正确的是：

```
lst = [1, 2, 3, 4, 5]
lst[1:3] = [10, 20, 30]
```

A. 列表长度不变

B. 列表长度可能改变

C. 报错

D. 创建新列表

答案 A. 列表长度不变

解析

13 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3]
lst[1:2] = []
print(lst)
```

A. [1, 3]

B. [1, 2, 3]

C. [1, [], 3]

D. 报错

答案 B. [1, 2, 3]

解析

14 • 单项选择

关于集合的copy()方法，以下说法正确的是：

A. 创建深拷贝

B. 创建浅拷贝

C. 不创建副本

D. 返回frozenset

答案 B. 创建浅拷贝

解析

15 • 单项选择

以下代码的输出是什么？

```
s1 = {1, 2, 3}
s2 = s1.copy()
s1.add(4)
print(len(s2))
```

A. 3

B. 4
C. 0
D. 报错

答案 A. 3

解析

16 • 单项选择

Pandas中，以下哪个操作最可能返回视图？

A. df'A', 'B'
B. df['A']
C. df[df['A'] > 0]
D. df.copy()

答案 A. df'A', 'B'

解析

17 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
series = df['A']
series.iloc[0] = 100
print(df['A'].iloc[0])
```

A. 1
B. 100
C. 可能是1或100
D. 报错

答案 B. 100

解析

18 • 单项选择

关于字典的clear()方法，以下说法正确的是：

A. 返回新字典
B. 清空原字典
C. 删除字典对象
D. 不修改字典

答案 A. 返回新字典

解析

19 • 单项选择

以下代码的输出是什么？

```
d1 = {'a': 1, 'b': 2}
d2 = d1
d1.clear()
print(len(d2))
```

A. 2
B. 0
C. 1
D. 报错

答案 B. 0

解析

20 • 单项选择

关于列表的sort()方法的key参数，以下说法正确的是：

A. key必须是函数
B. key可以是lambda表达式
C. key指定排序依据
D. 以上都正确

答案 B. key可以是lambda表达式

解析

21 • 单项选择

以下代码的输出是什么？

```
data = [('a', 3), ('b', 1), ('c', 2)]
data.sort(key=lambda x: x[1])
print(data[0][0])
```

A. 'a'
B. 'b'
C. 'c'
D. 1

答案 B. 'b'

解析

22 • 单项选择

关于元组的拆包，以下说法正确的是：

```
t = (1, 2, 3)
a, b, c = t
```

A. 修改了元组
B. 创建了三个新变量
C. 报错
D. a、b、c指向元组元素

答案 B. 创建了三个新变量

解析

23 • 单项选择

以下代码的输出是什么？

```
t = ([1, 2], [3, 4])
a, b = t
a.append(5)
```

```
print(t)
```

A. ([1, 2], [3, 4])
B. ([1, 2, 5], [3, 4])
C. ([1, 2], [3, 4, 5])
D. 报错

答案 A. ([1, 2], [3, 4])

解析

24 • 单项选择

关于字典推导式，以下说法正确的是：

A. 修改原字典
B. 创建新字典
C. 不能使用条件
D. 返回列表

答案 B. 创建新字典

解析

25 • 单项选择

以下代码的输出是什么？

```
original = {'a': 1, 'b': 2, 'c': 3}
new = {k: v*2 for k, v in original.items() if v > 1}
print(len(new))
```

A. 1
B. 2
C. 3
D. 0

答案 B. 2

解析

26 • 单项选择

关于列表的reverse()方法，以下说法正确的是：

A. 返回新列表
B. 修改原列表，返回None
C. 不修改列表
D. 返回迭代器

答案 B. 修改原列表，返回None

解析

27 • 单项选择

以下代码的输出是什么？

```
lst = [1, 2, 3]
result = lst.reverse()
print(result)
```

A. [3, 2, 1]

B. None
C. [1, 2, 3]
D. 报错

答案 B. None

解析

28 • 单项选择

关于集合的交集运算，以下说法正确的是：

A. s1 & s2 修改s1
B. s1.intersection(s2) 返回新集合
C. s1.intersection_update(s2) 返回新集合
D. 不能计算交集

答案 A. s1 & s2 修改s1

解析

29 • 单项选择

以下代码的输出是什么？

```
s1 = {1, 2, 3}
s2 = {2, 3, 4}
s3 = s1 & s2
print(len(s1), len(s3))
```

A. 3 2
B. 2 2
C. 3 3
D. 2 3

答案 B. 2 2

解析

30 • 单项选择

关于Pandas的loc和iloc的返回值，以下说法正确的是：

A. 总是返回副本
B. 总是返回视图
C. 可能返回视图或副本
D. 取决于inplace参数

答案 A. 总是返回副本

解析

二、不定项选择题

31 • 不定项选择

以下哪些情况会导致可变默认参数问题？

A. 默认参数是列表
B. 默认参数是字典

C. 默认参数是集合
D. 默认参数是整数

答案 A. 默认参数是列表；B. 默认参数是字典；C. 默认参数是集合

解析

32 • 不定项选择

关于浅拷贝的特点，以下说法正确的有：

A. 复制对象本身
B. 不复制嵌套对象
C. 比深拷贝快
D. 适合简单对象

答案 A. 复制对象本身；B. 不复制嵌套对象；C. 比深拷贝快；D. 适合简单对象

解析

33 • 不定项选择

以下哪些操作创建新对象？

A. `lst + [1]`
B. `lst.append(1)`
C. `s.upper()`
D. `t + (1,)`

答案 A. `lst + [1]`；C. `s.upper()`；D. `t + (1,)`

解析

34 • 不定项选择

关于元组的不可变性，以下说法正确的有：

A. 元组的长度不可变
B. 元组的元素引用不可变
C. 元组中的可变对象可以修改
D. 元组不能包含列表

答案 A. 元组的长度不可变；B. 元组的元素引用不可变；C. 元组中的可变对象可以修改

解析

35 • 不定项选择

以下哪些方法会修改原集合？

A. `add()`
B. `remove()`
C. `union()`
D. `update()`

答案 A. `add()`；B. `remove()`；D. `update()`

解析

36 • 不定项选择

关于字典的视图对象，以下说法正确的有：

A. 动态反映字典变化
B. 可以迭代
C. 不能转换为列表
D. 支持集合运算

答案 A. 动态反映字典变化；B. 可以迭代；D. 支持集合运算

解析

37 • 不定项选择

以下哪些是不可变类型？

A. frozenset
B. bytes
C. bytearray
D. tuple（包含不可变元素）

答案 A. frozenset；B. bytes；D. tuple（包含不可变元素）

解析

38 • 不定项选择

关于列表的切片，以下说法正确的有：

A. 切片创建新列表
B. 切片是浅拷贝
C. 可以用于赋值
D. 可以改变列表长度

答案 A. 切片创建新列表；B. 切片是浅拷贝；C. 可以用于赋值；D. 可以改变列表长度

解析

39 • 不定项选择

以下哪些操作不会修改原字符串？

A. replace()
B. split()
C. strip()
D. 所有字符串方法

答案 D. 所有字符串方法

解析

40 • 不定项选择

关于Pandas的inplace参数，以下说法正确的有：

A. True时修改原对象
B. False时返回新对象
C. 可能影响性能
D. 所有方法都支持

答案 A. True时修改原对象；B. False时返回新对象；C. 可能影响性能

解析

41 • 不定项选择

以下哪些类型可以作为字典的键？

A. int
B. str
C. tuple（包含不可变元素）
D. list

答案 A. int；B. str；C. tuple（包含不可变元素）

解析

42 • 不定项选择

关于函数参数传递，以下说法正确的有：

A. Python使用对象引用传递
B. 不可变对象表现像值传递
C. 可变对象表现像引用传递
D. 可以通过返回值避免修改

答案 A. Python使用对象引用传递；B. 不可变对象表现像值传递；C. 可变对象表现像引用传递；D. 可以通过返回值避免修改

解析

43 • 不定项选择

以下哪些操作会改变列表长度？

A. append()
B. extend()
C. pop()
D. sort()

答案 A. append()；B. extend()；C. pop()

解析

44 • 不定项选择

关于深拷贝的应用场景，以下说法正确的有：

A. 嵌套数据结构
B. 需要完全独立的副本
C. 避免意外修改
D. 所有情况都应使用

答案 A. 嵌套数据结构；B. 需要完全独立的副本；C. 避免意外修改

解析

45 • 不定项选择

以下哪些是Python的序列类型？

A. list
B. tuple
C. str
D. dict

答案 A. list；B. tuple；C. str

解析

三、填空题

46 • 填空 / 简答

可变对象作为函数默认参数时，会在 _____ 时创建，导致多次调用 _____ 同一对象。

答案 函数定义

解析 函数定义 ， 共享

共享

可变默认参数陷阱的核心原因：

- 默认参数在函数定义时

函数定义时创建（只创建一次）

- 所有函数调用共享

共享同一个默认参数对象

- 可变对象会累积修改

```
def func(lst=[]):
    # 在函数定义时创建列表
    lst.append(1)
    return lst
print(func())
# [1]
print(func())
# [1, 1] - 共享同一个列表!
```

47 • 填空 / 简答

列表的 _____ 方法返回None，而 _____ 函数返回新列表。

答案 reverse() ， reversed()

解析 列表反转的两种方式：

- reverse() ：就地反转

就地反转，修改原列表，返回None

- reversed() ：返回反转迭代器

反转迭代器，原列表不变

```
lst = [1, 2, 3]
lst.reverse()
# lst变为[3, 2, 1]
lst2 = [1, 2, 3]
rev = reversed(lst2)
# 返回迭代器，lst2不变
print(list(rev))
# [3, 2, 1]
```

48 • 填空 / 简答

元组包含可变对象时，元组 _____ 可哈希，因为可变对象的 _____ 可能改变。

答案 不 ， 哈希值

解析 哈希值

可变对象不能作为字典键的原因：

- 字典键必须是不可变
  不可变的
- 键必须可计算哈希值
  哈希值
- 可变对象的哈希值可能变化，导致字典内部结构混乱

```
# 错误示例
lst = [1, 2]
d = {lst: 'value'}
# TypeError: unhashable type: 'list'
# 正确示例
tup = (1, 2)
d = {tup: 'value'}
# 元组可以作为键
```

49 • 填空 / 简答

在Pandas中，避免SettingWithCopyWarning的方法是使用 _____ 或先调用 _____ 方法。

答案 loc/iloc ， copy()

解析 Pandas避免链式索引的方法：

- 使用 loc 或 iloc 进行明确的单次索引
- 使用 copy() 创建显式副本

```
# 错误：链式索引
df[df['A'] > 1]['B'] = 100
# SettingWithCopyWarning
# 正确方法1：使用loc
df.loc[df['A'] > 1, 'B'] = 100
# 正确方法2：显式复制
subset = df[df['A'] > 1].copy()
subset['B'] = 100
```

50 • 填空 / 简答

集合的 _____ 方法会修改原集合，而 _____ 运算符返回新集合。

答案 update() ， | （或

解析 （或 union() ）

集合的修改方法和非修改方法：

修改原集合：

- update() ：添加多个元素
- add() ：添加单个元素
- remove() 、 discard() ：删除元素

返回新集合：

- | （或 union() ）：并集
- & （或 intersection() ）：交集
- - （或 difference() ）：差集

```
s1 = {1, 2, 3}
s2 = {3, 4, 5}
# 修改原集合
s1.update(s2)
```

```
# s1变为{1, 2, 3, 4, 5}
# 返回新集合
s3 = s1 | s2
# 返回新集合，s1和s2不变
s4 = s1.union(s2)
# 同上
```

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class Counter:
    def __init__(self, start=0, history=[]):
        self.value = start
        self.history = history
        self.history.append(start)
    def increment(self):
        self.value += 1
        self.history.append(self.value)
c1 = Counter(0)
c1.increment()
c2 = Counter(10)
c2.increment()
print(c1.history)
print(c2.history)
```

问题：

- (1) 两个print的输出分别是什么？
- (2) 这个类有什么问题？
- (3) 如何修复？

答案 • (1) 输出：

```
[0, 1, 10, 11]
```

- (2) history是可变默认参数，所有实例共享同一个列表
- (3) 修复方法：

```
class Counter:
    def __init__(self, start=0, history=None):
        self.value = start
        self.history = history if history is not None else []
        self.history.append(start)
```

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
def merge_dicts(d1, d2, modify_original=False):
    if modify_original:
        d1.update(d2)
        return d1
```

```
    else:
        result = d1.copy()
        result.update(d2)
        return result
dict1 = {'a': 1, 'b': 2}
dict2 = {'b': 3, 'c': 4}
result1 = merge_dicts(dict1, dict2, False)
result2 = merge_dicts(dict1, dict2, True)
print(dict1)
print(result1)
print(result2)
```

问题：

- (1) 三个print的输出分别是什么？
- (2) 为什么result1和result2不同？
- (3) 这种设计的优缺点是什么？

答案 • (1) 输出：

```
{'a': 1, 'b': 3, 'c': 4}
{'a': 1, 'b': 2}
{'a': 1, 'b': 3, 'c': 4}
```

- (2) result1使用copy()创建副本，result2直接修改原字典
- (3) 优点：提供了灵活性，可选择是否修改原对象。缺点：容易出错，需要用户明确理解行为

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50]
})
# 方法1：链式索引
df[df['A'] > 2]['B'] = 999
# 方法2：使用loc
df.loc[df['A'] > 2, 'B'] = 888
print(df['B'].tolist())
```

问题：

- (1) 输出是什么？
- (2) 为什么方法1不生效？
- (3) 什么是链式索引？

答案 链式索引vs loc

- (1) 输出： [10, 20, 888, 888, 888]
- (2) 方法1是链式索引，产生临时副本，修改无效
- (3) 链式索引是连续的索引操作（如 df[condition]['column'] ），可能在副本上操作

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
def create_adders(n):
    adders = []
    for i in range(n):
        def adder(x):
            return x + i
        adders.append(adder)
    return adders
funcs = create_adders(3)
results = [f(10) for f in funcs]
print(results)
# 修复版本
def create_adders_fixed(n):
    adders = []
    for i in range(n):
        def adder(x, i=i):
            return x + i
        adders.append(adder)
    return adders
funcs_fixed = create_adders_fixed(3)
results_fixed = [f(10) for f in funcs_fixed]
print(results_fixed)
```

问题：

- (1) 两个print的输出分别是什么？
- (2) 第一个版本的问题是什么？
- (3) 为什么使用默认参数可以解决？

答案 • (1) 输出：

```
[12, 12, 12]
[10, 11, 12]
```

- (2) 所有函数共享同一个变量i，循环结束后i=2
- (3) 默认参数在函数定义时求值，捕获当时的i值

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
class DataContainer:
    def __init__(self, data):
        self._data = data
    @property
    def data(self):
        return self._data
    def process(self, func):
        self._data = func(self._data)
        return self
container = DataContainer([1, 2, 3])
```

```
result = container.process(lambda x: x + [4])
print(container.data)
print(result.data)
print(container is result)
```

问题：

- (1) 三个print的输出分别是什么？
- (2) 这种设计模式叫什么？
- (3) 有什么潜在问题？

答案 链式操作返回self

- (1) 输出：

```
[1, 2, 3, 4]
True
```

- (2) 流式接口（Fluent Interface）或方法链模式
- (3) 潜在问题：修改了原对象，可能造成意外副作用

解析

五、程序设计题

56 • 填空 / 简答

实现一个通用的不可变数据类：

1. ImmutableData类：
    - 初始化后不可修改
    - 支持任意属性
    - 实现 __hash__()
2. 功能要求

功能要求：

- 使用 __slots__ 优化内存
- 提供 replace() 方法创建修改版本
- 支持嵌套不可变对象

3. 应用示例

应用示例：

- 配置对象
- 数据传输对象
- 缓存键

答案

```
from typing import Any, Dict
class ImmutableData:
    """
    通用不可变数据类
    特性：
    1. 使用__slots__优化内存
    2. 初始化后完全不可修改
    3. 可哈希，能作为字典键
    4. 提供replace()方法创建修改版本
    使用示例：
    person = ImmutableData(name="Alice", age=30, city="Beijing")
```

```
    updated = person.replace(age=31)
    """
    __slots__ = ('_data', '_hash')
    def __init__(self, **kwargs):
        """
        初始化不可变数据对象
        参数：
        - **kwargs: 任意属性键值对
        """
        # 将所有参数转换为不可变形式
        immutable_data = {}
        for key, value in kwargs.items():
            immutable_data[key] = self._make_immutable(value)
        # 使用object.__setattr__绕过__setattr__限制
        object.__setattr__(self, '_data', immutable_data)
        object.__setattr__(self, '_hash', None)
    def _make_immutable(self, value):
        """递归转换为不可变对象"""
        if isinstance(value, dict):
        return ImmutableData(**value)
    elif isinstance(value, list):
        return tuple(self._make_immutable(v) for v in value)
    elif isinstance(value, set):
        return frozenset(self._make_immutable(v) for v in value)
    elif isinstance(value, ImmutableData):
        return value
    else:
        return value
def __getattr__(self, name):
    """属性访问"""
    if name.startswith('_'):
        raise AttributeError(f"'{type(self).__name__}' object has no attribute '{name}
    try:
        return self._data[name]
    except KeyError:
        raise AttributeError(f"'{type(self).__name__}' object has no attribute '{name}
def __setattr__(self, name, value):
    """禁止设置属性"""
    raise AttributeError("Cannot modify immutable object")
def __delattr__(self, name):
    """禁止删除属性"""
    raise AttributeError("Cannot modify immutable object")
def __hash__(self):
    """计算哈希值"""
    if self._hash is None:
        items = tuple(sorted(self._data.items()))
        hashable_items = self._to_hashable(items)
        object.__setattr__(self, '_hash', hash(hashable_items))
```

```
    return self._hash
def _to_hashable(self, obj):
    """转换为可哈希形式"""
    if isinstance(obj, ImmutableData):
        return self._to_hashable(tuple(sorted(obj._data.items())))
    elif isinstance(obj, (list, tuple)):
        return tuple(self._to_hashable(item) for item in obj)
    elif isinstance(obj, dict):
        return tuple(sorted((k, self._to_hashable(v)) for k, v in obj.items()))
    else:
        return obj
def __eq__(self, other):
    """相等性比较"""
    if not isinstance(other, ImmutableData):
        return False
    return self._data == other._data
def __repr__(self):
    """字符串表示"""
    items = ', '.join(f"{k}={repr(v)}" for k, v in self._data.items())
    return f"ImmutableData({items})"
def replace(self, **changes):
    """
    创建修改版本
    参数:
    - **changes: 要修改的属性
    返回:
    - ImmutableData: 新的不可变对象
    """
    new_data = self._data.copy()
    new_data.update(changes)
    return ImmutableData(**new_data)
def to_dict(self):
    """转换为字典"""
    result = {}
    for key, value in self._data.items():
        if isinstance(value, ImmutableData):
            result[key] = value.to_dict()
        elif isinstance(value, tuple):
            result[key] = list(value)
        elif isinstance(value, frozenset):
                result[key] = set(value)
            else:
                result[key] = value
        return result
# === 使用示例 ===
print("=== 不可变数据类示例 ===\n")
# 示例1：创建配置对象
config = ImmutableData(
```

```
    app_name="MyApp",
    version="1.0.0",
    database={"host": "localhost", "port": 5432},
    features=["auth", "api", "cache"]
)
print(f"配置: {config}")
print(f"应用名称: {config.app_name}")
print(f"数据库: {config.database}")
print(f"功能: {config.features}")
# 示例2：尝试修改（会失败）
try:
    config.app_name = "NewApp"
except AttributeError as e:
    print(f"\n   修改被阻止: {e}")
# 示例3：使用replace创建新版本
updated_config = config.replace(version="2.0.0")
print(f"\n更新版本后: {updated_config}")
print(f"原配置不变: {config.version}")
# 示例4：作为字典键
cache = {config: "cached_result"}
print(f"\n作为字典键: {len(cache)} 个条目")
# 示例5：数据传输对象
person = ImmutableData(
    name="Alice",
    age=30,
    address={"city": "Beijing", "district": "Haidian"}
)
print(f"\n人员信息: {person}")
print(f"地址: {person.address}")
print(f"转为字典: {person.to_dict()}")
```

解析

57 • 填空 / 简答

实现检测和分析对象可变性的工具：

1. 检测函数

检测函数：

- is_mutable(obj)
- find_mutable_parts(obj)
- mutability_report(obj)

2. 分析功能

分析功能：

- 递归检查嵌套结构
- 生成可变性树
- 提供修复建议

3. 可视化

可视化：

- 打印可变性结构

- 导出为JSON
- 生成图表

答案

```
import sys
from collections.abc import Mapping, Sequence
import json
def is_mutable(obj):
    """
    判断对象是否可变
    参数:
    - obj: 要检查的对象
    返回:
    - bool: 是否可变
    """
    immutable_types = (int, float, str, bytes, bool, type(None), frozenset)
    # 基本不可变类型
    if isinstance(obj, immutable_types):
        return False
    # 元组需要递归检查
    if isinstance(obj, tuple):
        return any(is_mutable(item) for item in obj)
    # 其他都是可变的
    return True
def find_mutable_parts(obj, path="root"):
    """
    查找对象中的可变部分
    参数:
    - obj: 要分析的对象
    - path: 当前路径
    返回:
    - list: 可变部分的列表
    """
    mutable_parts = []
    if is_mutable(obj):
        mutable_parts.append({
            'path': path,
            'type': type(obj).__name__,
            'value': repr(obj)[:50]
        })
    # 递归检查容器
    if isinstance(obj, dict):
        for key, value in obj.items():
            mutable_parts.extend(find_mutable_parts(value, f"{path}[{repr(key)}]"))
    elif isinstance(obj, (list, tuple)):
        for i, item in enumerate(obj):
            mutable_parts.extend(find_mutable_parts(item, f"{path}[{i}]"))
    elif isinstance(obj, set):
        for item in obj:
```

```
            mutable_parts.extend(find_mutable_parts(item, f"{path}.item"))
    return mutable_parts
def mutability_report(obj, name="object"):
    """
    生成可变性报告
    参数：
    - obj: 要分析的对象
    - name: 对象名称
    返回：
    - dict: 详细报告
    """
    report = {
        'name': name,
        'type': type(obj).__name__,
        'is_mutable': is_mutable(obj),
        'size_bytes': sys.getsizeof(obj),
        'mutable_parts': find_mutable_parts(obj, name)
    }
    # 添加建议
    if report['mutable_parts']:
        report['suggestion'] = "对象包含可变部分，不适合作为字典键或缓存键"
    else:
        report['suggestion'] = "对象完全不可变，可以安全用作字典键"
    return report
def print_mutability_tree(obj, indent=0, name="root"):
    """
    打印可变性结构树
    参数：
    - obj: 要打印的对象
    - indent: 缩进级别
    - name: 节点名称
    """
    prefix = "  " * indent
    mutable_marker = "✓ " if is_mutable(obj) else "✗ "
    print(f"{prefix}{mutable_marker} {name}: {type(obj).__name__}")
    # 递归打印子节点
    if isinstance(obj, dict) and indent < 5:
        for key, value in list(obj.items())[:5]:
            # 限制显示数量
            print_mutability_tree(value, indent + 1, f"[{repr(key)}]")
    elif isinstance(obj, (list, tuple)) and indent < 5:
        for i, item in enumerate(obj[:5]):
            # 限制显示数量
            print_mutability_tree(item, indent + 1, f"[{i}]")
def export_report_json(report, filename):
    """
    导出报告为JSON
    参数：
```

```
    - report: 报告字典
    - filename: 文件名
    """
    with open(filename, 'w', encoding='utf-8') as f:
        json.dump(report, f, indent=2, ensure_ascii=False)
    print(f"报告已导出到{filename}")
# === 使用示例 ===
print("=== 可变性检测工具示例 ===\n")
# 示例1：检测简单对象
test_objects = {
    '整数': 42,
    '字符串': "hello",
    '列表': [1, 2, 3],
    '元组': (1, 2, 3),
    '混合元组': ([1, 2], 3),
    '字典': {'a': 1, 'b': [2, 3]},
    'frozenset': frozenset([1, 2, 3])
}
print("基本类型可变性检测:")
for name, obj in test_objects.items():
    mutable = is_mutable(obj)
    marker = "  可变" if mutable else "  不可变"
    print(f"
            {marker} {name}: {type(obj).__name__}")
# 示例2：查找可变部分
print("\n复杂对象可变部分分析:")
complex_obj = {
    'name': 'config',
    'values': [1, 2, [3, 4]],
    'settings': {
        'debug': True,
        'data': [[5, 6], [7, 8]]
    }
mutable_parts = find_mutable_parts(complex_obj)
for part in mutable_parts:
    print(f"
            {part['path']}: {part['type']}")
# 示例3：生成详细报告
print("\n详细可变性报告:")
report = mutability_report(complex_obj, "complex_obj")
print(f" 名称: {report['name']}")
print(f" 类型: {report['type']}")
print(f" 可变: {report['is_mutable']}")
print(f" 大小: {report['size_bytes']} 字节")
print(f" 可变部分数量: {len(report['mutable_parts'])}")
print(f" 建议: {report['suggestion']}")
# 示例4：可变性树
print("\n可变性结构树:")
```

```
print_mutability_tree(complex_obj)
# 示例5：对比不同数据结构
print("\n\n数据结构对比:")
structures = {
    '纯不可变': (1, 2, (3, 4), "hello"),
    '混合结构': (1, [2, 3], {'a': 4}),
    '纯可变': [1, 2, [3, 4], {'a': 5}]
}
for name, struct in structures.items():
    print(f"\n{name}:")
    print_mutability_tree(struct, name=name)
```

解析

58 • 填空 / 简答

实现防止意外修改的数据容器：

1. SafeContainer类：

- 包装列表/字典
- 所有修改返回新对象
- 原对象保持不变

2. 操作追踪

操作追踪：

- 记录所有操作
- 支持撤销/重做
- 导出操作历史

3. 性能优化

性能优化：

- 写时复制
- 结构共享
- 延迟计算

答案

```
import copy
from datetime import datetime
class SafeContainer:
    """
    安全数据容器 - 所有修改返回新对象
    特性：
    1. 写时复制（Copy-on-Write）
    2. 操作追踪
    3. 撤销/重做支持
    4. 原对象永不变
    使用示例：
    container = SafeContainer([1, 2, 3])
    new_container = container.append(4) # 返回新对象
    """
    def __init__(self, data):
        """
        初始化容器
        参数：
```

```
        - data: 初始数据（列表或字典）
        """
        self._data = copy.deepcopy(data)
        self._history = []
        self._operation_log = []
        self._redo_stack = []
    # === 列表操作 ===
    def append(self, item):
        """添加元素，返回新容器"""
        if not isinstance(self._data, list):
            raise TypeError("append只能用于列表")
        new_data = self._data.copy()
        new_data.append(item)
    return self._create_new(new_data, f"append({repr(item)})")
def extend(self, items):
    """扩展列表，返回新容器"""
    if not isinstance(self._data, list):
        raise TypeError("extend只能用于列表")
    new_data = self._data.copy()
    new_data.extend(items)
    return self._create_new(new_data, f"extend({repr(items)})")
def remove(self, item):
    """移除元素，返回新容器"""
    if not isinstance(self._data, list):
        raise TypeError("remove只能用于列表")
    new_data = self._data.copy()
    new_data.remove(item)
    return self._create_new(new_data, f"remove({repr(item)})")
# === 字典操作 ===
def set(self, key, value):
    """设置键值，返回新容器"""
    if not isinstance(self._data, dict):
        raise TypeError("set只能用于字典")
    new_data = self._data.copy()
    new_data[key] = value
    return self._create_new(new_data, f"set({repr(key)}, {repr(value)})")
def delete(self, key):
    """删除键，返回新容器"""
    if not isinstance(self._data, dict):
        raise TypeError("delete只能用于字典")
    new_data = self._data.copy()
    del new_data[key]
    return self._create_new(new_data, f"delete({repr(key)})")
# === 辅助方法 ===
def _create_new(self, new_data, operation):
    """创建新容器并记录操作"""
    new_container = SafeContainer.__new__(SafeContainer)
    new_container._data = new_data
```

```
    new_container._history = self._history + [self._data]
    new_container._operation_log = self._operation_log + [operation]
    new_container._redo_stack = []
    return new_container
def undo(self):
    """撤销最后一次操作"""
    if not self._history:
        print("没有可撤销的操作")
        return self
    new_container = SafeContainer.__new__(SafeContainer)
    new_container._data = self._history[-1]
    new_container._history = self._history[:-1]
    new_container._operation_log = self._operation_log[:-1]
    new_container._redo_stack = self._redo_stack + [self._data]
    return new_container
def redo(self):
    """重做操作"""
    if not self._redo_stack:
        print("没有可重做的操作")
        return self
    new_container = SafeContainer.__new__(SafeContainer)
    new_container._data = self._redo_stack[-1]
    new_container._history = self._history + [self._data]
    new_container._operation_log = self._operation_log + ["redo"]
    new_container._redo_stack = self._redo_stack[:-1]
    return new_container
def get_data(self):
    """获取当前数据（副本）"""
    return copy.deepcopy(self._data)
def get_history(self):
        """获取操作历史"""
        return list(zip(self._operation_log, self._history))
    def __repr__(self):
        """字符串表示"""
        return f"SafeContainer({self._data})"
    def __len__(self):
        """长度"""
        return len(self._data)
# === 使用示例 ===
print("=== 安全数据容器示例 ===\n")
# 示例1：列表操作
print("示例1：列表操作（写时复制）")
container1 = SafeContainer([1, 2, 3])
print(f"初始: {container1}")
container2 = container1.append(4)
print(f"添加4后: {container2}")
print(f"原容器不变: {container1}")
container3 = container2.extend([5, 6])
```

```
print(f"扩展后: {container3}")
# 示例2：操作链
print("\n示例2：操作链")
result = (SafeContainer([])
    .append(1)
    .append(2)
    .append(3)
    .extend([4, 5])
    .remove(2))
print(f"链式操作结果: {result}")
# 示例3：撤销/重做
print("\n示例3：撤销/重做")
c1 = SafeContainer([1, 2])
c2 = c1.append(3)
c3 = c2.append(4)
print(f"c3: {c3}")
c2_restored = c3.undo()
print(f"撤销后: {c2_restored}")
c3_restored = c2_restored.redo()
print(f"重做后: {c3_restored}")
# 示例4：操作历史
print("\n示例4：操作历史")
history = c3.get_history()
for i, (op, state) in enumerate(history):
    print(f"
            {i+1}. {op}: {state}")
# 示例5：字典操作
print("\n示例5：字典操作")
dict_container = SafeContainer({'a': 1, 'b': 2})
print(f"初始: {dict_container}")
updated = dict_container.set('c', 3)
print(f"添加c后: {updated}")
print(f"原字典不变: {dict_container}")
deleted = updated.delete('b')
print(f"删除b后: {deleted}")
```

评分说明：

- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

  总分：100分

解析

59 • 填空 / 简答

实现完整的函数参数保护机制：
1. 装饰器集合
装饰器集合：
- @readonly_params ：参数只读
- @copy_params ：自动复制
- @validate_immutable ：验证不可变
2. 参数分析
参数分析：
- 检测可变参数
- 生成警告
- 自动修复
3. 测试工具
测试工具：
- 参数修改检测
- 副作用追踪
- 性能影响分析

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析

60 • 填空 / 简答

实现Pandas操作的自动优化：
1. 优化规则
优化规则：
- 避免链式索引

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析

第 10 套

60 题

一、单项选择题

01 • 单项选择

以下代码的输出是什么？

```
def build_list(val, lst=None):
    if lst is None:
        lst = []
    lst.append(val)
    return lst
a = build_list(1)
b = build_list(2)
print(len(a), len(b))
```

A. 1 1
B. 2 2
C. 1 2
D. 2 1

答案 A. 1 1

解析

02 • 单项选择

关于列表的extend()方法，以下说法正确的是：

```
lst = [1, 2]
lst.extend([3, 4])
```

A. 返回新列表
B. 修改原列表
C. 报错
D. 不做任何操作

答案 B. 修改原列表

解析

03 • 单项选择

以下代码的输出是什么？

```
original = [1, 2, 3]
copy1 = original
copy2 = original[:]
original.append(4)
print(len(copy1), len(copy2))
```

A. 3 3
B. 4 4

C. 4 3

D. 3 4

答案 C. 4 3

解析

04 • 单项选择

关于元组的+=操作，以下说法正确的是：

```
t = (1, 2)
t += (3,)
```

A. 修改原元组

B. 创建新元组

C. 报错

D. t保持不变

答案 B. 创建新元组

解析

05 • 单项选择

以下代码的输出是什么？

```
def modify_tuple(t):
    t = t + (4,)
    return t
original = (1, 2, 3)
result = modify_tuple(original)
print(len(original), len(result))
```

A. 3 3

B. 4 4

C. 3 4

D. 4 3

答案 C. 3 4

解析

06 • 单项选择

关于字典的pop()方法，以下说法正确的是：

A. 不修改原字典

B. 修改原字典并返回删除的值

C. 键不存在时返回None

D. 不能指定默认值

答案 B. 修改原字典并返回删除的值

解析

07 • 单项选择

以下代码的输出是什么？

```
d = {'a': 1, 'b': 2}
val = d.pop('c', 0)
```

```
print(val, len(d))
```

A. None 2
B. 0 2
C. KeyError
D. 0 3

答案 B. 0 2

解析

08 • 单项选择

关于集合的difference()和difference_update()，以下说法正确的是：

A. 两者都修改原集合
B. difference()返回新集合，difference_update()修改原集合
C. 两者都返回新集合
D. 完全相同

答案 B. difference()返回新集合，difference_update()修改原集合

解析

09 • 单项选择

以下代码的输出是什么？

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}
s3 = s1.difference(s2)
print(len(s1), len(s3))
```

A. 2 2
B. 4 2
C. 2 4
D. 4 4

答案 B. 4 2

解析

10 • 单项选择

关于字符串的split()方法，以下说法正确的是：

A. 修改原字符串
B. 返回列表
C. 返回元组
D. 返回字符串

答案 B. 返回列表

解析

11 • 单项选择

以下代码的输出是什么？

```
s = "a,b,c"
parts = s.split(',')
s = s.upper()
```

```
print(parts[0])
```

A. 'A'
B. 'a'
C. 'a,b,c'
D. 报错

答案 B. 'a'

解析

12 • 单项选择

关于列表的clear()和赋值为空列表，以下说法正确的是：

```
lst1 = [1, 2, 3]
lst2 = lst1
# 方法1: lst1.clear()
# 方法2: lst1 = []
```

A. 两种方法效果相同
B. clear()会影响lst2，赋值不会
C. 赋值会影响lst2，clear()不会
D. 都不影响lst2

答案 B. clear()会影响lst2，赋值不会

解析

13 • 单项选择

以下代码的输出是什么？

```
lst1 = [1, 2, 3]
lst2 = lst1
lst1 = []
print(len(lst2))
```

A. 0
B. 3
C. None
D. 报错

答案 B. 3

解析

14 • 单项选择

关于frozenset，以下说法正确的是：

A. 可以修改元素
B. 不可变的集合
C. 不能包含元素
D. 与set完全相同

答案 B. 不可变的集合

解析

15 • 单项选择

以下代码的输出是什么？

```
fs = frozenset([1, 2, 3])
s = {fs}
print(len(s))
```

A. 0
B. 1
C. 3
D. 报错

答案 B. 1

解析

16 • 单项选择

Pandas中，以下哪种操作最安全（不会产生警告）？

A. df[df['A'] > 0]['B'] = 1
B. df.loc[df['A'] > 0, 'B'] = 1
C. df['B'][df['A'] > 0] = 1
D. 都一样

答案 B. df.loc[df['A'] > 0, 'B'] = 1

解析

17 • 单项选择

以下代码的输出是什么？

```
import pandas as pd
df = pd.DataFrame({'A': [1, 2, 3]})
df_copy = df.copy(deep=True)
df['A'] = [10, 20, 30]
print(df_copy['A'].iloc[0])
```

A. 1
B. 10
C. None
D. 报错

答案 A. 1

解析

18 • 单项选择

关于列表的+和+=操作符，以下说法正确的是：

A. 两者都修改原列表
B. +创建新列表，+=修改原列表
C. +修改原列表，+=创建新列表
D. 两者都创建新列表

答案 B. +创建新列表，+=修改原列表

解析

19 • 单项选择

以下代码的输出是什么？

```
a = [1, 2]
b = a
a = a + [3]
print(id(a) == id(b))
```

A. True
B. False
C. 可能是True或False
D. 报错

答案 B. False

解析

20 • 单项选择

关于字典的get()方法的默认值，以下说法正确的是：

A. 默认值是None
B. 默认值是0
C. 默认值是空字符串
D. 必须指定默认值

答案 A. 默认值是None

解析

21 • 单项选择

以下代码的输出是什么？

```
d = {'a': 1}
val = d.get('b')
print(val)
```

A. 0
B. None
C. ''
D. KeyError

答案 B. None

解析

22 • 单项选择

关于元组的不可变性和列表的可变性，以下说法正确的是：

A. 元组总是比列表占用更少内存
B. 元组的访问速度总是更快
C. 元组可以作为字典键，列表不能
D. 以上都正确

答案 C. 元组可以作为字典键，列表不能

解析

23 • 单项选择

以下代码会发生什么？

```
d = {[1, 2]: 'value'}
```

A. 正常执行
B. 抛出TypeError
C. 列表被转换为元组
D. 创建空字典

答案 B. 抛出TypeError

解析

24 • 单项选择

关于集合的|、&、-运算符，以下说法正确的是：

A. 都修改原集合
B. 都返回新集合
C. 只有|修改原集合
D. 只有&修改原集合

答案 B. 都返回新集合

解析

25 • 单项选择

以下代码的输出是什么？

```
s1 = {1, 2, 3}
s2 = {2, 3, 4}
s3 = s1 | s2
print(len(s1), len(s3))
```

A. 3 4
B. 4 4
C. 3 3
D. 4 3

答案 A. 3 4

解析

26 • 单项选择

关于字符串的format()方法，以下说法正确的是：

A. 修改原字符串
B. 返回新字符串
C. 只能格式化数字
D. 不能使用

答案 B. 返回新字符串

解析

27 • 单项选择

以下代码的输出是什么？

```
template = "Hello, {}"
result = template.format("World")
```

```
print(template)
```

A. "Hello, World"
B. "Hello, {}"
C. "World"
D. 报错

答案 B. "Hello, {}"

解析

28 • 单项选择

关于列表的sort()方法的reverse参数，以下说法正确的是：

A. reverse=True降序排序
B. reverse=False降序排序
C. 默认是True
D. 不能使用

答案 A. reverse=True降序排序

解析

29 • 单项选择

以下代码的输出是什么？

```
lst = [3, 1, 4, 1, 5]
lst.sort(reverse=True)
print(lst[0])
```

A. 1
B. 3
C. 5
D. 报错

答案 C. 5

解析

30 • 单项选择

关于Pandas的copy()方法的deep参数，以下说法正确的是：

A. deep=True创建深拷贝
B. deep=False创建浅拷贝
C. 默认是deep=True
D. 以上都正确

答案 D. 以上都正确

解析

二、不定项选择题

31 • 不定项选择

以下哪些操作会创建对象的副本？

A. list.copy()

B. dict.copy()
C. set.copy()
D. str.copy()

答案 A. list.copy()；B. dict.copy()；C. set.copy()

解析

32 • 不定项选择

关于可变对象作为类属性，以下说法正确的有：

A. 所有实例共享同一对象
B. 可能导致意外行为
C. 应该在__init__中初始化
D. 没有问题

答案 A. 所有实例共享同一对象；B. 可能导致意外行为；C. 应该在__init__中初始化

解析

33 • 不定项选择

以下哪些类型的对象可以作为集合的元素？

A. int
B. str
C. tuple（包含不可变元素）
D. list

答案 A. int；B. str；C. tuple（包含不可变元素）

解析

34 • 不定项选择

关于列表的切片操作，以下说法正确的有：

A. 创建新列表
B. 是浅拷贝
C. 可以用于删除元素
D. 可以用于插入元素

答案 A. 创建新列表；B. 是浅拷贝；C. 可以用于删除元素；D. 可以用于插入元素

解析

35 • 不定项选择

以下哪些方法会修改原字典？

A. update()
B. pop()
C. popitem()
D. get()

答案 A. update()；B. pop()；C. popitem()

解析

36 • 不定项选择

关于元组包含列表的情况，以下说法正确的有：

A. 元组本身不可变
B. 列表内容可以修改
C. 这样的元组不可哈希
D. 不能创建这样的元组

答案 A. 元组本身不可变；B. 列表内容可以修改；C. 这样的元组不可哈希

解析

37 • 不定项选择

以下哪些是字符串的不可变性带来的影响？

A. 字符串拼接效率较低
B. 字符串可以作为字典键
C. 字符串方法都返回新字符串
D. 字符串占用更多内存

答案 A. 字符串拼接效率较低；B. 字符串可以作为字典键；C. 字符串方法都返回新字符串

解析

38 • 不定项选择

关于深拷贝和浅拷贝的选择，以下说法正确的有：

A. 简单对象用浅拷贝
B. 嵌套对象需要完全独立时用深拷贝
C. 深拷贝更慢
D. 总是应该用深拷贝

答案 A. 简单对象用浅拷贝；B. 嵌套对象需要完全独立时用深拷贝；C. 深拷贝更慢

解析

39 • 不定项选择

以下哪些操作不会修改原列表？

A. `sorted(lst)`
B. `reversed(lst)`
C. `lst[:]`
D. `list(lst)`

答案 A. `sorted(lst)`；B. `reversed(lst)`；C. `lst[:]`；D. `list(lst)`

解析

40 • 不定项选择

关于Pandas的视图和副本，以下说法正确的有：

A. 视图修改会影响原数据
B. 副本修改不影响原数据
C. 使用`copy()`确保获得副本
D. 链式索引可能产生副本

答案 A. 视图修改会影响原数据；B. 副本修改不影响原数据；C. 使用`copy()`确保获得副本；D. 链式索引可能产生副本

解析

41 • 不定项选择

以下哪些是不可变类型？

A. int
B. float
C. str
D. bool

答案 A. int；B. float；C. str；D. bool

解析

42 • 不定项选择

关于函数参数的可变性问题，以下说法正确的有：

A. 传递可变对象时要小心
B. 可以在函数内创建副本
C. 使用None作为默认值更安全
D. 不可变对象没有这个问题

答案 A. 传递可变对象时要小心；B. 可以在函数内创建副本；C. 使用None作为默认值更安全；D. 不可变对象没有这个问题

解析

43 • 不定项选择

以下哪些集合操作会修改原集合？

A. add()
B. union()
C. update()
D. intersection()

答案 A. add()；C. update()

解析

44 • 不定项选择

关于列表推导式和可变性，以下说法正确的有：

A. 创建新列表
B. 不修改原数据
C. 可以包含可变对象
D. 总是比循环安全

答案 A. 创建新列表；B. 不修改原数据；C. 可以包含可变对象

解析

45 • 不定项选择

以下哪些是避免可变默认参数问题的方法？

A. 使用None作为默认值
B. 在函数内创建新对象
C. 使用不可变对象

D. 使用global变量

答案 A. 使用None作为默认值；B. 在函数内创建新对象；C. 使用不可变对象

解析

三、填空题

46 • 填空 / 简答

可变对象作为默认参数的问题可以通过使用 _____ 作为默认值，然后在函数内部判断并创建 _____ 来解决。

答案 None ， 新对象

解析 新对象

就地修改方法vs返回新对象的方法：

- 就地修改方法（如 list.sort() 、 list.reverse() ）返回 None
- 返回新对象的方法（如 sorted() 、 reversed() ）返回新对象，原对象不变

47 • 填空 / 简答

列表的 _____ 方法会就地修改列表，而 _____ 函数返回排序后的新列表。

答案 sort() ， sorted()

解析 列表排序的两种方式（与第7、9套相同考点）

48 • 填空 / 简答

元组的不可变性是指 _____ 不可变，但元组中包含的 _____ 对象的内容可以被修改。

答案 元素引用

解析 元素引用 （或
（或 元组本身
元组本身 ），
），可变
可变

元组的不可变性是浅层的：

- 元组本身不可变（不能修改元组结构）
- 但元组内的可变对象可以被修改

49 • 填空 / 简答

在Pandas中，使用 _____ 索引可能产生SettingWithCopyWarning，应该使用 _____ 方法来避免。

答案 链式

解析 链式 ， copy()

Pandas避免链式索引警告的方法

50 • 填空 / 简答

字典的 _____ 方法会删除并返回键值对，而 _____ 方法只是查询不修改字典。

答案 pop() （或

解析 （或 popitem() ），
），get()
字典的修改方法vs查询方法

四、程序代码阅读题

51 • 填空 / 简答

阅读以下代码，回答问题：

```
class TaskQueue:
    tasks = []
    def add_task(self, task):
        self.tasks.append(task)
    def get_tasks(self):
        return self.tasks
    def clear_tasks(self):
        self.tasks = []
q1 = TaskQueue()
q2 = TaskQueue()
q1.add_task("Task 1")
q2.add_task("Task 2")
q1.clear_tasks()
print(len(q1.get_tasks()))
print(len(q2.get_tasks()))
```

问题：

- (1) 两个print的输出分别是什么？
- (2) 为什么clear_tasks()的行为不符合预期？
- (3) 如何正确实现这个类？

答案 类问题

- (1) 输出：0, 1
- (2) clear_tasks() 中的 self.tasks = [] 创建了新的实例属性，而不是清空类属性
- (3) 应该在 __init__ 中初始化实例属性：

```
def __init__(self):
    self.tasks = []
```

解析

52 • 填空 / 简答

阅读以下代码，回答问题：

```
import copy
class Node:
    def __init__(self, value, children=None):
        self.value = value
        self.children = children if children else []
    def add_child(self, child):
        self.children.append(child)
root = Node(1)
```

```
child1 = Node(2)
child2 = Node(3)
root.add_child(child1)
root.add_child(child2)
# 浅拷贝
root_shallow = copy.copy(root)
root_shallow.value = 10
# 深拷贝
root_deep = copy.deepcopy(root)
root_deep.children[0].value = 20
print(root.value)
print(root.children[0].value)
print(root_shallow.children[0].value)
```

问题：

- (1) 三个print的输出分别是什么？
- (2) 浅拷贝和深拷贝的区别在这里如何体现？
- (3) 什么时候需要使用深拷贝？

答案 浅拷贝vs深拷贝

深拷贝

- (1) 输出：1, 20, 20
- (2) 浅拷贝只复制对象本身，children列表仍是共享的；深拷贝递归复制所有对象
- (3) 当对象包含嵌套的可变对象，且需要完全独立的副本时

解析

53 • 填空 / 简答

阅读以下代码，回答问题：

```
import pandas as pd
df = pd.DataFrame({
    'group': ['A', 'A', 'B', 'B'],
    'value': [1, 2, 3, 4]
})
# 方法1
grouped = df.groupby('group')
for name, group in grouped:
    group['new_col'] = group['value'] * 2
# 方法2
df['new_col'] = df.groupby('group')['value'].transform(lambda x: x * 2)
print('new_col' in df.columns)
```

问题：

- (1) 输出是什么？
- (2) 为什么方法1不会修改原DataFrame？
- (3) transform()方法的作用是什么？

答案 分组操作

- (1) 输出：True
- (2) 方法1中的group是副本，修改不影响原DataFrame
- (3) transform()对每个分组应用函数并保持原DataFrame的形状

解析

54 • 填空 / 简答

阅读以下代码，回答问题：

```
def process_config(config, defaults={'timeout': 30, 'retries': 3}):
    defaults.update(config)
    return defaults
config1 = process_config({'timeout': 60})
config2 = process_config({'retries': 5})
print(config1)
print(config2)
print(config1 is config2)
```

问题：

- (1) 三个print的输出分别是什么？
- (2) 这个函数有什么严重问题？
- (3) 如何修复这个函数？

答案 • (1) 输出：

```
{'timeout': 60, 'retries': 5}
True
```

- (2) 默认参数defaults是可变字典，被所有调用共享并累积修改
- (3) 修复方法：

```
def process_config(config, defaults=None):
    if defaults is None:
        defaults = {'timeout': 30, 'retries': 3}
    result = defaults.copy()
    result.update(config)
    return result
```

解析

55 • 填空 / 简答

阅读以下代码，回答问题：

```
class ImmutablePoint:
    __slots__ = ('x', 'y')
    def __init__(self, x, y):
        object.__setattr__(self, 'x', x)
        object.__setattr__(self, 'y', y)
    def __setattr__(self, name, value):
        raise AttributeError("Cannot modify immutable object")
    def __hash__(self):
        return hash((self.x, self.y))
p1 = ImmutablePoint(1, 2)
p2 = ImmutablePoint(1, 2)
print(p1 == p2)
print(hash(p1) == hash(p2))
d = {p1: 'point1'}
d[p2] = 'point2'
```

print(len(d))
问题：
- (1) 三个print的输出分别是什么？
- (2) 为什么需要实现__hash__()？
- (3) 这个类还缺少什么方法？

答案 不可变Point类
- (1) 输出：False, True, 1
- (2) 实现 __hash__() 使对象可以作为字典键和集合元素
- (3) 缺少 __eq__() 方法，导致p1 == p2返回False

解析

五、程序设计题

56 • 填空 / 简答

实现一个写时复制（Copy-on-Write）的列表容器：
1. COWList类：
- 初始时不复制数据
- 第一次修改时才复制
- 多个实例可以共享数据
2. 功能要求
功能要求：
- 支持所有列表操作
- 追踪引用计数
- 优化内存使用
3. 性能测试
性能测试：
- 对比普通列表
- 测量内存占用
- 测量操作速度

答案
```
import copy
from weakref import WeakValueDictionary
class COWList:
    """
    写时复制（Copy-on-Write）列表容器
    设计思路：
    1. 多个实例共享同一份数据
    2. 只有在修改时才复制数据
    3. 使用引用计数追踪共享
    4. 优化内存使用
    特性：
    - 读操作：O(1)，无复制开销
    - 写操作：首次写时复制，后续O(1)
    - 内存优化：多个实例共享数据
    使用示例：
    list1 = COWList([1, 2, 3])
```

```
    list2 = COWList.from_cow(list1) # 共享数据
    list2.append(4) # 触发复制
    """
    # 类级别的数据存储池
    _data_pool = WeakValueDictionary()
    _next_id = 0
    def __init__(self, data=None):
        """
        初始化COW列表
        参数：
        - data: 初始数据
        """
        if data is None:
            data = []
    # 为数据分配唯一ID
    self._data_id = COWList._next_id
    COWList._next_id += 1
    # 存储数据
    self._data = list(data)
    self._is_shared = False
    self._ref_count = 1
@classmethod
def from_cow(cls, other):
    """
    从另一个COW列表创建共享实例
    参数：
    - other: 另一个COWList实例
    返回：
    - COWList: 新实例，共享数据
    """
    new_instance = cls.__new__(cls)
    new_instance._data = other._data
    new_instance._data_id = other._data_id
    new_instance._is_shared = True
    other._is_shared = True
    new_instance._ref_count = other._ref_count
    other._ref_count += 1
    return new_instance
def _ensure_unique(self):
    """确保数据独占（写时复制）"""
    if self._is_shared:
        print(f" [COW] 触发复制，原引用计数: {self._ref_count}")
        self._data = self._data.copy()
        self._is_shared = False
        self._ref_count = 1
# === 列表操作 ===
def append(self, item):
    """添加元素"""
```

```
    self._ensure_unique()
    self._data.append(item)
def extend(self, items):
    """扩展列表"""
    self._ensure_unique()
    self._data.extend(items)
def insert(self, index, item):
    """插入元素"""
    self._ensure_unique()
    self._data.insert(index, item)
def remove(self, item):
    """移除元素"""
    self._ensure_unique()
    self._data.remove(item)
def pop(self, index=-1):
    """弹出元素"""
    self._ensure_unique()
    return self._data.pop(index)
def clear(self):
    """清空列表"""
    self._ensure_unique()
    self._data.clear()
def sort(self, **kwargs):
    """排序"""
    self._ensure_unique()
    self._data.sort(**kwargs)
# === 只读操作（不触发复制）===
def __getitem__(self, index):
    """获取元素"""
    return self._data[index]
def __len__(self):
    """长度"""
        return len(self._data)
    def __iter__(self):
        """迭代"""
        return iter(self._data)
    def __repr__(self):
        """字符串表示"""
        shared_marker = " [共享]" if self._is_shared else ""
        return f"COWList({self._data}){shared_marker}"
    def get_memory_info(self):
        """获取内存信息"""
        import sys
        return {
            'data_size': sys.getsizeof(self._data),
            'is_shared': self._is_shared,
            'ref_count': self._ref_count,
            'data_id': id(self._data)
```

```
        }
# === 使用示例 ===
print("=== 写时复制容器示例 ===\n")
# 示例1：共享数据
print("示例1：数据共享")
list1 = COWList([1, 2, 3, 4, 5])
print(f"list1: {list1}")
print(f"内存信息: {list1.get_memory_info()}")
list2 = COWList.from_cow(list1)
print(f"\nlist2（共享自list1）: {list2}")
print(f"list1数据ID: {id(list1._data)}")
print(f"list2数据ID: {id(list2._data)}")
print(f"数据共享: {id(list1._data) == id(list2._data)}")
# 示例2：写时复制
print("\n示例2：写时复制触发")
print("修改list2...")
list2.append(6)
print(f"list1: {list1}")
print(f"list2: {list2}")
print(f"list1数据ID: {id(list1._data)}")
print(f"list2数据ID: {id(list2._data)}")
print(f"数据已分离: {id(list1._data) != id(list2._data)}")
# 示例3：性能对比
print("\n示例3：性能测试")
import time
import sys
def test_normal_copy(n):
    """测试普通复制"""
    lists = []
    start = time.time()
    base = list(range(n))
    for _ in range(100):
        lists.append(base.copy())
    return time.time() - start, sum(sys.getsizeof(lst) for lst in lists)
def test_cow(n):
    """测试COW"""
    lists = []
    start = time.time()
    base = COWList(list(range(n)))
    for _ in range(100):
        lists.append(COWList.from_cow(base))
    return time.time() - start, sum(sys.getsizeof(lst._data) for lst in lists)
n = 1000
normal_time, normal_memory = test_normal_copy(n)
cow_time, cow_memory = test_cow(n)
print(f"数据大小: {n}个元素")
print(f"普通复制: 时间={normal_time:.4f}s, 内存≈{normal_memory}字节")
print(f"COW复制: 时间={cow_time:.4f}s, 内存≈{cow_memory}字节")
```

```
print(f"内存节省: {(1 - cow_memory/normal_memory)*100:.1f}%")
```

解析

57 • 填空 / 简答

实现一个完整的不可变配置系统：

1. Config类：

- 完全不可变
- 支持嵌套配置
- 提供便捷访问

2. ConfigBuilder：

- 构建器模式
- 验证配置
- 生成不可变实例

3. 功能

功能：

- 配置合并
- 配置继承
- 配置导出

答案

```
from collections.abc import Mapping
import json
from pathlib import Path
class Config(Mapping):
    """
    完全不可变的配置对象
    特性：
    1. 初始化后完全不可修改
    2. 支持嵌套配置（点号访问）
    3. 可哈希，能作为字典键
    4. 支持配置合并和继承
    使用示例：
    config = Config({'database': {'host': 'localhost'}})
    print(config['database.host'])
    """
    __slots__ = ('_data', '_hash')
    def __init__(self, data):
        """
        初始化配置
        参数：
        - data: 配置字典
        """
        # 递归转换为不可变
        object.__setattr__(self, '_data', self._freeze(data))
        object.__setattr__(self, '_hash', None)
    def _freeze(self, obj):
        """递归冻结对象"""
        if isinstance(obj, dict):
            return {k: self._freeze(v) for k, v in obj.items()}
```

```
        elif isinstance(obj, list):
            return tuple(self._freeze(item) for item in obj)
    elif isinstance(obj, set):
        return frozenset(self._freeze(item) for item in obj)
    else:
        return obj
def get(self, path, default=None):
    """
    使用点号路径获取配置
    参数：
    - path: 配置路径，如 'database.host'
    - default: 默认值
    返回：
    - 配置值或默认值
    """
    keys = path.split('.')
    value = self._data
    for key in keys:
        if isinstance(value, dict) and key in value:
            value = value[key]
        else:
            return default
    # 如果是字典，包装为Config
    if isinstance(value, dict):
        return Config(value)
    return value
# === Mapping接口 ===
def __getitem__(self, key):
    """支持字典访问"""
    if '.' in key:
        return self.get(key)
    value = self._data[key]
    if isinstance(value, dict):
        return Config(value)
    return value
def __iter__(self):
    """迭代键"""
    return iter(self._data)
def __len__(self):
    """长度"""
    return len(self._data)
# === 禁止修改 ===
def __setattr__(self, name, value):
    """禁止设置属性"""
    raise AttributeError("Config对象不可变")
def __setitem__(self, key, value):
    """禁止设置项"""
    raise TypeError("Config对象不可变")
```

```
# === 哈希支持 ===
def __hash__(self):
    """计算哈希值"""
    if self._hash is None:
        object.__setattr__(self, '_hash', hash(self._to_hashable(self._data)))
    return self._hash
def _to_hashable(self, obj):
    """转换为可哈希对象"""
    if isinstance(obj, dict):
        return tuple(sorted((k, self._to_hashable(v)) for k, v in obj.items()))
    elif isinstance(obj, (list, tuple)):
        return tuple(self._to_hashable(item) for item in obj)
    elif isinstance(obj, set):
        return frozenset(self._to_hashable(item) for item in obj)
    else:
        return obj
def __repr__(self):
    """字符串表示"""
    return f"Config({self._data})"
def merge(self, other):
    """
    合并配置，返回新Config
    参数：
    - other: 另一个配置字典或Config对象
    返回：
    - Config: 合并后的新配置
    """
    if isinstance(other, Config):
        other = other._data
    merged = self._deep_merge(self._data, other)
    return Config(merged)
def _deep_merge(self, base, update):
    """深度合并字典"""
    result = base.copy()
    for key, value in update.items():
        if key in result and isinstance(result[key], dict) and isinstance(value, dict)
            result[key] = self._deep_merge(result[key], value)
        else:
            result[key] = value
    return result
def to_dict(self):
    """转换为普通字典"""
    return self._unfreeze(self._data)
def _unfreeze(self, obj):
    """递归解冻对象"""
    if isinstance(obj, dict):
        return {k: self._unfreeze(v) for k, v in obj.items()}
    elif isinstance(obj, tuple):
```

```
        return [self._unfreeze(item) for item in obj]
    elif isinstance(obj, frozenset):
        return {self._unfreeze(item) for item in obj}
    else:
        return obj
class ConfigBuilder:
    """
    配置构建器
    功能：
    1. 流式API构建配置
    2. 配置验证
    3. 从文件加载
    4. 配置继承
    使用示例：
    config = (ConfigBuilder()
        .set('app.name', 'MyApp')
        .set('app.version', '1.0')
        .validate()
        .build())
    """
    def __init__(self):
        """初始化构建器"""
        self._config = {}
        self._validators = []
    def set(self, path, value):
        """
        设置配置值
        参数：
        - path: 配置路径（支持点号）
        - value: 配置值
        返回：
        - self: 支持链式调用
        """
        keys = path.split('.')
        current = self._config
        for key in keys[:-1]:
        if key not in current:
            current[key] = {}
        current = current[key]
    current[keys[-1]] = value
    return self
def merge(self, config_dict):
    """
    合并配置字典
    参数：
    - config_dict: 要合并的配置
    返回：
    - self: 支持链式调用
```

```
    """
    self._deep_merge(self._config, config_dict)
    return self
def _deep_merge(self, base, update):
    """深度合并"""
    for key, value in update.items():
        if key in base and isinstance(base[key], dict) and isinstance(value, dict):
            self._deep_merge(base[key], value)
        else:
            base[key] = value
def load_from_file(self, filepath):
    """
    从JSON文件加载配置
    参数:
    - filepath: 文件路径
    返回:
    - self: 支持链式调用
    """
    path = Path(filepath)
    if path.exists():
        with path.open('r', encoding='utf-8') as f:
            config = json.load(f)
        self.merge(config)
    return self
def add_validator(self, validator_func, error_msg=None):
    """
    添加验证器
    参数:
    - validator_func: 验证函数，接收config字典，返回bool
    - error_msg: 错误消息
    返回:
    - self: 支持链式调用
    """
    self._validators.append((validator_func, error_msg))
    return self
def validate(self):
    """
    验证配置
    返回:
    - self: 支持链式调用
    抛出:
    - ValueError: 验证失败
    """
    for validator, error_msg in self._validators:
        if not validator(self._config):
            msg = error_msg or f"验证失败: {validator.__name__}"
            raise ValueError(msg)
    return self
```

```
def build(self):
    """
    构建不可变配置对象
    返回：
        - Config: 不可变配置
        """
        return Config(self._config)
    def save_to_file(self, filepath):
        """
        保存配置到文件
        参数：
        - filepath: 文件路径
        """
        path = Path(filepath)
        with path.open('w', encoding='utf-8') as f:
            json.dump(self._config, f, indent=2, ensure_ascii=False)
# === 使用示例 ===
print("=== 不可变配置系统示例 ===\n")
# 示例1：创建配置
print("示例1：创建和访问配置")
config = Config({
    'app': {
        'name': 'MyApp',
        'version': '1.0.0',
        'debug': False
    },
    'database': {
        'host': 'localhost',
        'port': 5432,
        'credentials': {
            'user': 'admin',
            'password': 'secret'
    },
    'features': ['auth', 'api', 'cache']
})
print(f"应用名称: {config['app']['name']}")
print(f"数据库主机: {config.get('database.host')}")
print(f"用户名: {config.get('database.credentials.user')}")
print(f"功能列表: {config['features']}")
# 示例2：配置合并
print("\n示例2：配置合并")
base_config = Config({'app': {'name': 'Base', 'debug': True}})
override_config = {'app': {'debug': False}, 'new_key': 'value'}
merged = base_config.merge(override_config)
print(f"基础配置: {base_config}")
print(f"合并后: {merged}")
# 示例3：使用构建器
print("\n示例3：使用ConfigBuilder")
```

```
def validate_port(config):
    """验证端口范围"""
    if 'database' in config and 'port' in config['database']:
        port = config['database']['port']
        return 1024 <= port <= 65535
    return True
def validate_required(config):
    """验证必需字段"""
    return 'app' in config and 'name' in config['app']
built_config = (ConfigBuilder()
    .set('app.name', 'BuilderApp')
    .set('app.version', '2.0.0')
    .set('database.host', 'localhost')
    .set('database.port', 5432)
    .add_validator(validate_port, "端口必须在1024-65535之间")
    .add_validator(validate_required, "缺少必需的app.name字段")
    .validate()
    .build())
print(f"构建的配置: {built_config}")
print(f"配置哈希: {hash(built_config)}")
# 示例4：配置作为字典键
print("\n示例4：配置作为缓存键")
cache = {
    built_config: "cached_data_1",
    merged: "cached_data_2"
}
print(f"缓存条目数: {len(cache)}")
```

解析

58 • 填空 / 简答

实现完整的函数参数验证框架：

1. 验证装饰器

验证装饰器：

- @validate_types ：类型验证
- @validate_range ：范围验证
- @validate_immutable ：不可变验证

2. 自动保护

自动保护：

- 自动复制可变参数
- 防止意外修改
- 详细错误信息

3. 性能优化

性能优化：

- 缓存验证结果
- 最小化复制
- 可配置验证级别

答案

```
from functools import wraps
import inspect
from typing import get_type_hints, Any
import copy
def validate_types(func):
    """
    类型验证装饰器
    功能：
    - 基于类型注解验证参数类型
    - 类型不匹配时抛出TypeError
    - 支持可选类型（Union, Optional）
    使用示例：
    @validate_types
    def func(name: str, age: int) -> str:
        return f"{name} is {age}"
    """
    @wraps(func)
    def wrapper(*args, **kwargs):
        # 获取类型提示
        hints = get_type_hints(func)
        sig = inspect.signature(func)
        bound = sig.bind(*args, **kwargs)
        bound.apply_defaults()
        # 验证每个参数
        for param_name, param_value in bound.arguments.items():
            if param_name in hints and param_name != 'return':
                expected_type = hints[param_name]
                # 处理泛型类型
                if hasattr(expected_type, '__origin__'):
                # Union, Optional等
                continue
                if not isinstance(param_value, expected_type):
                raise TypeError(
                f"参数'{param_name}'类型错误: "
                f"期望{expected_type.__name__}, "
                f"得到{type(param_value).__name__}"
                )
        return func(*args, **kwargs)
    return wrapper
def validate_range(**ranges):
    """
    范围验证装饰器
    功能：
    - 验证数值参数的范围
    - 支持min和max限制
    使用示例：
    @validate_range(age=(0, 150), score=(0, 100))
    def func(age, score):
```

```
        ...
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            sig = inspect.signature(func)
            bound = sig.bind(*args, **kwargs)
            bound.apply_defaults()
            for param_name, (min_val, max_val) in ranges.items():
                if param_name in bound.arguments:
                value = bound.arguments[param_name]
                if not (min_val <= value <= max_val):
                raise ValueError(
                f"参数'{param_name}'超出范围: "
                f"{value} 不在 [{min_val}, {max_val}] 内"
                )
            return func(*args, **kwargs)
        return wrapper
    return decorator
def validate_immutable(*param_names):
    """
    不可变验证装饰器
    功能:
    - 确保指定参数是不可变对象
    - 可变对象会抛出TypeError
    使用示例:
    @validate_immutable('config', 'settings')
    def func(config, settings):
        ...
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            sig = inspect.signature(func)
            bound = sig.bind(*args, **kwargs)
            immutable_types = (int, float, str, tuple, frozenset, bytes, bool, type(None))
            for param_name in param_names:
                if param_name in bound.arguments:
                value = bound.arguments[param_name]
                if not isinstance(value, immutable_types):
                # 检查元组是否包含可变对象
                if isinstance(value, tuple):
                if any(not isinstance(item, immutable_types) for item in value
                raise TypeError(
                f"参数'{param_name}'必须是不可变对象, "
                f"但元组包含可变元素"
                )
                else:
```

```
                raise TypeError(
                f"参数'{param_name}'必须是不可变对象, "
                f"得到{type(value).__name__}"
                )
            return func(*args, **kwargs)
        return wrapper
    return decorator
def auto_copy_params(deep=False):
    """
    自动复制可变参数装饰器
    功能:
    - 自动检测并复制可变参数
    - 防止函数修改外部对象
    - 可配置深拷贝或浅拷贝
    使用示例:
    @auto_copy_params(deep=True)
    def func(data):
        data.append(1) # 不会影响外部
    """
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            def is_mutable(obj):
                immutable = (int, float, str, tuple, frozenset, bytes, bool, type(None))
                return not isinstance(obj, immutable)
            # 复制可变参数
            new_args = []
            for arg in args:
                if is_mutable(arg):
                new_args.append(copy.deepcopy(arg) if deep else copy.copy(arg))
                else:
                new_args.append(arg)
            new_kwargs = {}
            for key, value in kwargs.items():
                if is_mutable(value):
                new_kwargs[key] = copy.deepcopy(value) if deep else copy.copy(value)
                else:
                new_kwargs[key] = value
            return func(*new_args, **new_kwargs)
        return wrapper
    return decorator
# === 使用示例 ===
print("=== 参数验证框架示例 ===\n")
# 示例1：类型验证
print("示例1：类型验证")
@validate_types
def greet(name: str, age: int, active: bool = True) -> str:
    return f"{name}, {age}岁, 活跃:{active}"
```

```
try:
    print(greet("Alice", 30))
    print(greet("Bob", "25"))
# 类型错误
except TypeError as e:
    print(f"  {e}")
# 示例2：范围验证
print("\n示例2：范围验证")
@validate_range(age=(0, 150), score=(0, 100))
def register_student(name, age, score):
    return f"{name}: {age}岁, 分数{score}"
try:
    print(register_student("Alice", 20, 95))
    print(register_student("Bob", 200, 95))
# 年龄超出范围
except ValueError as e:
    print(f"  {e}")
# 示例3：不可变性验证
print("\n示例3：不可变性验证")
@validate_immutable('config')
def process_config(config):
    return f"处理配置: {config}"
try:
    print(process_config((1, 2, 3)))
    print(process_config([1, 2, 3]))
# 可变对象
except TypeError as e:
    print(f"  {e}")
# 示例4：自动复制参数
print("\n示例4：自动参数保护")
@auto_copy_params(deep=True)
def modify_data(data):
    data.append(999)
    return data
original = [1, 2, 3]
result = modify_data(original)
print(f"原数据: {original}")
print(f"返回值: {result}")
# 示例5：组合使用
print("\n示例5：组合多个验证器")
@validate_types
@validate_range(age=(0, 150))
@auto_copy_params(deep=True)
def update_user(name: str, age: int, hobbies: list):
    hobbies.append("reading")
    return {'name': name, 'age': age, 'hobbies': hobbies}
user_hobbies = ['coding', 'music']
result = update_user("Alice", 25, user_hobbies)
```

```
print(f"原爱好列表: {user_hobbies}")
print(f"更新结果: {result}")
```

评分说明：

- 单选题：每题1分，共30分
- 多选题：每题2分，选对但不全得1分，有错选不得分，共30分
- 填空题：每空1分，共10分
- 代码阅读题：每题3分，共15分
- 程序设计题：5选3，每题5分，共15分
- 总分：

总分：100分

解析

59 • 填空 / 简答

实现数据对象的完整版本管理：

1. VersionedObject类：

- 自动版本控制
- 修改历史记录
- 版本回滚

2. 差异追踪

差异追踪：

- 记录增量变化
- 最小化存储
- 快速恢复

3. 分支管理

分支管理：

- 创建分支
- 合并分支
- 冲突解决

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析

60 • 填空 / 简答

实现Pandas的安全操作封装：

答案 （原卷为 5 选 3，未提供本题参考答案；考试时任选 3 题作答即可）

解析